

Password strength code:

```
import re
```

```
def check_password_strength(password):
```

```
    if len(password) < 8:
```

```
        return "Weak: Password must be at least 8 characters long."
```

```
    if not any(char.isdigit() for char in password):
```

```
        return "Weak: Password must include at least one number."
```

```
    if not any(char.isupper() for char in password):
```

```
        return "Weak: Password must include at least one uppercase letter."
```

```
    if not any(char.islower() for char in password):
```

```
        return "Weak: Password must include at least one lowercase letter."
```

```
    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
```

```
        return "Medium: Add special characters to make your password stronger."
```

```
    return "Strong: Your password is secure!"
```

```
def password_checker():
```

```
    print("Welcome to the Password Strength Checker!")
```

```
    while True:
```

```
        password = input("\nEnter your password (or type 'exit' to quit): ")
```

```
        if password.lower() == "exit":
```

```

        print("Thank you for using the Password Strength Checker! Goodbye!")
        break

    result = check_password_strength(password)
    print(result)

if __name__ == "__main__":
    password_checker()

```

Experiment 10: Password Strength Testing using Python

Write a Python program that accepts a password from the user and checks whether it is weak, medium, or strong based on the following rules:

- Minimum 8 characters
- Must contain at least one digit
- Must contain at least one uppercase letter
- Must contain at least one lowercase letter
- Must contain at least one special character

Program: (File name: password_checker.py)

```

import re

def check_password_strength(password):
    if len(password) < 8:
        return "Weak: Password must be at least 8 characters long."
    if not any(char.isdigit() for char in password):

```

```

return "Weak: Password must include at least one number."
if not any(char.isupper() for char in password):
return "Weak: Password must include at least one uppercase letter."
if not any(char.islower() for char in password):
return "Weak: Password must include at least one lowercase letter."

if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password)

return "Medium: Add special characters to make your password stronger."
return "Strong: Your password is secure!"
def password_checker():
print("Welcome to the Password Strength Checker!")
while True:
password = input("\nEnter your password (or type 'exit' to quit): ")
if password.lower() == "exit":
print("Thank you for using the Password Strength Checker! Goodbye!") break
result = check_password_strength(password) print(result)
if __name__ == "__main__": password_checker()

```

Execution Steps:-

1. Open Command Prompt or VS Code Terminal.
2. Navigate to the folder where password_checker.py is saved
3. Run the program: python password_checker.py

The program will display:

Welcome to the Password Strength Checker!

Enter your password (or type 'exit' to quit):

4. Type a password and press Enter.
5. The program will display whether the password is:
 - Weak
 - Medium
 - Strong

6. To stop the program, type: exit

Sample Input & Output:

Input:

Enter your password: abc123

Output:

Weak: Password must be at least 8 characters long.

Input:

Enter your password: Abcdef12

Output:

Medium: Add special characters to make your password stronger.

Input:

Enter your password: Abc@1234

Output:

Strong: Your password is secure!

Experiment 11: Analyzing Phishing Emails

To analyze a suspicious email using EML Analyzer and VirusTotal and identify whether it is phishing based on technical indicators.

Tools Used:

- Online EML Analyzer
- VirusTotal
- Sample file: 2020-05-05-phishing-email-example-01.eml

Questions to Answer:

1. What is the full email address of the sender?
2. What domain is used to send this email?
3. What is the sender's IP address from the header?
4. Is the sender IP blacklisted?
5. What is the result of SPF authentication?
6. What is one suspicious URL found in the email body?

Part A: EML Analyzer Results

The file 2020-05-05-phishing-email-example-01.eml was uploaded to the EML Analyzer. EML Analyzer Results:

Key Observations:

- Subject: Warning: Final Notice
- From: malware-traffic-analysis.net Support sues@nnwifi.com
- To: brad@malware-traffic-analysis.net
- Content Type: text/html
- Message-ID: Missing

Header Analysis:

- Sender IP: 94.100.31.27
- Reverse DNS: 94-100-31-27.static.hvvc.us
- Mail Server: mail.nnwifi.com

Authentication:

- SPF: Failed
- DKIM: Not signed
- DMARC: Not aligned

Part B: VirusTotal Analysis

The sender IP 94.100.31.27 was checked on VirusTotal.

VirusTotal Result:

- Detection Ratio: 1 / 93 vendors flagged as malicious
- Location: Netherlands
- ASN: AS29802 (HVC-AS)

This means the IP is not widely blacklisted but has suspicious reputation.

Suspicious Link Identified

From EML Analyzer, the following URL was extracted:

<https://servervirto.com.co/ed/trn/update?email=brad@malware-traffic-analysis.net> Reasons it is suspicious:

- Does not match sender domain (nnwifi.com)
- Uses foreign domain (.com.co)
- Requests confirmation of ownership (credential harvesting pattern)

Answers to Given Questions

1. Full sender email address: sues@nnwifi.com
2. Domain used to send the email: nnwifi.com
3. Sender's IP address: 94.100.31.27
4. Is sender IP blacklisted? :Yes (1/93 vendors flagged it as malicious in VirusTotal)
5. SPF authentication result: Fail

6. One suspicious URL in email body:

<https://servervirto.com.co/ed/trn/update?email=brad@malware-traffic-analysis.net>

Conclusion:-

Phishing indicators found:

- Urgent subject: "Warning: Final Notice"
- Fake display name pretending to be malware-traffic-analysis.net
- Actual sender domain is unrelated (nnwifi.com)
- SPF authentication failed
- Message-ID missing
- HTML-only email
- Suspicious external link
- IP partially flagged by VirusTotal

Final Verdict:-

This email is classified as PHISHING.

Experiment 12: Packet Sniffing and Network Traffic Analysis with Wireshark

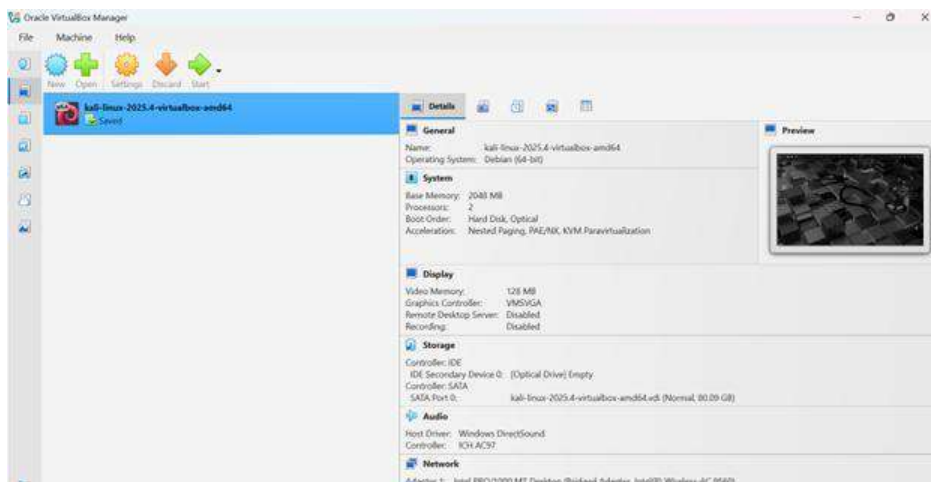
In this experiment, capture live network packets, analyze them, and understand what information an attacker can see, a local HTTP server running on port 8080 is used. Since HTTP is not encrypted, all transmitted data can be viewed in plain text by anyone who captures the traffic. Tools like tcpdump are used to capture the packets, and Wireshark is used to analyze them.

Requirements

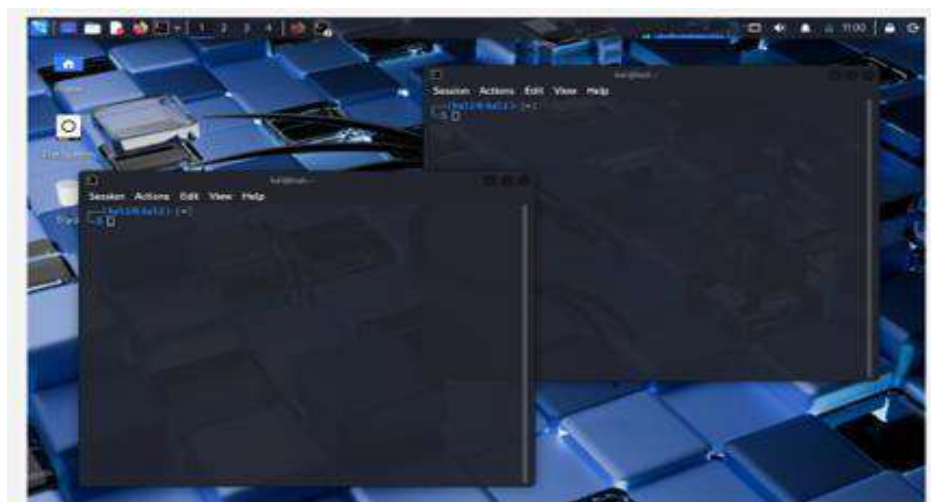
- Kali Linux (in Oracle VirtualBox) [comes with build in Wireshark]
- Local HTTP Server (Python)

Procedure:

Step 1: Open Kali Linux.



Step 2: Open Terminal in Kali Linux.

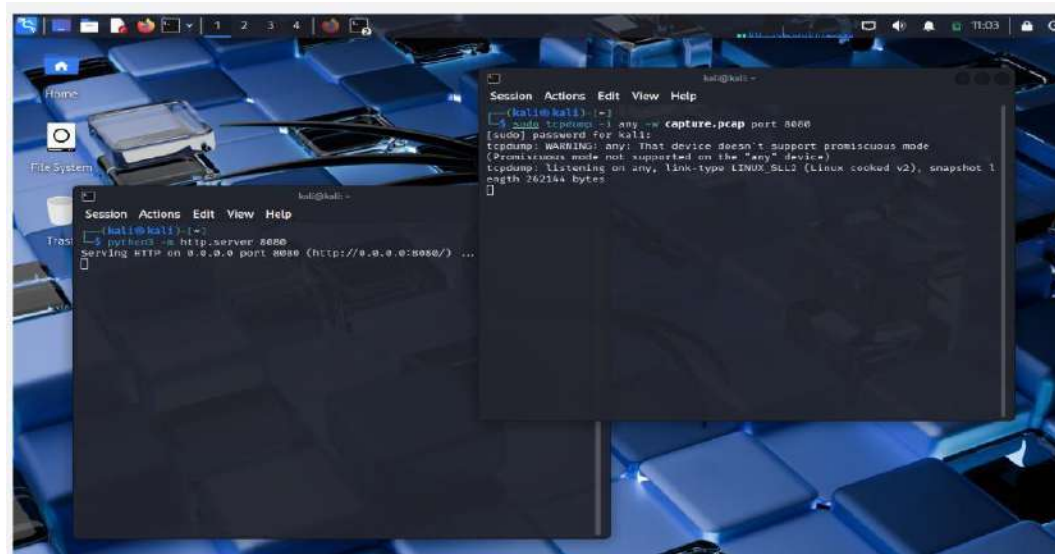


Step 3:

Start a local HTTP server on port 8080 using : `python3 -m http.server 8080`

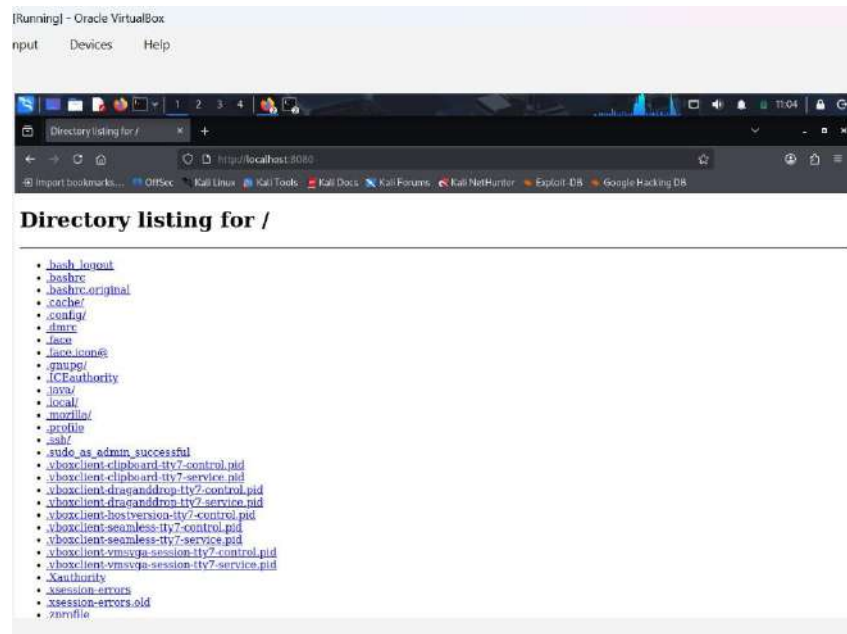


Step 4: In another new terminal start packet capture: `sudo tcpdump -i any -w capture.pcap port 8080`



Step 5: Open Firefox in kali Linux and go to: <http://localhost:8080>

Refresh the page to generate traffic.

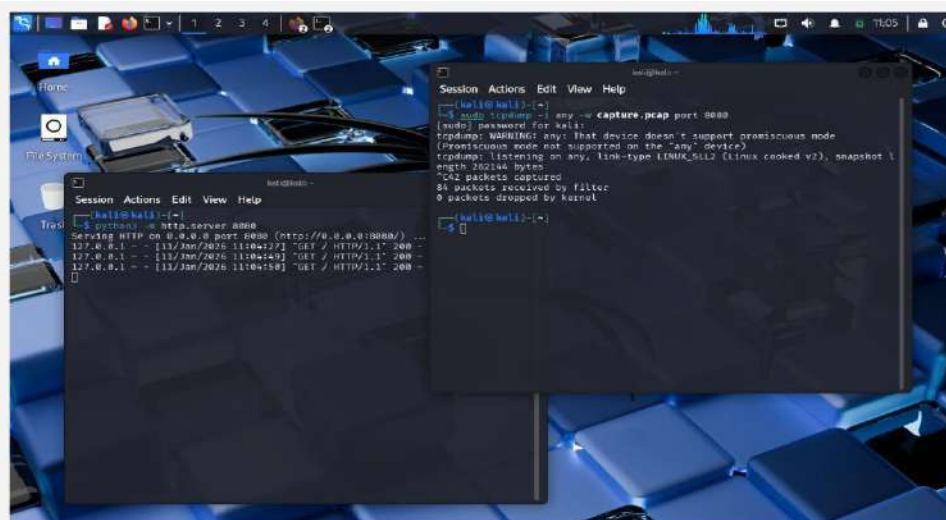


Step 6:

Go back to tcpdump terminal.

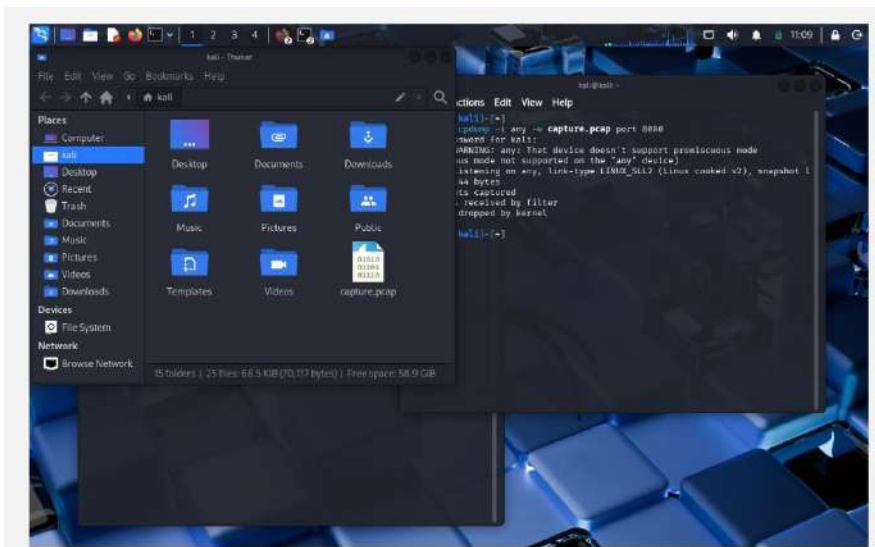
Stop packet capturing by using Ctrl + C.

It will stop and show how many packets were captured The packets are saved as capture.pcap.



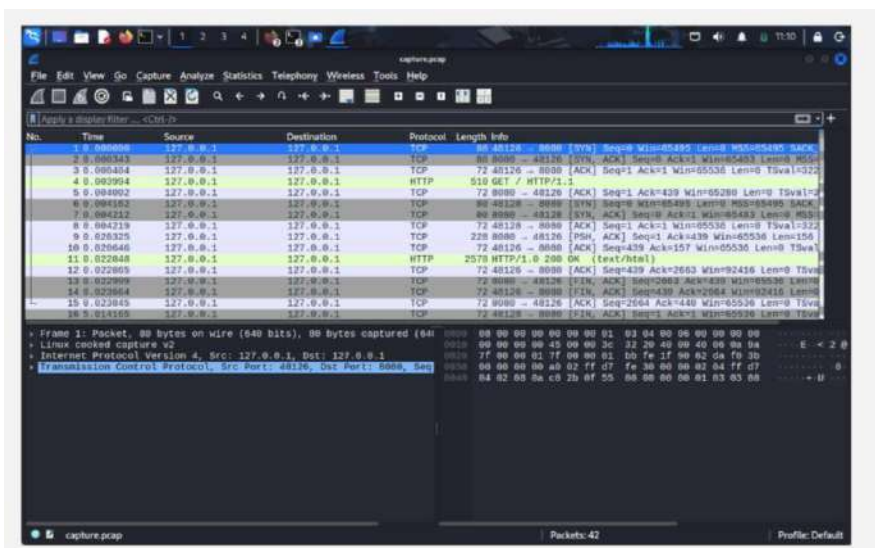
Step 7:

- Click the Kali Linux dragon icon (top left).
- Type: File Manager and open it.
- Your Home folder will open.
- You will see the file: capture.pcap.



Step 8:-

- Since Wireshark is pre-installed in Kali, just double-click capture.pcap.
- The file will open directly in Wireshark for analysis.

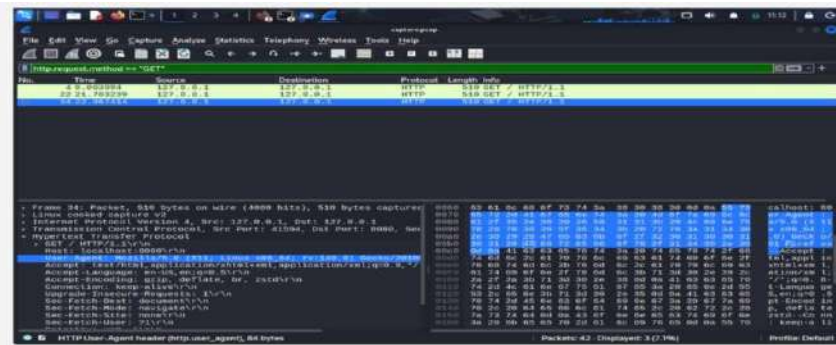


Step 9:-

Filter Login Packets

In Wireshark filter bar, type: `http.request.method == "POST"` Press Enter.

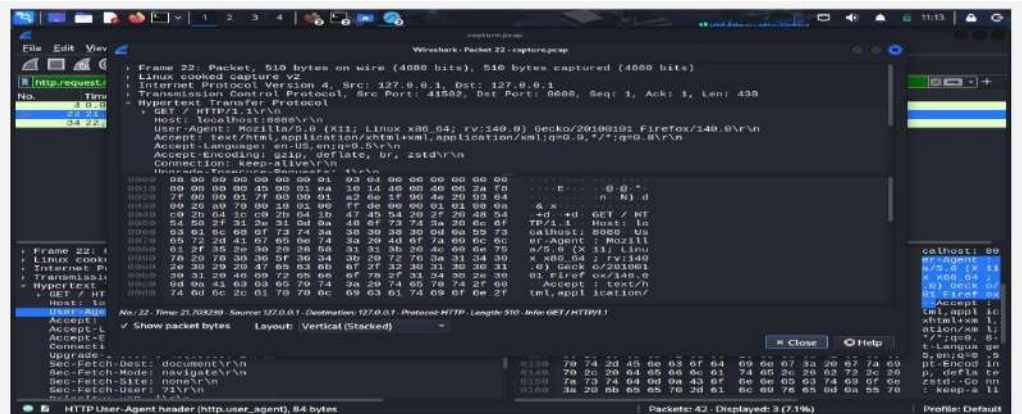
Now only important packets will show.



Step 10:-

Click on any one of the packet and the following data is displayed.

Browser details such as OS, browser version, language, and visited URLs are visible. If a form is submitted, username and password can be seen in plain text. This proves HTTP is insecure



Conclusion :-

Packets were successfully captured and analyzed. Sensitive data is visible when HTTP is used. Packet sniffing and network traffic analysis show that unencrypted communication is unsafe. HTTPS is necessary to protect data.

Experiment 1: SQL Injection (SQLi)

Tool: WebGoat / OWASP Juice Shop / Burp Suite

Objective: Exploit SQL injection vulnerabilities to retrieve unauthorized data from a database.

Tasks: Identify SQLi vulnerabilities, bypass authentication, and extract user details.

SQL Injection using Burp Suite in OWASP Juice Shop

AIM: To perform SQL Injection attack using **Burp Suite** on Juice Shop.

REQUIREMENTS

- Kali Linux
- Burp Suite
- Web browser
- Juice Shop (local or online)

STEP-BY-STEP PROCESS

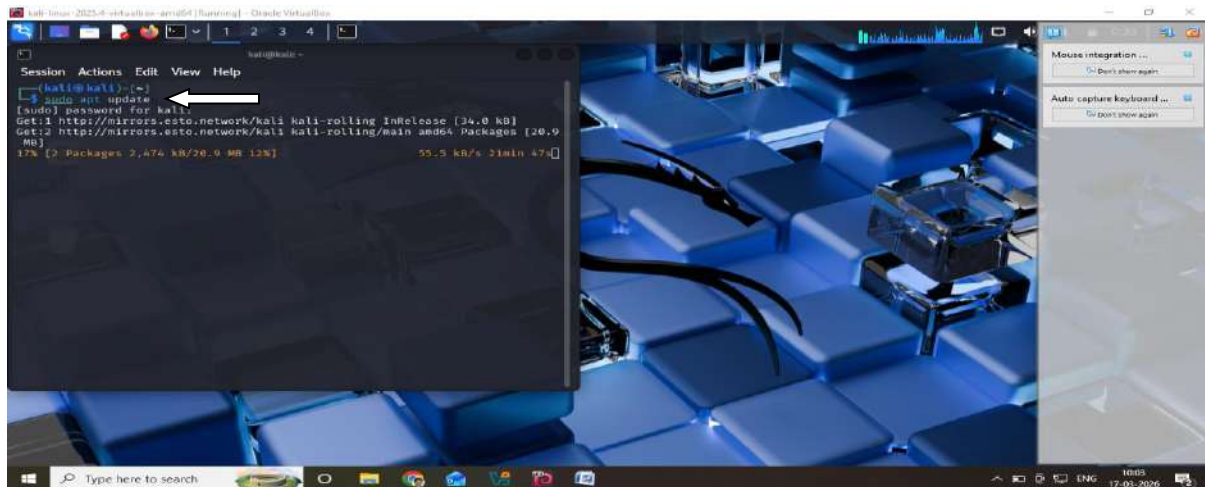
1. Start Juice Shop in Kali Linux

Juice Shop can run with Docker or Node.js. Docker is easiest.

Step 1: Install Docker (if not installed)

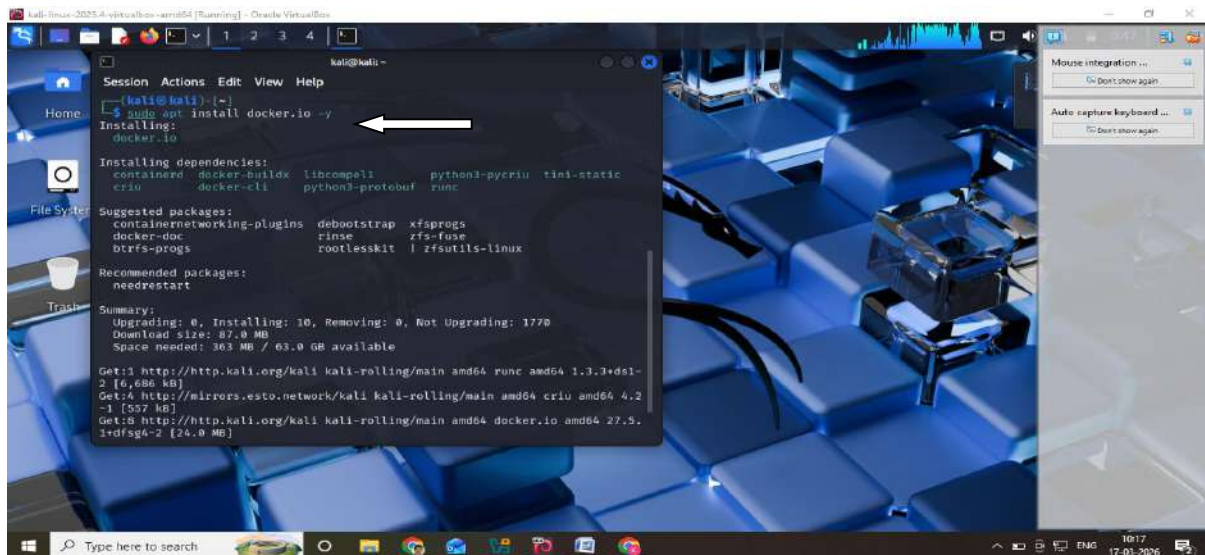
Open terminal in Kali:

sudo apt update



```
kali@kali:~$ sudo apt update
[sudo] password for kali:
Get:1 http://mirrors.esto.network/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirrors.esto.network/kali kali-rolling/main amd64 Packages [20.9 MB]
17% [2 Packages 2,474 kB/20.9 MB 12%] 55.5 kB/s 21min 47s
```

sudo apt install docker.io -y



```
kali@kali:~$ sudo apt install docker.io -y
Installing:
docker.io

Installing dependencies:
containerd docker-buildx libcompelt python3-pycrui tini-static
criu docker-cli python3-protebuf runc

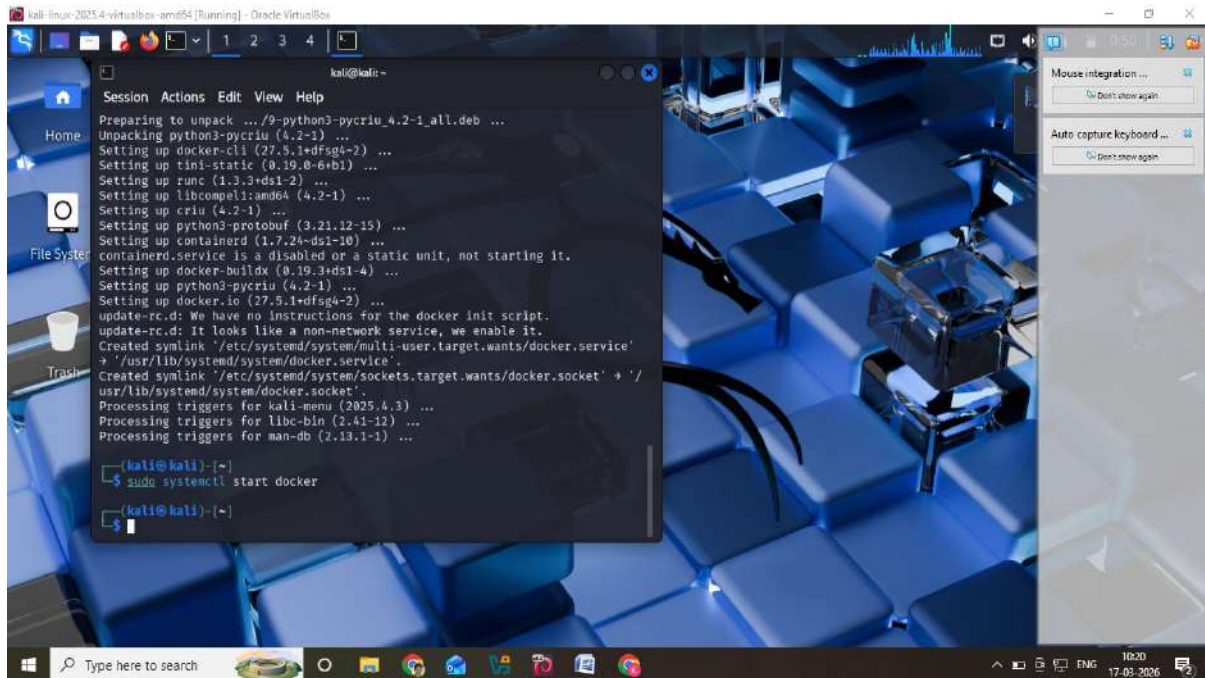
Suggested packages:
containernetworking-plugins debootstrap xfsprogs
docker-doc fuse zfs-fuse
btrfs-progs rootlesskit | zfsutils-linux

Recommended packages:
needrestart

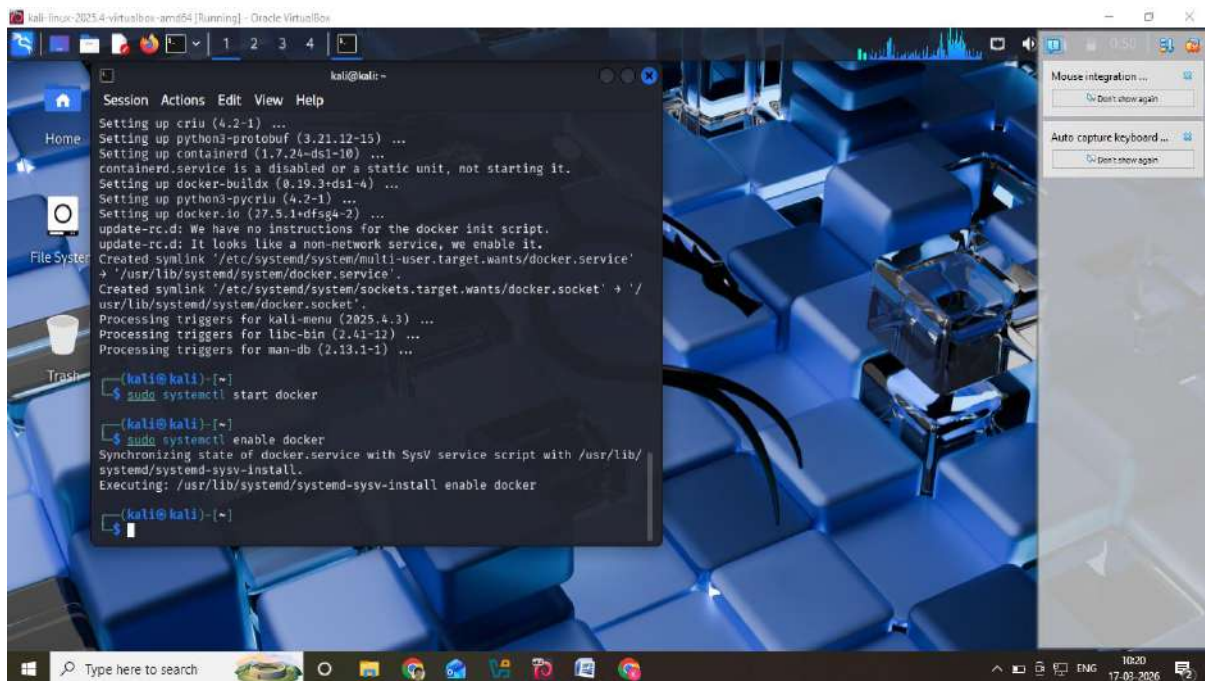
Summary:
Upgrading: 0, Installing: 10, Removing: 0, Not Upgrading: 1770
Download size: 87.0 MB
Space needed: 363 MB / 63.0 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 runc amd64 1.3.3+ds1-2 [6,686 kB]
Get:4 http://mirrors.esto.network/kali kali-rolling/main amd64 criu amd64 4.2-1 [557 kB]
Get:8 http://http.kali.org/kali kali-rolling/main amd64 docker.io amd64 27.5.1+dfsg4-2 [24.0 MB]
```

sudo systemctl start docker

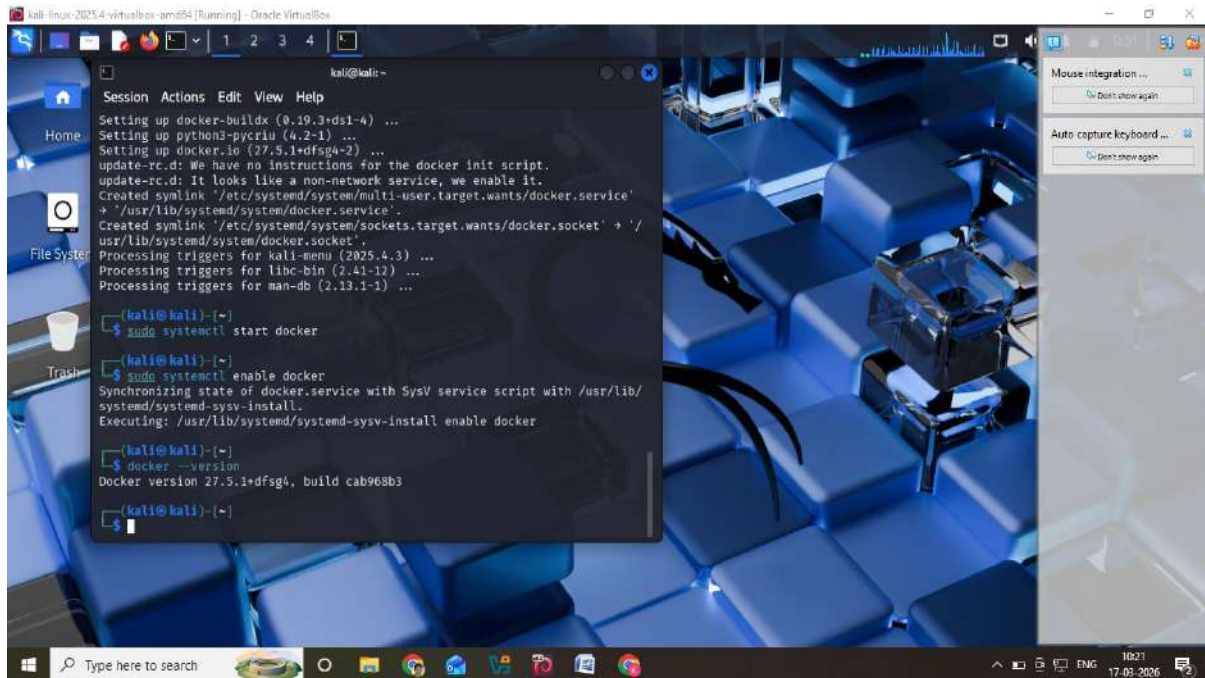


sudo systemctl enable docker



Verify:

docker --version



Run:

sudo usermod -aG docker \$USER

Restart the session (important):

reboot

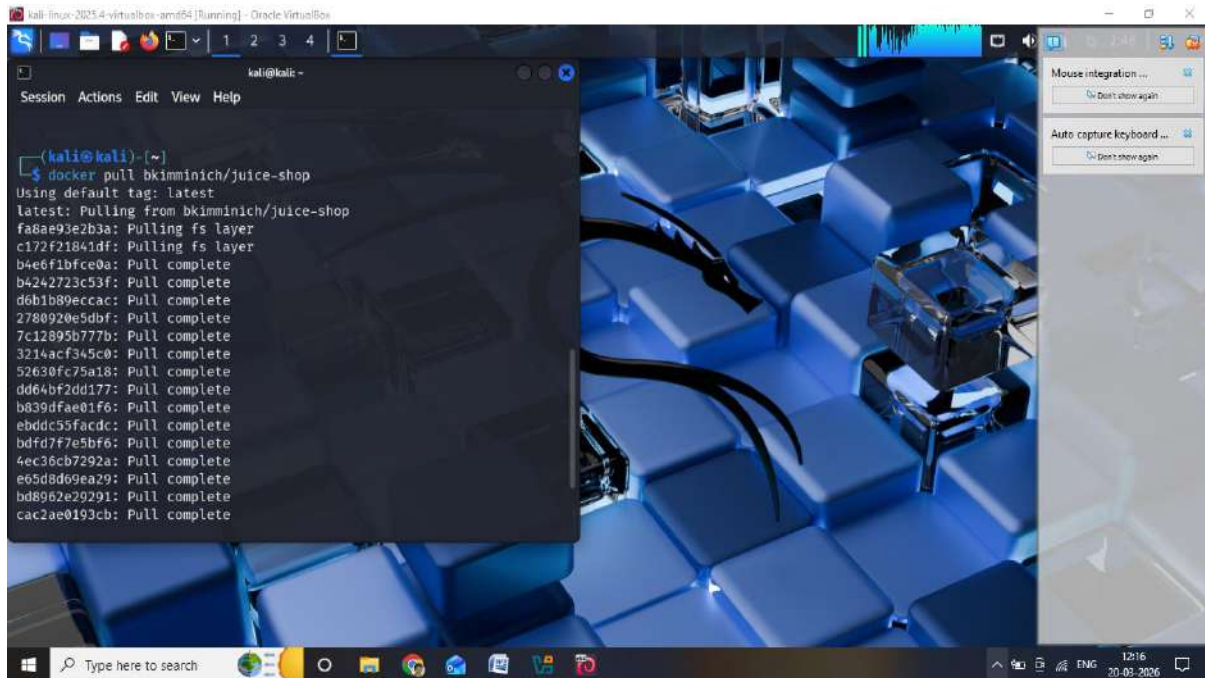
OR logout and login again.

Test Docker:

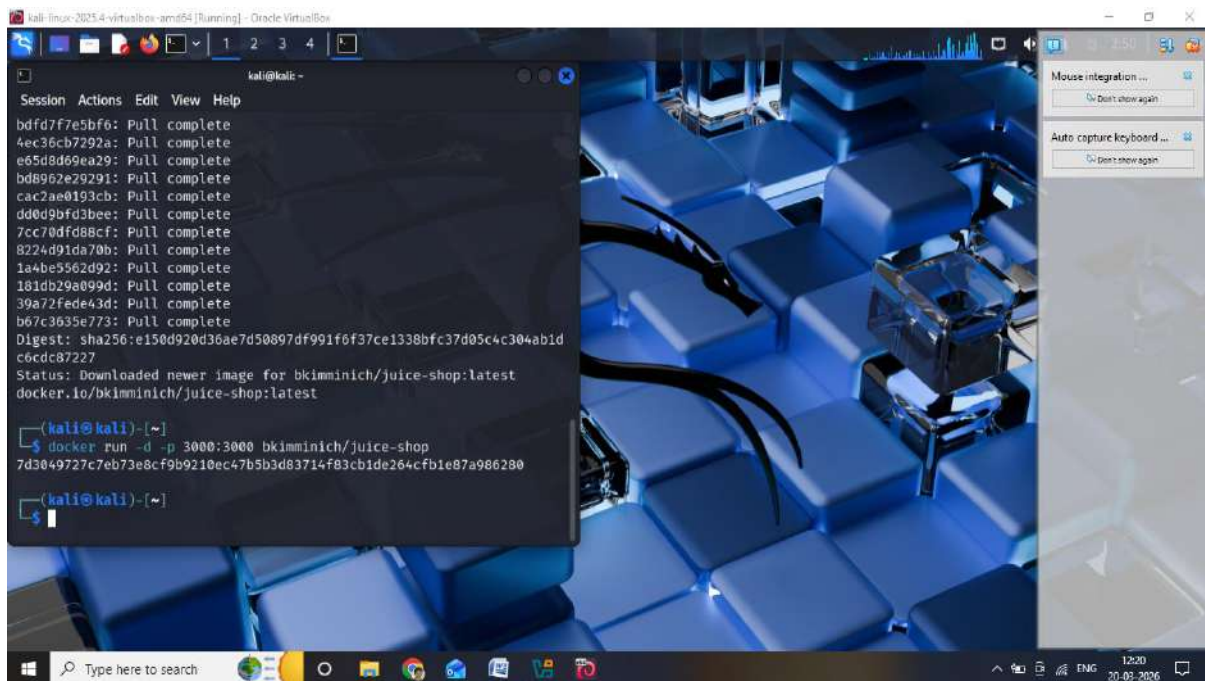
docker run hello-world

Download and Run Juice Shop

docker pull bkimminich/juice-shop



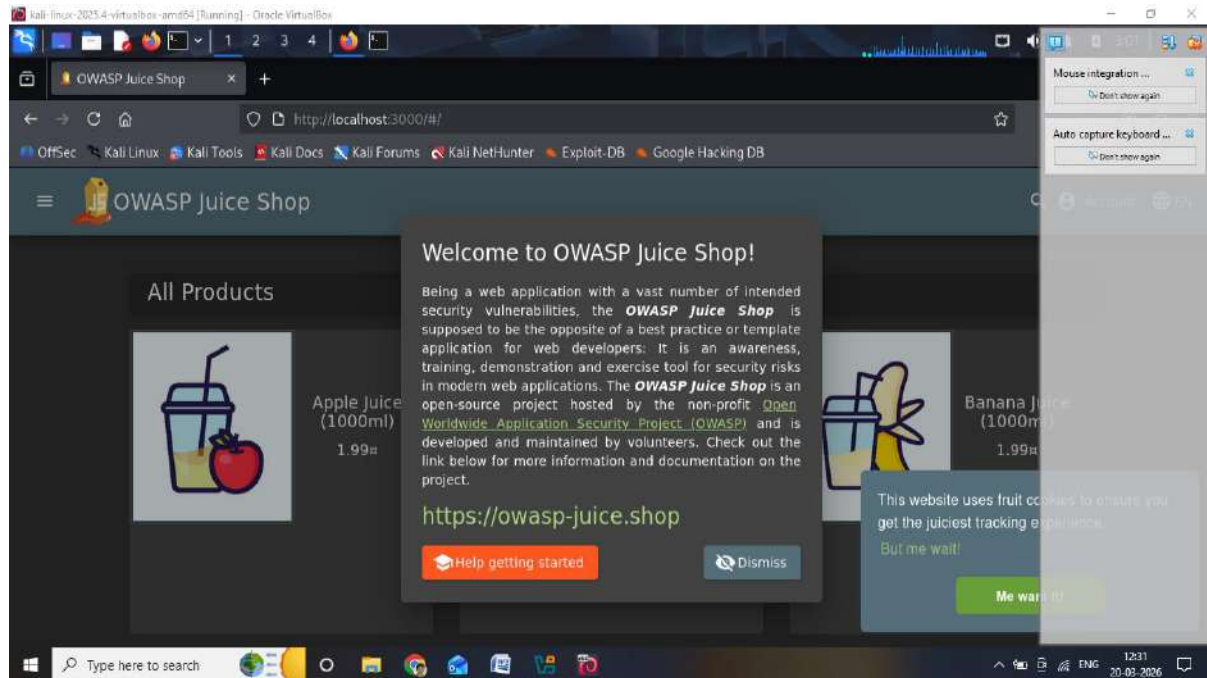
docker run -d -p 3000:3000 bkimminich/juice-shop



Step 3: Open the Application

Open browser in Kali and go to:

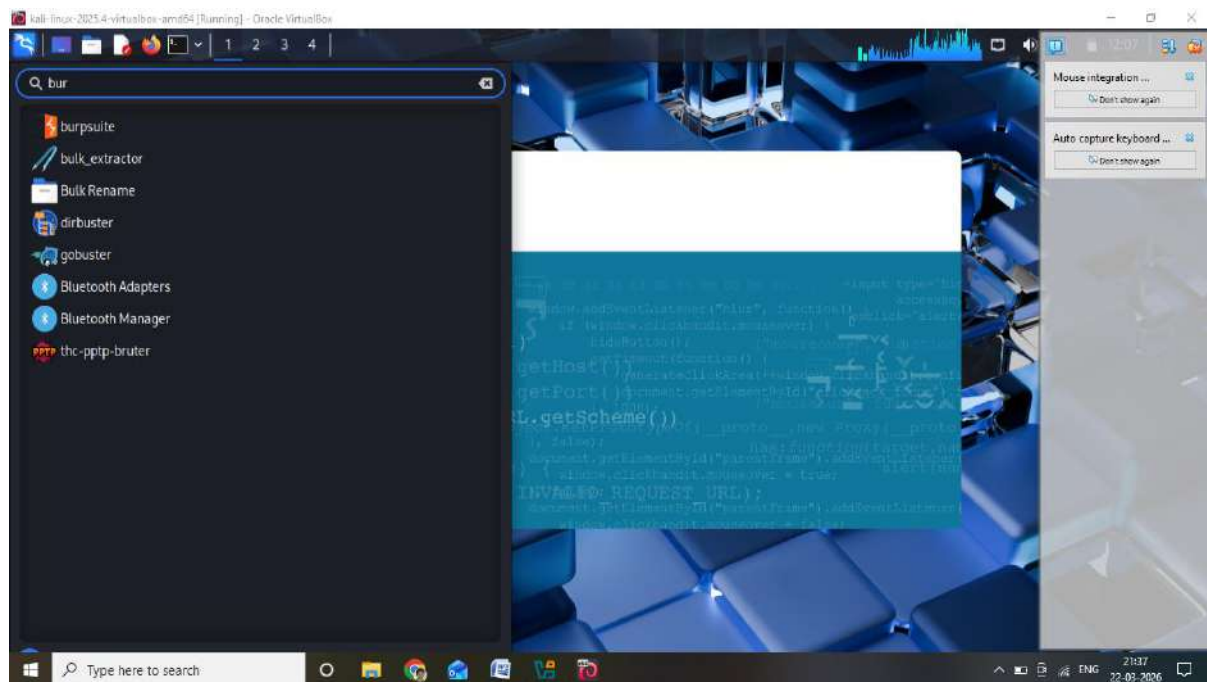
<http://localhost:3000>



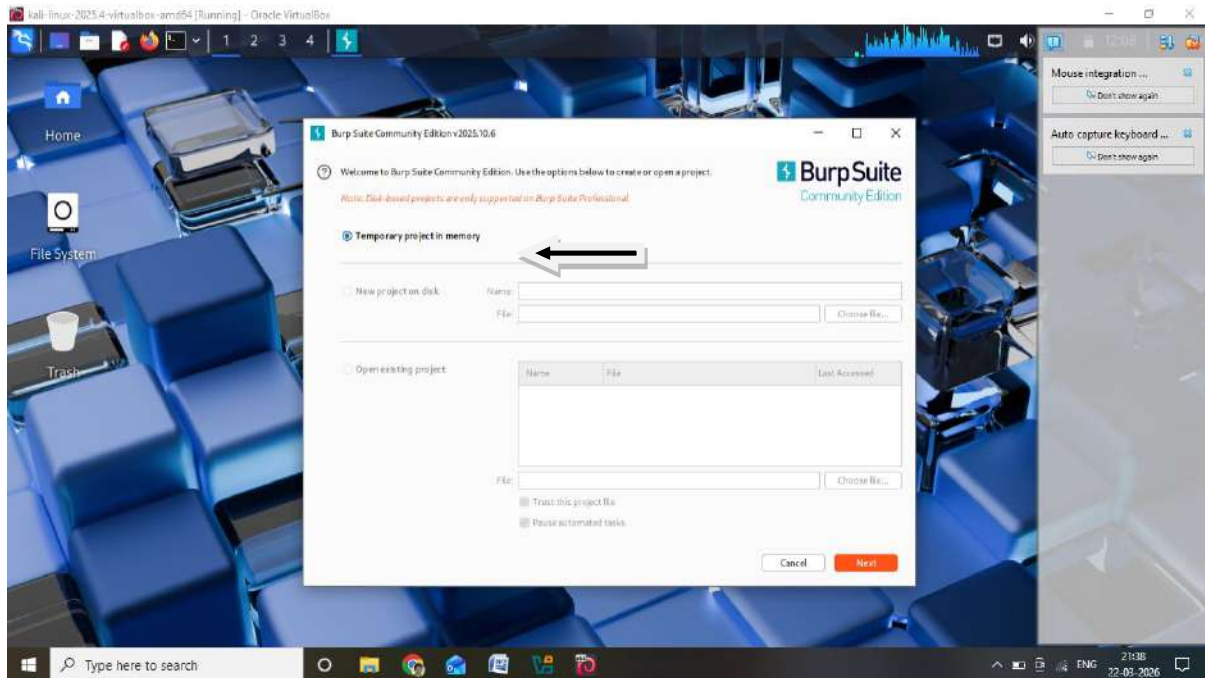
STEP 1: Open Burp Suite

Open terminal:

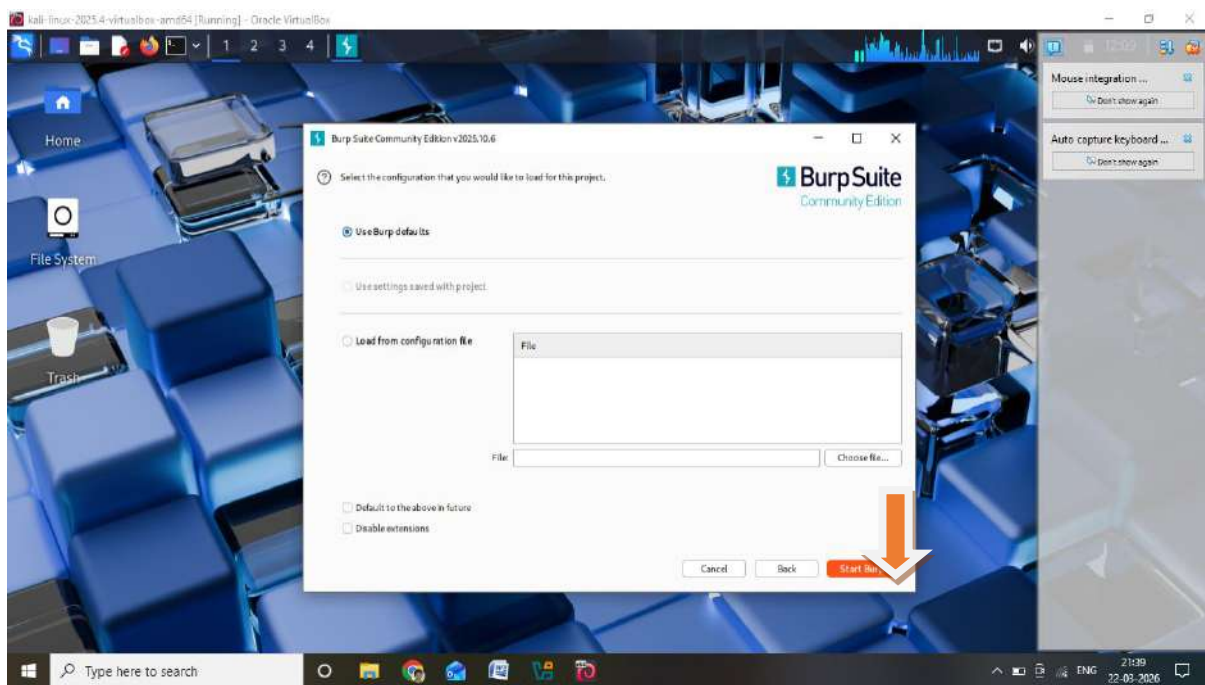
Burpsuite



- Select Temporary Project



- Click Start Burp

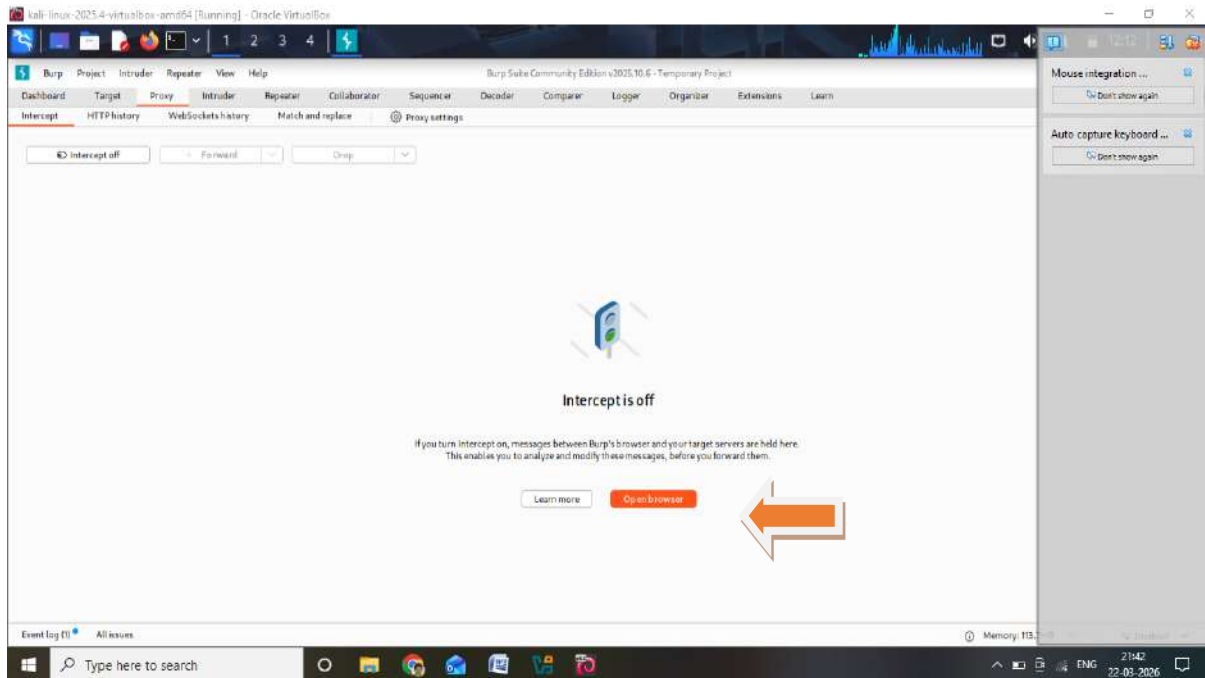


STEP 2: Open Burp Browser

In Burp:

Go to **Proxy tab**

Click **"Open Browser"**



This browser is already configured with proxy (no manual setup needed)

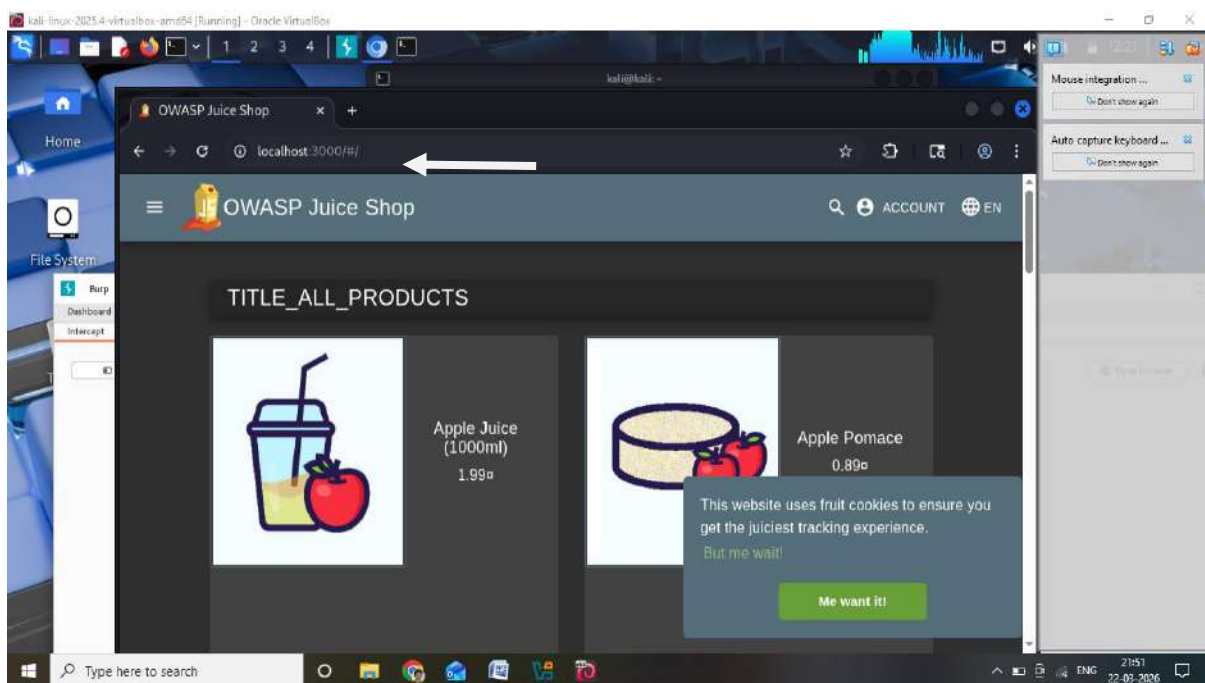
STEP 3: Open Juice Shop Website

In Burp browser, open:

<https://juice-shop.herokuapp.com/#/login>

OR (local):

<http://localhost:3000/#/login>



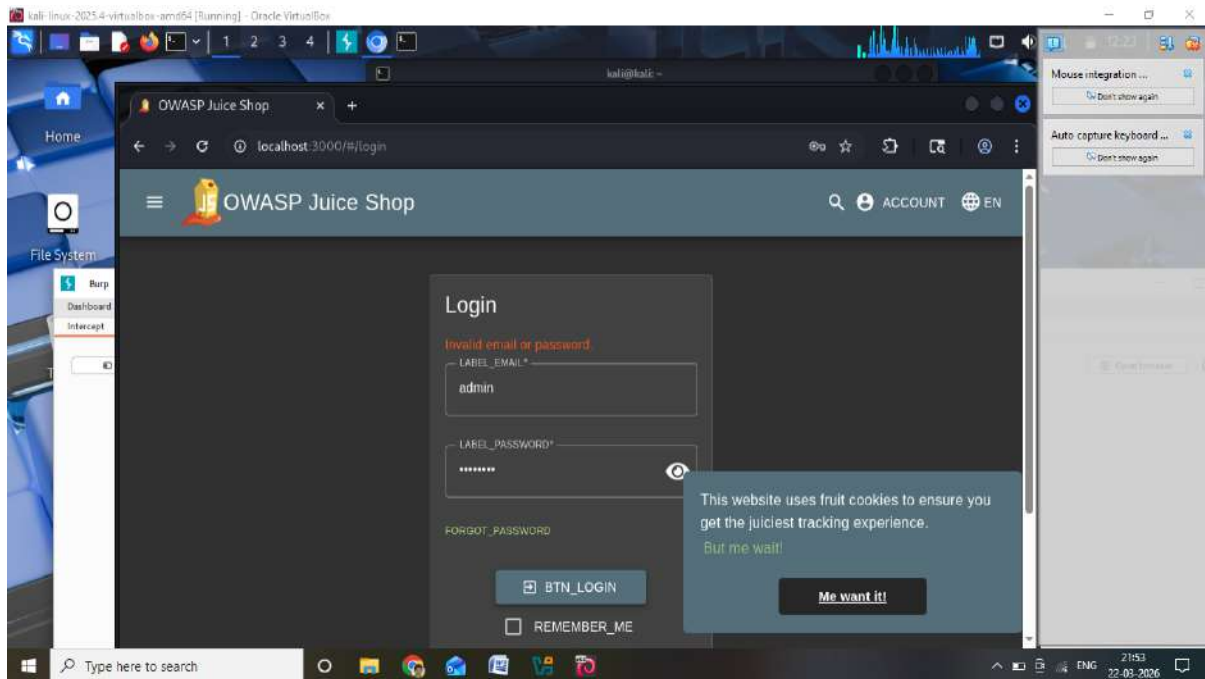
STEP 4: Enter Dummy Login Details

Enter any random values:

Email: admin

Password: password

Click **Login**



STEP 5: Capture Request in Burp

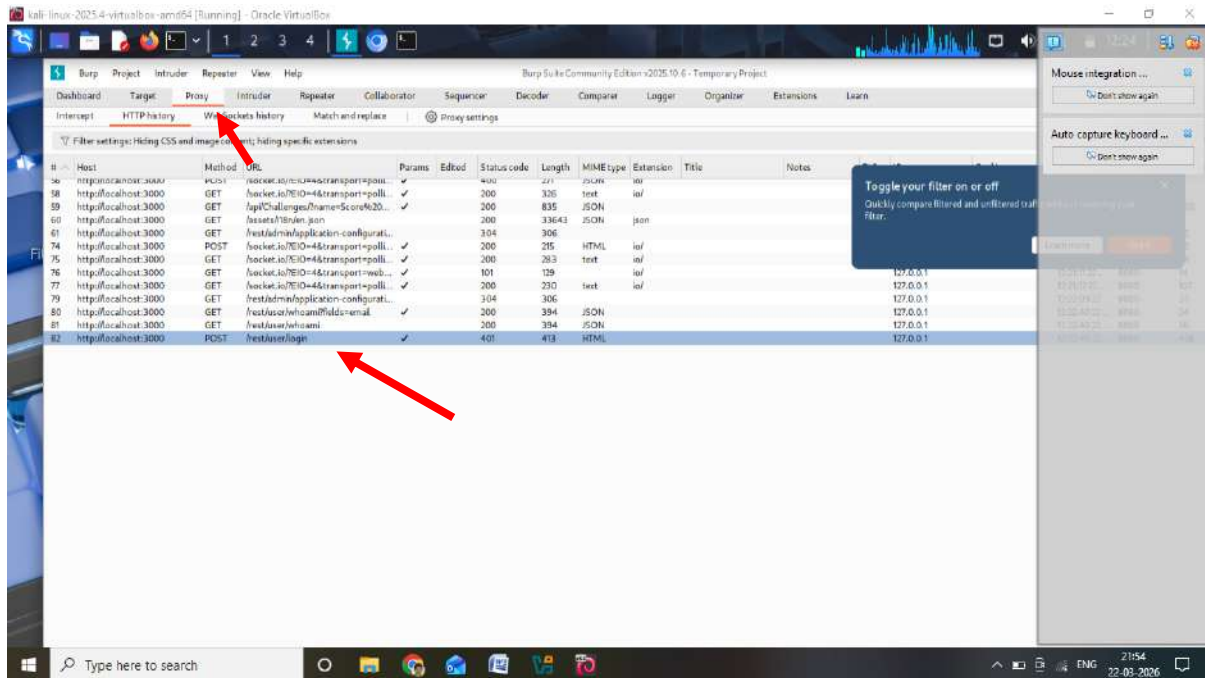
Now go to Burp:

Click **HTTP History**

Look for:

/rest/user/login

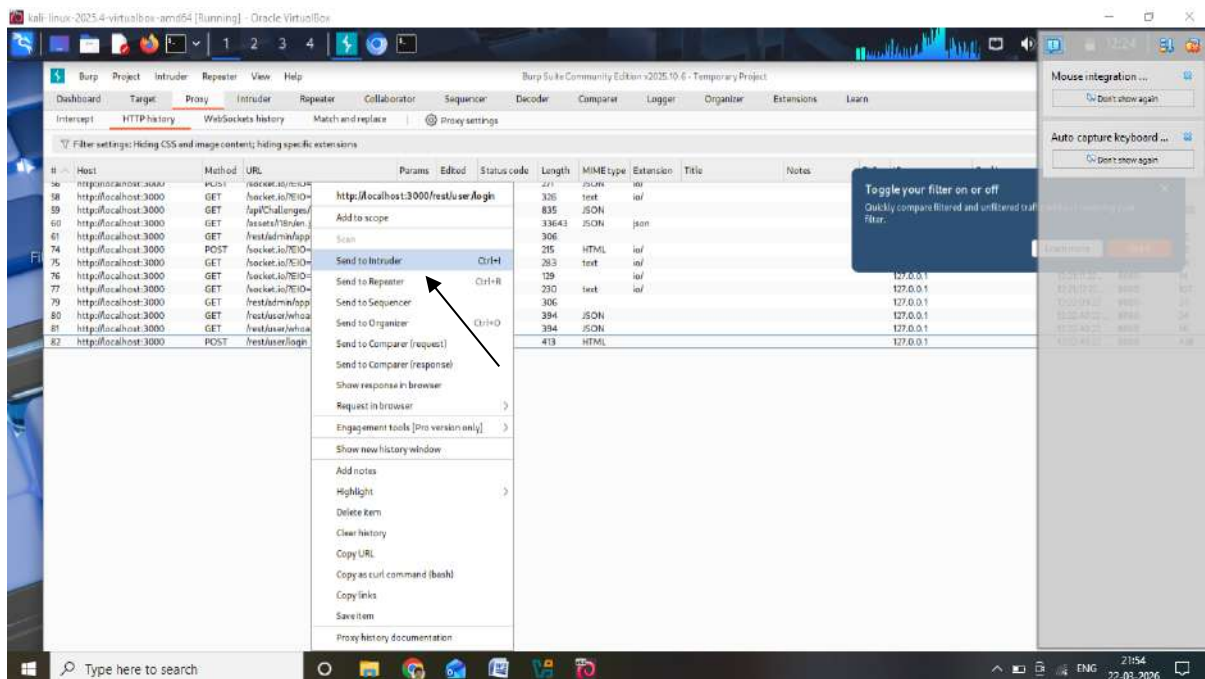
This is the login request



STEP 6: Send Request to Intruder

- Right-click on `/rest/user/login`
- Click:

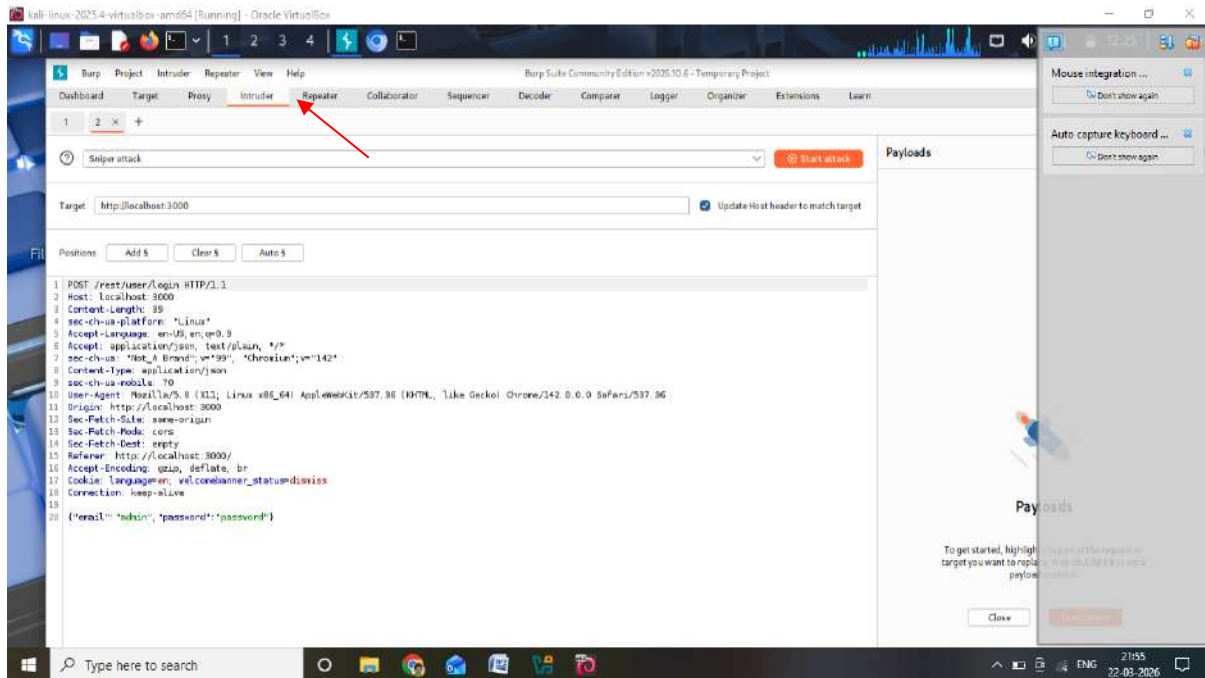
Send to Intruder



STEP 7: Configure Intruder Attack

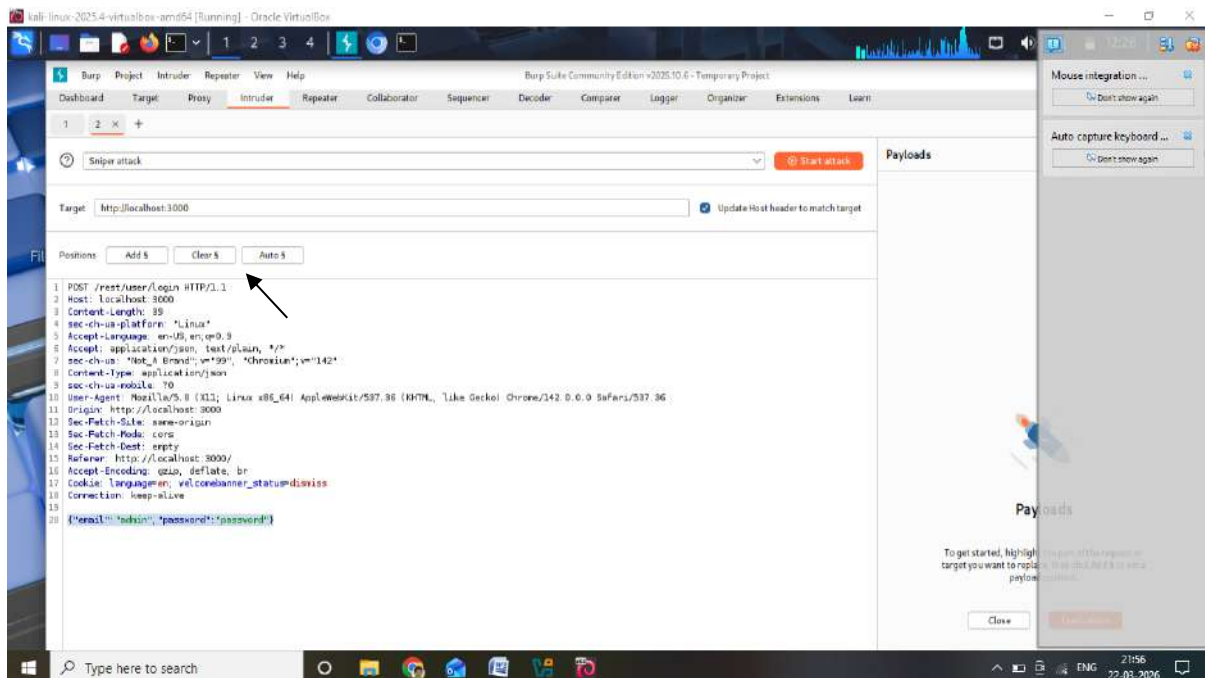
Go to:

Intruder tab



1. Positions Tab

- Click Clear \$
- Select email value:



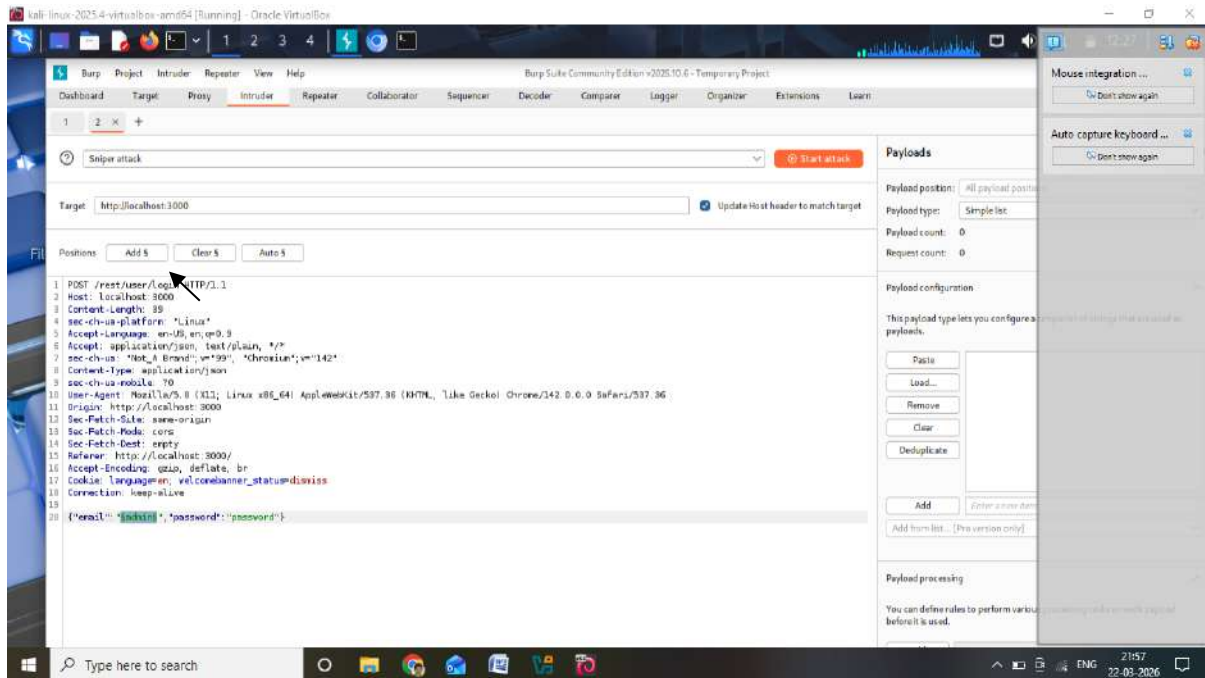
Example:

"email":"admin"

- Click Add \$

Now it becomes:

"email": "\$admin\$"



STEP 8: Add SQL Payloads

Go to:

Payloads tab

Open the website: <https://github.com/Ninja-Yubaraj/SQL-Injection-Payloads-List>
copy and paste from: **Generic SQL Injection Payloads** Section

After unchecking URL encoder

Now paste SQL injection payloads.

Example Payload List:

' OR 1=1 --

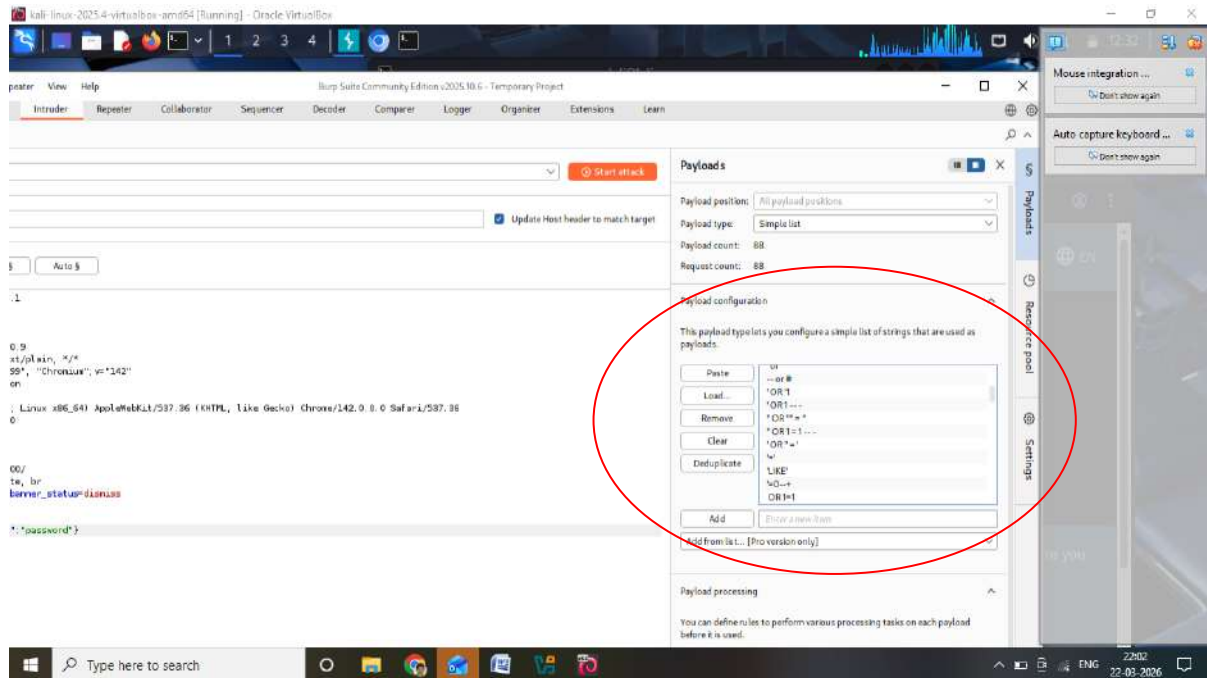
' OR '1'='1

admin' --

' OR 1=1#

' OR '='

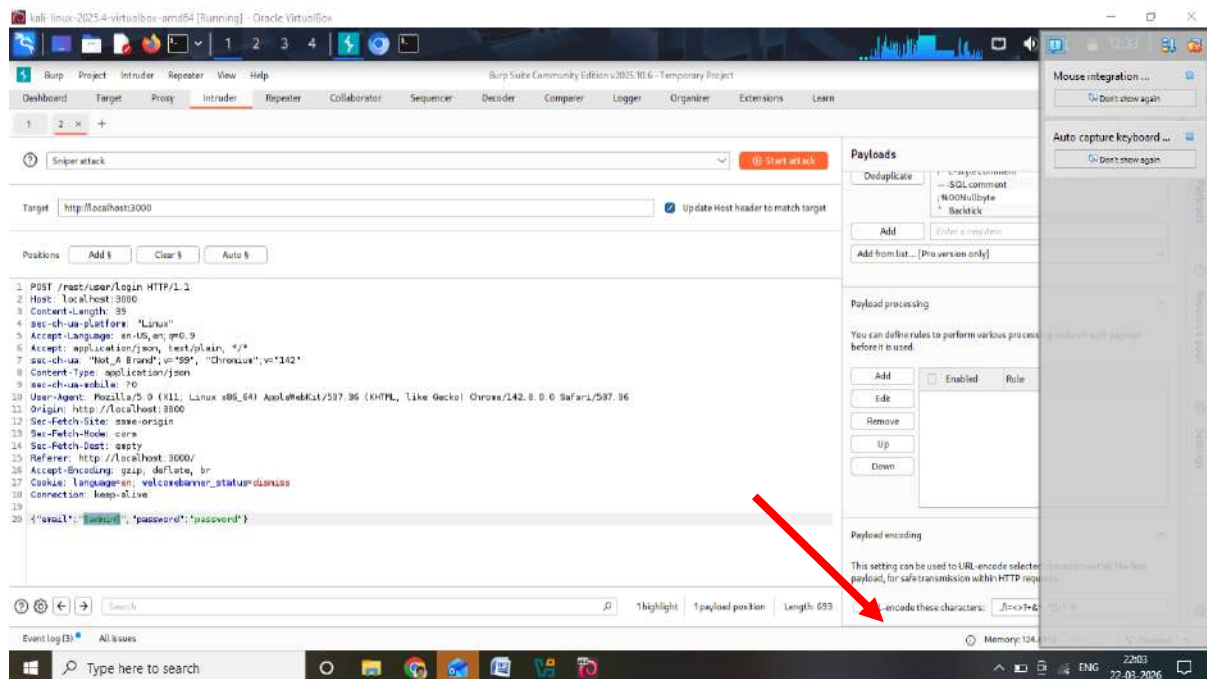
These payloads will be tested automatically



STEP 9: Disable URL Encoding

Very important:

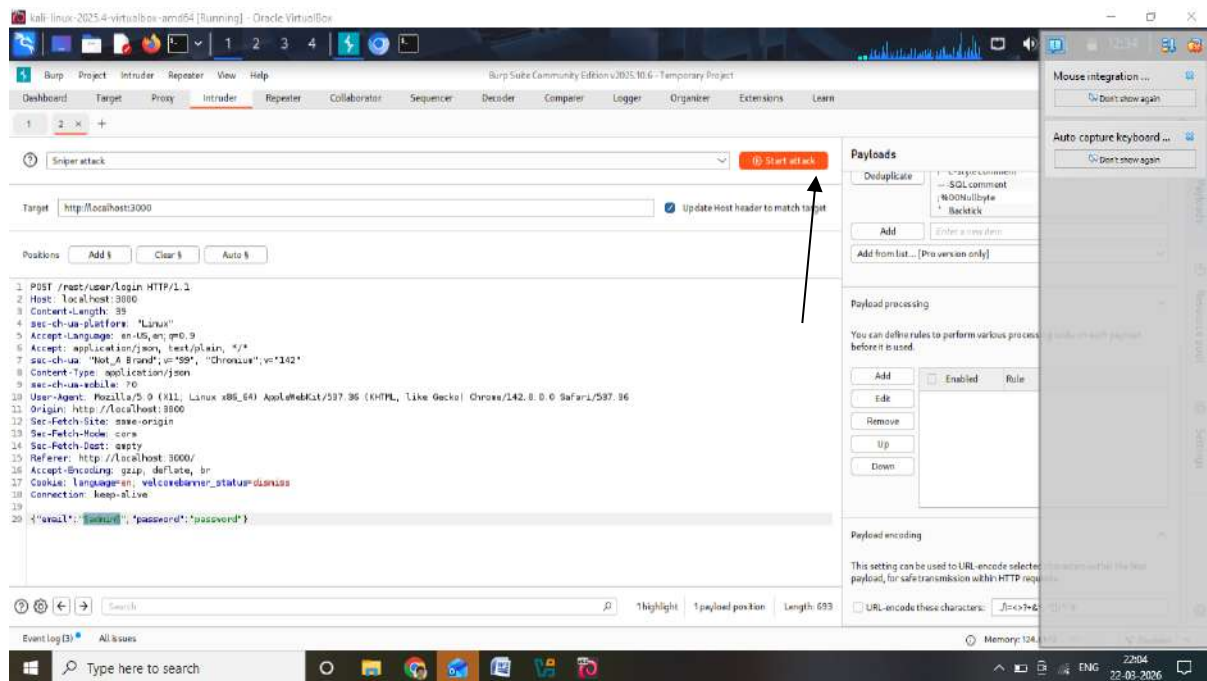
- Uncheck:
URL Encode these characters



STEP 10: Start Attack

Click:

Start Attack



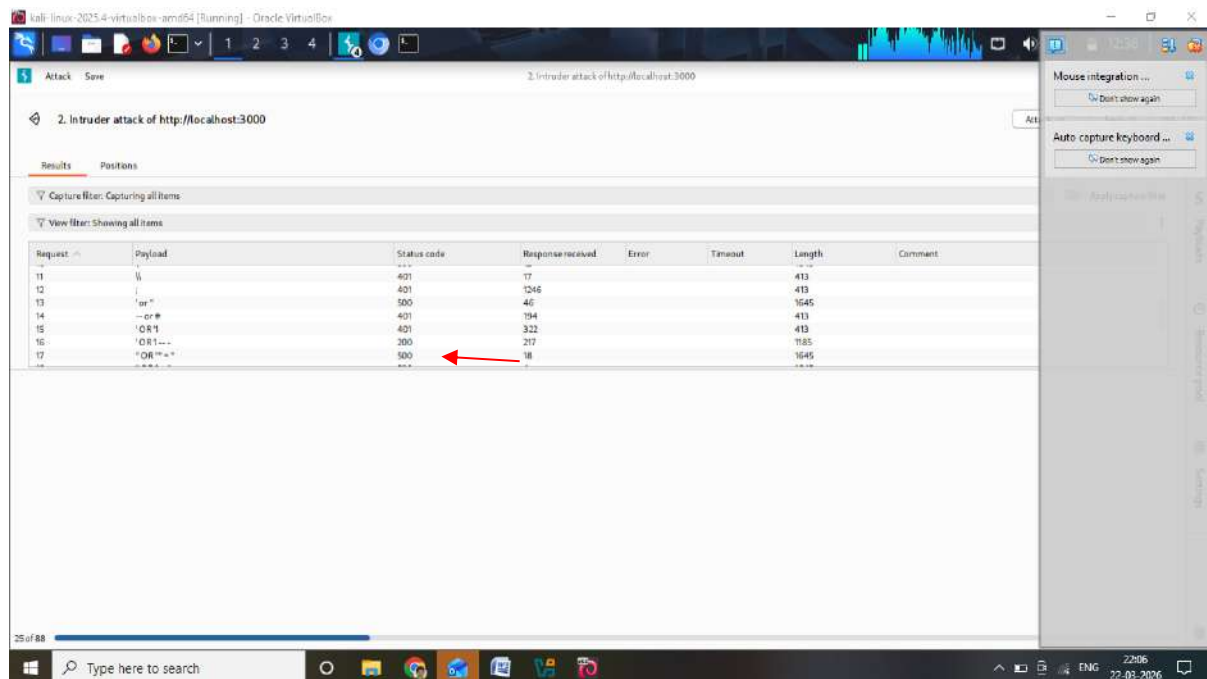
STEP 11: Analyze Results

You will see many responses.

Focus on:

- **Status Code = 200** ✓ (Success)
- Other codes = Failed

If 200 appears → SQL Injection worked



STEP 12: Extract Authentication Token

Click the **200** response

Go to:

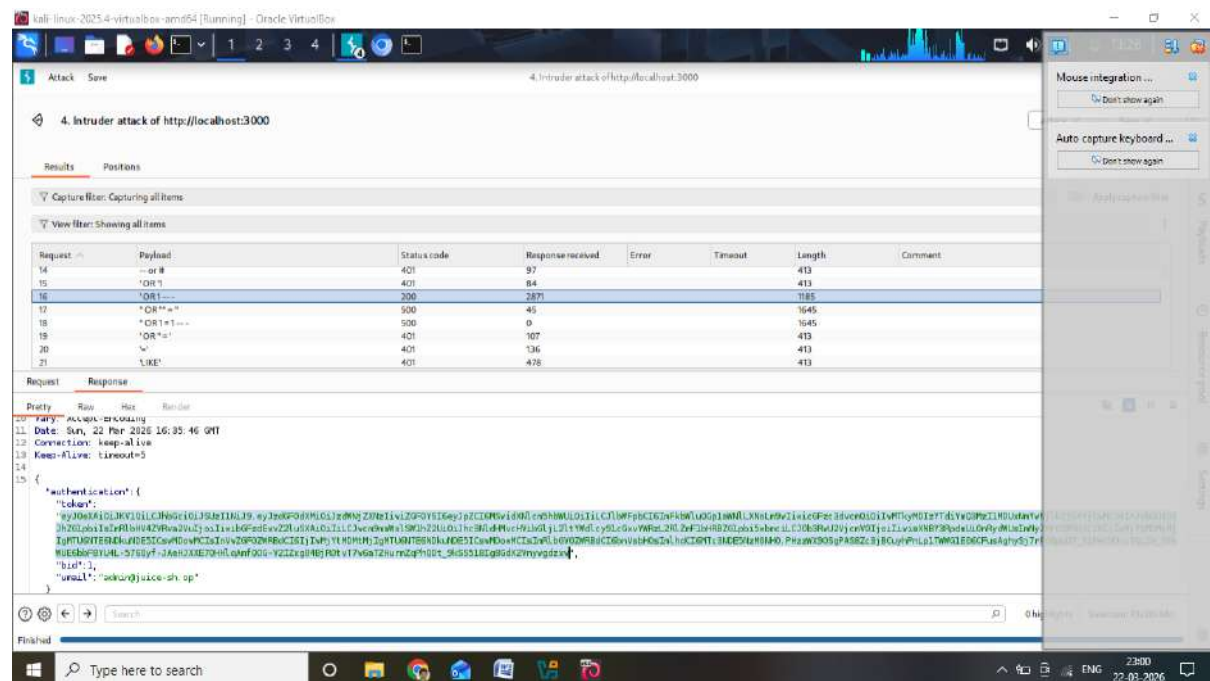
Response tab

Look for:

authentication token

Copy it

"eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1Ni9y.ejZdGF0dXMiOiJzdWNjZXNzliwiZGF0YSI6eyJpZCI6MSwidXNlc
m5hbWUiOiOilCJlbnWFpbCl6ImFkbWluQGp1aWNILXNoLm9wliwicGFzc3dvcmQilwMTkyMDIzYTdiYmQ3
Mzl1MDUxNmYwNjkzE4YjUwMCIsInJvbGUiOiJhZG1pbilisImRlbHV4ZVRva2VuljoiliwibGFzdExvZ2luSXAiOi
liLCJwcm9maWxlSW1hZ2UuOiJhc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RIzmF1bHRBZG1pbi5wbmc
iLCJ0b3RwU2VjcmV0IjoiliwiaXNBZjRpdmdUiOnRydWUsImNyZWFOZWRRBdCl6ljlwMjYtMDMtMjlgMTU6NT
E6NDkuNDE5ICswMDowMCIsInVwZGF0ZWRBdCl6ljlwMjYtMDMtMjlgMTU6NTE6NDkuNDE5ICswMDow
MCIsImRlbGV0ZWRBdCl6bnVsbH0slmlhdCl6MTc3NDE5NzM0NH0.PHzWX9OSgPAS8Zc3jBCuyhPnLp1TW
WG1E06CFusAghySj7rPGEpuU7_91MetNQ-cTQilw_3EkWUE6bbf8YU4L-576Qyf-
JAeHXXE7QHHLqAmfQOG-V2IZxg04BJRotvT7w6aTZHurmqPhQDt_9kSS518lg8GdX2Vnywgdxzw"



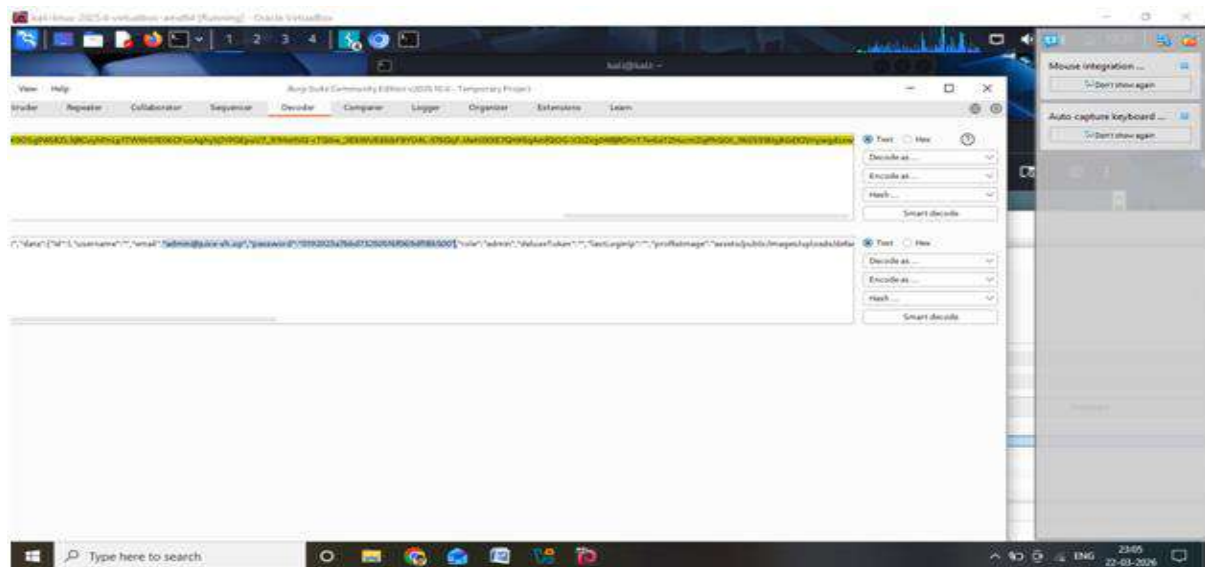
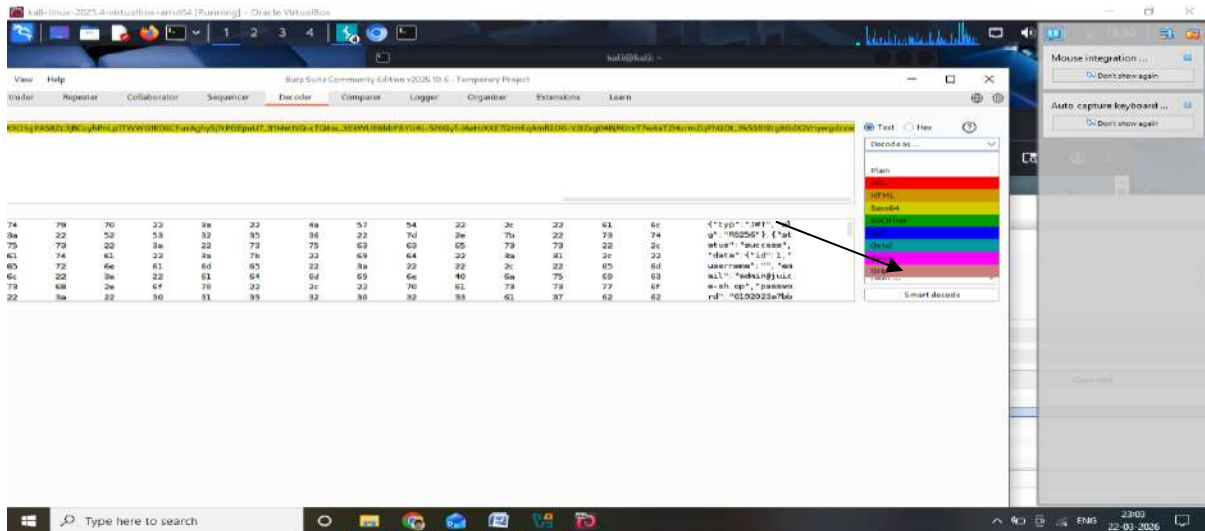
STEP 13: Decode Token

Go to:

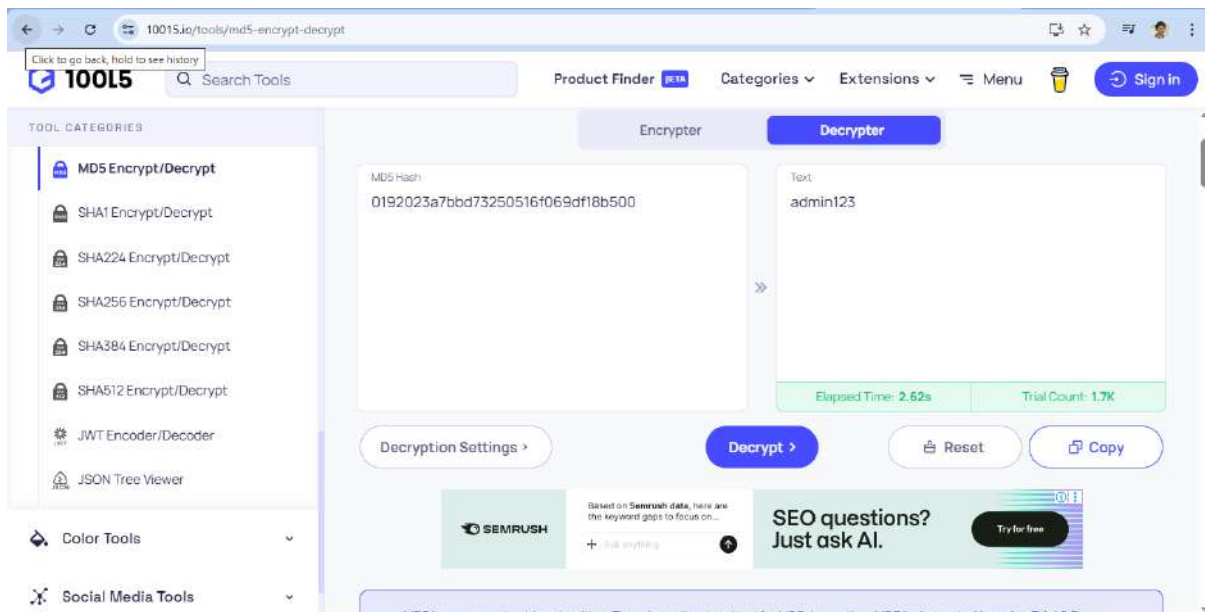
Decoder tab

1. Paste token
2. Select:

Decode as → Base64



Password purpose: Use online MD5 decoder



STEP 14: Get Credentials

After decoding, you will see:

email: admin@juice-sh.op

password: admin123

These are extracted credentials

STEP 15: Login Using Extracted Data

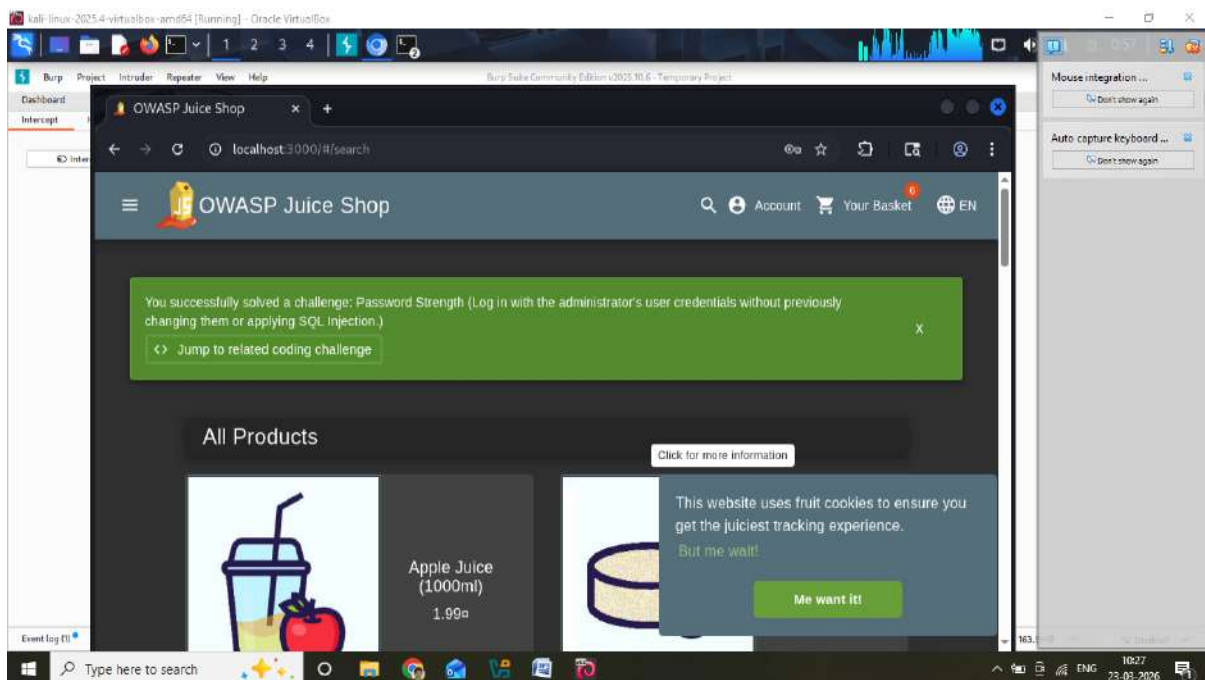
Go back to Juice Shop:

Enter:

- Email
- Password (decoded one)

Click Login

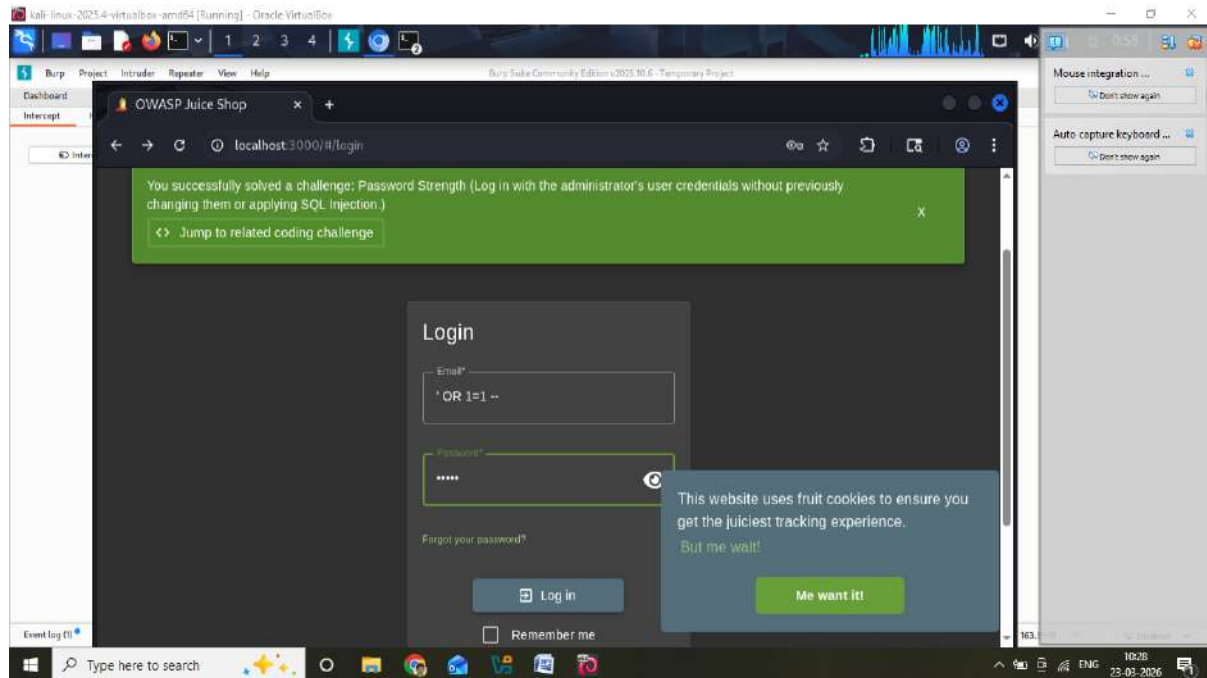
Successfully logged in



TYPES OF SQL INJECTION USED HERE

1. Authentication Bypass

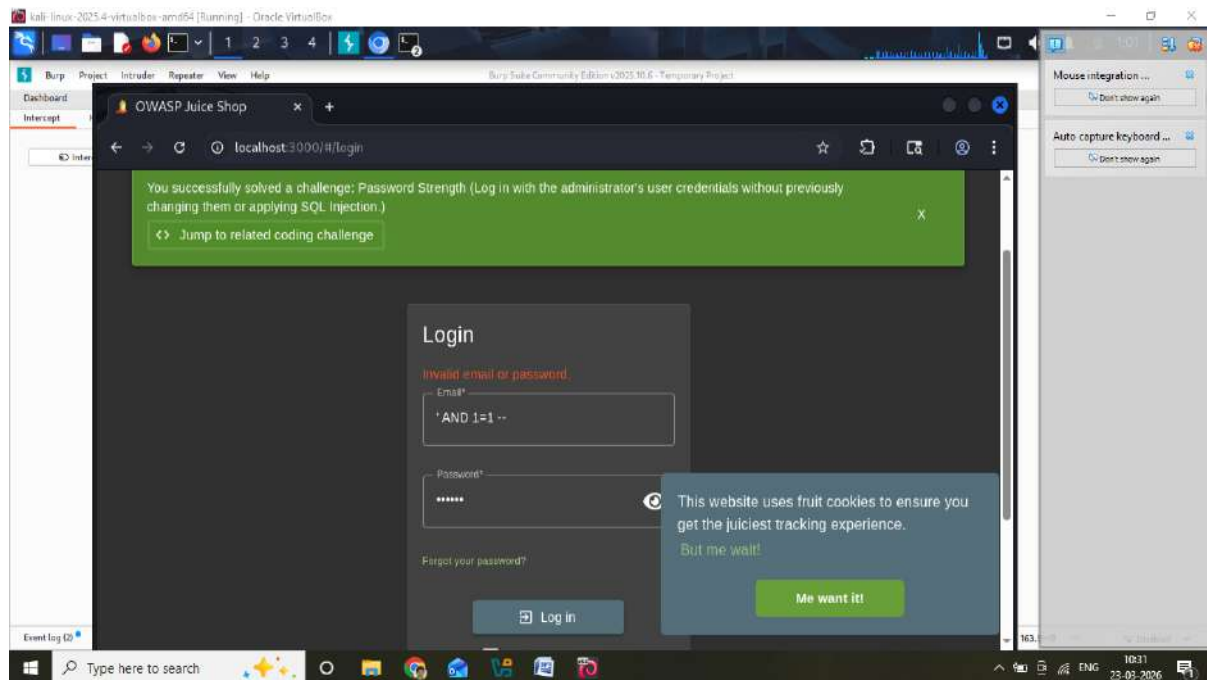
' OR 1=1 --



2. Boolean-Based (Blind)

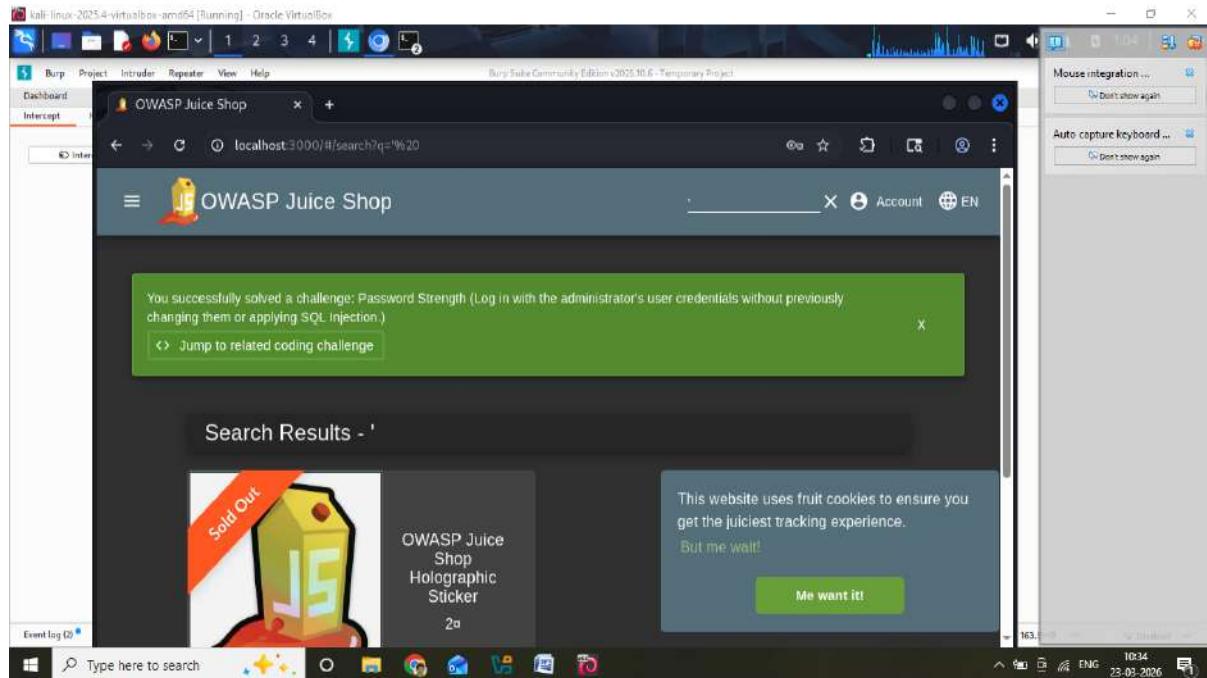
' AND 1=1 --

' AND 1=2 --



3. Error-Based

'



4. Automated Injection (Intruder)

✓ Testing multiple payloads automatically

RESULT

SQL Injection vulnerability successfully identified and exploited using Burp Suite.

Experiment 2: Cross-Site Scripting (XSS)

Tool: WebGoat / OWASP Juice Shop

Objective: Understand and execute reflected, stored, and DOM-based XSS attacks.

Tasks: Inject malicious scripts, steal cookies, and execute JavaScript on victim browsers.

OWASP Juice Shop is installed and running on <http://localhost:3000>

Pull the OWASP Juice Shop Docker Image

Open **Terminal / Command Prompt** and run:

`docker pull bkimminich/juice-shop`

```
harivigneshrao@HARIS-MacBook-Pro ~ % docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
c172f21842df: Pull complete
b4242723c53f: Pull complete
3214acf346c8: Pull complete
887255a8f795: Pull complete
52b2cf54d8e1: Pull complete
463b5dc7de96: Downloading [=====>] 25.17MB/124.9MB
77d3479ca1b9: Pull complete
278acde295e: Pull complete
7c1289eb777b: Pull complete
52438fc75a1b: Pull complete
d6b1b9e0ccac: Pull complete
8cd92d02ae08: Downloading [=====>] 19.92MB/43.3MB
3fd8a6f794cf: Pull complete
76274508eac4: Download complete
d044b72d4177: Pull complete
b4e6f1bfce0a: Pull complete
bdf677fe8b76: Pull complete
ff4331a1212f: Pull complete
b539d1fa81f6: Pull complete
f3e519d19e1f: Pull complete
ebddc55fadc0: Pull complete
2788920e5cbf: Pull complete
cd44a6882c7c: Pull complete
9a70f641b965: Pull complete
2b885e1b42d1: Download complete
```

Run the Juice Shop Container

After the image is downloaded, run:

`docker run -d -p 3000:3000 bkimminich/juice-shop`

```

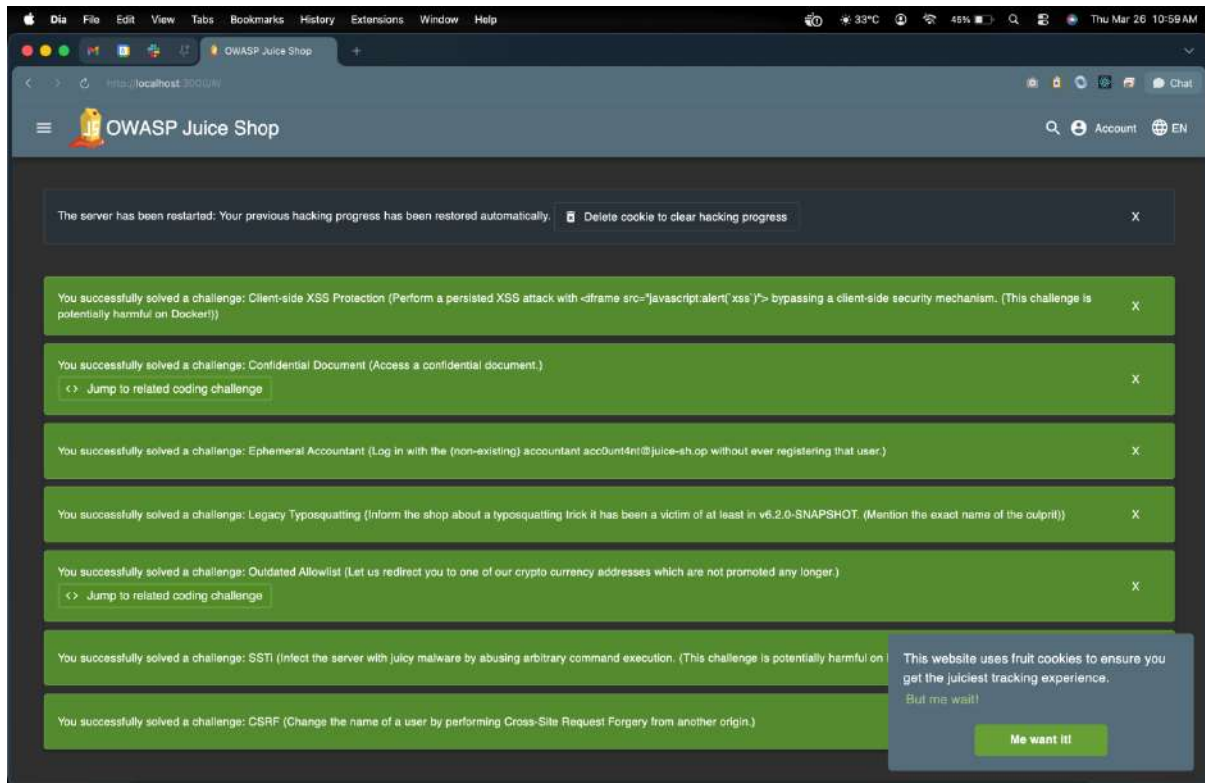
b4242728c53f: Pull complete
3214edf345cd: Pull complete
8872b5ae8795: Pull complete
82b2ef64aee5: Pull complete
483b5dc7de96: Pull complete
77d3b79c81b9: Pull complete
278aeedc995e: Pull complete
7e12896b777b: Pull complete
82630fc76a18: Pull complete
d601b89ecce0: Pull complete
5c92c022ae88: Pull complete
39d8e9f294cf: Pull complete
76274538eac4: Pull complete
d664b720d177: Pull complete
b46df18f0c8e: Pull complete
bdf4777e5bfa: Pull complete
ff43311d121d: Pull complete
b839d9ae81f6: Pull complete
fae519d19e1f: Pull complete
e0d8c58facde: Pull complete
278a920e0dbf: Pull complete
cc446682cc7c: Pull complete
0e7df641b955: Pull complete
28586b1b421c: Download complete
Digest: sha256:224988f8e839bc422445435b272743bee7ad9f364a7d944a88b7594f3bc529d8
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest
harivigneshrao@HARIS-MacBook-Pro: ~ % docker run -d --p 3000:3000 bkimminich/juice-shop
c32133eaz20a8e39ac771db5c8094897094779c7bel0681f5e4bfc1b952be7
harivigneshrao@HARIS-MacBook-Pro: ~ %

```

Access OWASP Juice Shop

Open your browser and navigate to:

<http://localhost:3000>

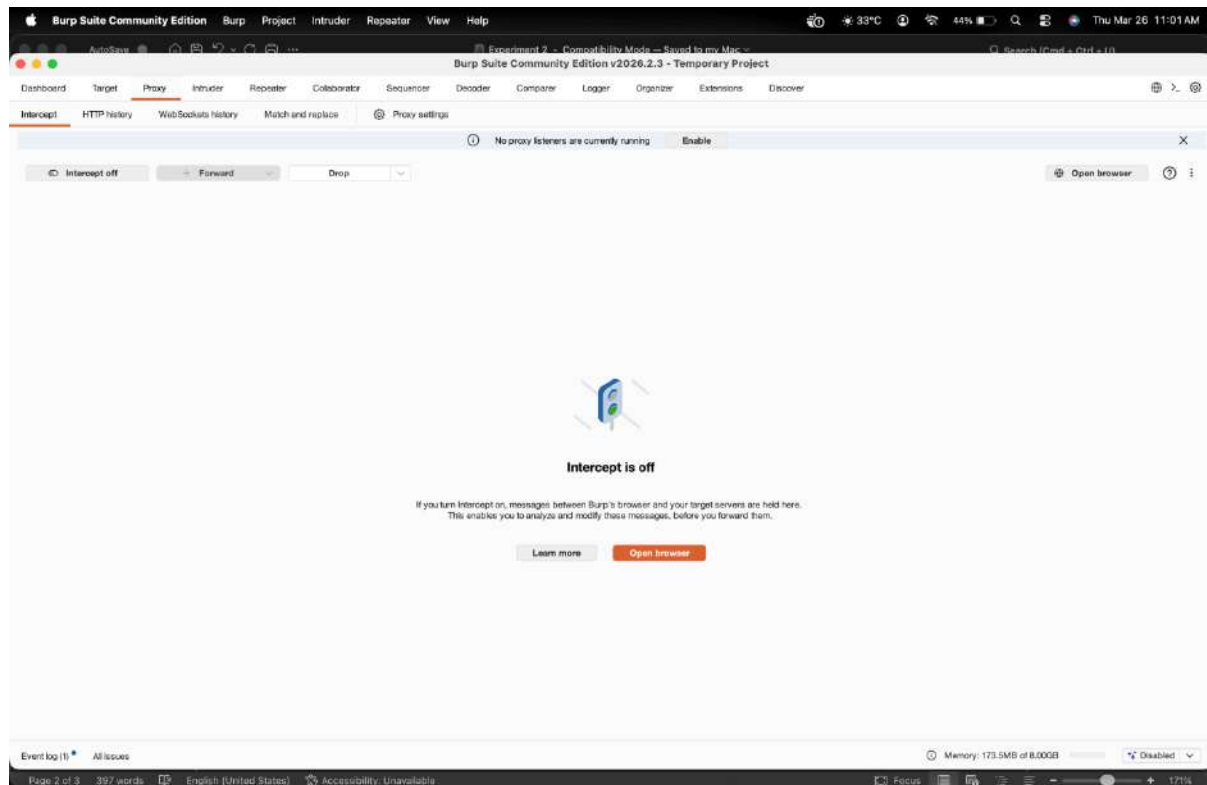


Burp Suite is installed and configured to intercept traffic

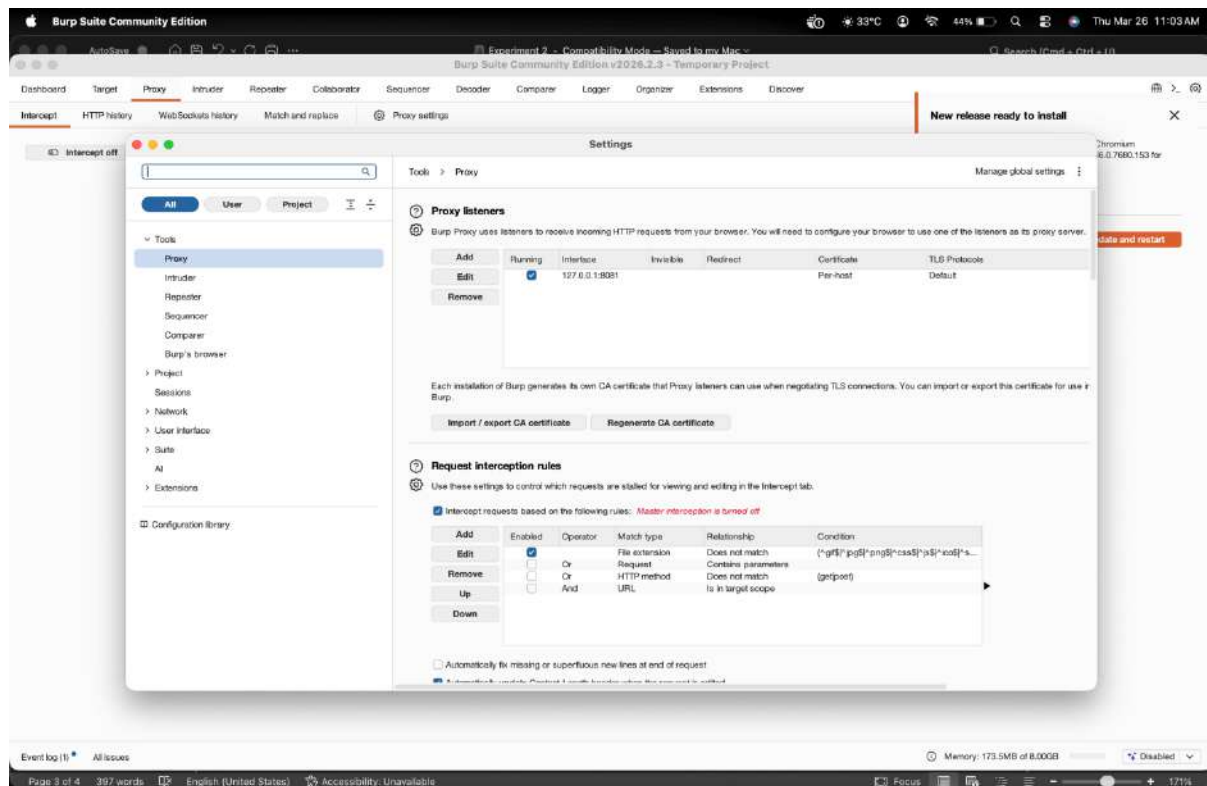
A modern web browser (Chrome/Firefox)

Step 1: Configure Burp Suite Proxy

1. Open Burp Suite and navigate to Proxy > Intercept.
2. Ensure Intercept is off.



3. Go to Proxy > Options, under Proxy Listeners, check that Burp is listening on 127.0.0.1:8081.



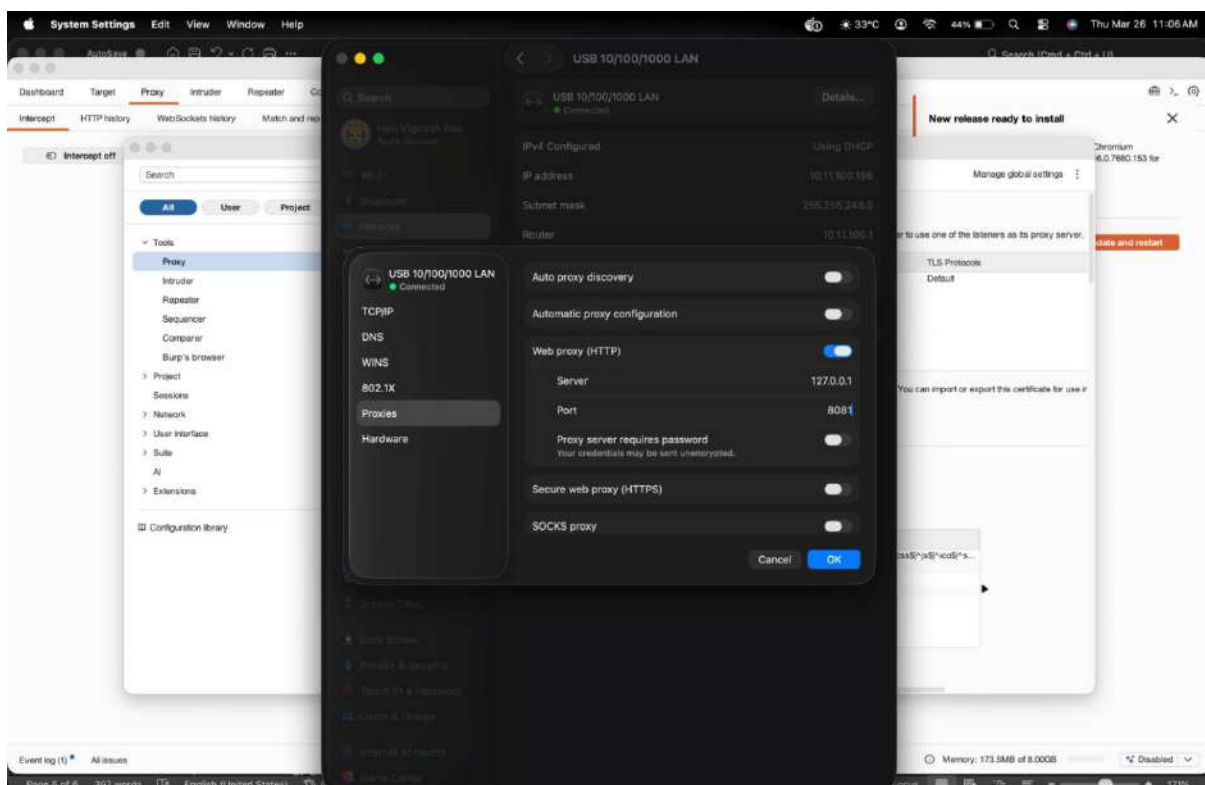
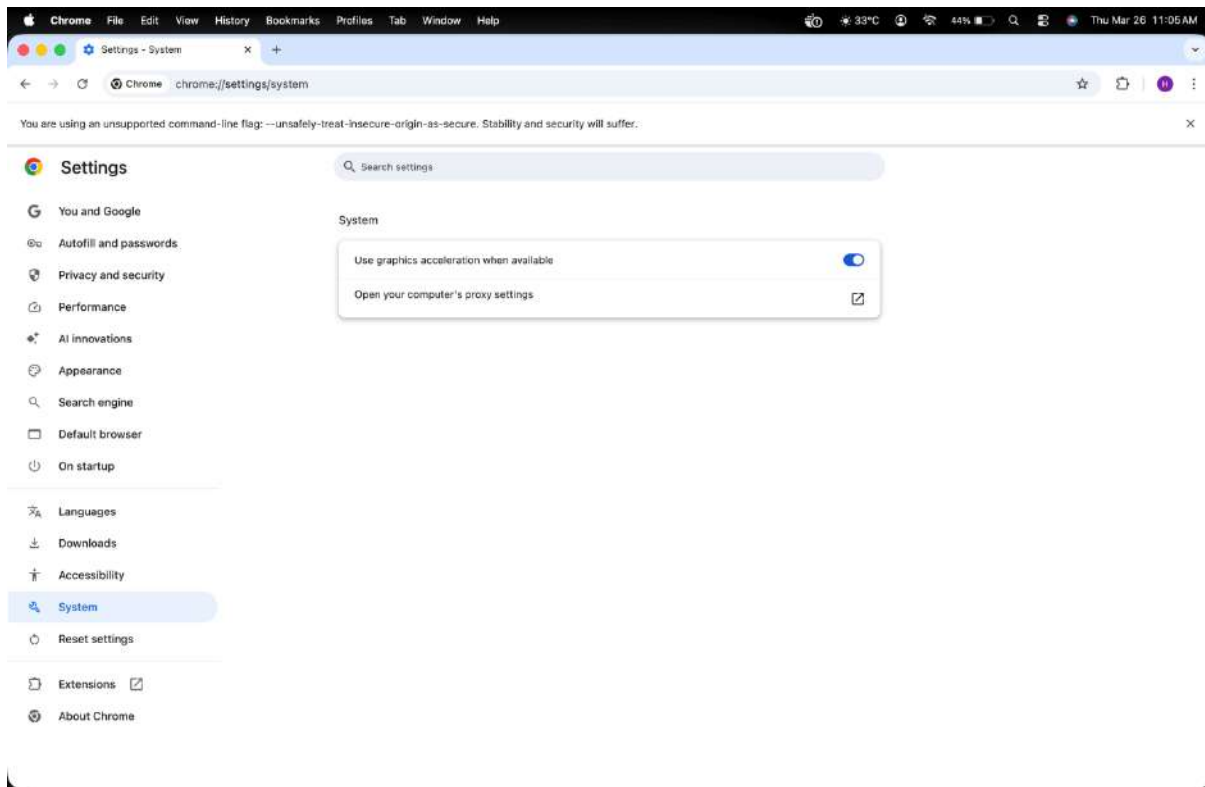
4. Open your browser and configure it to use Burp Suite as a proxy:

O Chrome: Settings > Proxy settings > Manual proxy > Set HTTP Proxy to

127.0.0.1, Port 8081.

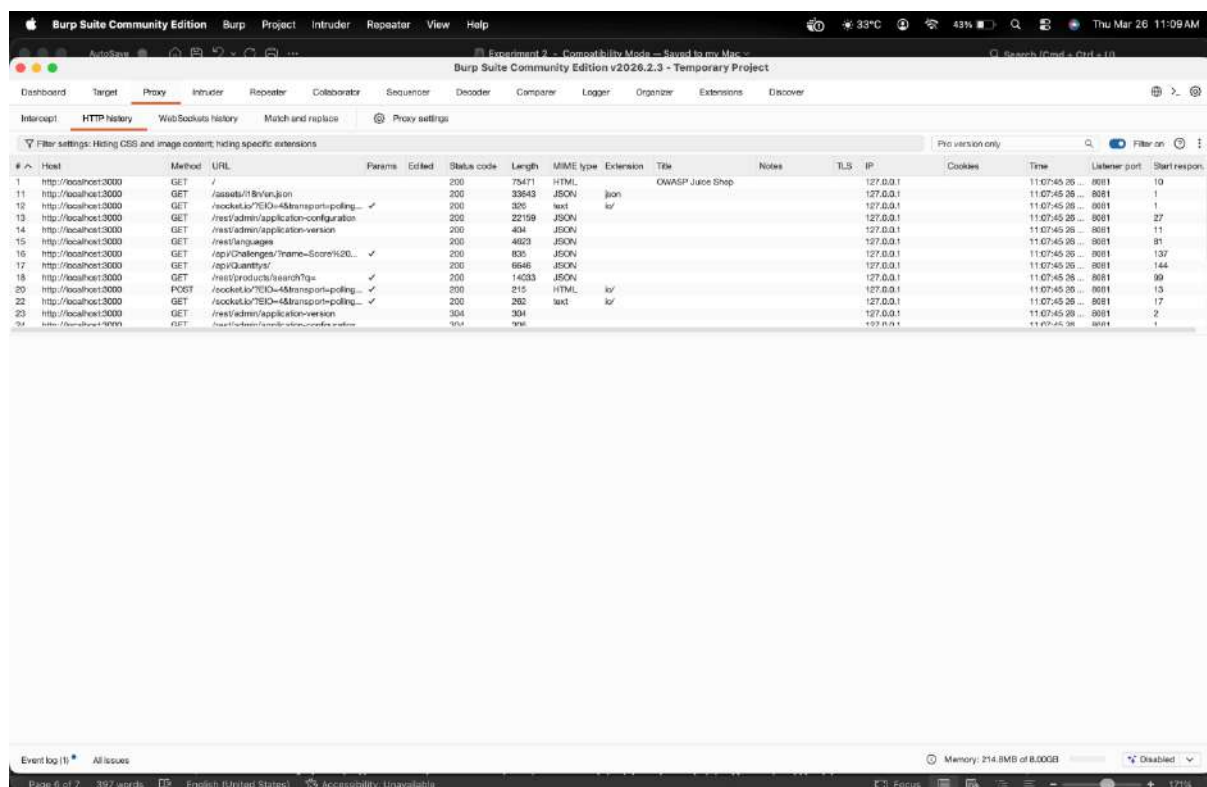
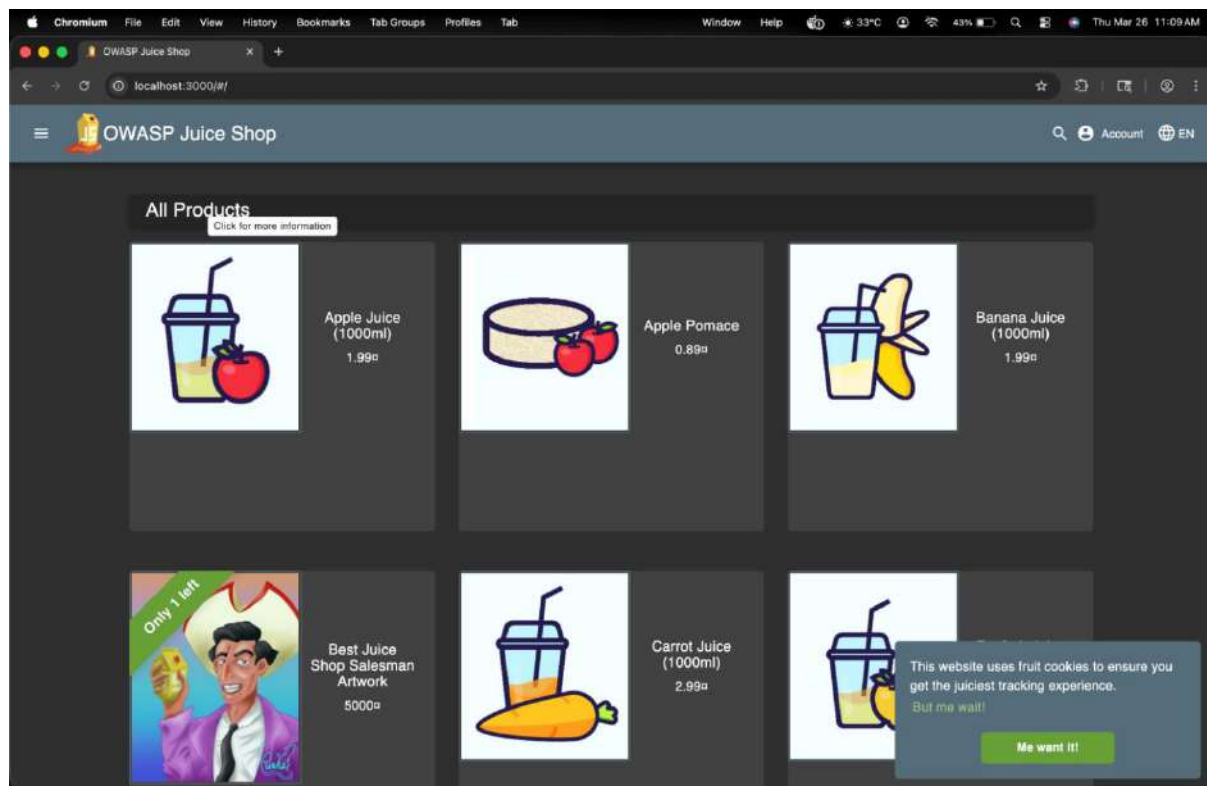
O Firefox: Preferences > Network Settings > Manual proxy > Set HTTP Proxy to

127.0.0.1, Port 8081.



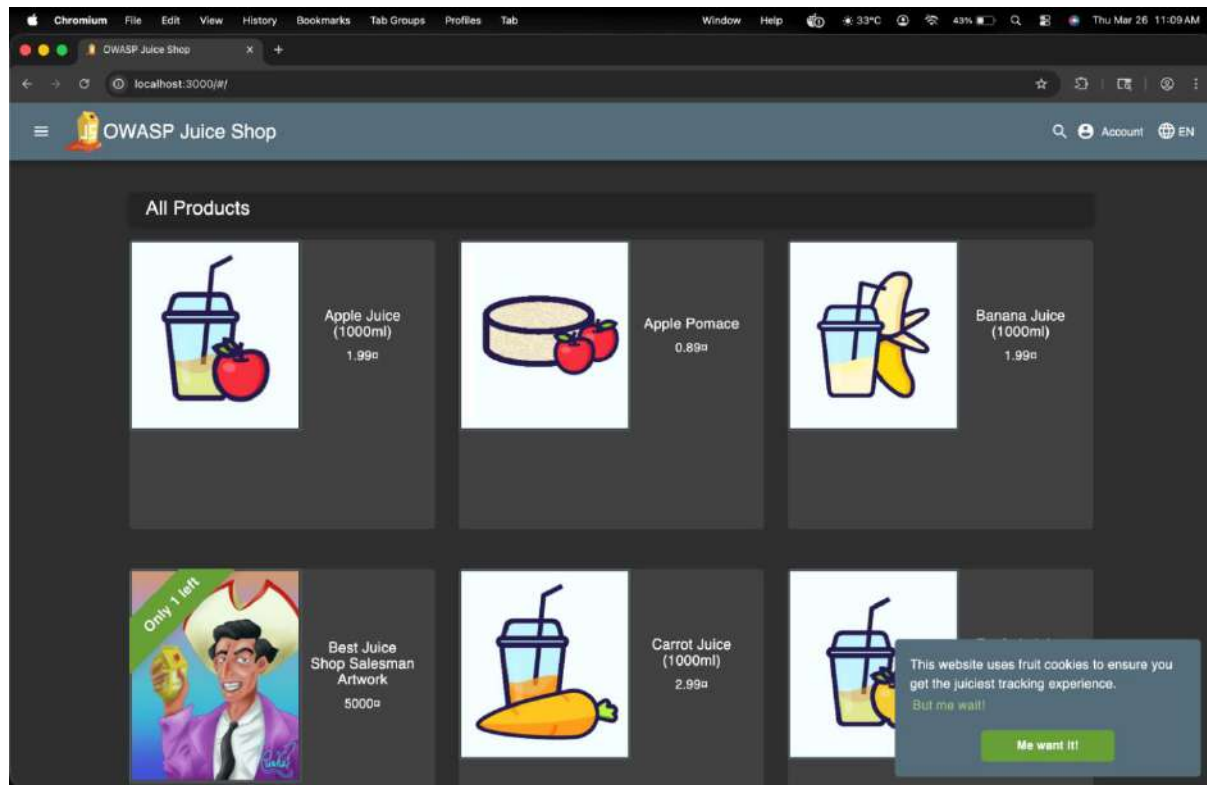
5. Verify by opening <http://localhost:3000> in your browser while Burp Suite is running.

Requests should appear in HTTP history.

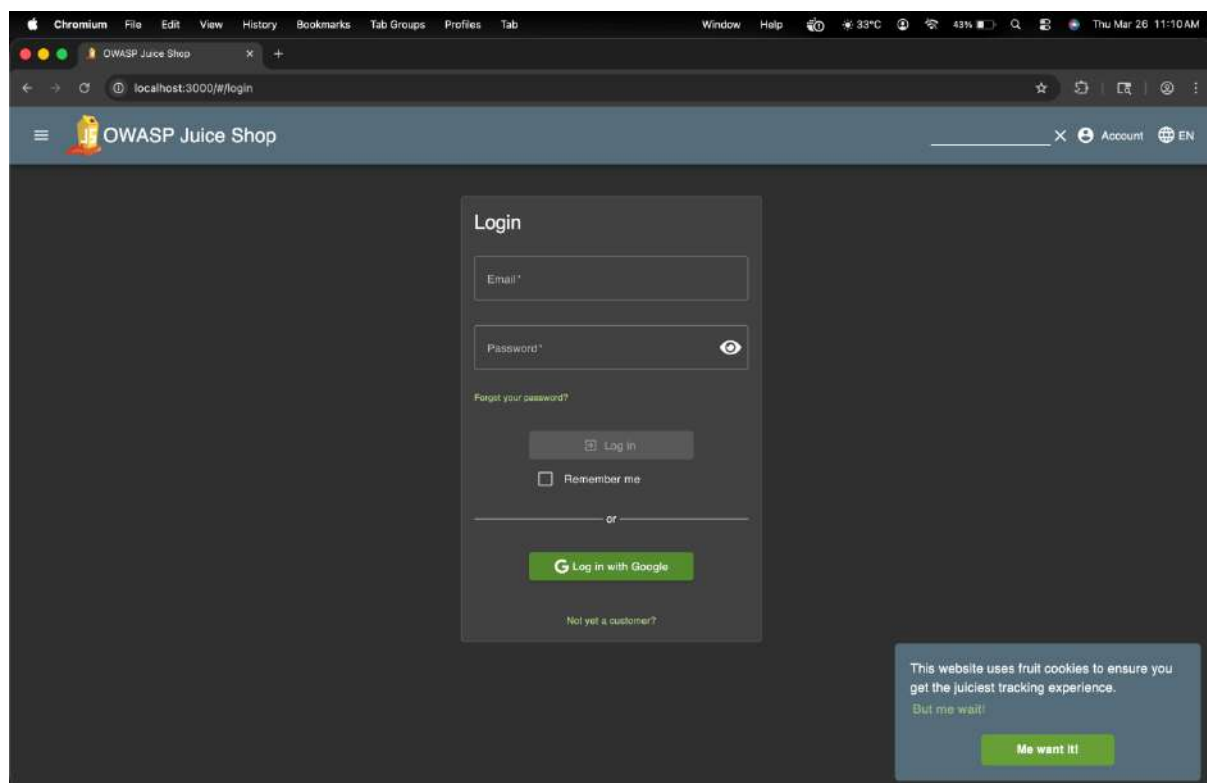


Step 2: Identify XSS Vulnerabilities

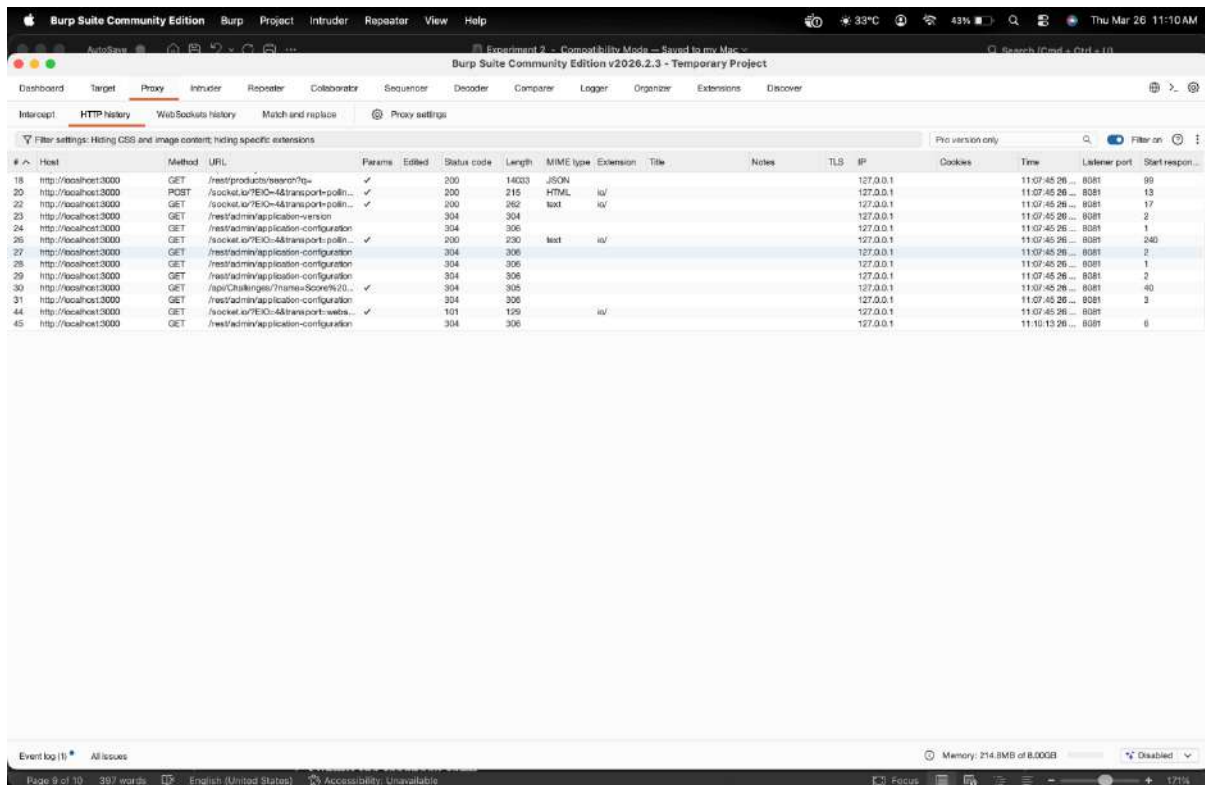
1. Open <http://localhost:3000> in the browser.



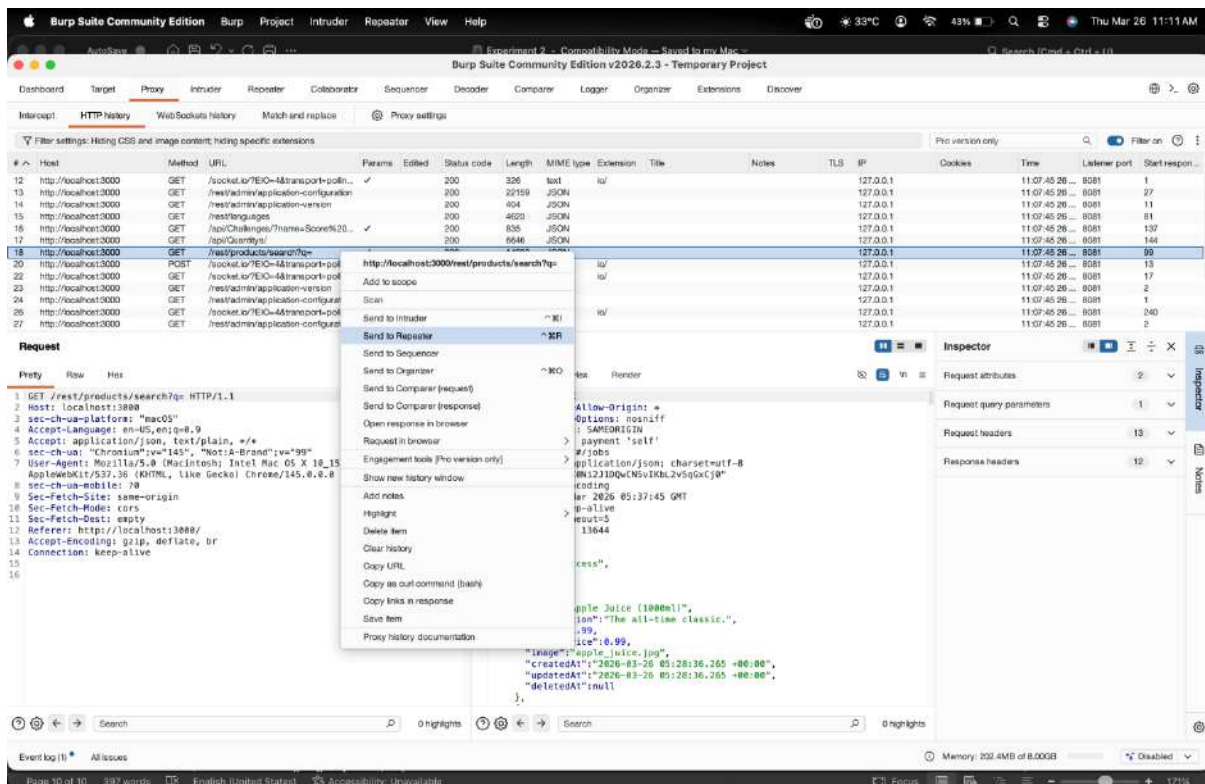
2. Browse through different sections of Juice Shop and look for input fields (such as search bars, login forms, and feedback forms).



2. Open Burp Suite and navigate to Proxy > HTTP history.

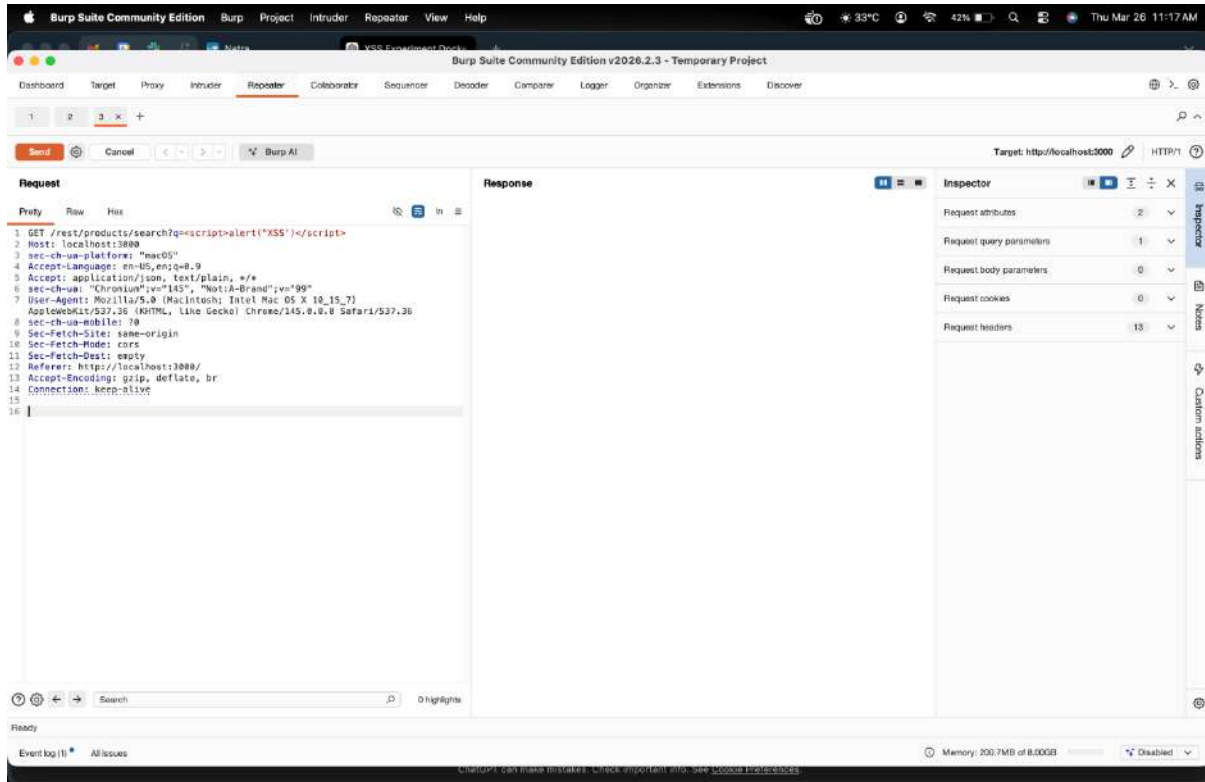


4. Locate the request corresponding to an input field and send it to Repeater (Right-click > Send to Repeater).

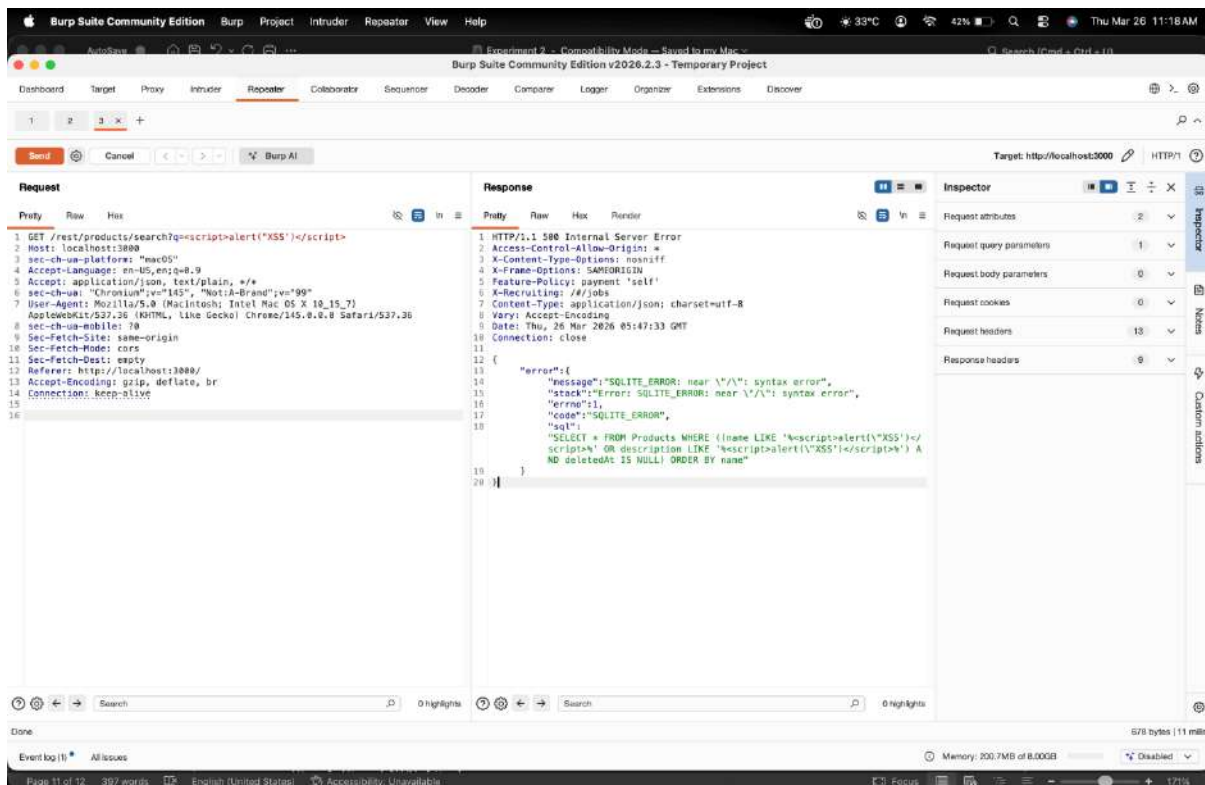


5. In the Repeater tab, experiment by modifying the input fields with JavaScript payloads such as:

```
<script>alert("XSS")</script>
```

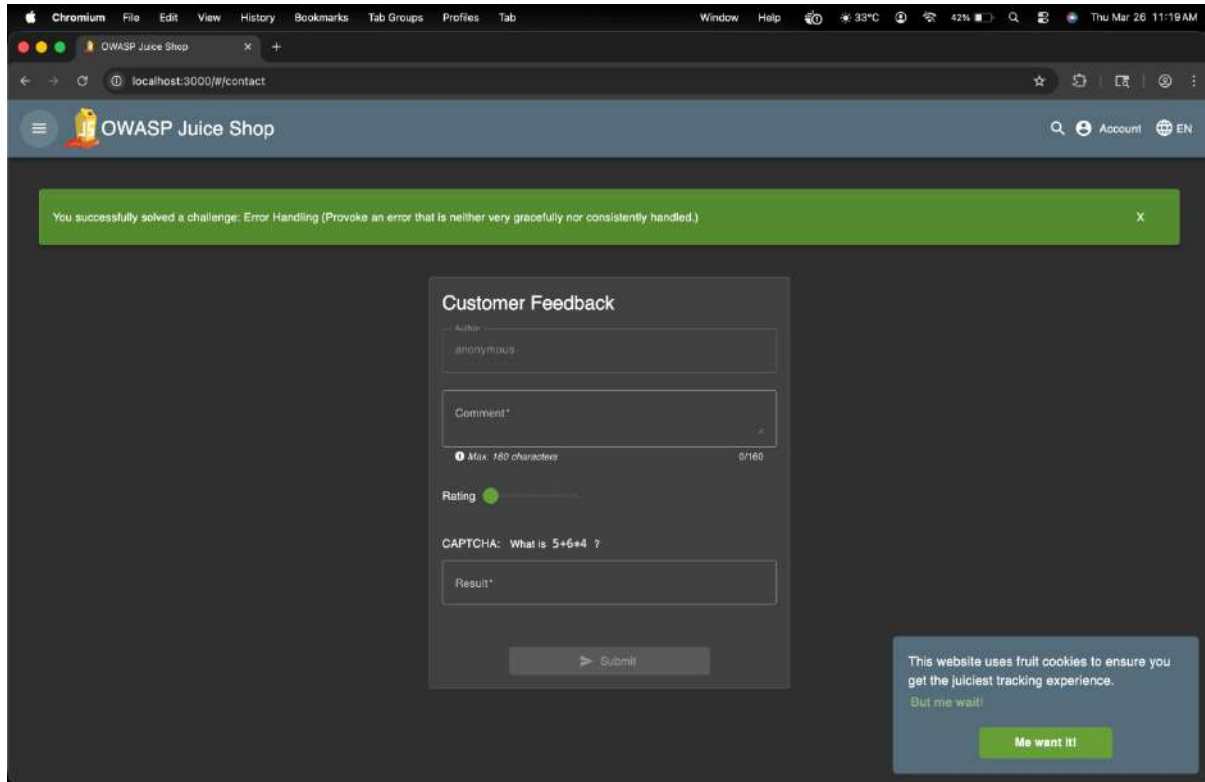


6. Click Send and observe the response for execution of the script.

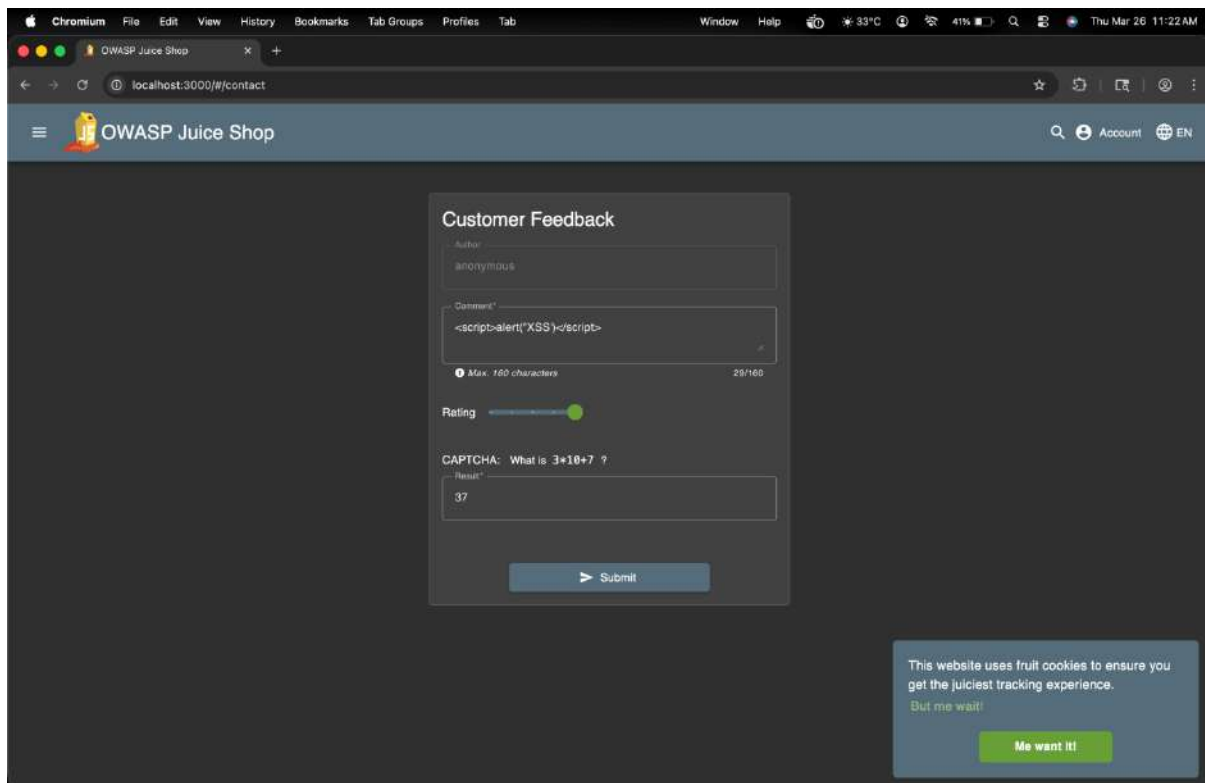


Step 3: Exploiting XSS in Feedback Form

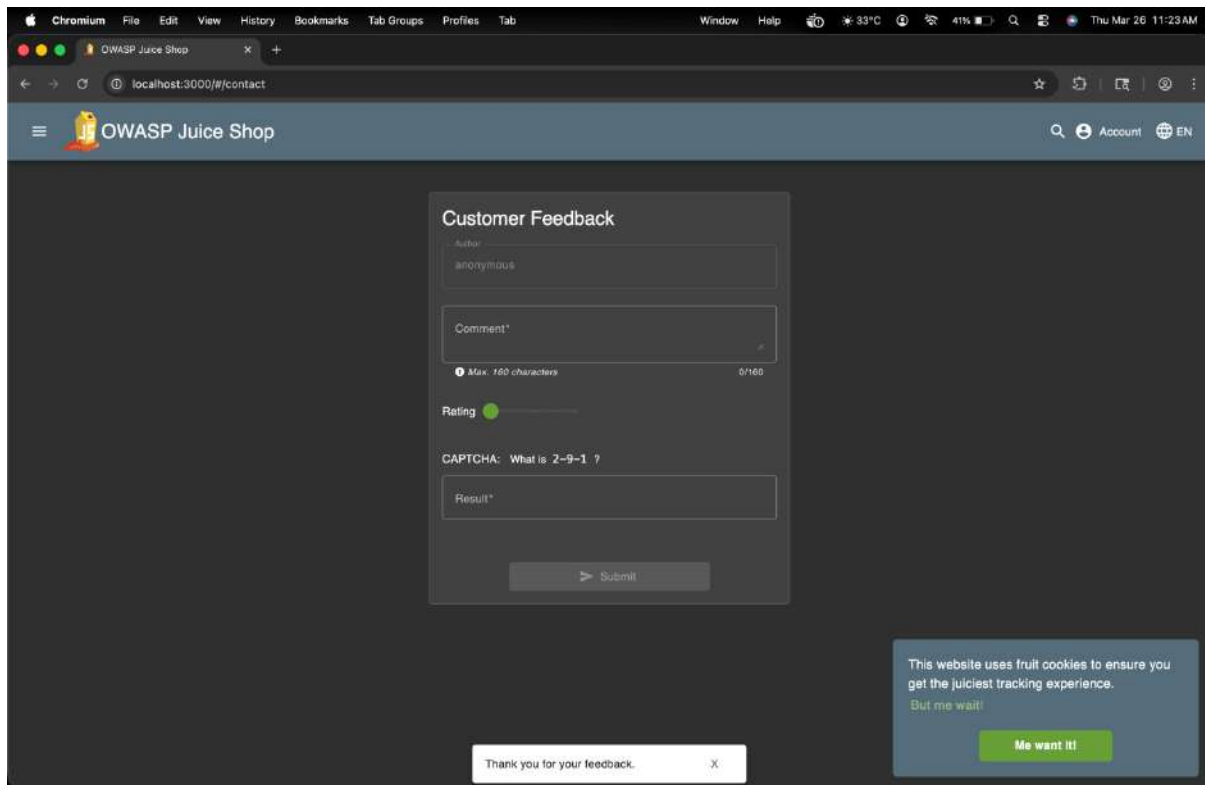
1. In Juice Shop, navigate to Contact Us or Feedback page.



2. Enter the following payload in the comment field: `<script>alert('XSS')</script>`



3. Submit the feedback form.



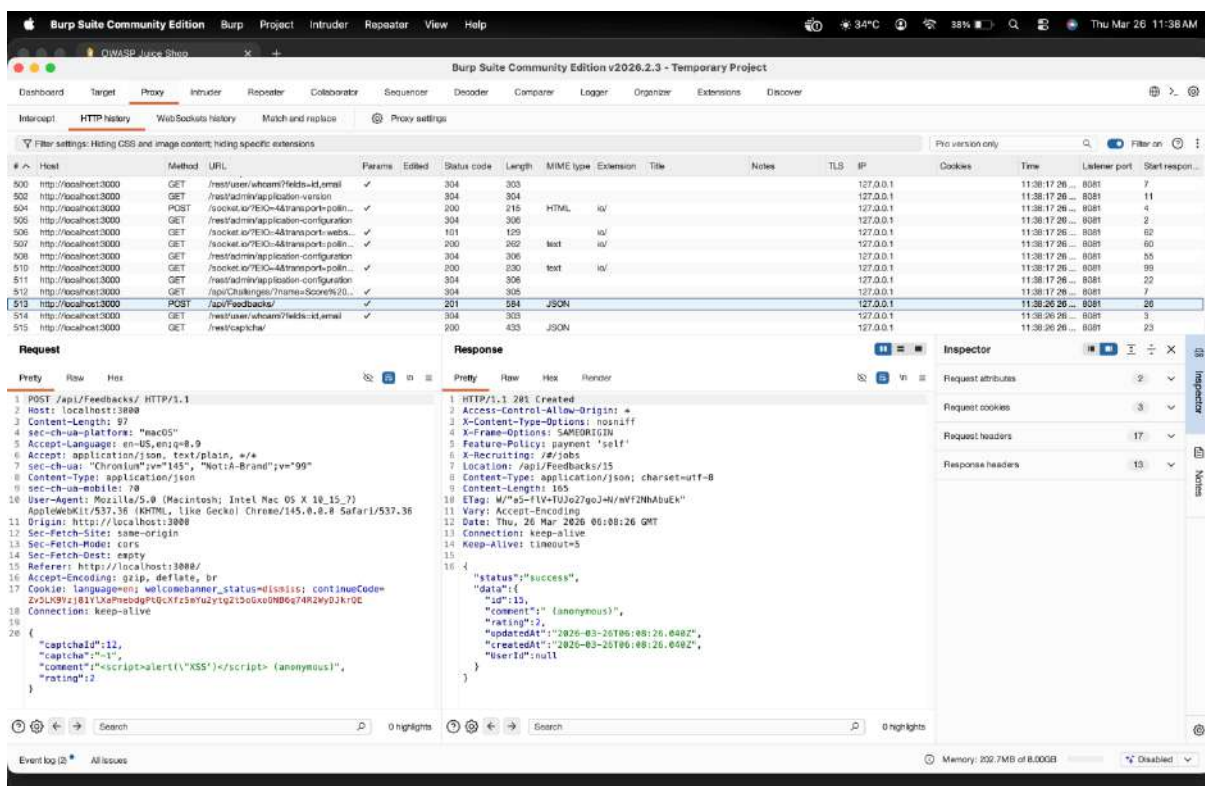
4. If the site is vulnerable, an alert box will pop up.

5. Confirm the XSS attack by checking:

○ The browser console (F12> Console).

○ The HTTP history tab in Burp Suite.

Inspecting the request and response in Burp Suite.



Step 4: Stored XSS Attack

1. Locate a form where the input is stored and displayed later (e.g., user profile, reviews, or feedback section).

2. Enter the payload:

```
<script>document.write('<img src=x onerror=alert("Stored XSS")>');</script>
```

Burp Suite Community Edition v2026.2.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Camper Logger Organizer Extensions Discover

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
579	http://localhost:3000	GET	/rest/captcha			200	433	JSON				127.0.0.1			11:40:18.26	8081	89
580	http://localhost:3000	GET	/rest/user/whoami?fields=id,email			304	303					127.0.0.1			11:40:18.26	8081	19
582	http://localhost:3000	POST	/socket.io/?EIO=4&transport=pol...			200	215	HTML	io/			127.0.0.1			11:40:18.26	8081	2
583	http://localhost:3000	GET	/socket.io/?EIO=4&transport=web...			101	126		io/			127.0.0.1			11:40:18.26	8081	3
585	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1			11:40:18.26	8081	6
586	http://localhost:3000	GET	/socket.io/?EIO=4&transport=pol...			200	262	text	io/			127.0.0.1			11:40:18.26	8081	3
587	http://localhost:3000	GET	/rest/admin/application-version			304	304					127.0.0.1			11:40:18.26	8081	15
588	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1			11:40:18.26	8081	17
590	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1			11:40:18.26	8081	1
591	http://localhost:3000	GET	/api/challenges/?name=Score%20...			304	305					127.0.0.1			11:40:18.26	8081	6
592	http://localhost:3000	POST	/api/feedbacks/			201	384	JSON				127.0.0.1			11:40:52.26	8081	24
593	http://localhost:3000	GET	/rest/captcha			200	433	JSON				127.0.0.1			11:40:52.26	8081	22
594	http://localhost:3000	GET	/rest/user/whoami?fields=id,email			304	303					127.0.0.1			11:40:52.26	8081	8

Request

1 POST /api/feedbacks/ HTTP/1.1

2 Host: localhost:3000

3 Content-Length: 145

4 sec-ch-ua-platform: "macOS"

5 Accept-Language: en-US,en;q=0.9

6 Accept: application/json, text/plain, */*

7 sec-ch-ua: "Chromium" v="145", "Not-A-Brand" v="99"

8 Content-Type: application/json

9 sec-ch-ua-mobile: ?0

10 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.8.8.8 Safari/537.36

11 Origin: http://localhost:3000

12 Sec-Fetch-Site: same-origin

13 Sec-Fetch-Mode: cors

14 Sec-Fetch-Dest: empty

15 Referer: http://localhost:3000/

16 Accept-Encoding: gzip, deflate, br

17 Cookie: language=; welcomeBanner_status=dismiss; continueCode=ZvSLK9zj81YLxahnebdgP0GKfz5ntu2ytq2t3o6x0M86q74R2WY0Jk10E

18 Connection: keep-alive

20 {

21 "captchaId":15,

22 "captcha":"64",

23 "comment":

24 "<script>document.write('');</script> [anonymous]",

25 "rating":2

Response

1 HTTP/1.1 201 Created

2 Access-Control-Allow-Origin: *

3 X-Content-Type-Options: nosniff

4 X-Frame-Options: SAMEORIGIN

5 Feature-Policy: payment 'self'

6 X-Recruiting: /#/jobs

7 Location: /api/feedbacks/15

8 Content-Type: application/json; charset=utf-8

9 Content-Length: 165

10 ETag: W/"a5-130meN/+3uTPC8M3J1JnXF6v15o"

11 Vary: Accept-Encoding

12 Date: Thu, 26 Mar 2026 05:18:52 GMT

13 Connection: keep-alive

14 Keep-Alive: timeout=5

15

16 {

17 "status":"success",

18 "data":{

19 "id":15,

20 "comment":" [anonymous]",

21 "rating":2,

22 "updatedAt":"2026-03-26T06:18:52.668Z",

23 "createdAt":"2026-03-26T06:18:52.668Z",

24 "userId":null

25 }

26 }

Inspector

Request attributes 2

Request cookies 3

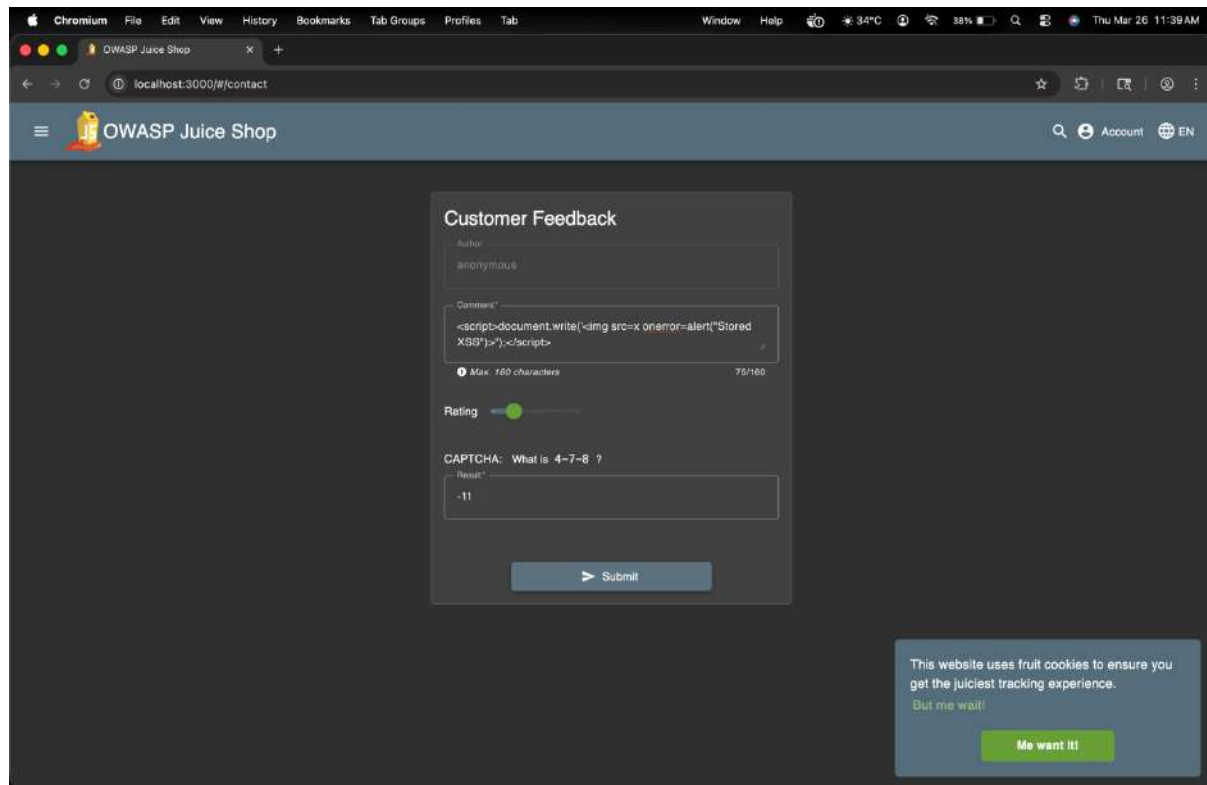
Request headers 17

Response headers 13

Event log (2) All issues

Memory: 214.5MB of 8.00GB

Disabled



3. Submit the form.
4. Reload the page where the stored input is displayed.

Experiment 3: Broken Authentication and Session Management

Tool: Juice Shop / Burp Suite

Objective: Exploit weak authentication mechanisms and hijack sessions.

Tasks: Capture and modify session cookies, brute force login credentials, and exploit session fixation.

AIM

To identify and analyze **authentication vulnerabilities** and **session management flaws** in a web application using Docker and Burp Suite.

REQUIREMENTS

- Kali Linux
- Docker
- Burp Suite
- Web Browser (Chrome/Firefox)
- Internet connection

THEORY

Authentication verifies user identity (login system).

Session Management maintains user state using session IDs or tokens.

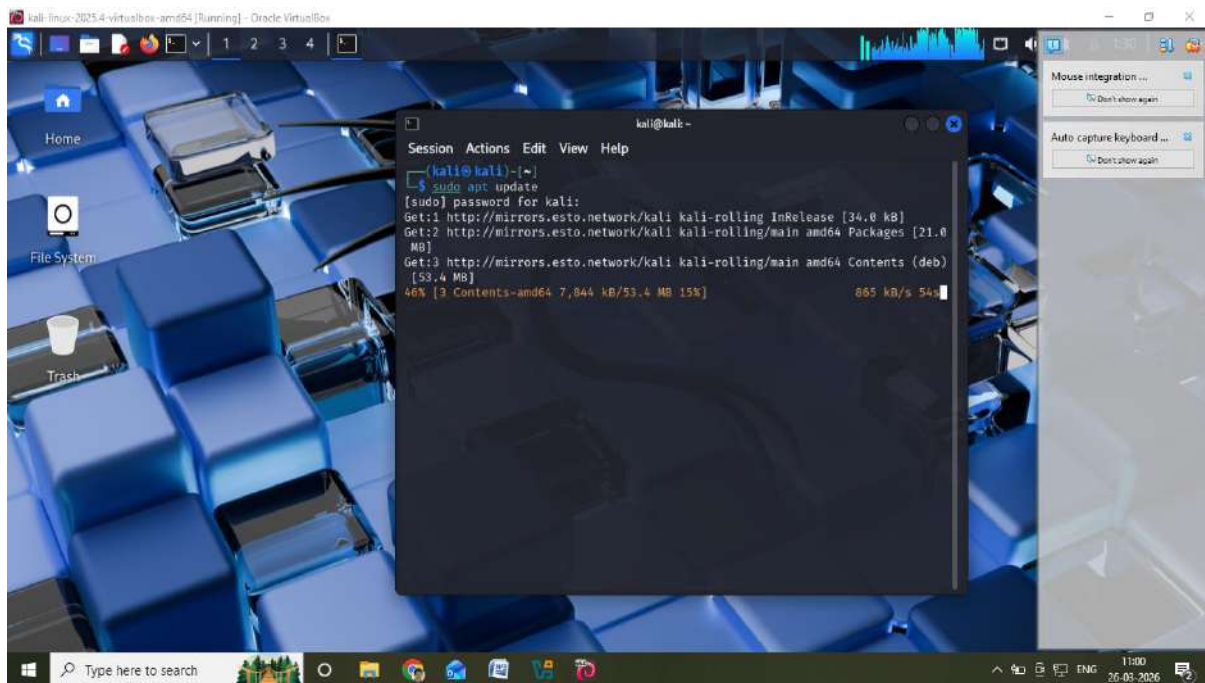
Common vulnerabilities:

- SQL Injection
- Weak passwords
- Brute force attacks
- Session hijacking
- Session fixation
- Improper logout

PROCEDURE

STEP 1: Install Docker

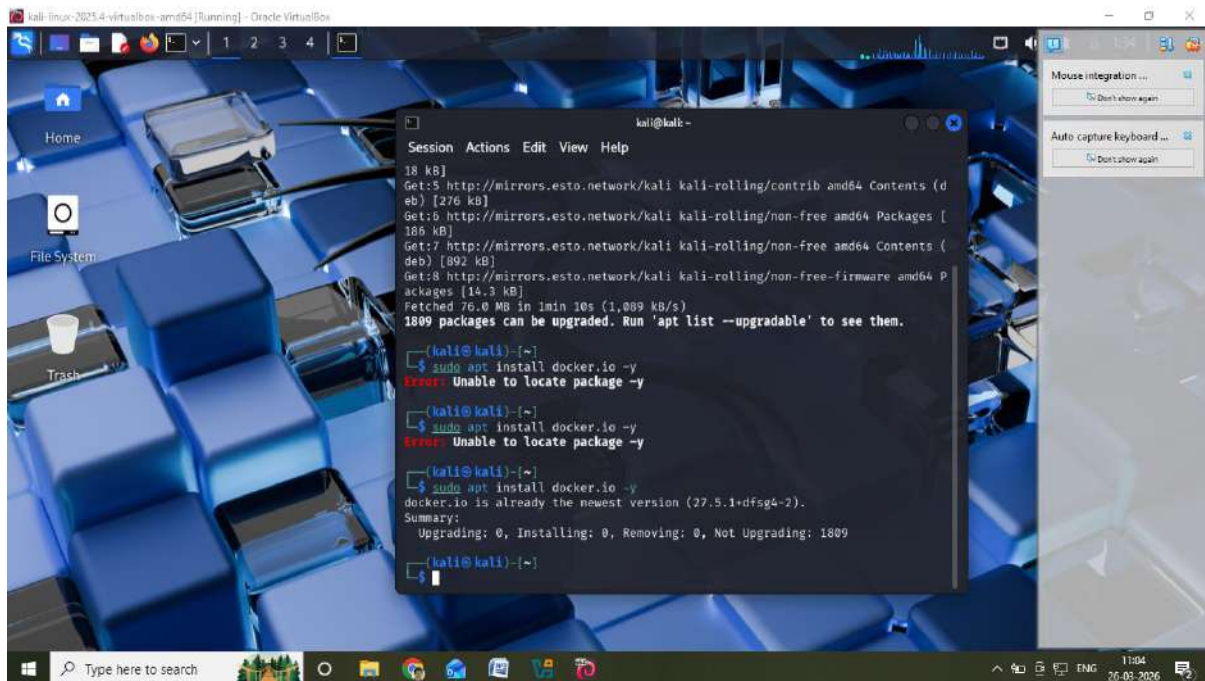
sudo apt update



The screenshot shows a Kali Linux desktop environment with a terminal window open. The terminal displays the output of the 'sudo apt update' command. The output shows the system is updating its package lists from mirrors. The progress bar indicates that 46% of the packages are being updated, with a total of 7,844 kB/53.4 MB. The terminal window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The desktop background is a blue and white keyboard. The taskbar at the bottom shows the search bar and several application icons. The system clock in the bottom right corner shows 11:00 on 26-03-2026.

```
kali@kali:~$ sudo apt update
[sudo] password for kali:
Get:1 http://mirrors.esto.network/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirrors.esto.network/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:3 http://mirrors.esto.network/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]
46% [3 Contents-amd64 7,844 kB/53.4 MB 15%] 865 kB/s 54%
```

sudo apt install docker.io -y



The screenshot shows the same Kali Linux desktop environment. The terminal window now displays the output of the 'sudo apt install docker.io -y' command. The output shows that the system is fetching 76.0 MB in 1min 10s (1,089 kB/s) and that 1809 packages can be upgraded. The terminal window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The desktop background is a blue and white keyboard. The taskbar at the bottom shows the search bar and several application icons. The system clock in the bottom right corner shows 11:04 on 26-03-2026.

```
kali@kali:~$ sudo apt install docker.io -y
18 kB]
Get:5 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Contents (deb) [276 kB]
Get:6 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:7 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Contents (deb) [892 kB]
Get:8 http://mirrors.esto.network/kali kali-rolling/non-free-firmware amd64 Packages [14.3 kB]
Fetched 76.0 MB in 1min 10s (1,089 kB/s)
1809 packages can be upgraded. Run 'apt list --upgradable' to see them.

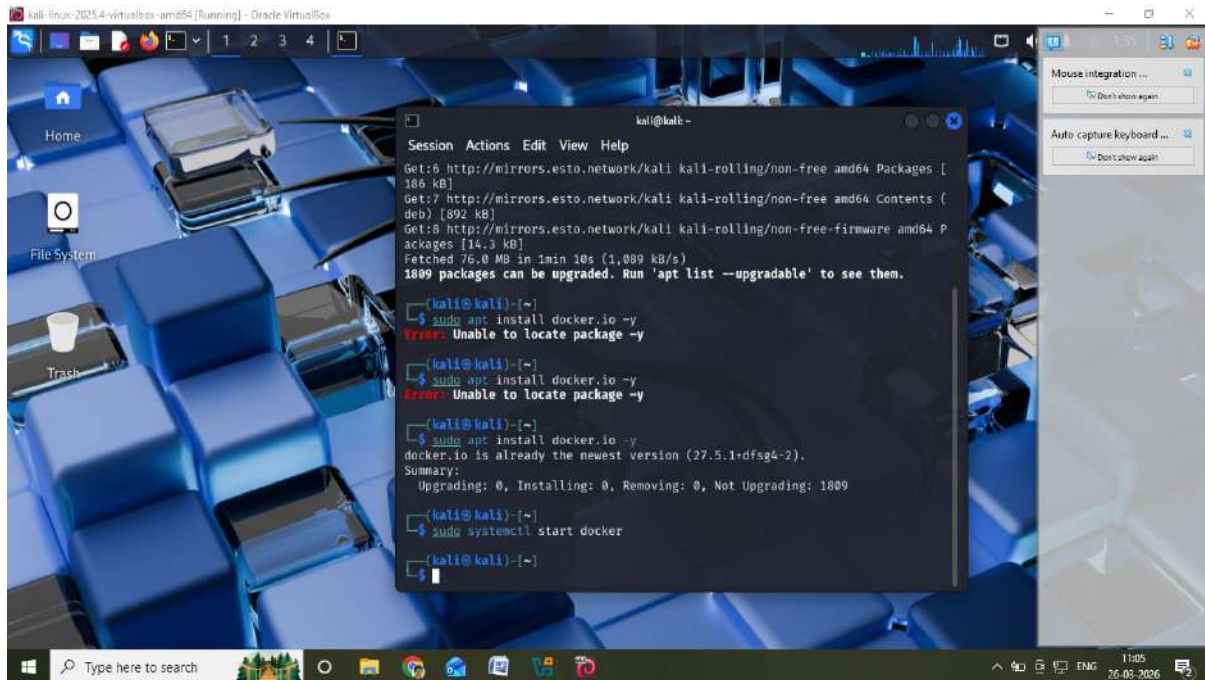
(kali@kali)~$ sudo apt install docker.io -y
Error: Unable to locate package -y

(kali@kali)~$ sudo apt install docker.io -y
Error: Unable to locate package -y

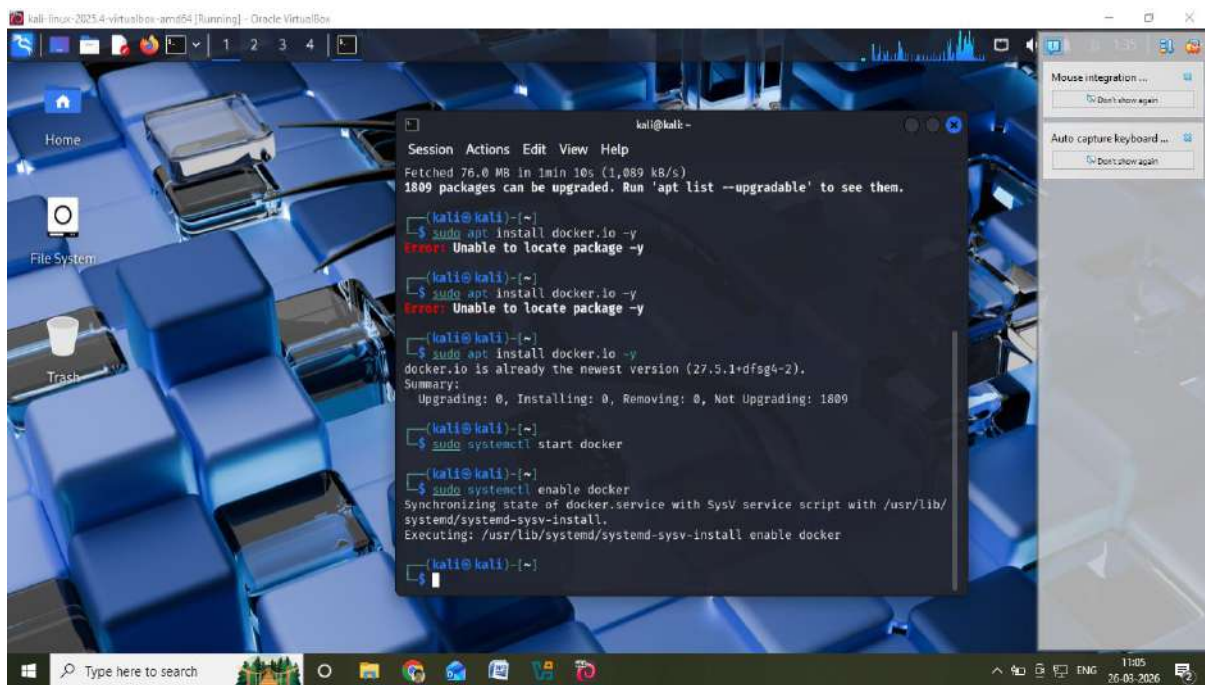
(kali@kali)~$ sudo apt install docker.io -y
docker.io is already the newest version (27.5.1-dfsg4-2).
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1809

(kali@kali)~$
```

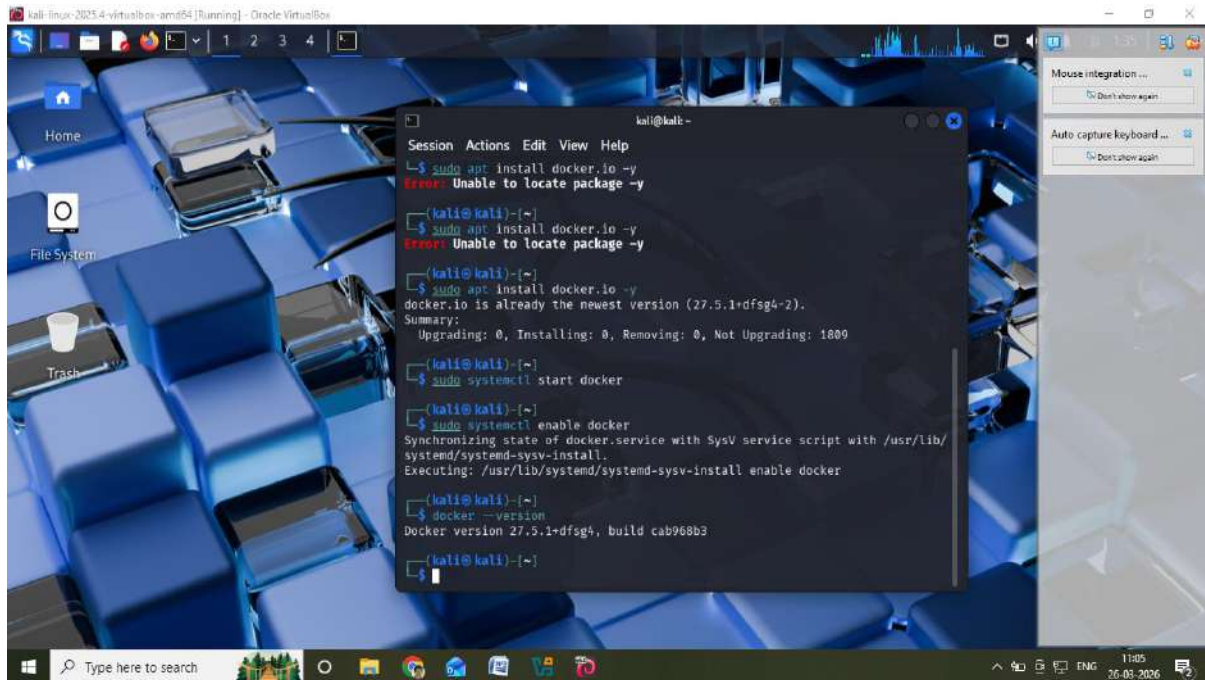
sudo systemctl start docker



sudo systemctl enable docker

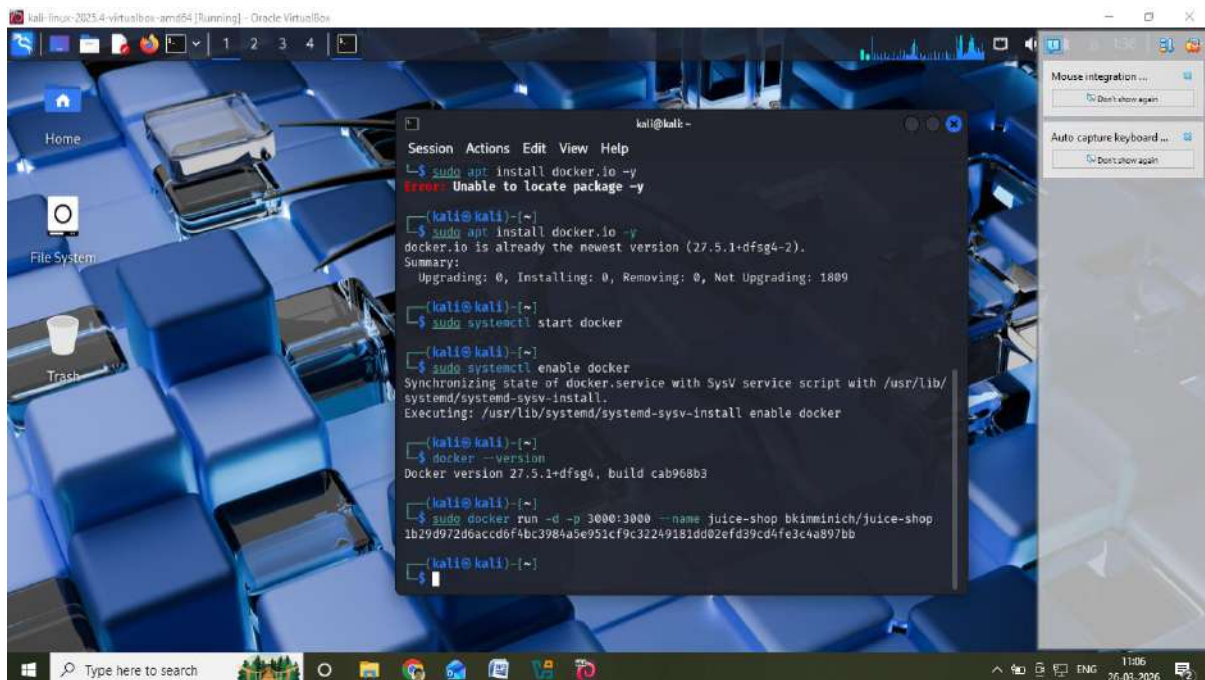


docker --version



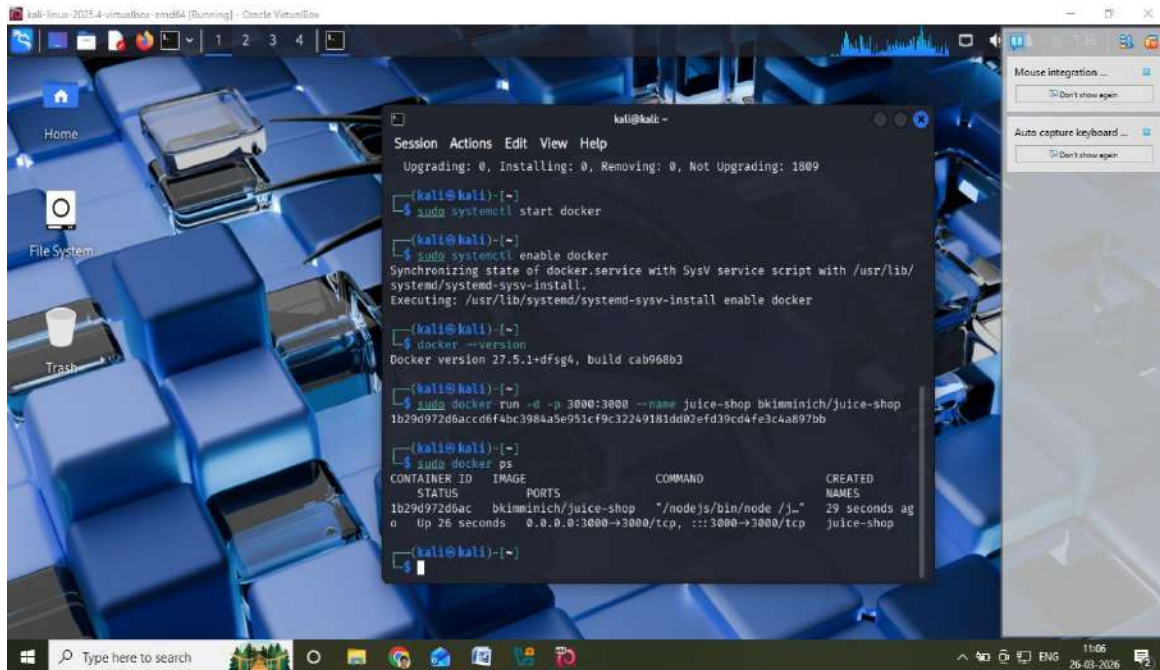
STEP 2: Run Juice Shop using Docker

`sudo docker run -d -p 3000:3000 --name juice-shop bkimminich/juice-shop`



Check container:

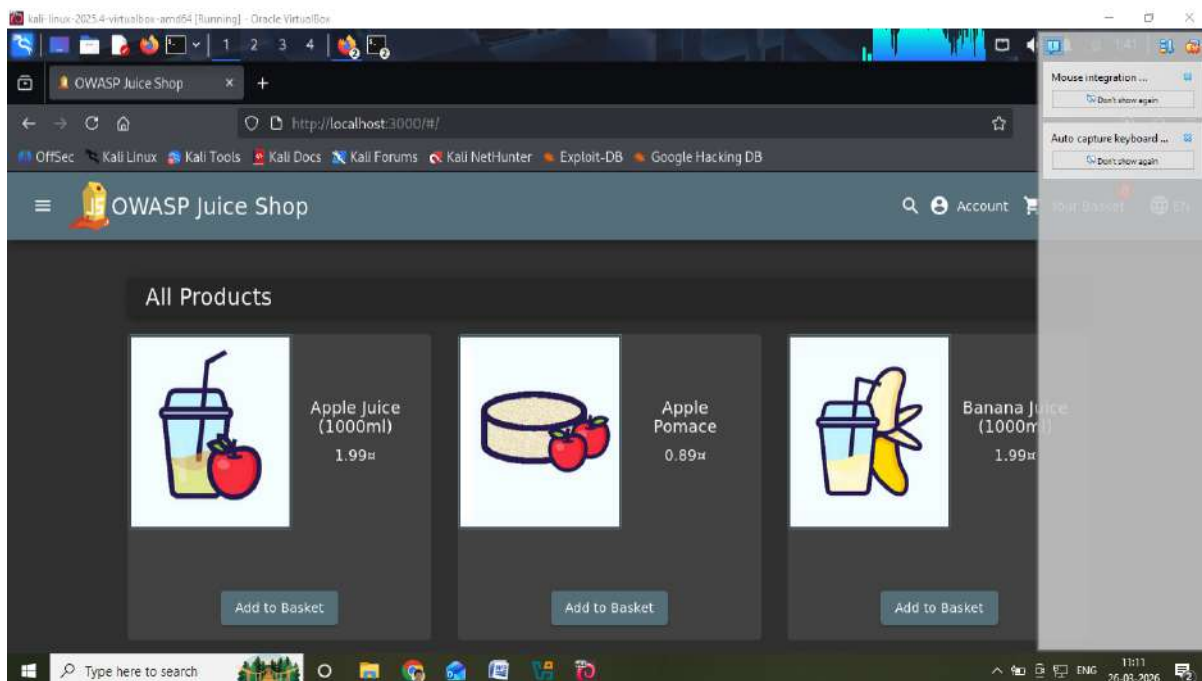
`sudo docker ps`



STEP 3: Open Application

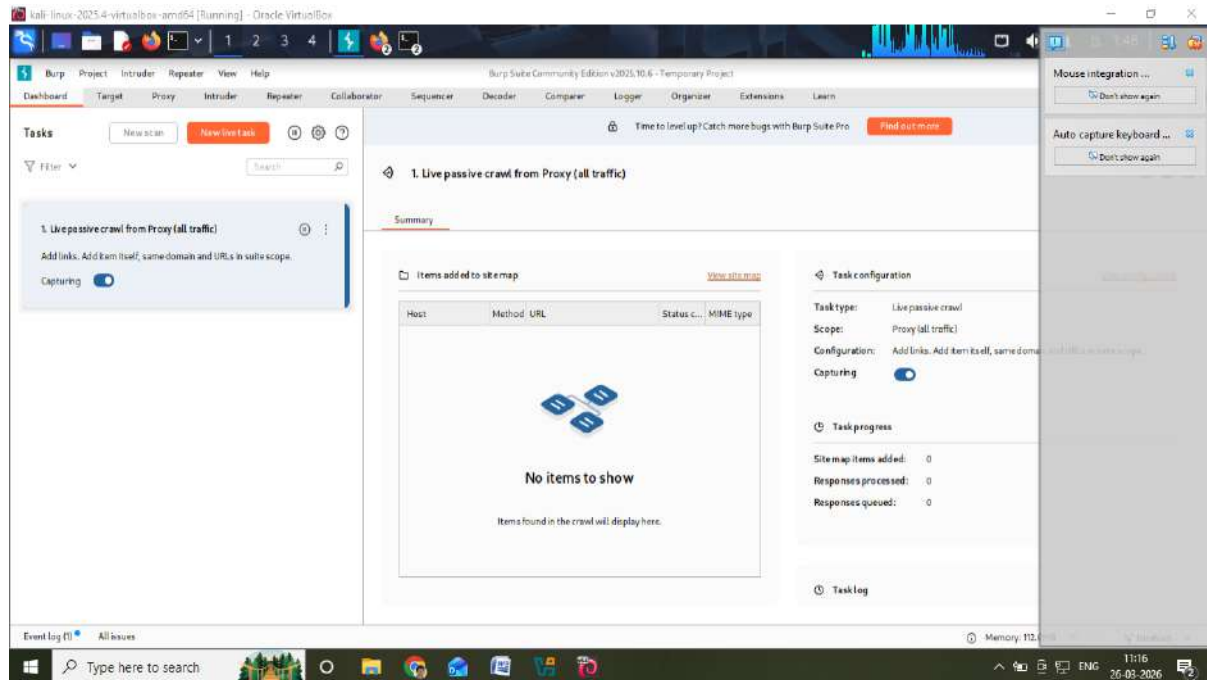
Open browser:

<http://localhost:3000>

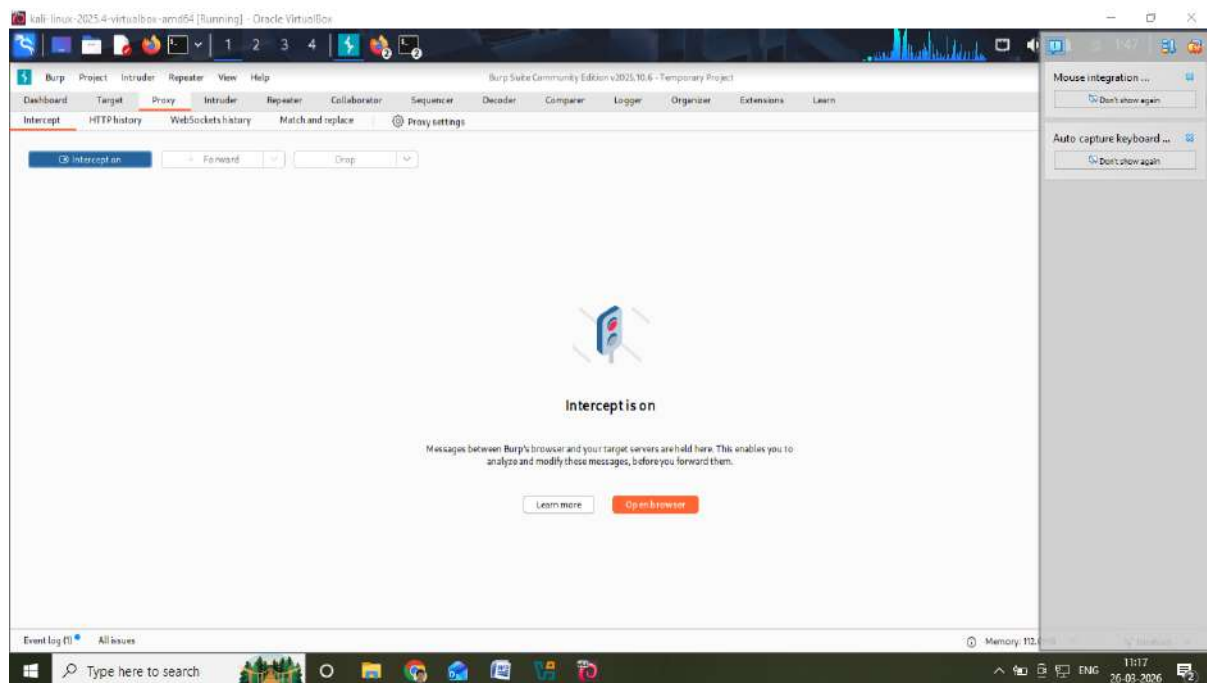


STEP 4: Configure Burp Suite

1. Open Burp Suite

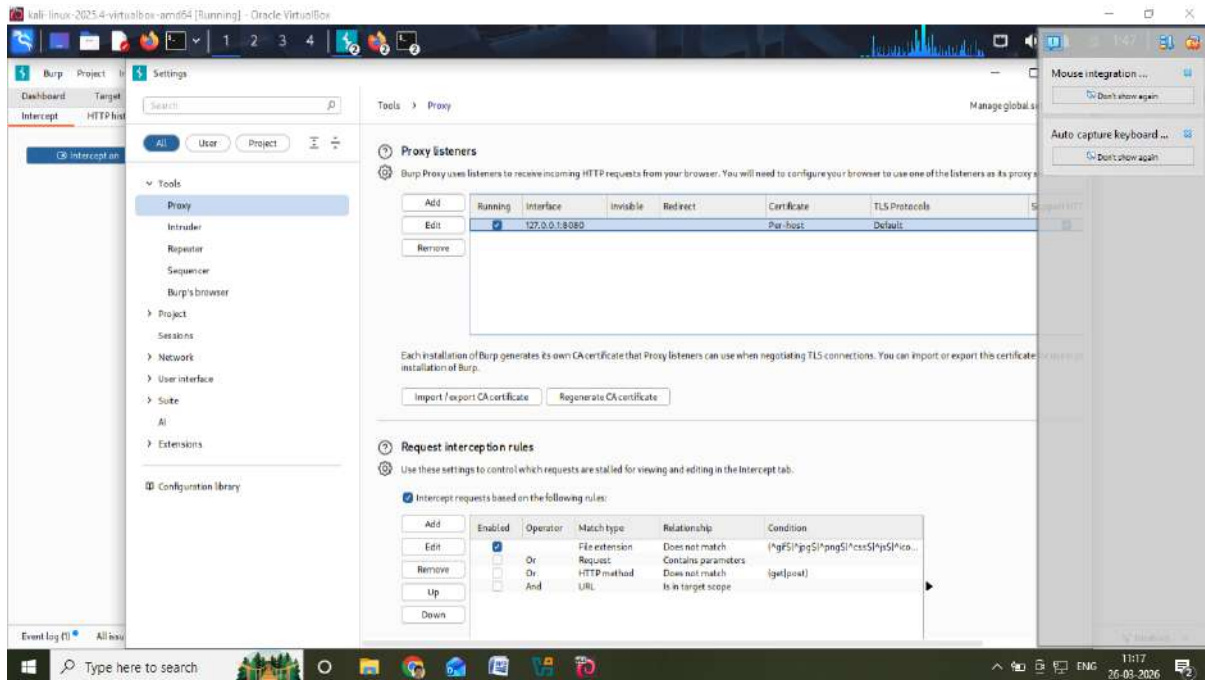


2. Proxy → Intercept ON



3. Set browser proxy:

- IP: 127.0.0.1
- Port: 8080



PART A: AUTHENTICATION TESTING

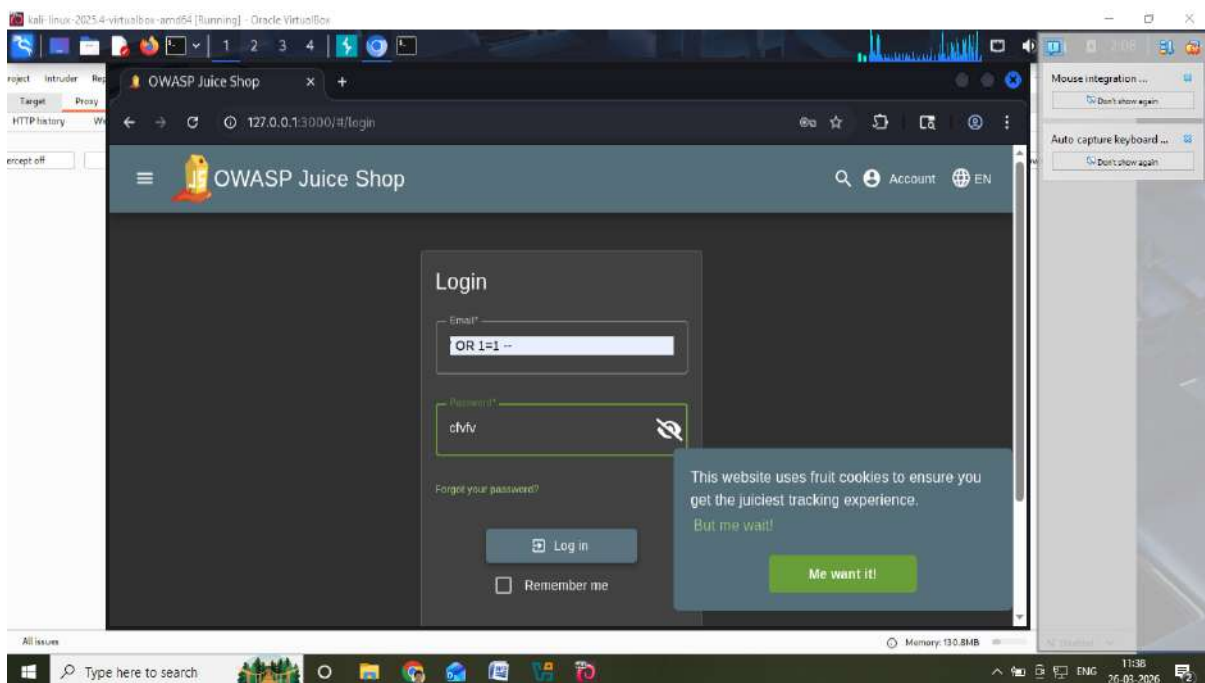
Test 1: SQL Injection Login Bypass

Steps:

1. Open Login page
2. Enter:

Email: ' OR 1=1 --

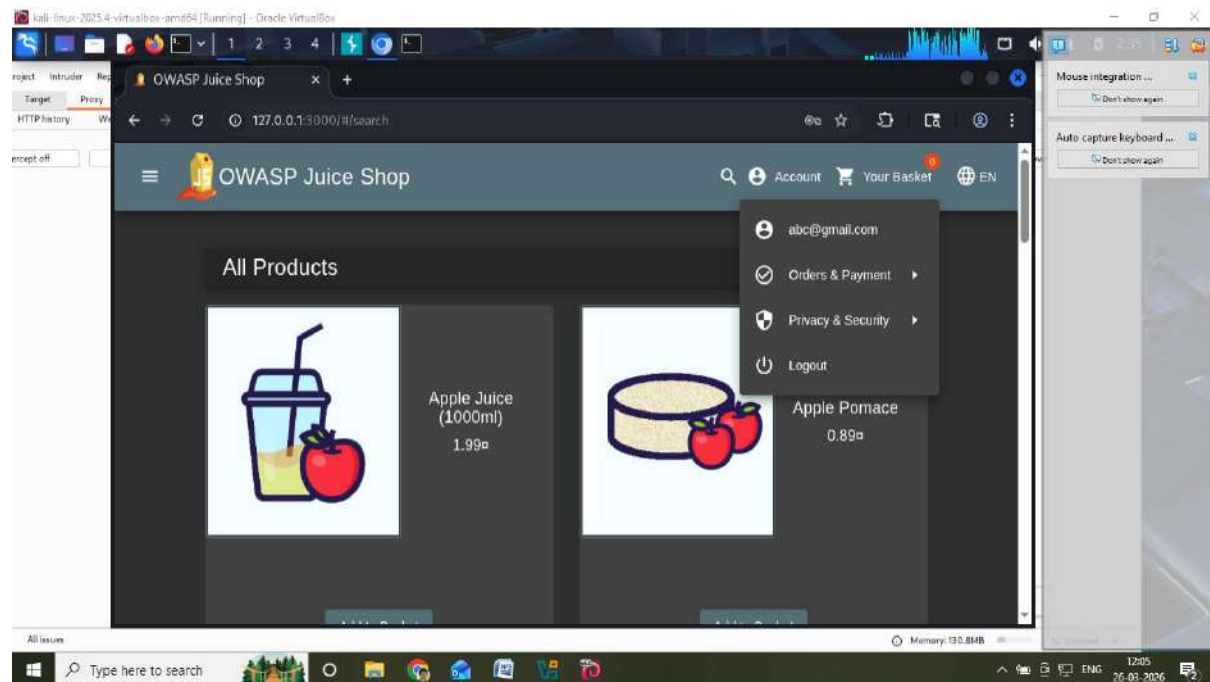
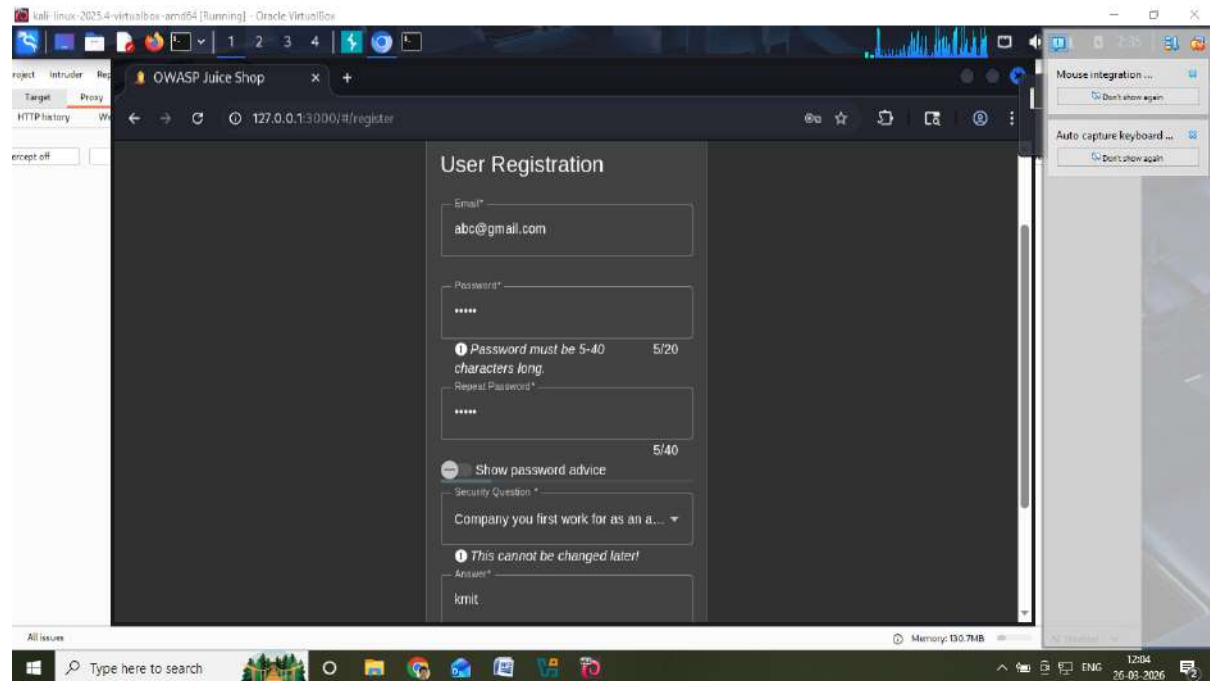
Password: anything



2. Use password:

12345

Result:



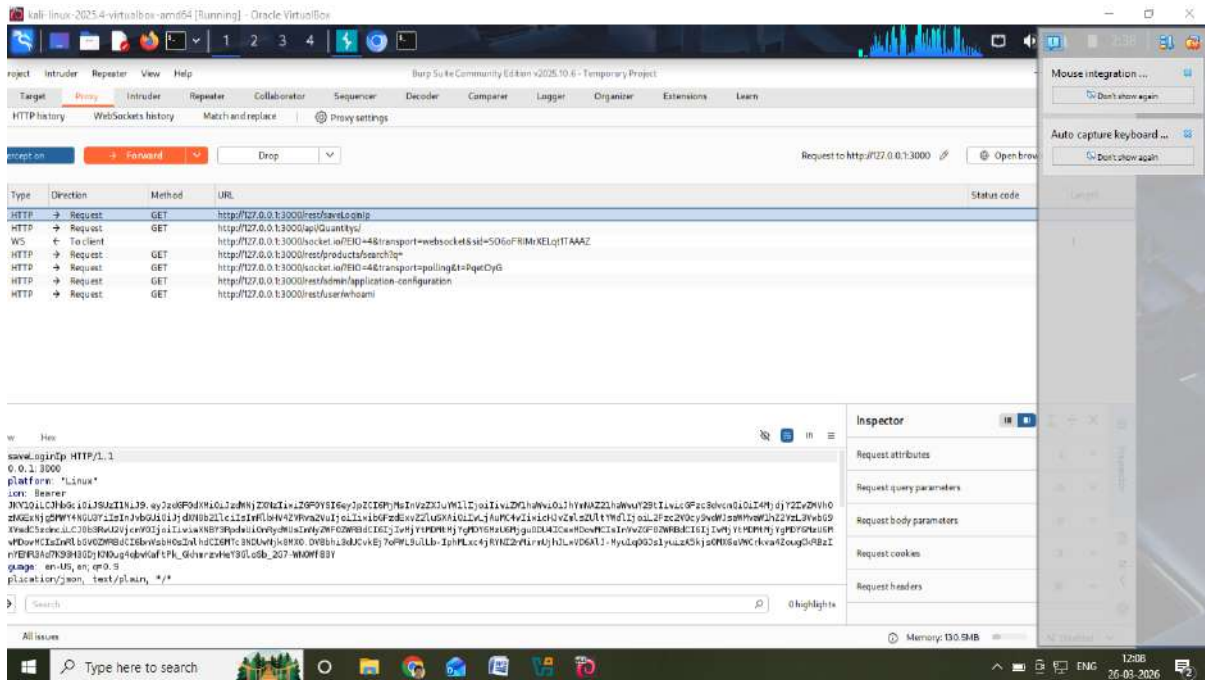
Inference:

No password complexity enforcement

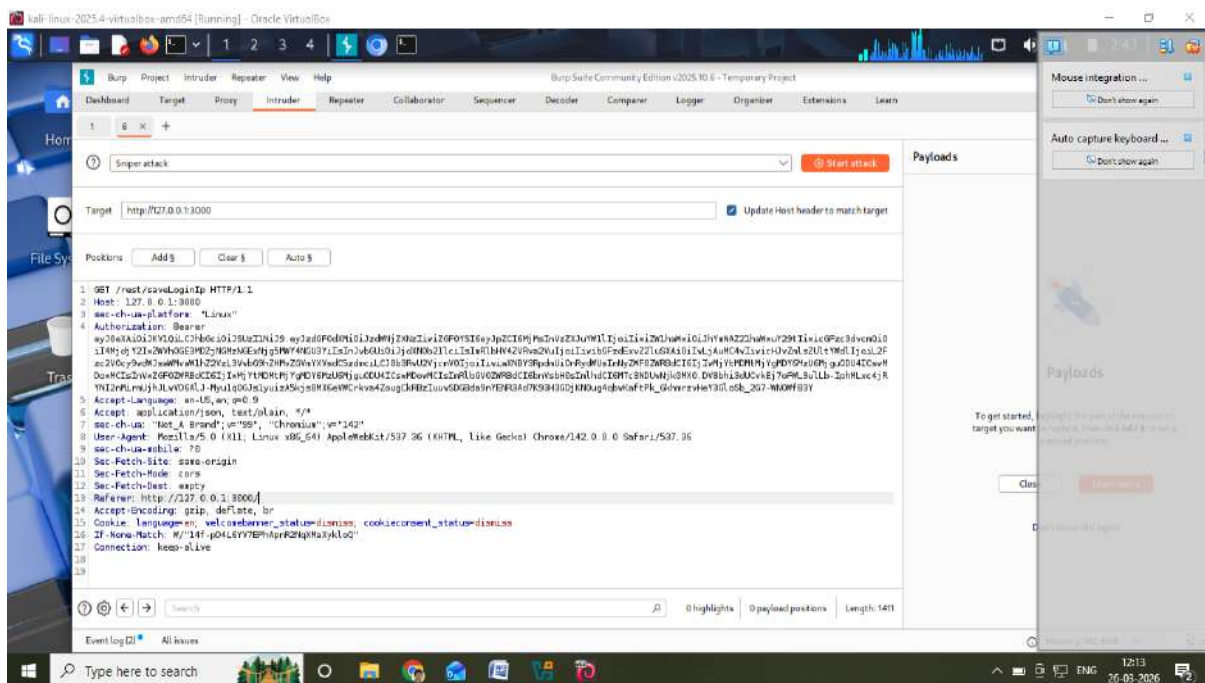
Test 3: Brute Force Attack

Steps:

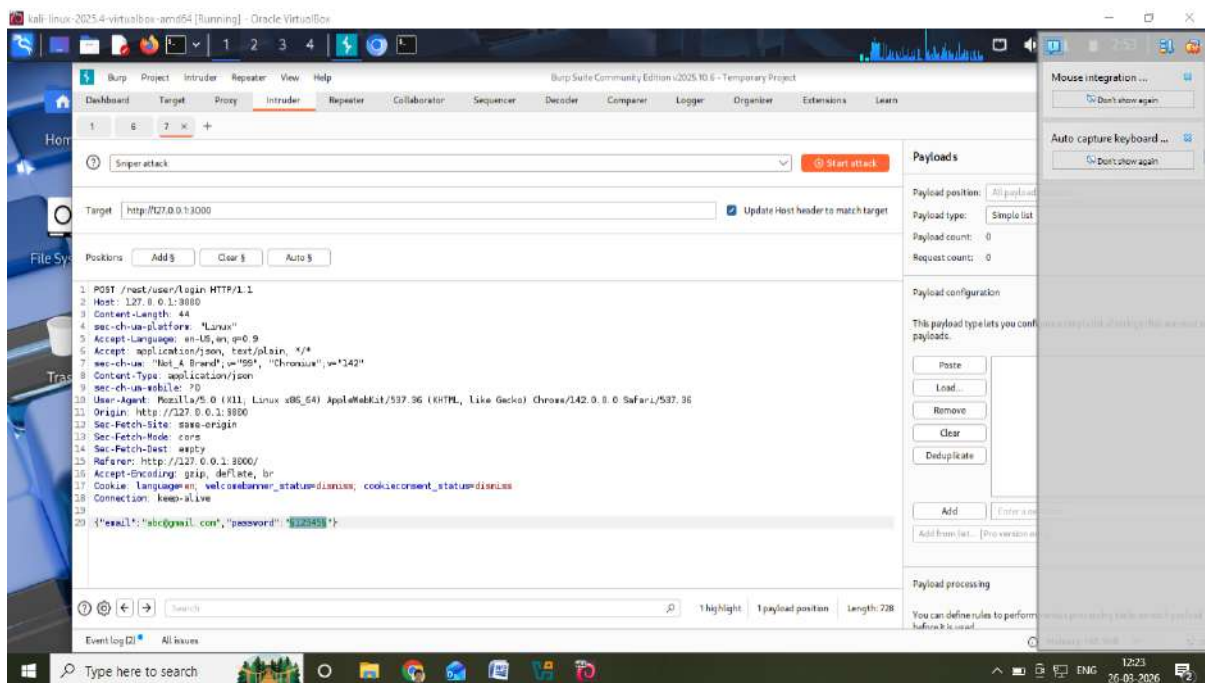
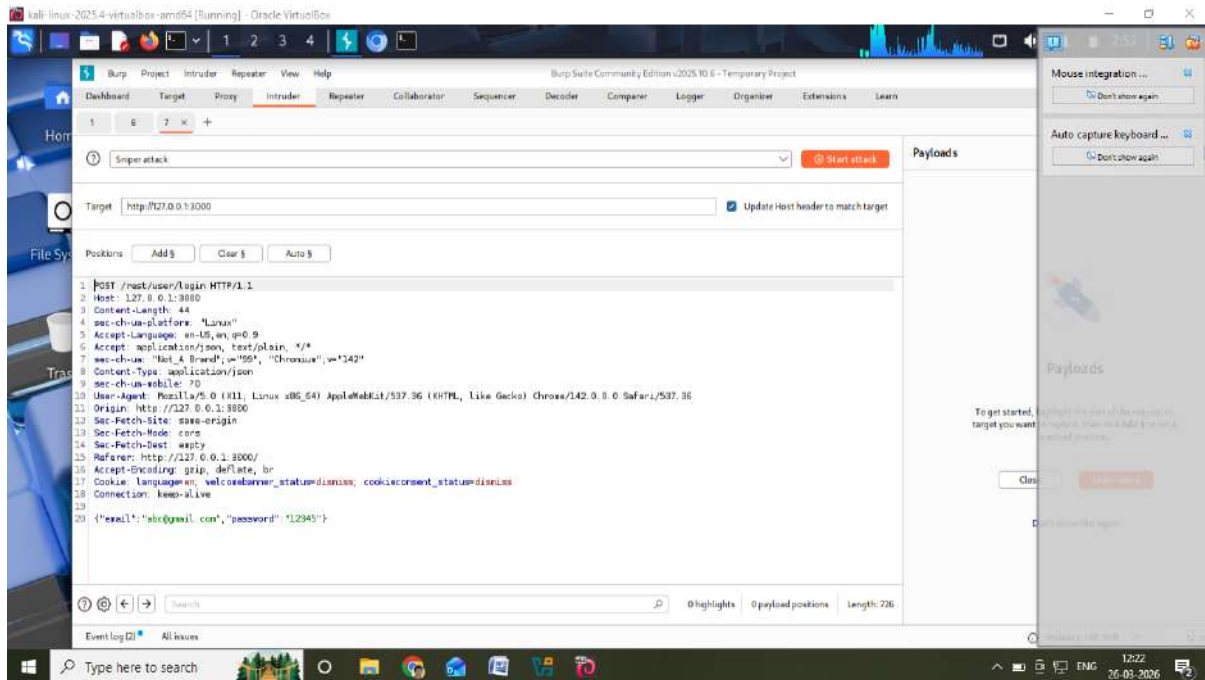
1. Capture login request in Burp



2. Send to Intruder



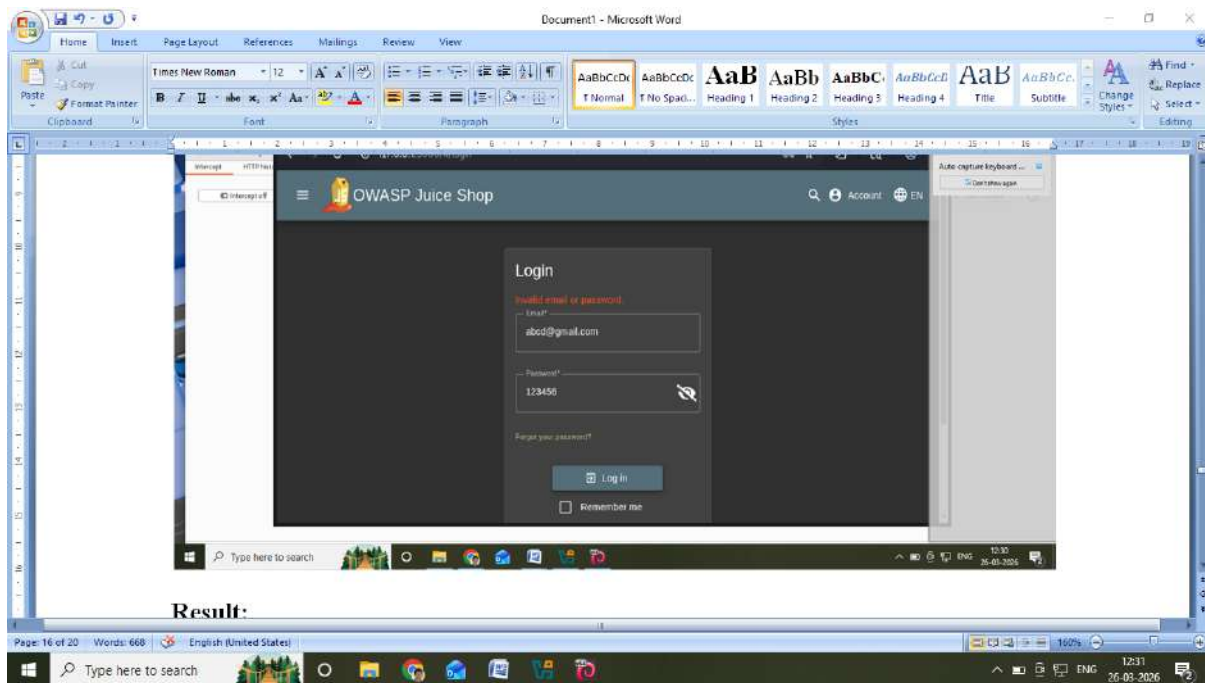
3. Select password field



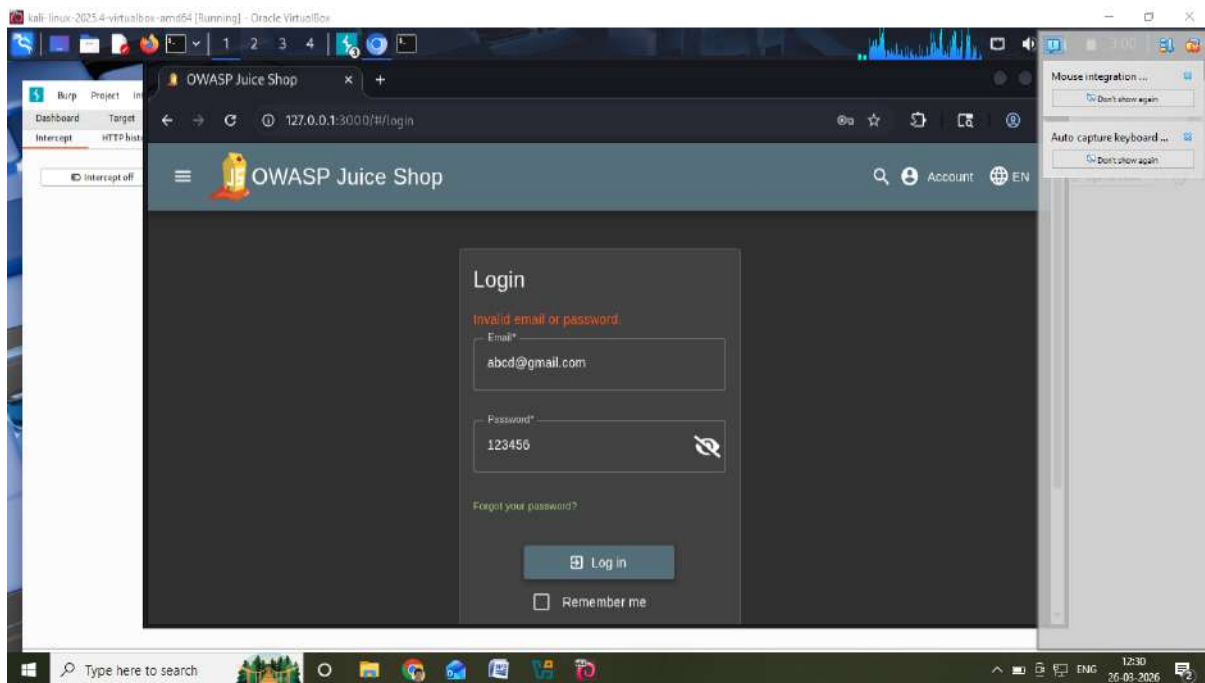
4. Load wordlist:

/usr/share/wordlists/rockyou.txt

- Try valid email + wrong password



- Try invalid email + wrong password



Result:

Different error messages

Inference:

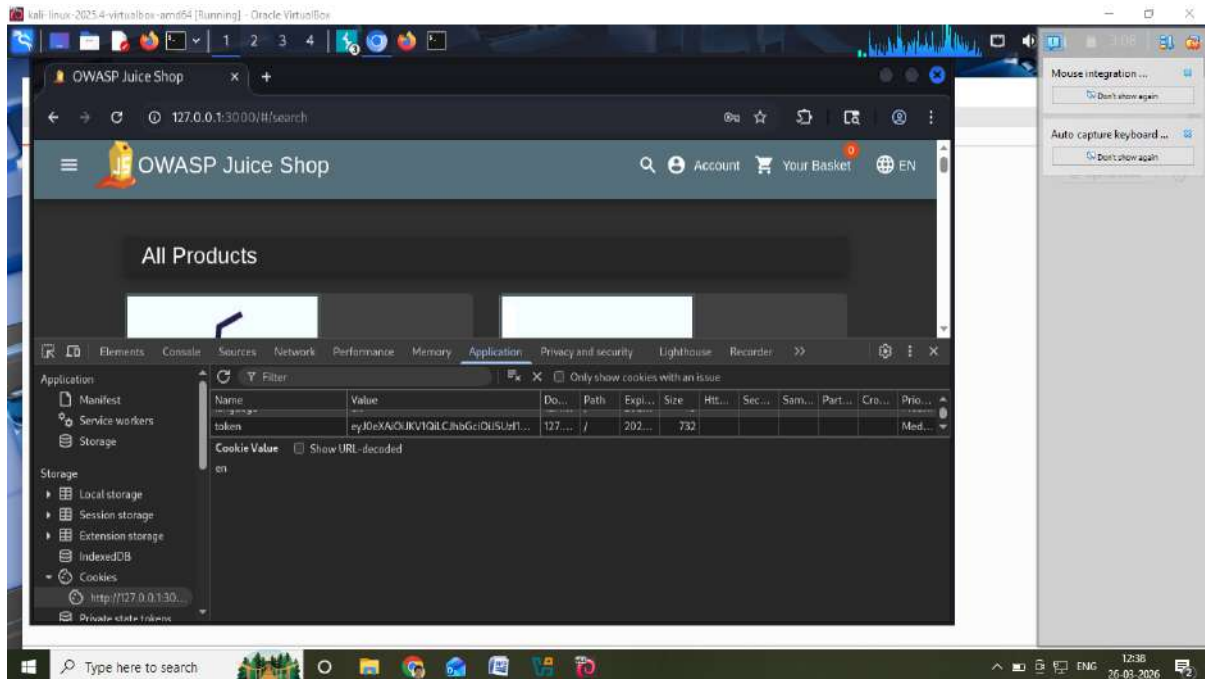
System reveals valid usernames

PART B: SESSION MANAGEMENT TESTING

Test 5: Session Cookie Analysis

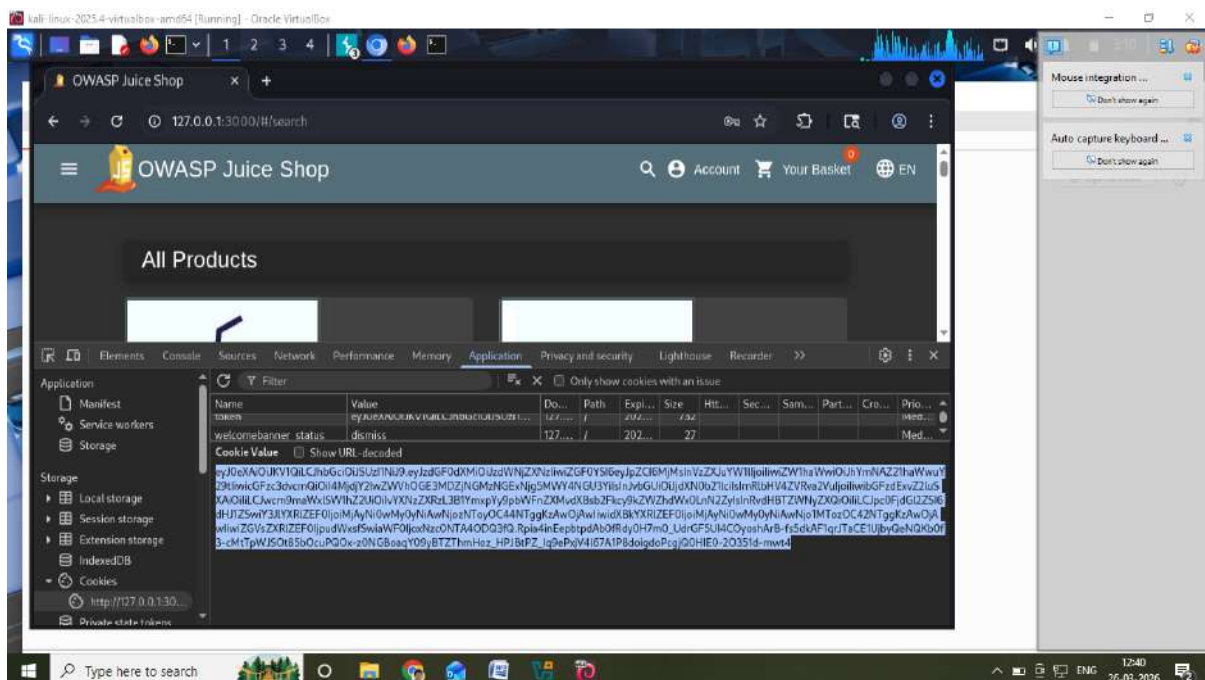
Steps:

1. Login
2. Press F12 → Application → Cookies



Result:

Session token visible

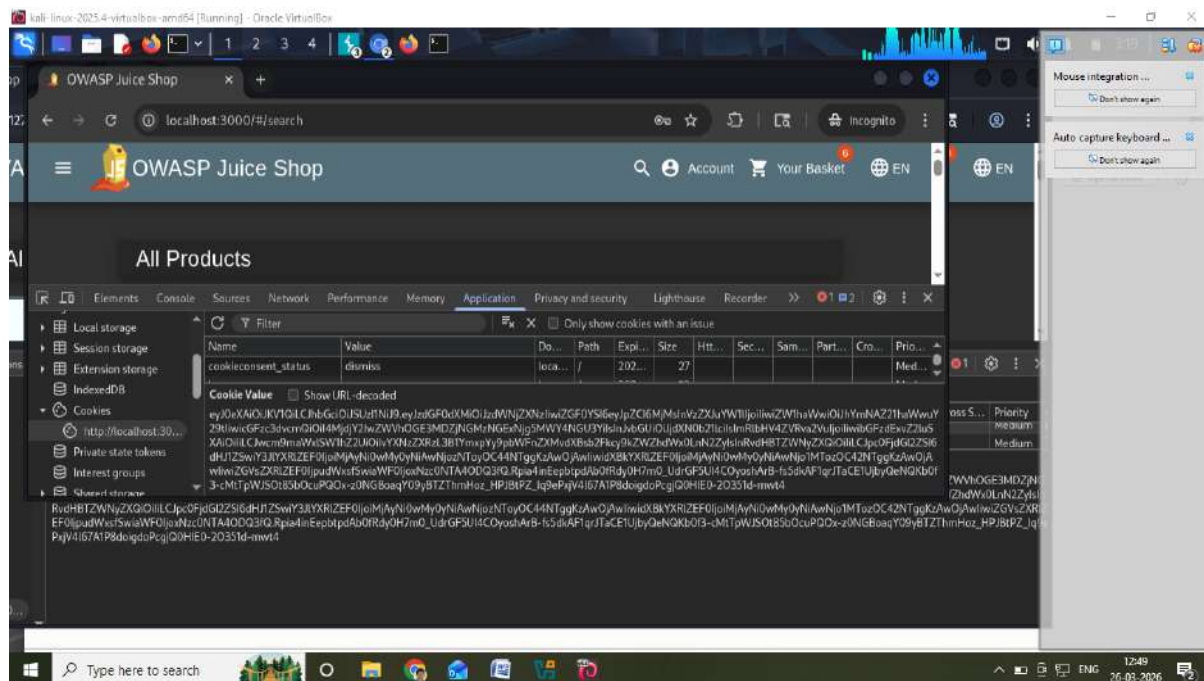


Inference:

Check for HTTPOnly & Secure flags

Steps:

1. Copy session cookie
2. Open another browser
3. Replace cookie



Result:

Access without login

Inference:

Session not properly protected

Test 7: Session Fixation

Steps:

1. Note session ID before login
2. Login
3. Compare session ID

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1Ni9.eyJzdGF0dXMiOiJkdWNjZXNzIiwiaWF0YSI6eyJPZCI6MSwidXNlcm5hbWUiOiiliLCJlbWpCbGl6mFkbWluQGp1aWNlXNoLm9wliwicGFzc3dvcmQoOiIwMTkyMDIzYTdiYmQ3Mzl1MDUxNmYwNjkKzJE4YjUwMCIsInJvbGUoiOiJhZG1pbilScmRlbHV4ZVRva2VuljoiliwiibGFzdExvZ2luSXAiOiIiLCJwcm9maWxlSW1hZ2UuOiJhc3NldHMvcHViIGljL2ltYWdlcy91cGxvYWRzL2RIZmF1bHRBRBG1pbj5wbmcilCj0b3RwU2VjcmlV0ljoiliwiaXNB3RpdmlUOnRydWUsImNyZWFOZWRBdCl6IjlwMjYtMDMtMjYgMDU6Mzc6MTUuNDU2ICswMDowMCIsInVwZGF0ZWRBdCl6IjlwMjYtMDMtMjYgMDU6Mzc6MTUuNDU2ICswMDowMCIsImRlbGV0ZWRBdCl6bnVsblH0slmhdCl6MTc3NDUwOTg4MH0.ZSQs6sHIZMsZ0JsHgprRfXESQ2ERIj2zpx93QQMvCd2Jm3qKGWtVkY8H8acafz1B9HmS7lvoDZ-

K2ST0RT2b5xV6wDd8J1hdlhoGlleM1ZCGVt8yuuBaAs36Mfakm2wOq7nrNowYRKcQf5rwnPUme9N5_Yy3
oBhNi-Zk3CxJQSM

Using Burp Suite

Step 1: Intercept Login Request

- Turn **Intercept ON**
- Login in browser

Step 2: Check Response

Go to:

Proxy → HTTP History

Click login request

Step 3: Find Token

Check:

- **Response tab**
- OR Headers:

Set-Cookie: token=

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwia2F0YSI6eyJpZCI6MSwidXNlcm5hbWUiOiIiLCJlbWpCbCI6ImFkbWluQGp1aWNILXNoLm9wliwicGFzc3dvcmlhbmMTkyMDIzYTdiYmQ3MzI1MDUxNmYwYmJkZjE4YjUwMCIsInJvbGUiOiJhZG1pbilslmRlbHV4ZVRva2VuljoiliwibGFzdExvZ2luSXAiOiIiLCJwcm9maWxISW1hZ2UuOiJhc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHRBZG1pbjE5bmciLCJ0b3RwU2VjcmV0IjoiliwiaXNBY3RpdmlhbmRydWUslmNyZWFOZWRBdCI6IjIwMjYyYmM5MDU0MzI1MDUxNmYwYmJkZjE4YjUwMCIsInJvbGUiOiJhZG1pbilslmRlbGV0ZWRBdCI6bnVsbH0slmlhdCI6MTc3NDUwOTg0MjY0LjZSQS6sHIZMsZ0JshGpRrfXESQ2ERIj2zpx93QQMvCd2Jm3qKGWtVky8H8acafz1B9HmS7lvoDZ-

K2ST0RT2b5xV6wDd8J1hdlhoGileM1ZCGVt8yuuBaAs36Mfakm2wOq7nrNowYRKcQf5rwnPUme9N5_Yy3
oBhNi-Zk3CxJQSM

That value = **Session ID**

Result:

Same session ID

Inference:

Session fixation vulnerability exists

Test 8: Logout Mechanism

Steps:

1. Login → copy cookie
2. Logout

3. Reuse cookie

Result:

Session still active

Inference:

Session not invalidated properly

Test 9: JWT Token Analysis**Steps:**

1. Copy token
2. Decode in jwt.io

Result:

Payload visible

Screenshot 13: JWT decoding**Inference:**

Improper token validation possible

DOCKER COMMANDS USED

docker ps

docker stop juice-shop

docker start juice-shop

docker rm juice-shop

docker logs juice-shop

RESULT

Authentication and session vulnerabilities were successfully identified in the application, including:

- SQL Injection
- Weak password policy
- Brute force vulnerability
- Session hijacking
- Session fixation

Experiment: 15 Analysing Android App Permissions and Mobile Traffic

1. Aim

To analyse Android application network traffic and permissions by intercepting HTTP and HTTPS requests using an Android Emulator and Burp Suite.

2. Objective

- To create and run an Android Virtual Device (AVD).
- To configure a proxy between the Android Emulator and Burp Suite.
- To capture HTTP traffic from the Android device.
- To intercept and analyse HTTPS traffic by installing the Burp Suite CA certificate.

3. Required Tools / Software

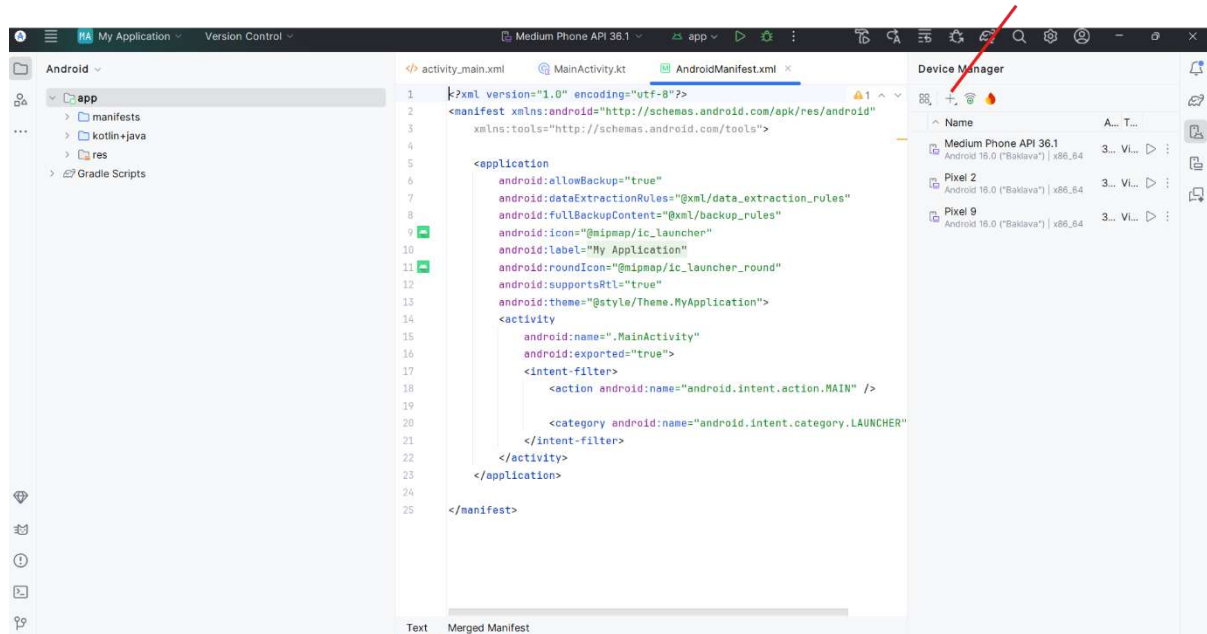
- Android Studio
- Android Emulator (AVD Manager)
- Burp Suite
- ADB (Android Debug Bridge)
- Google Chrome (inside emulator)
- Internet connection

4. Procedure

Step 1: Create Android Virtual Device

1. Install **Android Studio**.
2. Open **AVD Manager**.
3. Create a new Virtual Device.

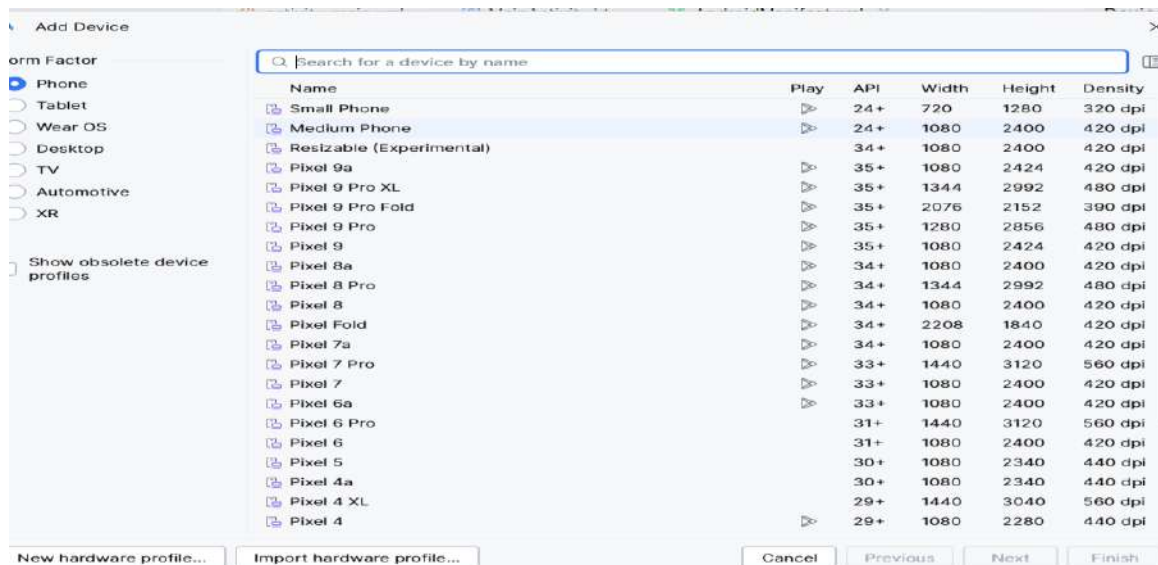
Step-1: Download Android Studio->Create Virtual Device



Step 2: Select Device

Choose any device such as:

- Pixel 2
- Pixel 9



Step-3: Start AVD from Command Prompt1

C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\emulator

C:\Users\Admin\AppData\Local\Android\Sdk\emulator>emulator -list-avds

Medium_Phone_API_36.1

Pixel_2

Pixel_9

```
C:\Users\Admin\AppData\Local\Android\Sdk\emulator>emulator -avd Pixel_9 -http-proxy  
http://10.0.2.2:8080
```



Step-4: Setup Proxy:

In AVD

Settings-> Network and Internet-> internet->select icon  -> top right corner there is edit->advanced options -> proxy->Change proxy None->Manual->Hostname as 10.0.2.2 ->save



Note:if save option is not working then follow the steps in command prompt

Command Prompt2:

```
C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools
```

```
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb devices
```

List of devices attached

emulator-5554 device

```
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings put global http_proxy  
10.0.2.2:8080
```

```
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings get global http_proxy  
10.0.2.2:8080
```

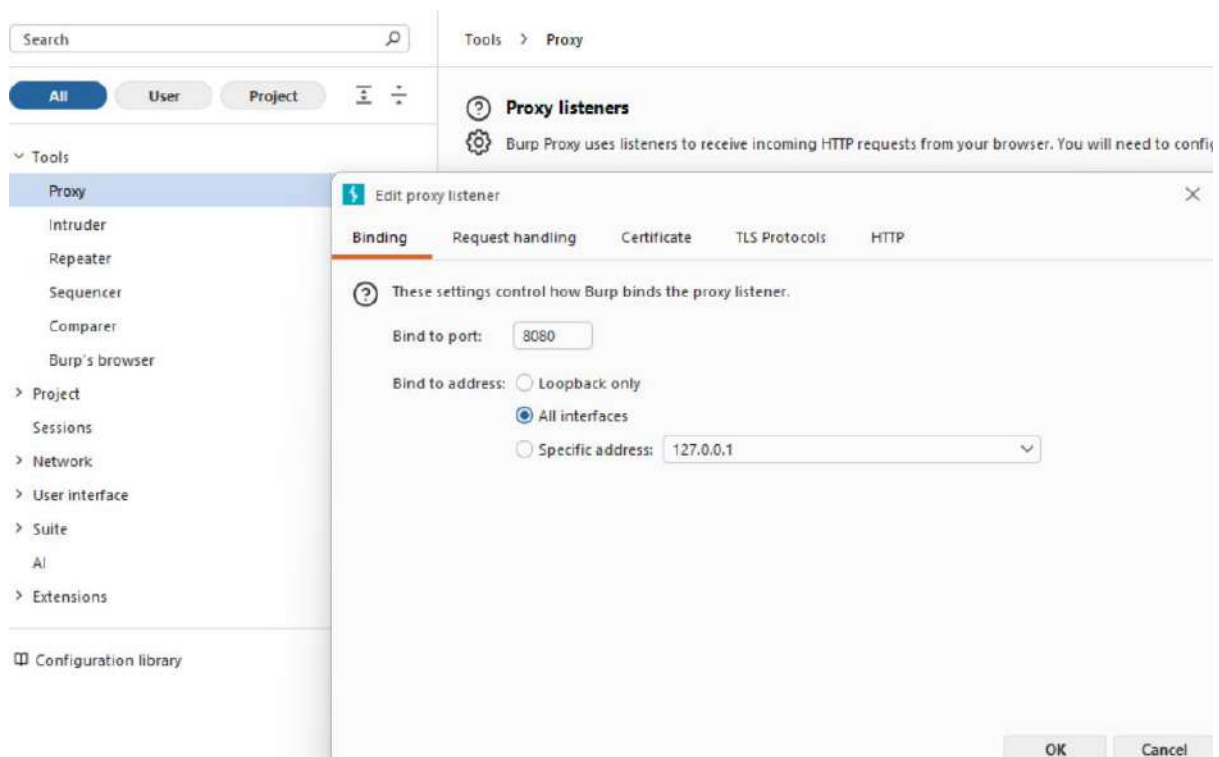
```
C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb devices
List of devices attached
emulator-5554    device

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings put global http_proxy 10.0.2.2:8080
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings get global http_proxy
10.0.2.2:8080
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>|
```

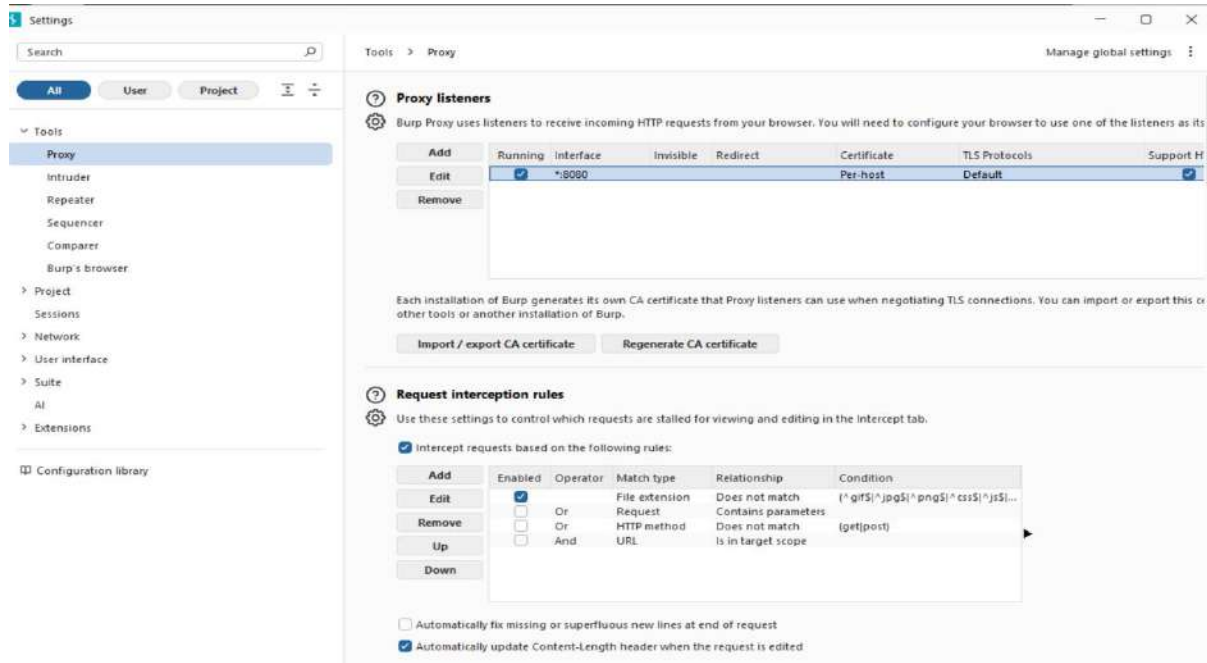
Step 5: In BurpSuit

Go proxy->options/proxy settings->under Proxy Listeners->Edit->select All interfaces->ok

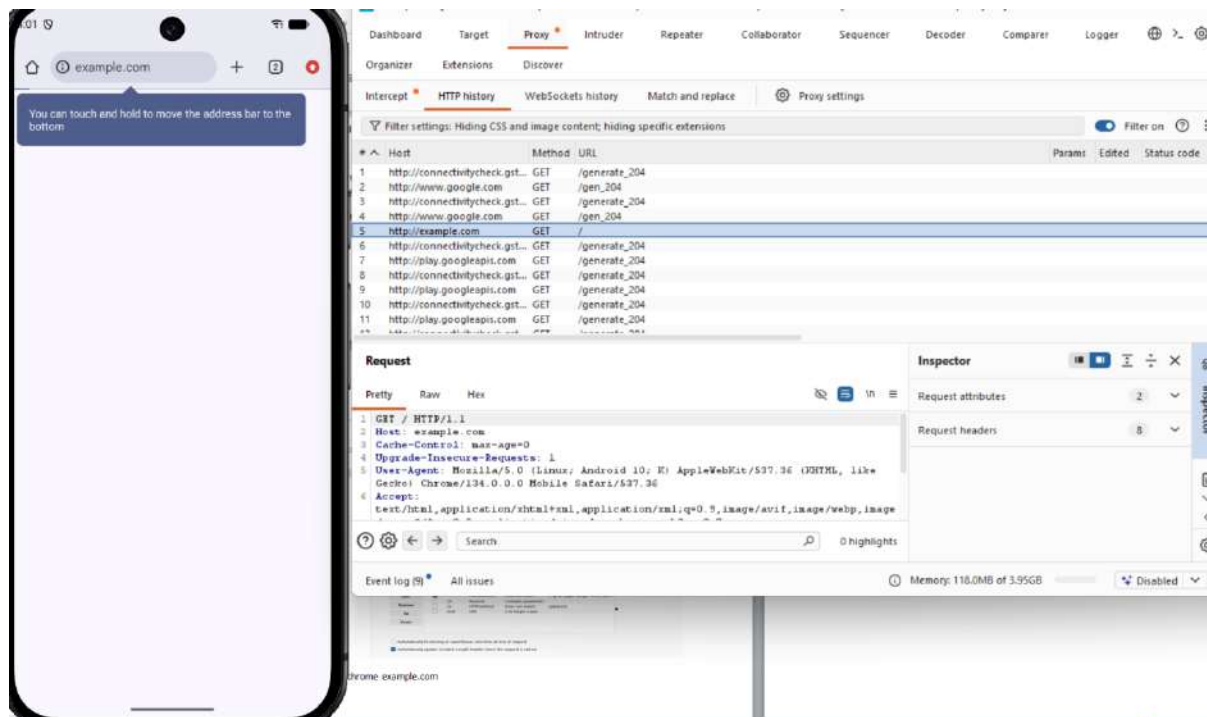
Go proxy->intercept on



You can see BurpSuite is accepting all interfaces traffic

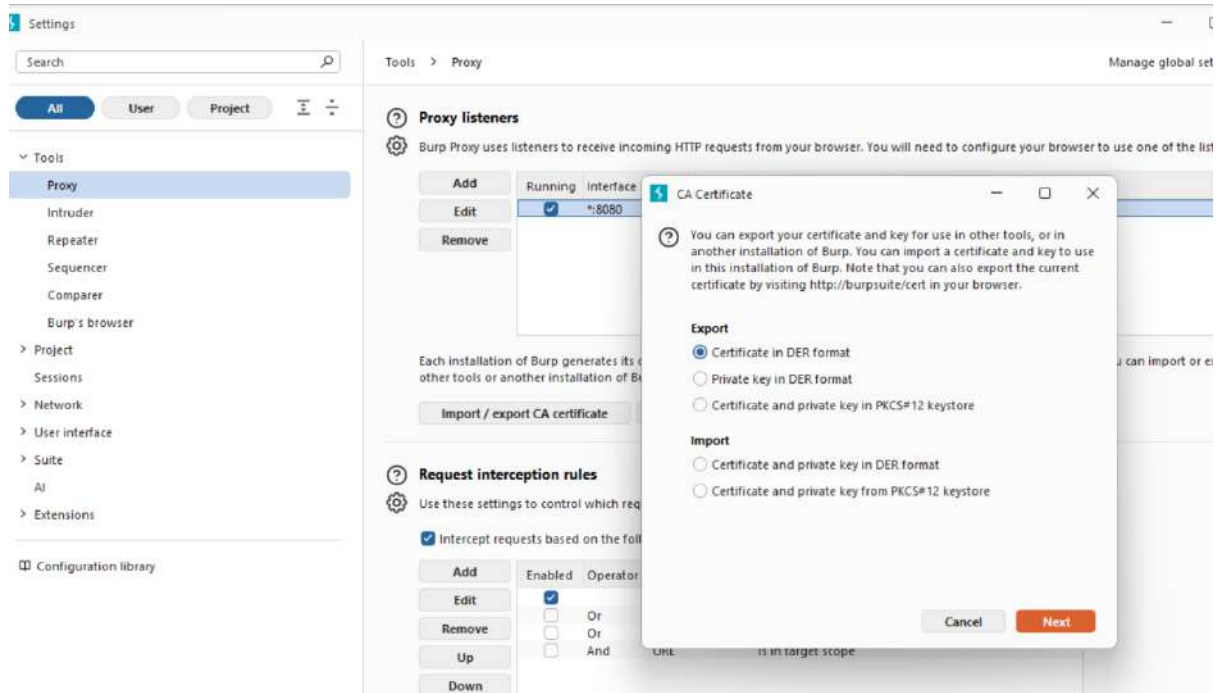


Step 6: in AVD type in chrome example.com and you can see burpsuit HTTP history



Step 7: in Step 5 we only able to http request and response only but if you want to analyse https request and response follow the Procedure to download CA certificate from Bunrpsuit.

Go procy->options/proxy settings->import/export CA certificate->Certificate in DER format->select file->Downloads->give name as burpcer.der



Step 8: Sending CA certificate to AVD

Go to command Prompt2

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb push
C:\Users\Admin\Downloads\burpcer.cer /sdcard/Download/

```
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb push C:\Users\Admin\Downloads\burpcer.cer /sdcard/Download/
C:\Users\Admin\Downloads\burpcer.cer: 1 file pushed, 0 skipped. 0.0 MB/s (939 bytes in 0.022s)
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>
```

To install Certificate in AVD

Settings

→ Security & privacy

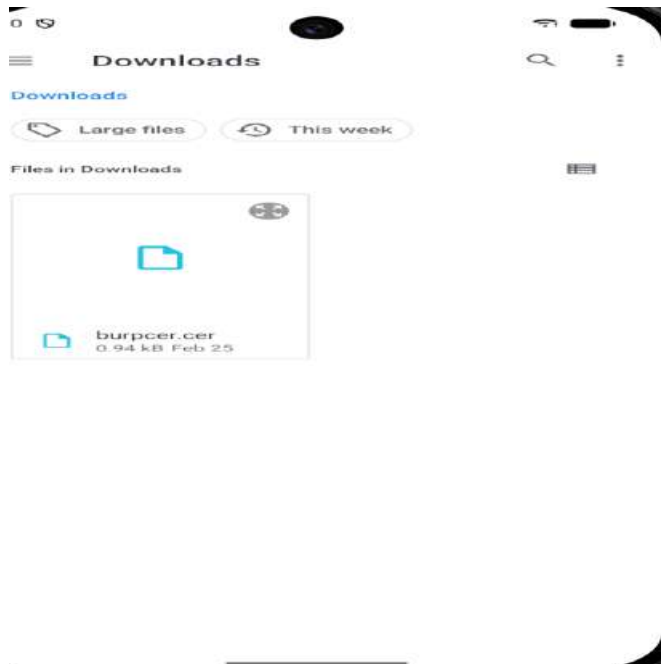
→ More security & privacy

→ Encryption & credentials

→ Install a certificate

→ CA certificate

In AVD we can see burpcer.der certificate



Step 9: verify HTTPS Interception

1. Open browser in emulator.
2. Visit any HTTPS website.

Example:

<https://example.com> or <https://wikipedia.com>

3. In Burp Suite, check **Proxy → HTTP History**.

Now HTTPS requests and responses will be visible.

5. Result

Android application network traffic was successfully intercepted and analysed using Burp Suite by configuring proxy settings in the Android emulator and installing the Burp Suite CA certificate.

6. Precautions

- Ensure Burp Suite proxy port (8080) is correctly configured.
- Verify the emulator is running before executing ADB commands.
- Install the CA certificate properly to capture HTTPS traffic.
- Disable intercept when not required to allow traffic to pass normally.

Experiment 16: Testing IoT Device Security (Default Passwords & Open Ports)

Aim

To analyze security weaknesses in a simulated IoT device by identifying:

- Open ports
- Running services
- Service version disclosure
- Default/weak authentication
- Unencrypted communication

Tools Used

- Windows 10 (Host System)
- Docker Desktop
- OWASP Juice Shop
- Kali Linux
- Nmap

Experimental Setup

- A vulnerable web application (Juice Shop) was deployed using Docker on Windows.
- Kali Linux was used as the attacker machine.
- The Windows system IP address was scanned from Kali Linux to simulate IoT network testing.

Attack Scenario Description

In this experiment:

- Kali Linux acted as the attacker machine.
- Windows system hosting the Docker container acted as the IoT device.

- The vulnerable web application (Juice Shop) simulated the IoT dashboard.
- Network scanning was performed from Kali to the Windows IP address over LAN.

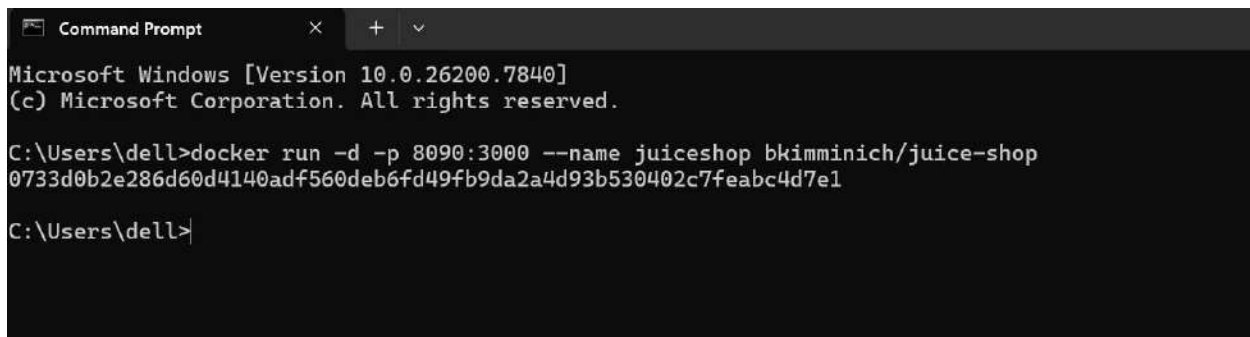
This simulates a real-world scenario where an attacker scans an IoT device connected to the same network.

Procedure

Step 1: Deploy Vulnerable IoT Simulation

The following command was used to run Juice Shop in Docker:

```
docker run -d -p 8090:3000 --name juiceshop bkimminich/juice-shop
```



```
Command Prompt
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\dell>docker run -d -p 8090:3000 --name juiceshop bkimminich/juice-shop
0733d0b2e286d60d4140adf560deb6fd49fb9da2a4d93b530402c7feabc4d7e1

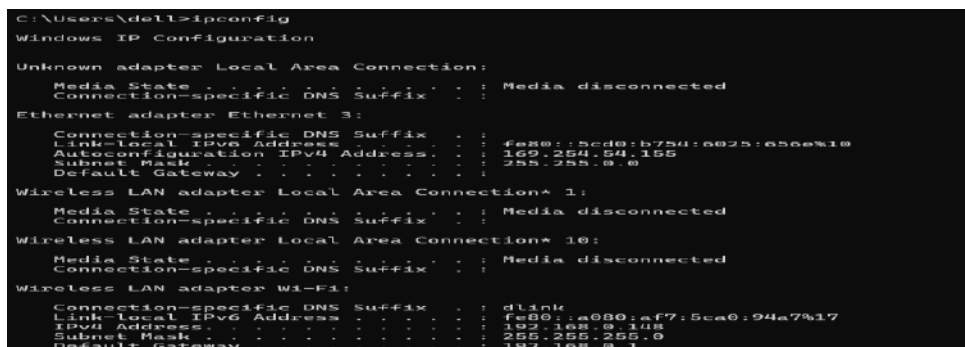
C:\Users\dell>
```

Figure 1: Deployment of simulated IoT device using Docker The application was accessed at:

<http://localhost:8090>

Step 2: Identify Host IP Address

On Windows: ipconfig



```
C:\Users\dell>ipconfig
Windows IP Configuration

Unknown adapter Local Area Connection:
   Media State . . . . . : Media disconnected
   Connection-specific DNS Suffix . :

Ethernet adapter Ethernet 3:
   Connection-specific DNS Suffix . :
   Link-local IPv6 Address . . . . . : fe80::5cd0:b75d:a025:a50e%10
   Autoconfiguration IPv4 Address. . . : 169.254.54.155
   Subnet Mask . . . . . : 255.255.0.0
   Default Gateway . . . . . :

Wireless LAN adapter Local Area Connection* 1:
   Media State . . . . . : Media disconnected
   Connection-specific DNS Suffix . :

Wireless LAN adapter Local Area Connection* 10:
   Media State . . . . . : Media disconnected
   Connection-specific DNS Suffix . :

Wireless LAN adapter Wi-Fi:
   Connection-specific DNS Suffix . : dlink
   Link-local IPv6 Address . . . . . : fe80::a080:af7:5ca0:94a7%17
   IPv4 Address. . . . . : 192.168.0.148
   Subnet Mask . . . . . : 255.255.255.0
   Default Gateway . . . . . : 192.168.0.1
```

Figure 2: Host system IP address identification

Step 3: Perform Network Scan from Kali Linux

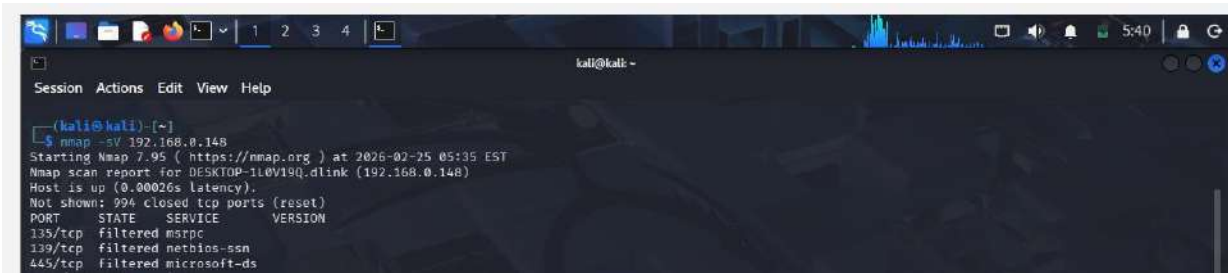


```
kali@kali: ~  
$ nmap localhost  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-25 05:30 EST  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000015s latency).  
Other addresses for localhost (not scanned): ::1  
Not shown: 998 closed tcp ports (reset)  
PORT      STATE SERVICE  
80/tcp    open  http  
3306/tcp  open  mysql  
  
Nmap done: 1 IP address (1 host up) scanned in 0.31 seconds  
  
kali@kali: ~
```

Figure 3: Open ports and service enumeration using Nmap

Step 4: Nmap Version Scan (-sV)

- `nmap -sV 192.168.x.x` output
- Showing service fingerprint / version detection



```
kali@kali: ~  
$ nmap -sV 192.168.0.148  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-25 05:35 EST  
Nmap scan report for DESKTOP-1L0V19Q.dlink (192.168.0.148)  
Host is up (0.00026s latency).  
Not shown: 994 closed tcp ports (reset)  
PORT      STATE SERVICE      VERSION  
135/tcp    filtered msrpc  
139/tcp    filtered netbios-ssn  
445/tcp    filtered microsoft-ds
```

The simulated IoT device dashboard was accessed through the browser using: <http://localhost:8090>



Figure 5: Simulated IoT device web dashboard running on port 8090

Step 6: Testing Default/Weak Credentials

The login functionality was tested using administrator credentials to check for default password vulnerability

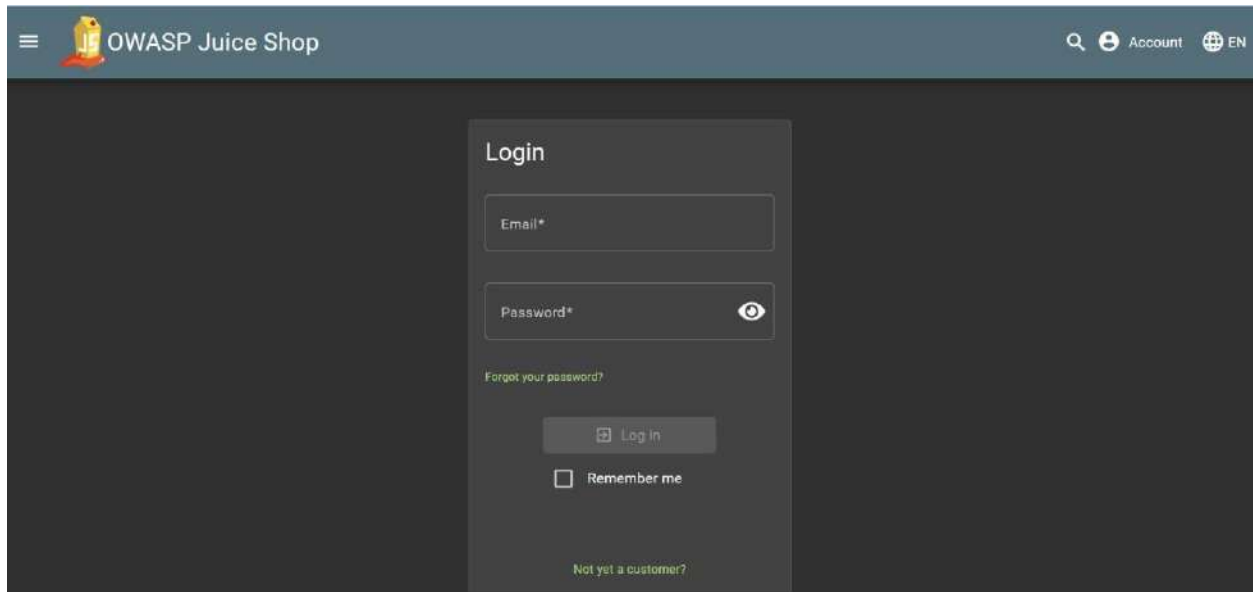


Figure 6: IoT device login interface

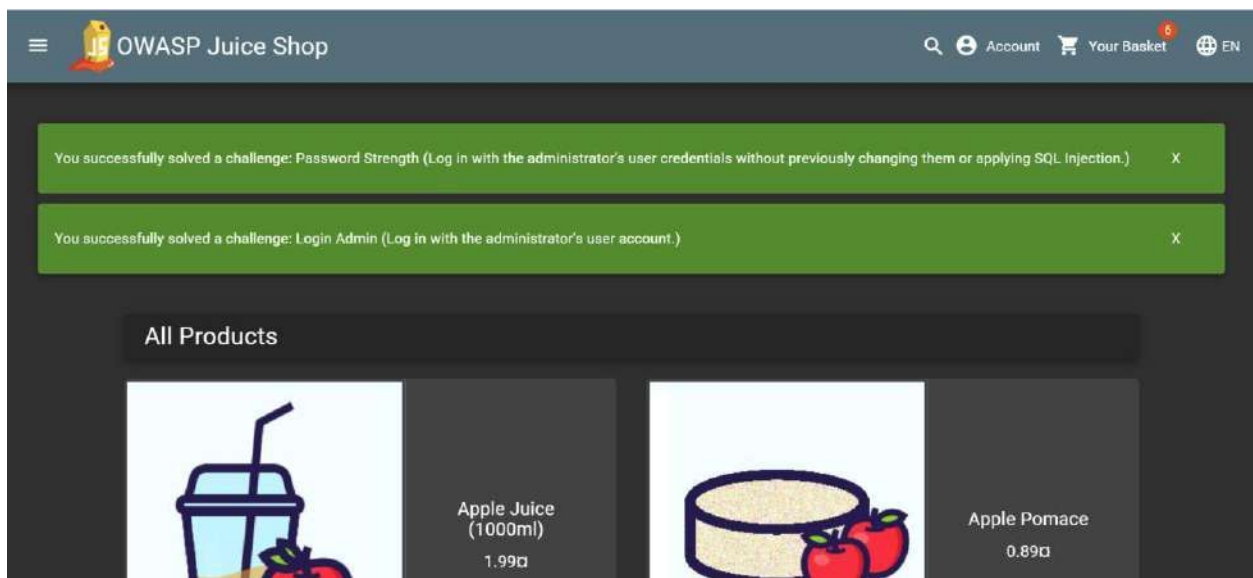


Figure 7: Successful login using default/weak credentials

Successful login using administrator credentials demonstrates the risk of default or weak passwords in IoT devices. If default credentials are not changed after deployment, attackers can gain unauthorized administrative access.

Step 7: Developer Tools → Network Tab

- Network tab showing HTTP requests
- Plain text traffic visible

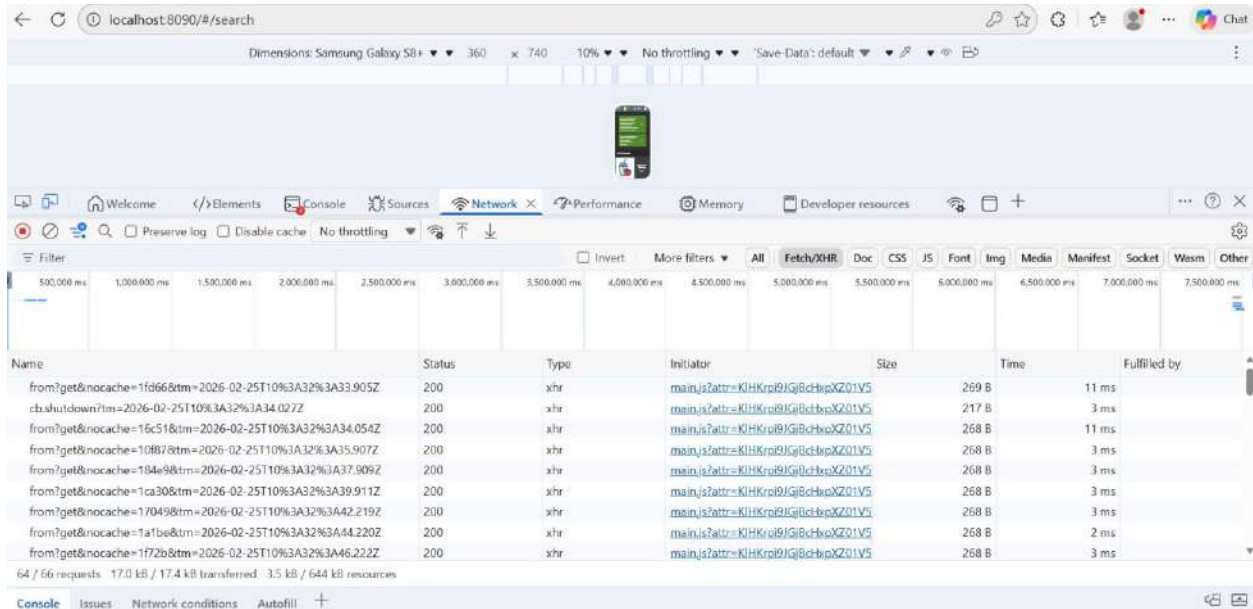


Figure 8: Unencrypted HTTP communication analysis

Observations

The scan revealed:

- Port 8090 open (HTTP service running).
- Service fingerprint identified as Juice Shop.
- Windows system ports (135, 139, 445) visible.
- Service version information exposed.
- MAC address detected.

- Web communication running over HTTP (not HTTPS). Using Browser Developer Tools:
- Network requests visible in plain text.
- No SSL/TLS encryption detected.

Vulnerabilities Identified

1. Open Port Exposure

- Port 8090 accessible over network.
- Increases attack surface.

2. Service Enumeration

- Service version and fingerprint visible.
- Enables targeted exploit attempts.

3. Unencrypted Communication

- Application runs over HTTP.
- Data transmitted in plain text.

4. Weak Authentication Risk

- Login interface exposed.
- No strict account lockout mechanism.

5. Information Disclosure

- Banner reveals application identity.
- Server headers visible.

Conclusion

The experiment demonstrated how an attacker on the same network can identify open ports, detect running services, and test default credentials on an IoT device. The presence of exposed services and unencrypted communication highlights common IoT security risks. Proper configuration, strong authentication, and secure communication protocols are essential to protect IoT devices from exploitation.

Experiment 17: Creating and Analyzing Disk Images Using dc3dd and Autopsy (Alternative to FTK Imager)

1. Aim

To create a forensic disk image and analyze digital evidence using **dc3dd** and **Autopsy** in **Kali Linux**.

2. Objectives

After completing this experiment, students will be able to:

- Understand the concept of **digital forensics and disk imaging**
- Create a **bit-by-bit forensic copy of a storage device**
- Maintain **data integrity using hashing**
- Analyze disk images to identify **files, deleted files, and system artifacts**

3. Introduction / Theory Digital Forensics

Digital forensics is the process of **identifying, collecting, preserving, analyzing, and presenting digital evidence** from computers or storage devices.

Digital evidence may include:

- Documents
- Emails
- Browser history
- Logs
- Deleted files

To preserve evidence, investigators create a **forensic disk image**.

Disk Imaging

Disk imaging is the process of creating an **exact replica of a storage device**. Characteristics of forensic disk images:

- Bit-by-bit copy of storage
- Preserves deleted files

- Maintains data integrity
- Allows investigation without altering original evidence

Hashing in Forensics

Hashing is used to verify the **integrity of digital evidence**. Common hash algorithms:

Algorithm	Purpose
MD5	evidence integrity
SHA1	verification
SHA256	stronger security

If the hash value of the original disk and the disk image match, the **evidence is considered intact**.

4. Tools Used

Tool	Purpose
Kali Linux	forensic analysis environment
dc3dd	disk image acquisition
Autopsy	forensic investigation and analysis

5. System Requirements

Component	Specification
Processor	Intel i3 or higher
RAM	Minimum 4 GB
Disk	20 GB free space
Software	Kali Linux VM

6. Algorithm

1. Start the Kali Linux environment.
2. Generate activity on the system to create forensic artifacts.
3. Identify the disk partition using system commands.
4. Create a forensic disk image using dc3dd.
5. Generate hash values for integrity verification.
6. Start the Autopsy forensic analysis tool.
7. Import the disk image into Autopsy.
8. Analyze files, logs, and deleted artifacts.
9. Extract relevant digital evidence.

7. Procedure

Step 1: Open Terminal in Kali Linux

Press:

Ctrl + Alt + T

This opens the terminal.

Step 2: Create Sample Evidence

Generate a file that will later be discovered during forensic analysis.

```
echo "Cybersecurity Lab Evidence" > evidence.txt
```

Verify the file creation.

```
Ls
```

Step 3: Identify Disk Partition

List storage devices.

```
lsblk
```

Example output:

```
sda
```

```
└--sda1
```

```
└--sda2
```

```

kali@kali:~$ zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~$ echo "Cybersecurity Lab Evidence" > evidence.txt
kali@kali:~$ ls
capture.pcap  Downloads  Pictures  Videos
Desktop      evidence.txt  Public
Documents    Music      Templates
kali@kali:~$ cat evidence.txt
Cybersecurity Lab Evidence
kali@kali:~$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 80.1G 0 disk
└─sda1 8:1    0 80.1G 0 part /
sr0   11:0    1 1024M 0 rom

```

```
(kali@kali)-[~]
└─$ sudo apt install dc3dd
Installing:
  dc3dd

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not U
pgrading: 1757
  Download size: 117 kB
  Space needed: 489 kB / 60.2 GB available

Get:1 http://kali.download/kali kali-rolling/main
amd64 dc3dd amd64 7.3.1-4 [117 kB]
Fetched 117 kB in 5s (21.9 kB/s)
Selecting previously unselected package dc3dd.
(Reading database ... 454289 files and directorie
s currently installed.)
Preparing to unpack .../dc3dd_7.3.1-4_amd64.deb .
..
Unpacking dc3dd (7.3.1-4) ...
Setting up dc3dd (7.3.1-4) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...

(kali@kali)-[~]
└─$ dc3dd --help

usage:
_____

    dc3dd [OPTION 1] [OPTION 2] ... [OPTION N
]

    *or*

    dc3dd [HELP OPTION]

    where each OPTION is selected from the ba
sic or advanced
    options listed below, or HELP OPTION is s
elected from the
    help options listed below.
```

Step 4: Create Disk Image

Step 1: Create a Small Disk File

```
dd if=/dev/zero of=practice_disk.dd bs=1M count=100
```

```
(kali@kali)-[~]
└─$ dd if=/dev/zero of=practice_disk.dd bs=1M cou
nt=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.28316
5 s, 370 MB/s
```

Explanation:

Parameter	Description
if	input disk
of	output image file
hash	generate MD5 hash
log	acquisition log file

Creates a **100 MB disk**.

Step 2: Mount the Disk

mkfs.ext4 practice_disk.dd

```
(kali㉿kali)-[~]  
$ mkfs.ext4 practice_disk.dd  
mke2fs 1.47.2 (1-Jan-2025)  
Discarding device blocks: done  
  
Creating filesystem with 102400 1k blocks and 255  
84 inodes  
Filesystem UUID: 514f4c47-066d-4a31-8af3-f13b33b7  
5b90  
Superblock backups stored on blocks:  
8193, 24577, 40961, 57345, 73729  
  
Allocating group tables: done  
  
Writing inode tables: done  
  
Creating journal (4096 blocks): done  
Writing superblocks and filesystem accounting inf  
ormation: done
```

Step 5: Verify Image Creation

Check the created image.

ls -lh /home/kali/disk_image.dd

```
(kali㉿kali)-[~]  
$ ls -lh /home/kali/disk_image.dd  
-rw-r--r-- 1 root root 9.8G Mar 15 11:59 /home/ka  
li/disk_image.dd
```

Check log file.

```
cat /home/kali/acquisition.log
```

The log will contain:

- image size
- hash value
- acquisition details

```

(kali㉿kali)-[~]
$ cat /home/kali/acquisition.log

dc3dd 7.2.646 started at 2026-03-15 11:52:30 -0400
0
compiled options:
command line dc3dd if=/dev/sda1 of=/home/kali/disk_image.dd hash=md5 log=/home/kali/acquisition.log
device size: 167966702 sectors (probed), 85,998,951,424 bytes
sector size: 512 bytes (probed)
10503356416 bytes ( 9.8 G ) copied ( 12% ), 432.546 s, 23 M/s

input results for device `/dev/sda1':
20514368 sectors in
0 bad sectors replaced by zeros
ce827a581c4dfce5b976603df85bb5e0 (md5)

output results for file `/home/kali/disk_image.dd':
20514368 sectors out

dc3dd aborted at 2026-03-15 11:59:42 -0400

```

Step 6: Start Autopsy

Launch the forensic analysis tool.

Autopsy

```

(kali㉿kali)-[~]
$ autopsy

=====
Autopsy Forensic Browser
http://www.sleuthkit.org/autopsy/
ver 2.24
=====

Evidence Locker: /var/lib/autopsy
Start Time: Sun Mar 15 12:01:18 2026
Remote Host: localhost
Local Port: 9999

Open an HTML browser on the remote host and paste
this URL in it:

http://localhost:9999/autopsy

Keep this process running and use <ctrl-c> to exit

```

```
(kali㉿kali)-[~]
$ sudo autopsy
[sudo] password for kali:

Autopsy Forensic Browser
http://www.sleuthkit.org/autops
y/
ver 2.24

Evidence Locker: /var/lib/autopsy
Start Time: Sun Mar 15 12:16:57 2026
Remote Host: localhost
Local Port: 9999

Open an HTML browser on the remote host and paste
this URL in it:

http://localhost:9999/autopsy

Keep this process running and use <ctrl-c> to exit
```

The terminal will display a URL: <http://localhost:9999/autopsy> Open this link in the browser.



Step 7: Create a New Case

In the Autopsy interface:

1. Click **Create New Case**
2. Enter case details:

The screenshot shows a web browser window with the title 'Create A New Case'. The address bar shows the URL 'http://127.0.0.1:9999/autopsy?mod=0&view=2'. The browser's tab bar includes 'OffSec', 'Kali Linux', 'Kali Tools', 'Kali Docs', 'Kali Forums', 'Kali NetHunter', 'Exploit-DB', and 'Google Hacking DB'. The main content area has a light green background and a yellow form titled 'CREATE A NEW CASE'. The form contains three sections: 1. 'Case Name' with a text input field containing 'CyberForensicsLab'; 2. 'Description' with a text input field containing 'Creating a new forensic investi'; 3. 'Investigator Names' with two columns of five text input fields each. The first field in the left column contains 'Lavanya'. At the bottom of the form are three buttons: 'NEW CASE', 'CANCEL', and 'HELP'.

Step 8: Add Disk Image

Select:

Add Host → Add Image

Browse and select:

/home/kali/disk_image.dd

Autopsy will begin analyzing the disk image.

The screenshot shows a web browser window with the title 'Creating Case: cyberForensicsLab'. The address bar shows the URL 'http://127.0.0.1:9999/autopsy?mod=0&view=2&case=CyberForensicsLab&desc=+Creating+a+new+fo'. The browser's tab bar includes 'OffSec', 'Kali Linux', 'Kali Tools', 'Kali Docs', 'Kali Forums', 'Kali NetHunter', 'Exploit-DB', and 'Google Hacking DB'. The main content area has a light green background. It displays the following text: 'Case directory (/var/lib/autopsy/CyberForensicsLab/) created', 'Configuration file (/var/lib/autopsy/CyberForensicsLab/case.aut) created', and 'We must now create a host for this case.'. Below this is a prompt 'Please select your name from the list:' followed by a dropdown menu showing 'lavanya'. At the bottom is a yellow button labeled 'ADD HOST'.

ADD A NEW HOST

1. **Host Name:** The name of the computer being investigated. It can contain only letters, numbers, and symbols.

KaliLabMachine

2. **Description:** An optional one-line description or note about this computer.

forensic disk image investigatio

3. **Time zone:** An optional timezone value (i.e. EST5EDT). If not given, it defaults to the local setting. A list of time zones can be found in the help files.

4. **Timeskew Adjustment:** An optional value to describe how many seconds this computer's clock was out of sync. For example, if the computer was 10 seconds fast, then enter -10 to compensate.

0

5. **Path of Alert Hash Database:** An optional hash database of known bad files.

6. **Path of Ignore Hash Database:** An optional hash database of known good files.

ADD HOST

CANCEL

HELP

Recommended Values

Host Name

KaliLabMachine

Description

Kali Linux virtual machine used for forensic disk image investigation

Time Zone

Leave Blank

Time Skew Adjustment

Leave Blank

Alert Hash Database

Leave Blank

Ignore Hash Database

Leave Blank

Then click:

Next

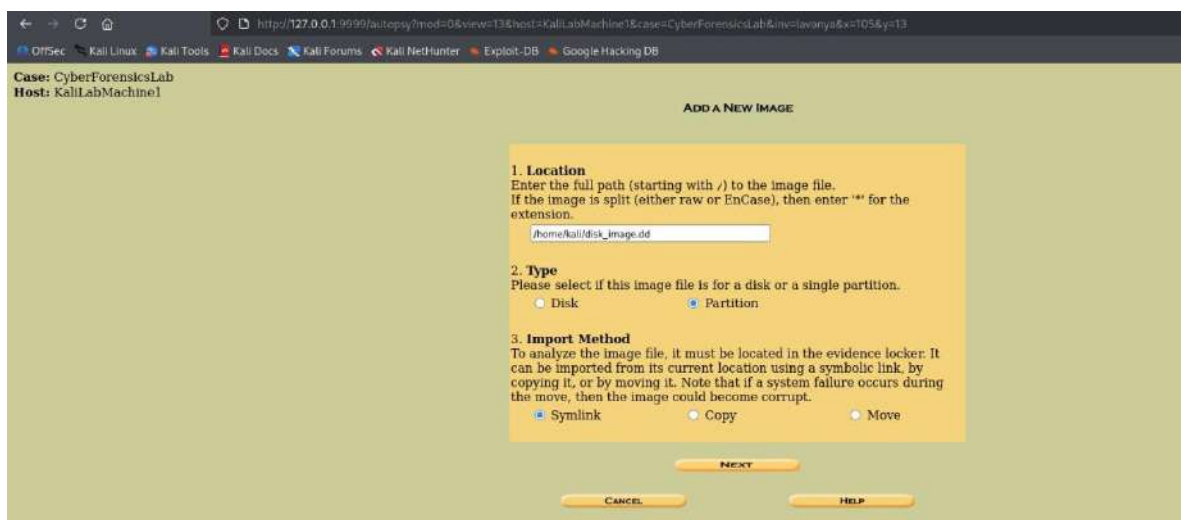
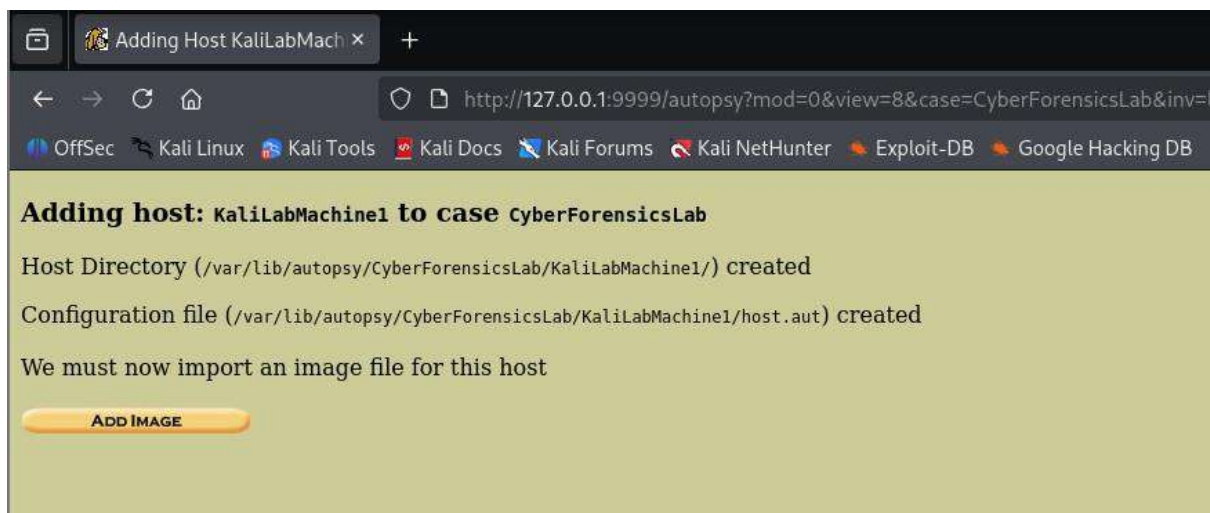


Image File Details

Local Name: images/disk_image.dd

Data Integrity: An MD5 hash can be used to verify the integrity of the image. (With split images, this hash is for the full image file)

☐ Ignore the hash value for this image.
☒ Calculate the hash value for this image.
☐ Add the following MD5 hash value for this image:

☒ Verify hash after importing?

File System Details

Analysis of the image file shows the following partitions:

Partition 1 (Type: ext4)

Mount Point: File System Type:

Calculating MD5 (this could take a while)
 Current MD5: CE827A581C4DFCE589766930F85885E9
 Testing partitions
 Linking image(s) into evidence locker
 Image file added with ID img1
 Volume image (0 to 0 - ext - /1/) added with ID vol1

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Case: CyberForensicsLab
 Host: KaliLabMachine1

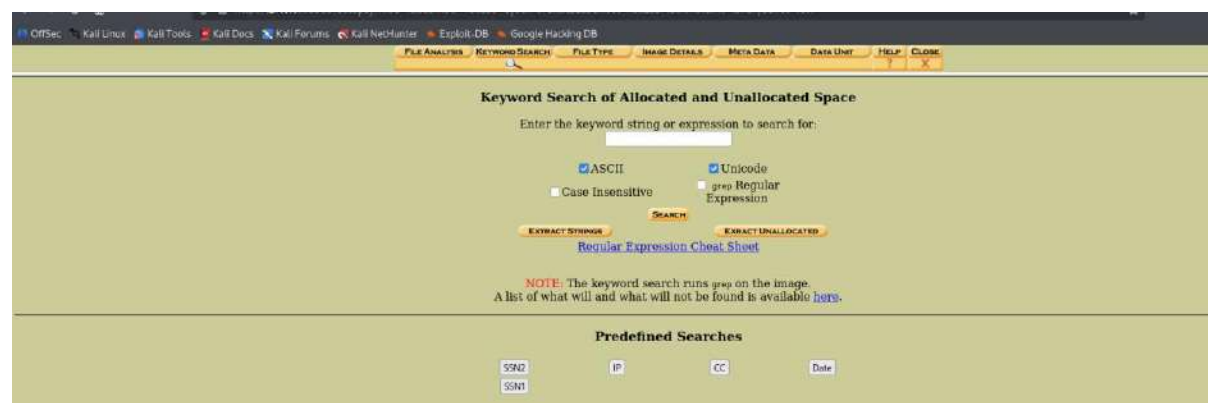
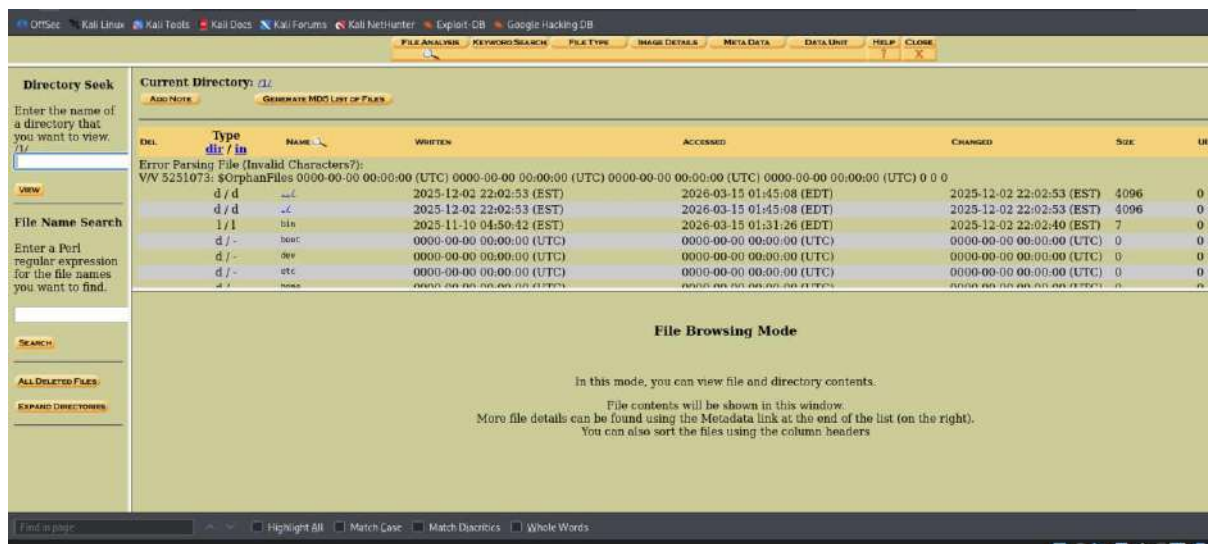
Select a volume to analyze or add a new image file.

CASE GALLERY		HOST GALLERY		HOST MANAGER	
mount	name		fs type		
/1/	disk_image.dd-0-0		ext		details

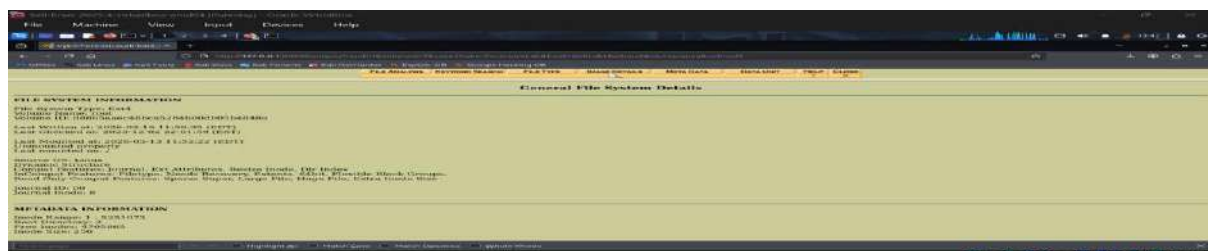
← → ↺ Ⓜ http://127.0.0.1:9999/autopsy?mod=0&view=17&host=KaliLabMachine1&case=CyberForensicsLab&inv=lavanya&vol=vol1&x=78&y=16 ☆

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

To start analyzing this volume, choose an analysis mode from the tabs above.



Step 9: Analyze Evidence



Experiment 18 : LOG FILE ANALYSIS FOR INCIDENT DETECTION LAB

Aim

To analyze system log files and detect suspicious activities such as failed login attempts and identify suspicious IP addresses.

Objective

- Understand system log files
- Detect suspicious activities and incidents
- Analyze logs using Linux tools
- Identify attack patterns (failed logins, brute force, etc.)

Requirements

- Kali Linux / Ubuntu VM
- Basic Linux command knowledge
- Tools:
- Kali Linux
- journalctl
- ssh
- grep, awk, sort
- grep
- awk
- less
- cat
- journalctl

Understanding Log Files Navigate to Log Directory `cd /var/log`

ls

The `/var/log` directory contains system and application logs such as Apache, Nginx, and login records used for security analysis.

```
kali@kali: /var/log/apache2
Session Actions Edit View Help
(kali@kali)-[~]
$ cd /var/log

(kali@kali)-[/var/log]
$ ls
alternatives.log  inetsim      openvpn      stunnel4
apache2          journal      postgresql   sysstat
apt              lastlog     private      wtmp
boot.log         lightdm      README       wtmp.db
btmp            macchanger.log  redis       Xorg.0.log
dpkg.log        mosquitto   runit
fontconfig.log  nginx       samba
gvm            notus-scanner speech-dispatcher

(kali@kali)-[/var/log]
$ sudo less /var/log/auth.log
[sudo] password for kali:
/var/log/auth.log: No such file or directory

(kali@kali)-[/var/log]
$ sudo lastb
[sudo] password for kali:
sudo: lastb: command not found

(kali@kali)-[/var/log]
$ last
kali      tty7      :0                Tue Mar 24 06:25 - still logged in
lightdm   tty7      :0                Tue Mar 24 06:24 - 06:25 (00:00)
```

What Your Output Shows From your ls:

- ✓ dpkg.log → package activity
- ✓ apache2 → web server logs
- ✓ nginx → web logs
- ✓ journal → system logs (IMPORTANT)
- ✓ wtmp, btmp → login records

Successful Logins (wtmp)

Command:

last

Capture:

- Login history

This displays successful login sessions. It helps identify legitimate and suspicious user access.

System Logs using journalctl Command:

`journalctl | less`

Capture:

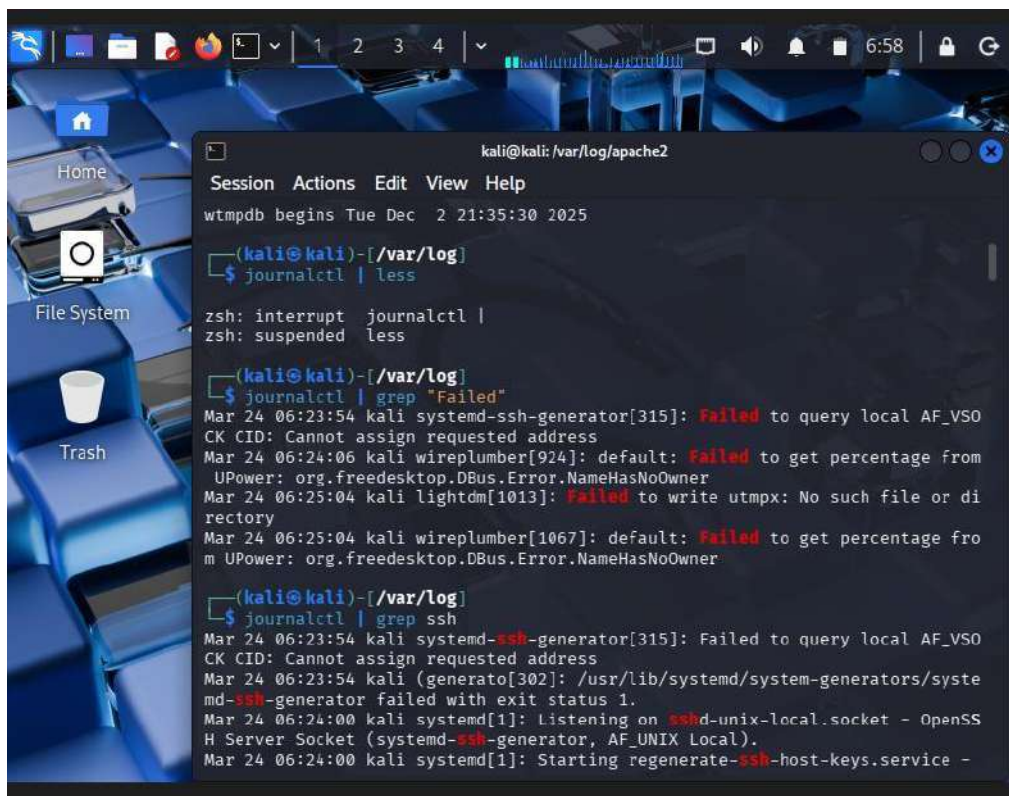
- Logs output

Failed SSH Attempts Command:

`journalctl | grep "Failed"`

SSH Login Activity Command:

`journalctl | grep ssh`

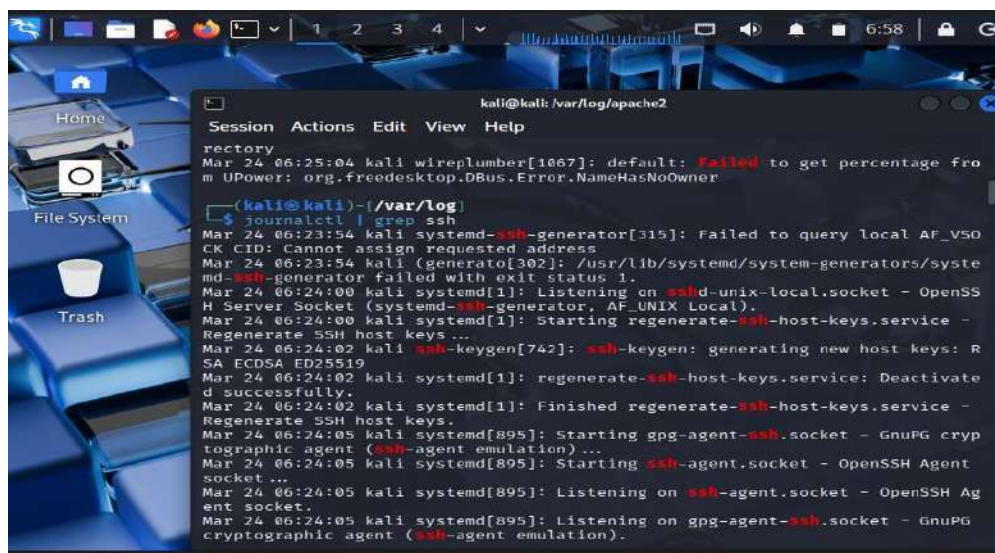
A screenshot of a Kali Linux desktop environment. The desktop background is a blue-tinted image of a computer keyboard. On the left side, there are icons for 'Home', 'File System', and 'Trash'. A terminal window is open in the center, titled 'kali@kali: /var/log/apache2'. The terminal shows the command 'journalctl | less' being executed, followed by 'zsh: interrupt journalctl |' and 'zsh: suspended less'. Then, the command 'journalctl | grep "Failed"' is executed, showing several log entries with 'Failed' in red. Finally, the command 'journalctl | grep ssh' is executed, showing log entries related to SSH, with 'ssh' in red. The terminal window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The desktop has a top bar with various icons and a clock showing '6:58'.

Explanation:

The `journalctl` command provides centralized system logs including authentication, services, and system events.

This screenshot shows failed authentication attempts, which may indicate unauthorized access attempts.

This screenshot also displays SSH-related logs, including login attempts and connections.

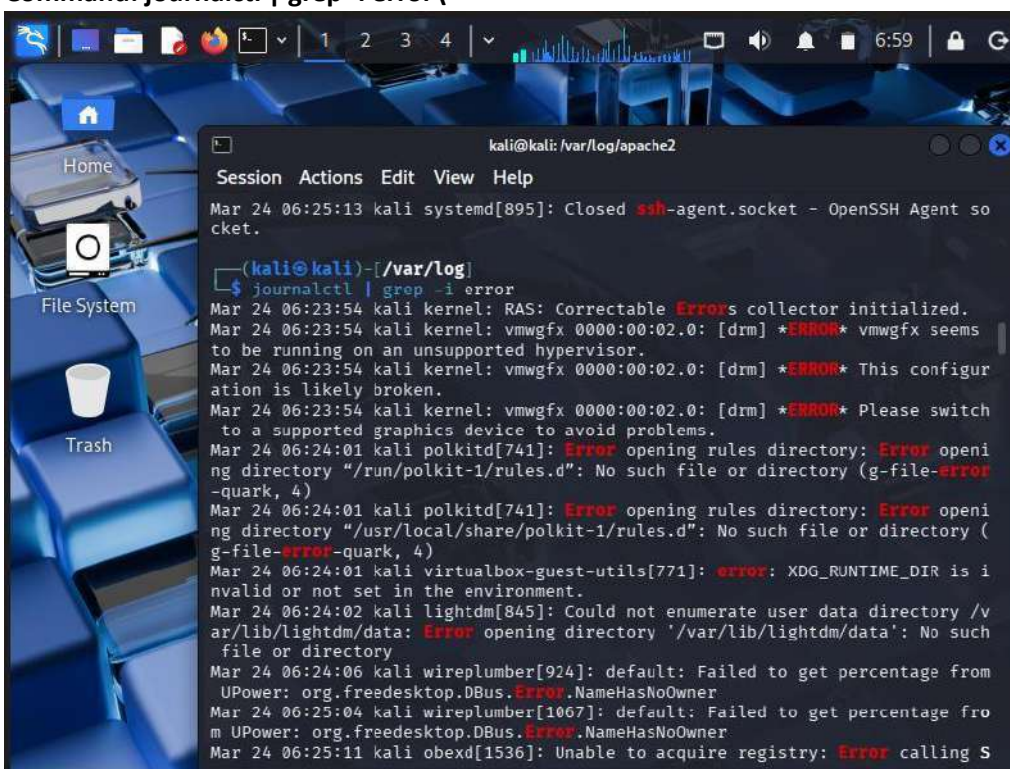


```
kali@kali: /var/log/apache2
Session Actions Edit View Help
rectory
Mar 24 06:25:04 kali wireplumber[1067]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner

(kali@kali)-[/var/log]
$ journalctl | grep ssh
Mar 24 06:23:54 kali systemd-ssh-generator[315]: Failed to query local AF_VSOCK CID: Cannot assign requested address
Mar 24 06:23:54 kali (generato[302]: /usr/lib/systemd/system-generators/systemd-ssh-generator failed with exit status 1.
Mar 24 06:24:00 kali systemd[1]: Listening on sshd-unix-local.socket - OpenSSH Server Socket (systemd-ssh-generator, AF_UNIX Local).
Mar 24 06:24:00 kali systemd[1]: Starting regenerate-ssh-host-keys.service - Regenerate SSH host keys...
Mar 24 06:24:02 kali ssh-keygen[742]: ssh-keygen: generating new host keys: RSA ECDSA ED25519
Mar 24 06:24:02 kali systemd[1]: regenerate-ssh-host-keys.service: Deactivated successfully.
Mar 24 06:24:02 kali systemd[1]: Finished regenerate-ssh-host-keys.service - Regenerate SSH host keys.
Mar 24 06:24:05 kali systemd[895]: Starting gpg-agent-ssh.socket - GnuPG cryptographic agent (ssh-agent emulation)...
Mar 24 06:24:05 kali systemd[895]: Starting ssh-agent.socket - OpenSSH Agent socket...
Mar 24 06:24:05 kali systemd[895]: Listening on ssh-agent.socket - OpenSSH Agent socket.
Mar 24 06:24:05 kali systemd[895]: Listening on gpg-agent-ssh.socket - GnuPG cryptographic agent (ssh-agent emulation).
```

Error Detection

Command: `journalctl | grep -i error`



```
kali@kali: /var/log/apache2
Session Actions Edit View Help
Mar 24 06:25:13 kali systemd[895]: Closed ssh-agent.socket - OpenSSH Agent socket.

(kali@kali)-[/var/log]
$ journalctl | grep -i error
Mar 24 06:23:54 kali kernel: RAS: Correctable Errors collector initialized.
Mar 24 06:23:54 kali kernel: vmwgfx 0000:00:02.0: [drm] *ERROR* vmwgfx seems to be running on an unsupported hypervisor.
Mar 24 06:23:54 kali kernel: vmwgfx 0000:00:02.0: [drm] *ERROR* This configuration is likely broken.
Mar 24 06:23:54 kali kernel: vmwgfx 0000:00:02.0: [drm] *ERROR* Please switch to a supported graphics device to avoid problems.
Mar 24 06:24:01 kali polkitd[741]: Error opening rules directory: Error opening directory "/run/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Mar 24 06:24:01 kali polkitd[741]: Error opening rules directory: Error opening directory "/usr/local/share/polkit-1/rules.d": No such file or directory (g-file-error-quark, 4)
Mar 24 06:24:01 kali virtualbox-guest-utils[771]: error: XDG_RUNTIME_DIR is invalid or not set in the environment.
Mar 24 06:24:02 kali lightdm[845]: Could not enumerate user data directory /var/lib/lightdm/data: Error opening directory '/var/lib/lightdm/data': No such file or directory
Mar 24 06:24:06 kali wireplumber[924]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Mar 24 06:25:04 kali wireplumber[1067]: default: Failed to get percentage from UPower: org.freedesktop.DBus.Error.NameHasNoOwner
Mar 24 06:25:11 kali obexd[1536]: Unable to acquire registry: Error calling S
```

The above screenshot highlights system errors, which may indicate abnormal behavior or system issues.

Apache Log Analysis Command:

`cd /var/log/apache2 ls`

Then:

sudo less access.log

Apache logs record web requests. These logs help identify suspicious web activity such as repeated requests or attacks.

Suspicious Requests Command:

grep "404" /var/log/apache2/access.log

This shows failed web requests (404 errors), which may indicate scanning or probing attacks.

Package Activity

Command:

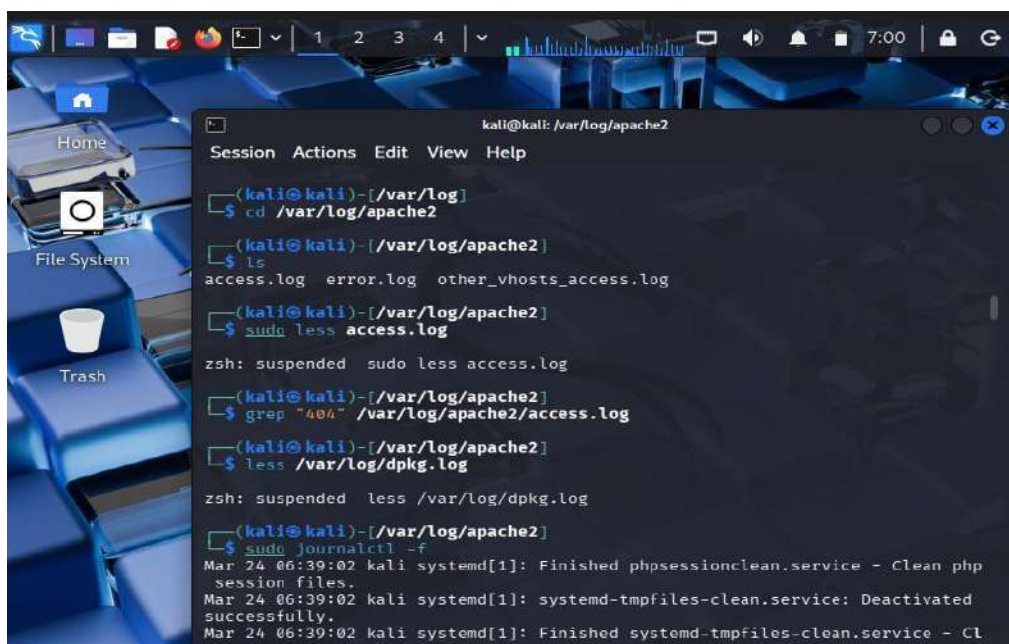
less /var/log/dpkg.log

This shows software installation and update logs, useful for detecting unauthorized changes.

Real-Time Monitoring Command:

sudo journalctl -f

This shows real-time system log monitoring, allowing detection of incidents as they occur.



```
kali@kali: /var/log/apache2
Session Actions Edit View Help
(kali@kali)-[/var/log]
$ cd /var/log/apache2
(kali@kali)-[/var/log/apache2]
$ ls
access.log  error.log  other_vhosts_access.log
(kali@kali)-[/var/log/apache2]
$ sudo less access.log
zsh: suspended  sudo less access.log
(kali@kali)-[/var/log/apache2]
$ grep "404" /var/log/apache2/access.log
(kali@kali)-[/var/log/apache2]
$ less /var/log/dpkg.log
zsh: suspended  less /var/log/dpkg.log
(kali@kali)-[/var/log/apache2]
$ sudo journalctl -f
Mar 24 06:39:02 kali systemd[1]: Finished phpsessionclean.service - Clean php
session files.
Mar 24 06:39:02 kali systemd[1]: systemd-tmpfiles-clean.service: Deactivated
successfully.
Mar 24 06:39:02 kali systemd[1]: Finished systemd-tmpfiles-clean.service - Cl
```

QUICK CHECKLIST

Check	Command	Suspicious If
Failed logins	lastb	Many attempts

SSH activity	`journalctl	grep ssh`
Login history	last	Odd timing
Errors	`journalctl	grep error`
Web logs	grep 404	Many failed requests

STEP-BY-STEP ANALYSIS OF YOUR LOG

1. System Services (NORMAL)

Finished phpsessionclean.service Finished systemd-tmpfiles-clean.service

✓ These are routine background tasks

✓ No issue

Regular system maintenance services were observed, indicating normal system operation.

2. CRON Jobs (NORMAL)

CRON: session opened for user root (root) CMD (debian-sa1)

✓ Automated scheduled task

✓ Runs periodically

Scheduled cron jobs were observed, which are normal automated system processes.

3. Your Own SUDO Activity (NORMAL)

kali : COMMAND=/usr/bin/less access.log kali : COMMAND=/usr/bin/journalctl -f

✓ This is YOU running commands

✓ Not suspicious

Sudo activity corresponds to legitimate administrative actions performed by the user.

4. XFCE / GUI Services (NORMAL)

xfconfd.service started

✓ Desktop environment service

✓ Normal

5. IMPORTANT PART

● Repeated Kernel Messages

dndHGCM: Received message HOST_DND_FN_GH_REQ_PENDING dnd: No guest source window

These lines are:

Repeated MANY times

Coming from VirtualBox (VBoxDRM / DnD)

Q Is This Suspicious?

Technically:

- NOT an attack
- But **abnormal repetitive behavior**

🔗 Why it matters:

- Repeated logs = unusual system activity
- Could indicate:
 - Misconfiguration
 - Resource issue
 - VirtualBox interaction problem

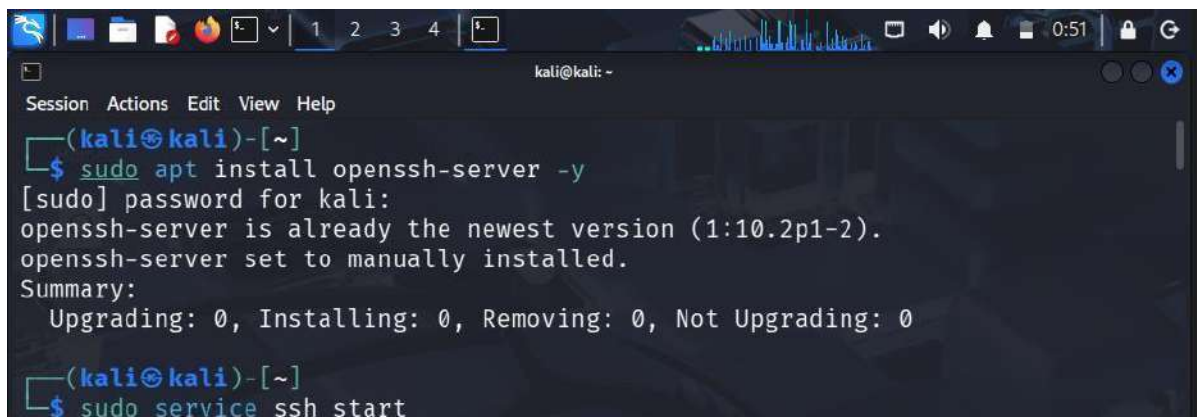
METHOD 1: SIMULATE FAILED SSH LOGIN

Step 1: Ensure SSH is Installed

sudo apt install openssh-server -y

Step 2: Start SSH Service

sudo service ssh start

A screenshot of a Kali Linux terminal window. The window has a title bar with 'kali@kali: ~' and standard window controls. The terminal shows the command 'sudo apt install openssh-server -y' being executed. The output indicates that openssh-server is already the newest version (1:10.2p1-2) and is set to manually installed. A summary shows 0 upgrades, 0 installations, 0 removals, and 0 not-upgrading actions. The prompt then changes to '(kali@kali)-[~]'. The next command 'sudo service ssh start' is entered at the bottom of the terminal.

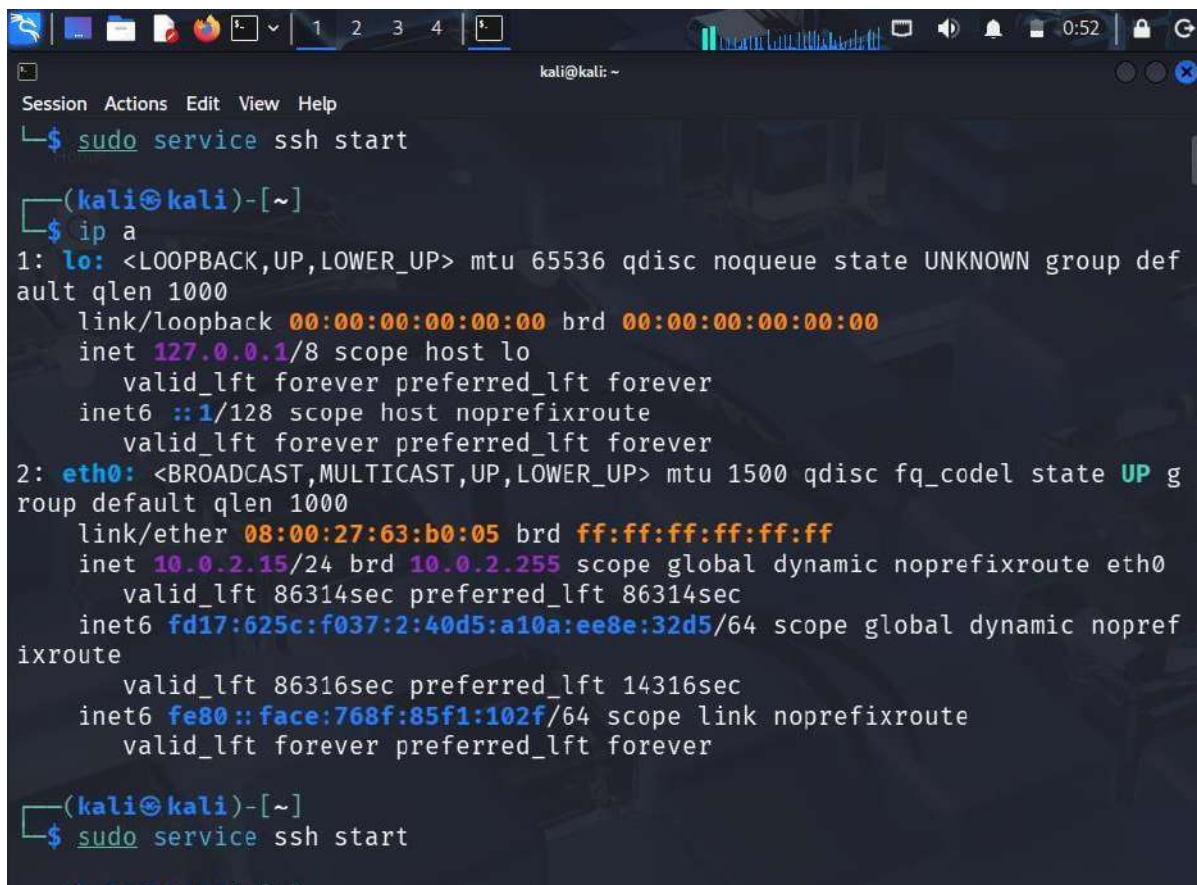
```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sudo apt install openssh-server -y  
[sudo] password for kali:  
openssh-server is already the newest version (1:10.2p1-2).  
openssh-server set to manually installed.  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0  
(kali@kali)-[~]  
$ sudo service ssh start
```

Step 3: Find Your IP Address

ip a

Look for something like:

inet 192.168.x.x

A terminal window titled 'kali@kali: ~' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows the command 'sudo service ssh start' being executed. Below it, the output of 'ip a' is displayed, showing details for the loopback interface 'lo' (127.0.0.1) and the ethernet interface 'eth0' (10.0.2.15). The terminal ends with the command 'sudo service ssh start' being entered again.

```
kali@kali: ~  
Session Actions Edit View Help  
$ sudo service ssh start  
  
(kali@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def  
ault qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g  
roup default qlen 1000  
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff  
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0  
        valid_lft 86314sec preferred_lft 86314sec  
    inet6 fd17:625c:f037:2:40d5:a10a:ee8e:32d5/64 scope global dynamic nopref  
ixroute  
        valid_lft 86316sec preferred_lft 14316sec  
    inet6 fe80::face:768f:85f1:102f/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
  
(kali@kali)-[~]  
$ sudo service ssh start
```

Step 4: Attempt Wrong Login (GENERATE FAILURES)

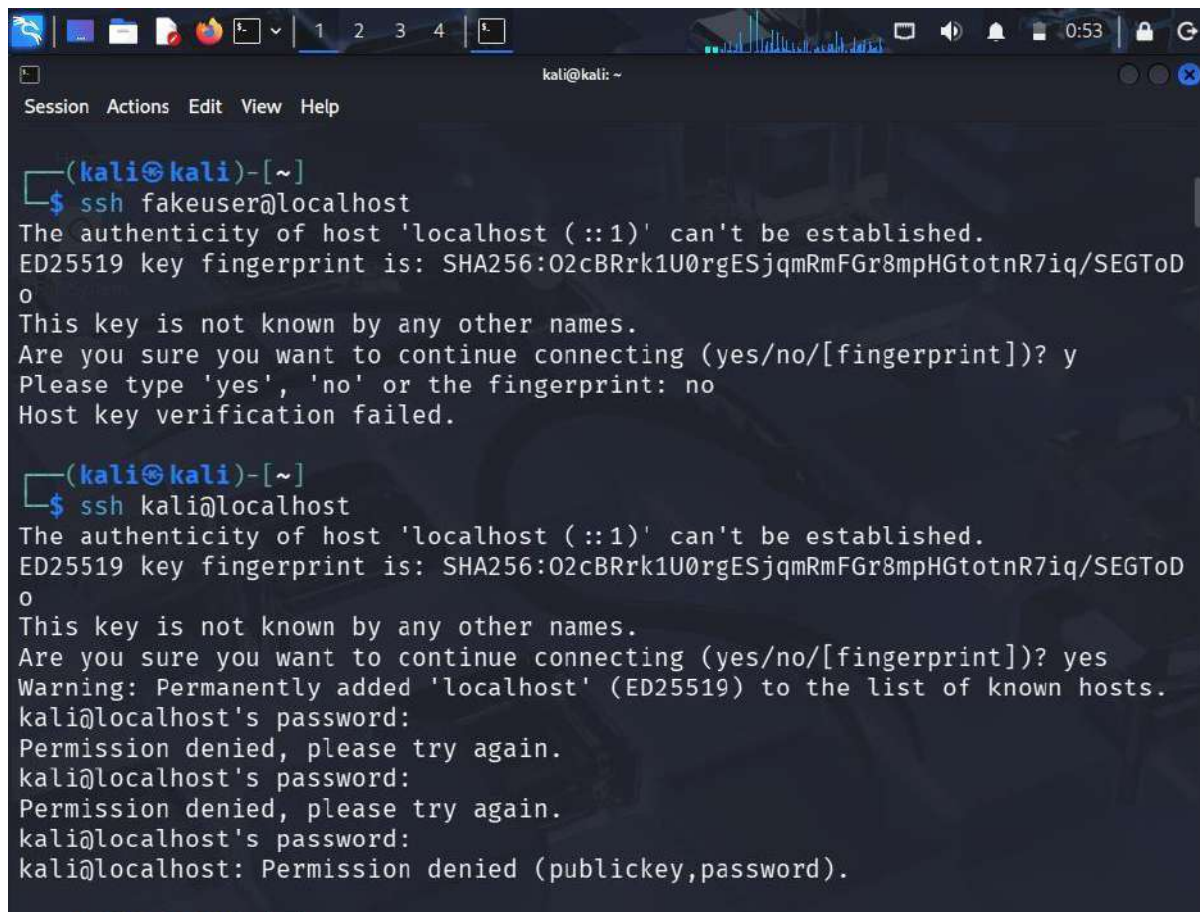
run:

ssh fakeuser@localhost

OR

ssh kali@localhost

Enter **wrong password multiple times (5–10 times)**



```
(kali㉿kali)-[~]
$ ssh fakeuser@localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is: SHA256:02cBRrk1U0rgESjqmRmFGr8mpHGtoR7iq/SEGToD
0
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: no
Host key verification failed.

(kali㉿kali)-[~]
$ ssh kali@localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is: SHA256:02cBRrk1U0rgESjqmRmFGr8mpHGtoR7iq/SEGToD
0
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
kali@localhost's password:
Permission denied, please try again.
kali@localhost's password:
Permission denied, please try again.
kali@localhost's password:
kali@localhost: Permission denied (publickey,password).
```

Now Analyze Logs

Step 5: Check Failed Logins

journalctl | grep "Failed password"

👉 You should now see:

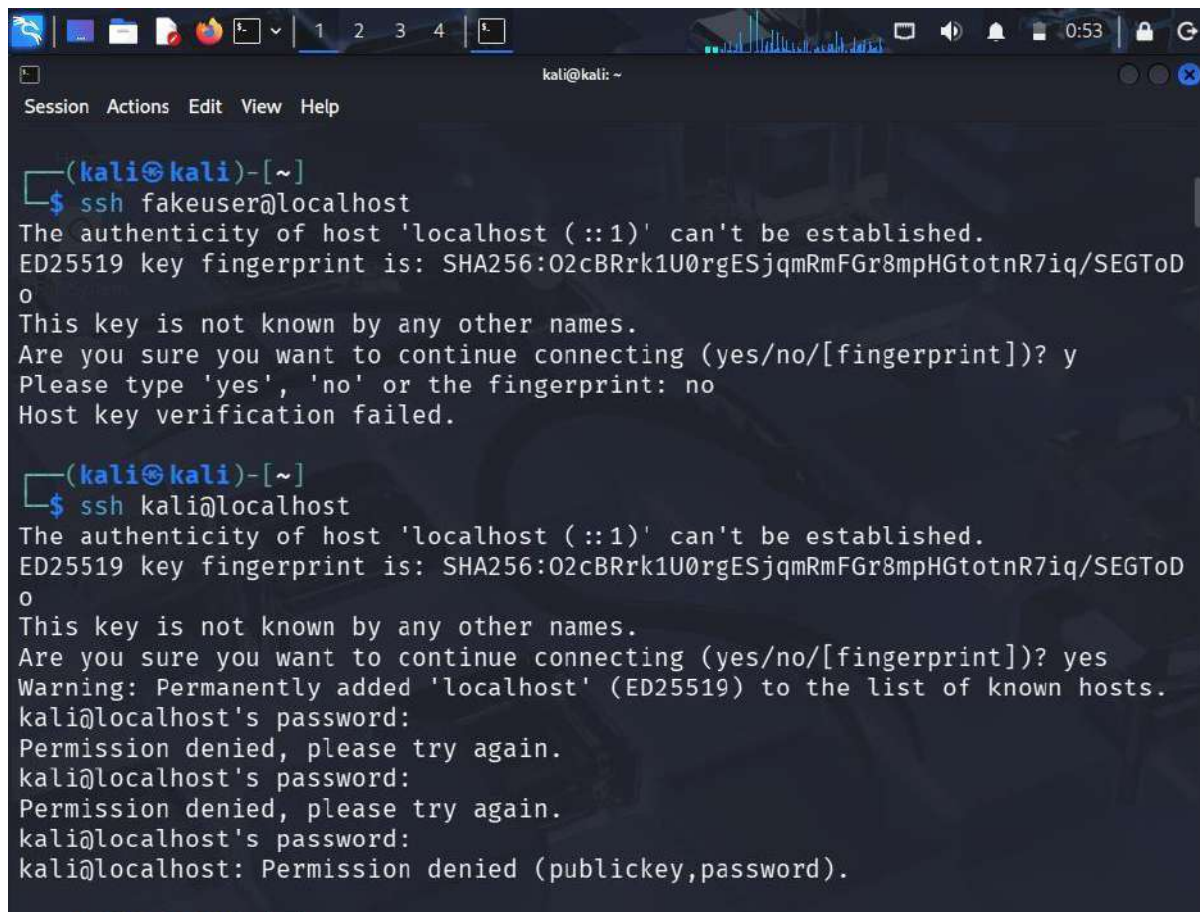
Failed password for kali from 127.0.0.1 port ...

Step 6: Extract Suspicious IP

journalctl | grep "Failed password" | awk '{print \$11}'

Output:

127.0.0.1



```
(kali㉿kali)-[~]
$ ssh fakeuser@localhost
The authenticity of host 'localhost (:::1)' can't be established.
ED25519 key fingerprint is: SHA256:02cBRrk1U0rgESjqmRmFGr8mpHGtoR7iq/SEGToD
0
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: no
Host key verification failed.

(kali㉿kali)-[~]
$ ssh kali@localhost
The authenticity of host 'localhost (:::1)' can't be established.
ED25519 key fingerprint is: SHA256:02cBRrk1U0rgESjqmRmFGr8mpHGtoR7iq/SEGToD
0
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
kali@localhost's password:
Permission denied, please try again.
kali@localhost's password:
Permission denied, please try again.
kali@localhost's password:
kali@localhost: Permission denied (publickey,password).
```

Step 7: Count Attempts Per IP

```
journalctl | grep "Failed password" | awk '{print $11}' | sort
| uniq -c | sort -nr
```

Example:

```
10 127.0.0.1
```

```
kali@kali: ~  
Session Actions Edit View Help  
└─$ journalctl | grep "Failed password"  
Mar 26 00:42:21 kali sshd-session[3399]: Failed password for kali from ::1 port 36238 ssh2  
Mar 26 00:42:29 kali sshd-session[3399]: Failed password for kali from ::1 port 36238 ssh2  
Mar 26 00:42:37 kali sshd-session[3399]: Failed password for kali from ::1 port 36238 ssh2  
  
└─(kali@kali)-[~]  
└─$ journalctl | grep "Failed password" | awk '{print $11}'  
::1  
::1  
::1  
  
└─(kali@kali)-[~]  
└─$ journalctl | grep "Failed password" | awk '{print $11}' | sort | uniq -c | sort -nr  
3 ::1
```

INTERPRETATION (VERY IMPORTANT)

Multiple failed login attempts were detected from IP address 127.0.0.1, indicating a simulated brute force attack.

METHOD 2: Use lastb (Alternative)

sudo lastb

Shows failed login records

HOW TO IDENTIFY "SUSPICIOUS IP"

Simple rule:

Condition	Meaning
Same IP repeated many times	🚩 Suspicious
Unknown IP	🚩 Suspicious
High frequency attempts	🚩 Brute force

Run

[illegible]

Network forensics is a branch of digital forensics that deals with monitoring, capturing, and analyzing network traffic to detect security breaches and malicious activities. Wireshark is a widely used open-source tool that helps in capturing real-time packets and analyzing them in detail.

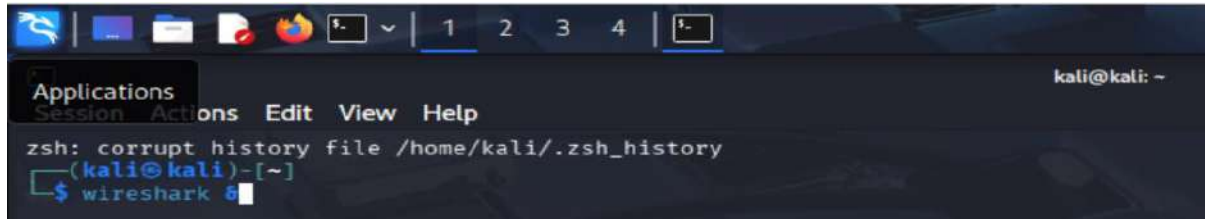
Tools Required:

1. Kali Linux
2. Wireshark

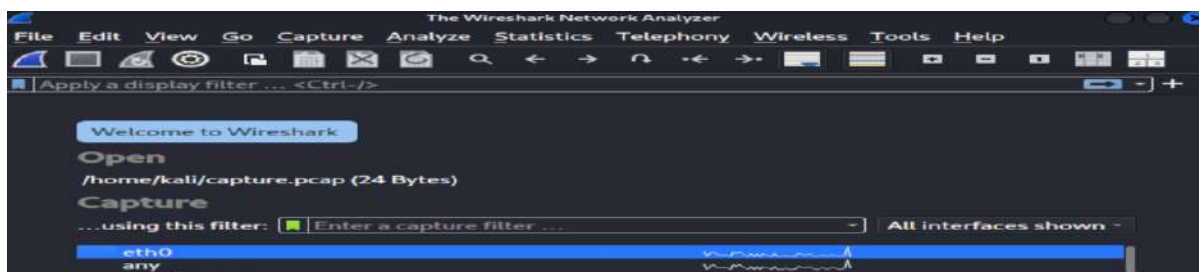
Procedure:

Step1:Open kali linux terminal

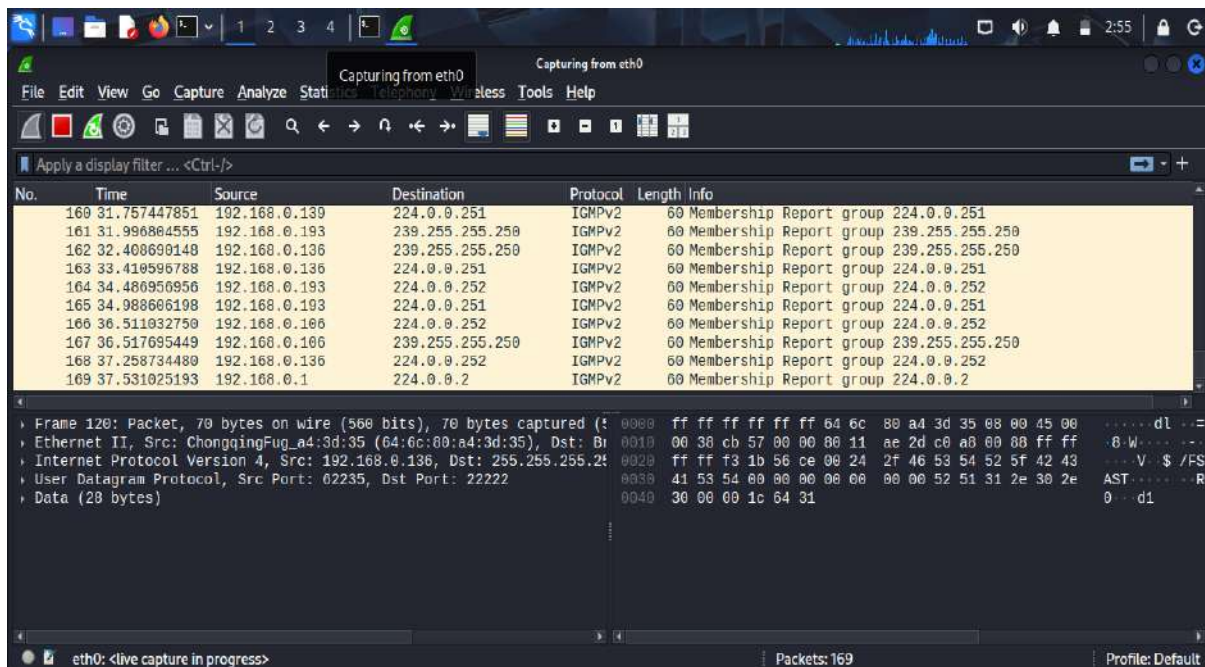
Type wireshark &



Step 2: Select eth0

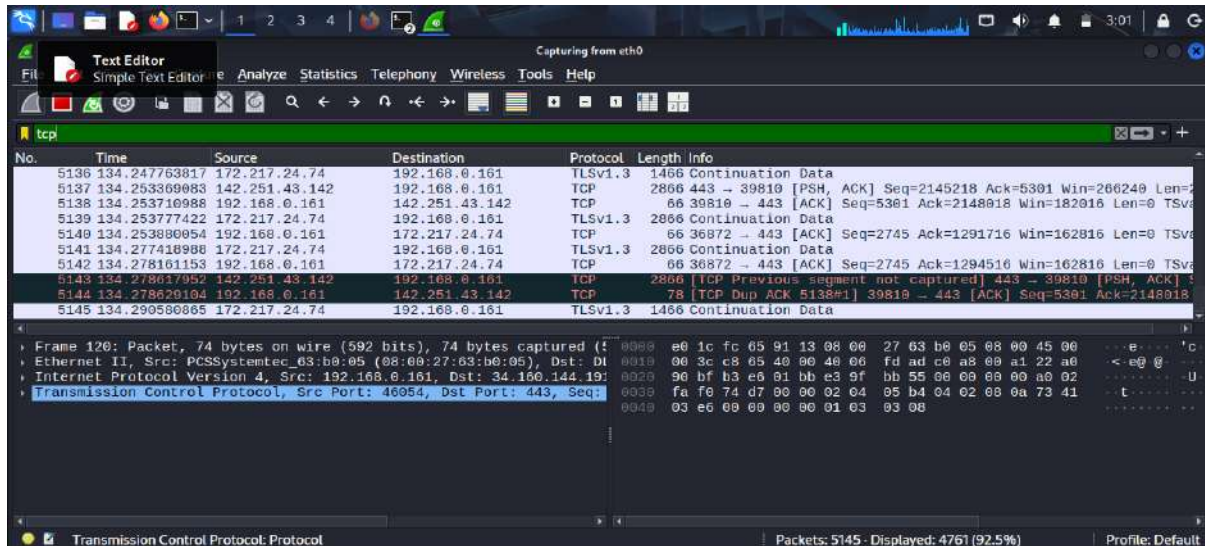


- Wireshark is launched to begin packet analysis. It provides a graphical interface to monitor all incoming and outgoing packets in a network. It supports multiple protocols and helps in detailed inspection.



Step 3: Protocol Analysis (TCP/UDP/HTTP): Wireshark supports deep inspection of protocols such as TCP, UDP, and HTTP. TCP ensures reliable communication, while UDP is faster but connectionless. HTTP packets help analyze web traffic and identify requests and responses.

- Apply filter as tcp...now you can able to see the only tcp packets. You can also try with UDP,HTTP

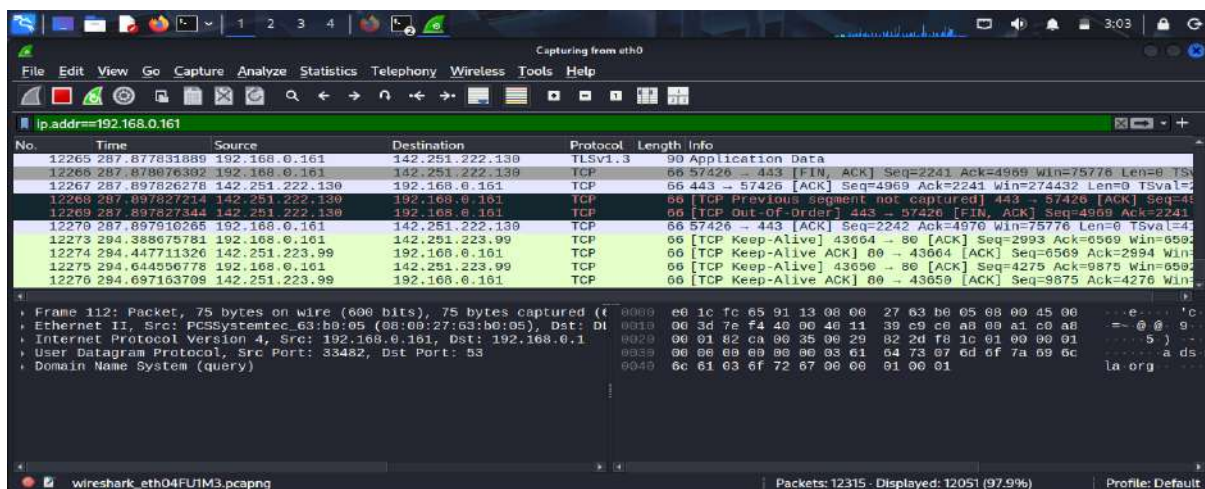


Step 4: Applying Filters

Filters are used to narrow down specific packets. For example:

- `ip.addr == 192.168.0.161` → filters packets of a specific IP

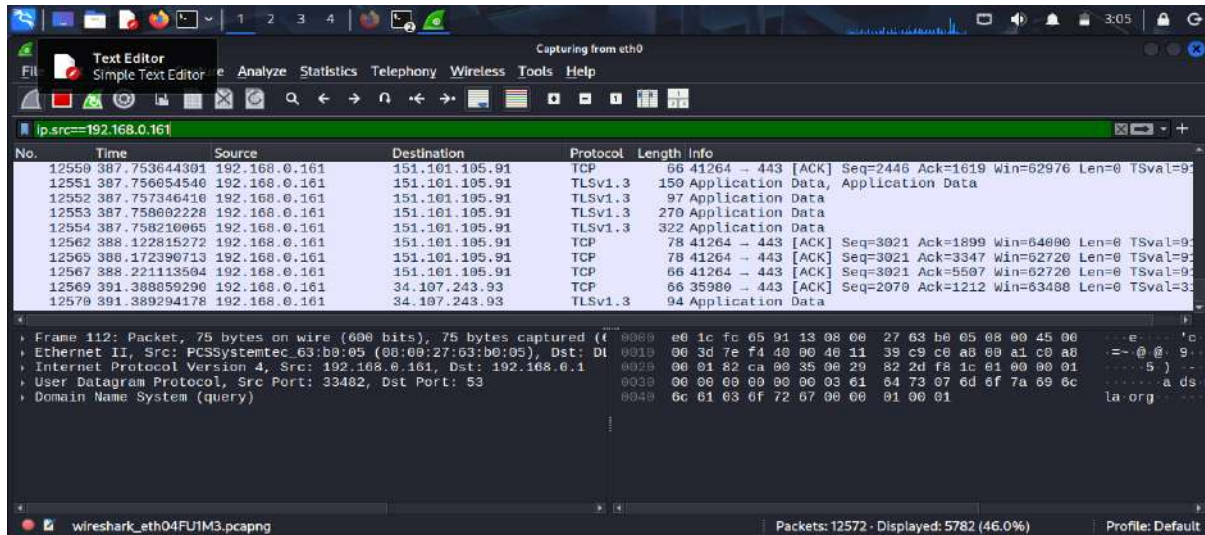
This simplifies analysis by focusing only on relevant data.



Step 5 :for particular Source address use

`Ip.src==<IP_Address>` like

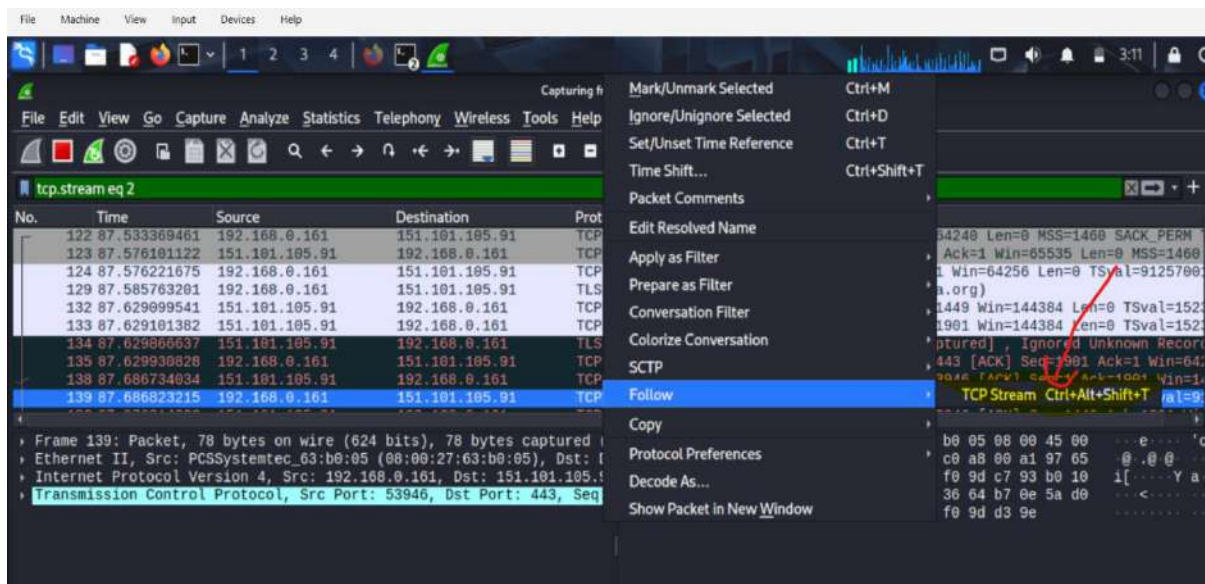
`Ip.src== 192.168.0.161`

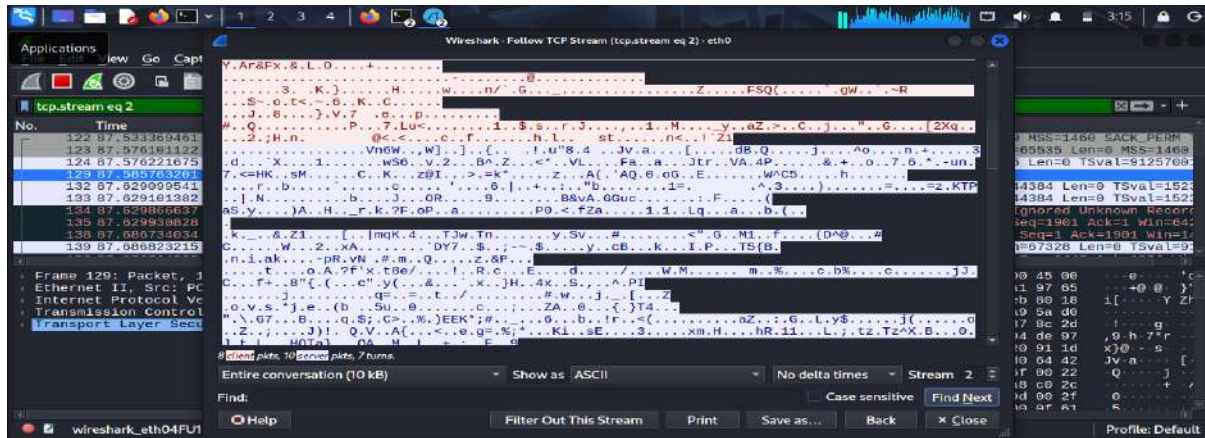


Step 6: Following TCP Stream

The "Follow TCP Stream" feature reconstructs communication between two endpoints. It helps in analyzing complete conversations, such as HTTP requests and responses, making it useful in forensic investigations.

- Right click on any TCP packet->Follow->TCP Stream





➤ Observation on above tcp stream:

Complete conversation between two devices over TCP

- **Red text** → Data sent from one device
- **Blue text** → Data sent from the other device

It reconstructs the **entire communication stream**

Mixed Content (Binary + Text)

- You see:
 - Random characters → **binary data**
 - Some readable text → **application data**

This means:

- It is **not plain HTTP**
- Likely:
 - Encrypted traffic (HTTPS)
 - Or application-level data

TCP Flags Visible

You can see:

[SYN], [ACK], [PSH], [FIN]

Meaning:

- **SYN** → Connection start
- **ACK** → Acknowledgement
- **PSH** → Data transfer
- **FIN** → Connection close

So this stream shows:

✓ Full TCP session lifecycle

Step 7: Identifying Suspicious Traffic

Open kali linux terminal

Type `nmap -sS <target_IP>`

Eg: `nmap -sS 192.168.0.156` (use your windows ip)

Open wireshark

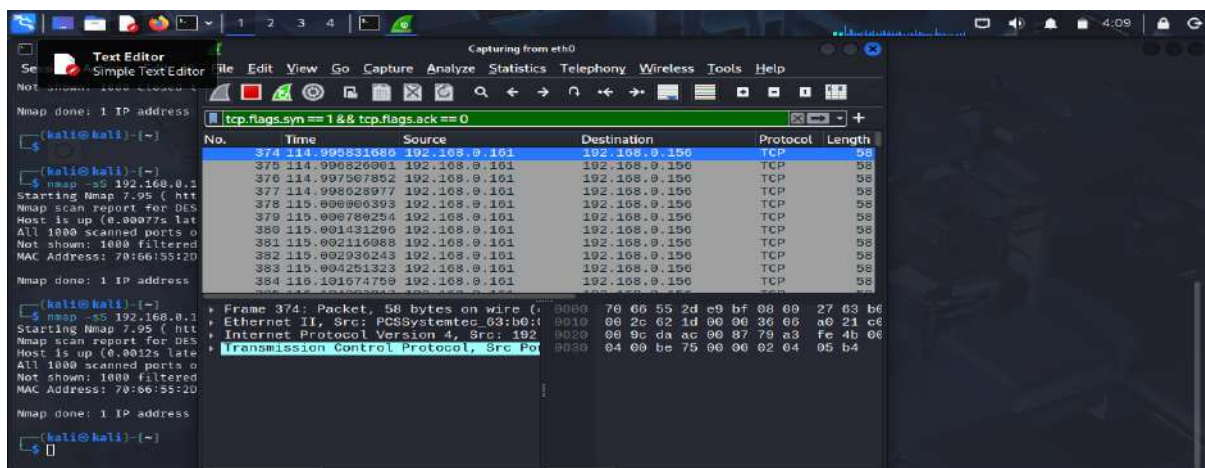
Use this filter:

`tcp.flags.syn == 1 && tcp.flags.ack == 0`

Now you will observe:

- Multiple SYN packets
- Different ports
- Incomplete connections
- Source address is Kali linux and Destination Address is Windows

This is **port scanning behavior (suspicious)**. Any way our system firewall stops this.



Step 8:

In wireshark

Go to Statistics->Capture File Properties

Details

File

Name:

/tmp/wireshark_eth06HJ1M3.pcapng

Length:

68 kB

Hash (SHA256):

38ceda9e5420d298d73f80a665aa44b6047ebb6f29adbfd95ae329655684abe3

Hash (SHA1):

058858daa62b363000ec4b029f9832e0d14cdd57

Format:

Wireshark/... - pcapng

Encapsulation:

Ethernet

Time

First packet:

2026-03-30 04:54:32

Last packet:

2026-03-30 04:57:06

Elapsed:

00:02:34

Capture

Hardware:

Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz (with SSE4.2)

OS:

Linux 6.16.8+kali-amd64

Application:

Dumpcap (Wireshark) 4.6.0

Interfaces

Interface	Dropped packets	Capture filter	Link type	Packet size limit (snaplen)
eth0	Unknown	none	Ethernet	262144 bytes

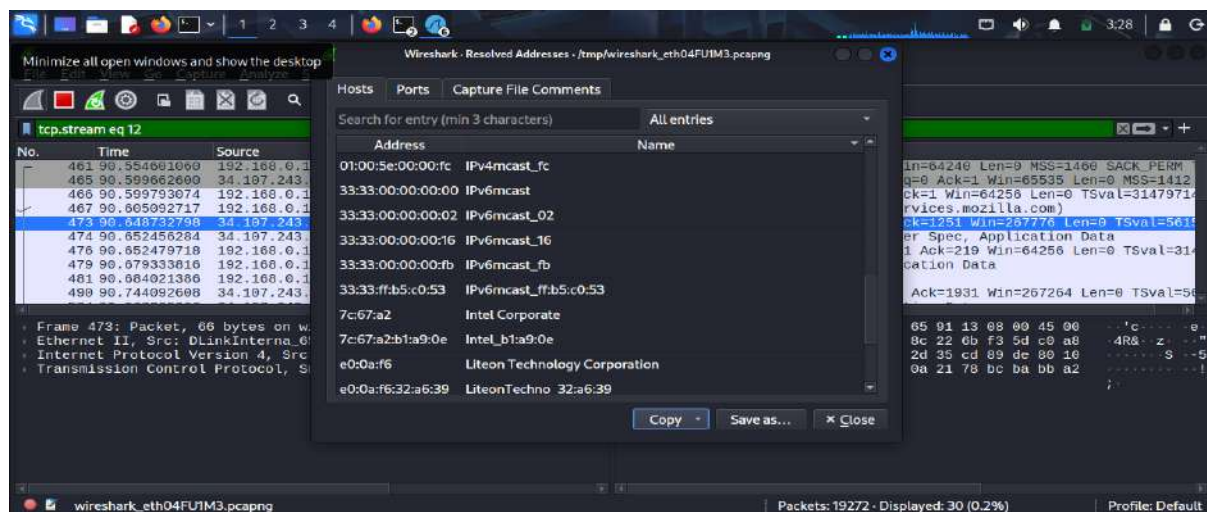
- It provides a quick overview of network traffic and helps detect suspicious activities without analyzing each packet individually.

Step 9:

In wireshark **Go To**

Statistics->Resolved Address

Resolve Address in Wireshark converts numerical network addresses (like IP or MAC) into **human-readable names** such as domain names or device names.



Step 10: Go to

Statistics-> Protocol Hierarchy Statistics

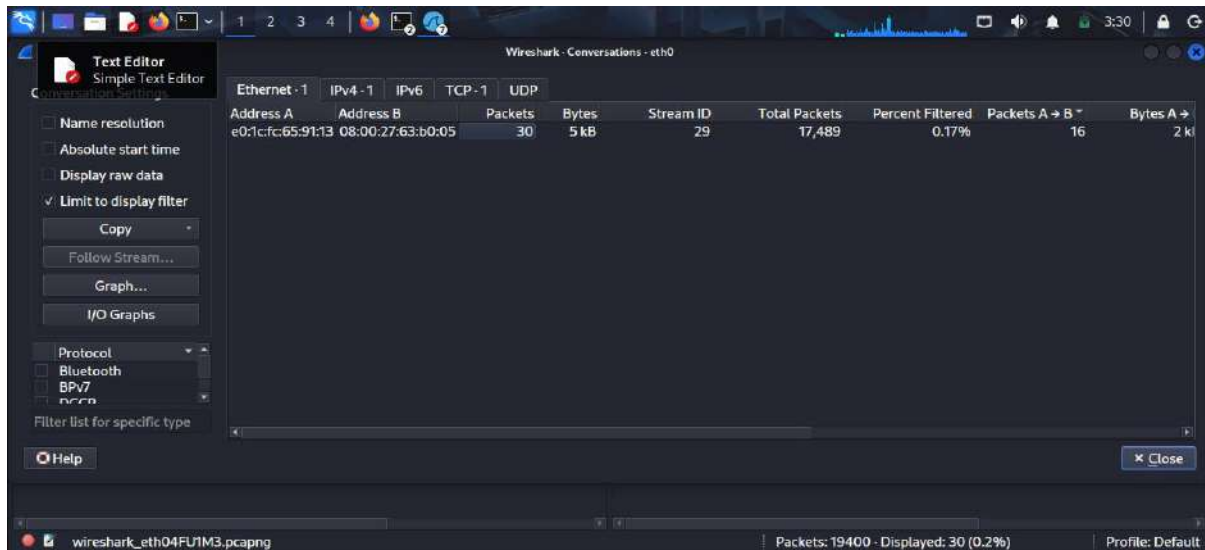
Protocol Hierarchy Statistics shows the breakdown of all network protocols in the captured traffic in a hierarchical (layer-wise) format along with their percentage and packet count.



Step 11: Go to

Statistics->Conversations

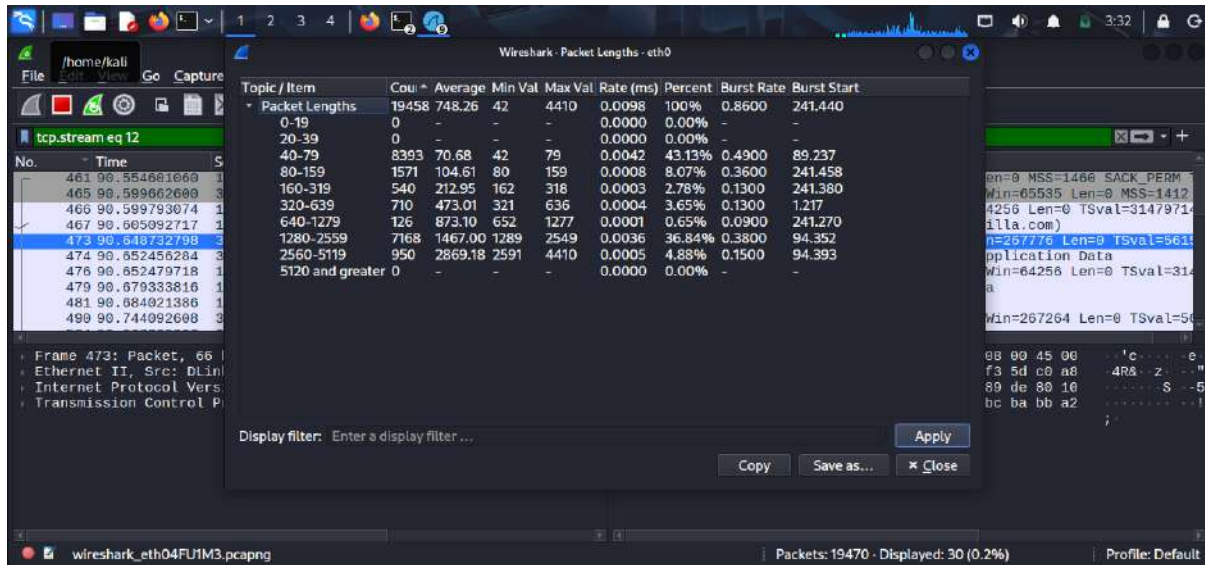
Conversations in Wireshark shows communication between two endpoints (devices), including source and destination addresses, number of packets, and data transferred.



Step 12:

Statistics-> Packet Length

Packet Length refers to the **size of a network packet** measured in bytes, as captured by Wireshark.



Step 13:

Statistics-> End Points

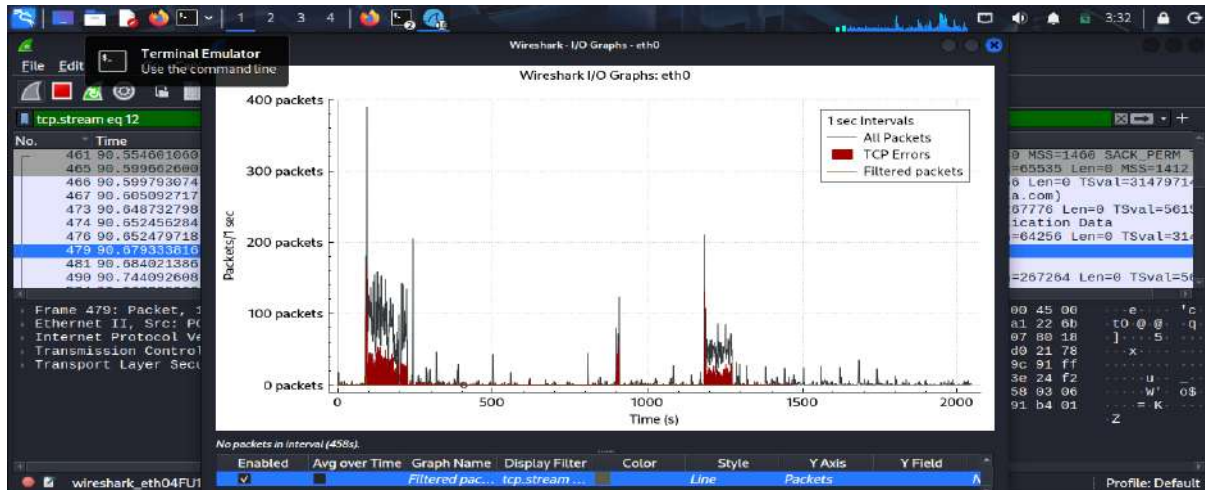
Endpoints in Wireshark represent individual network devices (hosts) involved in communication. It shows each device separately along with the number of packets sent/received and data usage.



Step 14:

Statistics-> I/O Graphs

I/O Graphs (Input/Output Graphs) in Wireshark are used to display network traffic over time in the form of a graph, showing the number of packets or bytes captured per second.



Result

Network packets were successfully captured and analyzed using Wireshark. Different protocols and communication patterns were observed and studied.

Conclusion

Wireshark is an effective tool for network forensics. It helps in capturing and analyzing network traffic, identifying suspicious activities, and understanding communication between systems. It plays a vital role in cybersecurity and incident investigation.

Viva Points

- What is packet sniffing?
- Difference between TCP and UDP
- What is a protocol analyzer?
- What is a filter in Wireshark?

EXPERIMENT 20:

Title: Privacy Audit of Popular Apps (Desktop **WhatsApp**) and Websites (**Facebook**) & Data Breach Case Study Analysis

Aim

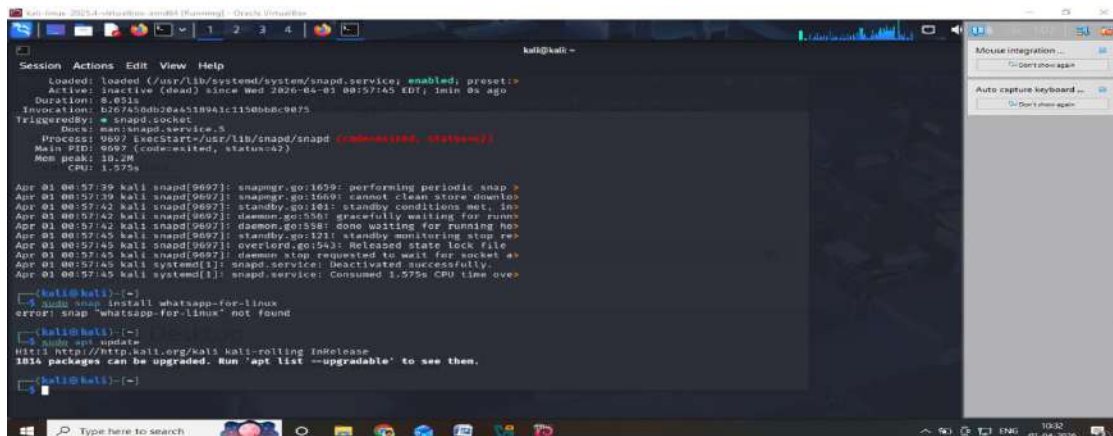
To analyze privacy, network behavior, and security risks of applications and websites using Kali Linux tools.

Tools Required

- Wireshark
- Burp Suite
- Kali Linux (VirtualBox)
- Desktop WhatsApp (Nativefier)
- Web Browser (Firefox)

Step 1: Install Node.js & npm

sudo apt update

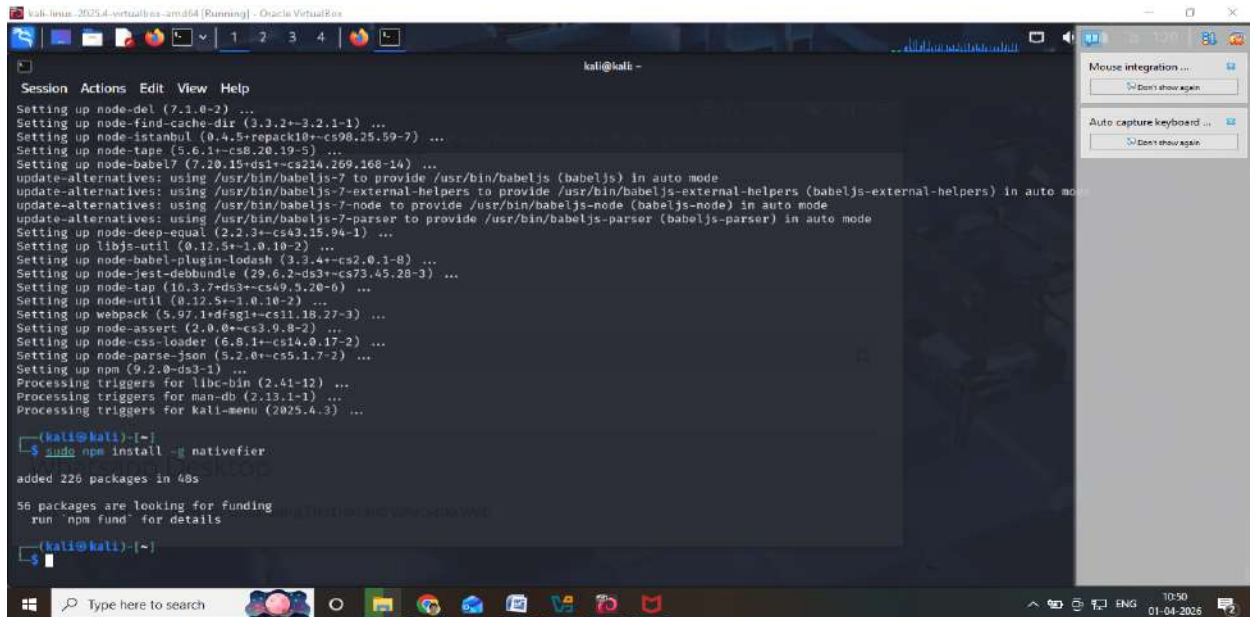


The screenshot shows a Kali Linux terminal window with a dark background. The terminal displays system logs for the snapd service, including messages about periodic snap updates, standby conditions, and daemon status. Below the logs, the user enters the command `sudo snap install whatsapp-for-linux`, which results in an error: `error: snap 'whatsapp-for-linux' not found`. The user then enters `sudo apt update`, which outputs the URL `http://http.kali.org/kali kali-rolling InRelease` and states that 1014 packages can be upgraded. The terminal window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. On the right side, there are two panels: 'Mouse Integration...' and 'Auto capture keyboard...'. The bottom of the window shows a taskbar with various application icons and a system tray with the date '01-04-2020' and time '10:32'.

```
kali@kali:~$ sudo snap install whatsapp-for-linux
error: snap 'whatsapp-for-linux' not found

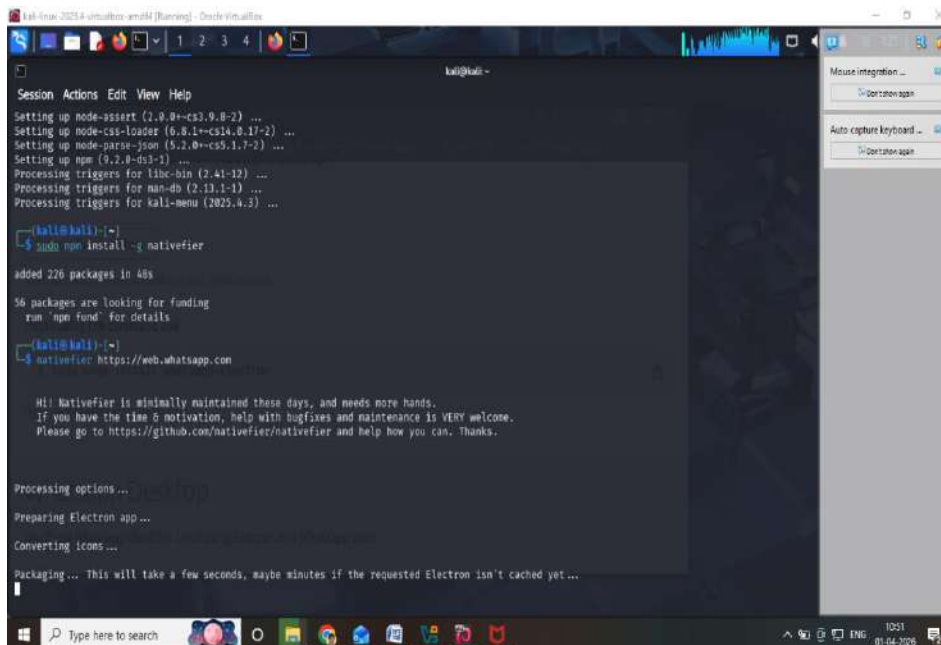
kali@kali:~$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
1014 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

sudo apt install nodejs npm -y



Step 3: Create WhatsApp Desktop App

nativefier <https://web.whatsapp.com>



It will create a folder like:

web-whatsapp-linux-x64

Step 4: Run WhatsApp

cd WhatsAppWeb-linux-x64

```
kali@kali:~/WhatsAppWeb-linux-x64
Session Actions Edit View Help
run 'npm fund' for details

(kali@kali)~$ nativefier https://web.whatsapp.com

Hi! Nativefier is minimally maintained these days, and needs more hands.
If you have the time & motivation, help with bugfixes and maintenance is VERY welcome.
Please go to https://github.com/nativefier/nativefier and help how you can. Thanks.

Processing options ...
Preparing Electron app ...
Converting icons ...
Packaging... This will take a few seconds, maybe minutes if the requested Electron isn't cached yet ...
Finalizing build...
App built to /home/kali/WhatsAppWeb-linux-x64, move to wherever it makes sense for you and run the contained executable file (prefixing with ./ if necessary)
Menu/desktop shortcuts are up to you, because Nativefier cannot know where you're going to move the app. Search for "linux .desktop file" for help, or see https://wiki.archlinux.org/index.php/Desktop_entries

(kali@kali)~$ cd web-whatsapp-linux-x64
cd: no such file or directory: web-whatsapp-linux-x64

(kali@kali)~$ cd WhatsAppWeb-linux-x64
cd: no such file or directory: WhatsAppWeb-linux-x64

(kali@kali)~$ cd ~/WhatsAppWeb-linux-x64
```

./WhatsAppWeb

```
kali@kali:~/WhatsAppWeb-linux-x64
Session Actions Edit View Help
Hi! Nativefier is minimally maintained these days, and needs more hands.
If you have the time & motivation, help with bugfixes and maintenance is VERY welcome.
Please go to https://github.com/nativefier/nativefier and help how you can. Thanks.

Processing options ...
Preparing Electron app ...
Converting icons ...
Packaging... This will take a few seconds, maybe minutes if the requested Electron isn't cached yet ...
Finalizing build...
App built to /home/kali/WhatsAppWeb-linux-x64, move to wherever it makes sense for you and run the contained executable file (prefixing with ./ if necessary)
Menu/desktop shortcuts are up to you, because Nativefier cannot know where you're going to move the app. Search for "linux .desktop file" for help, or see https://wiki.archlinux.org/index.php/Desktop_entries

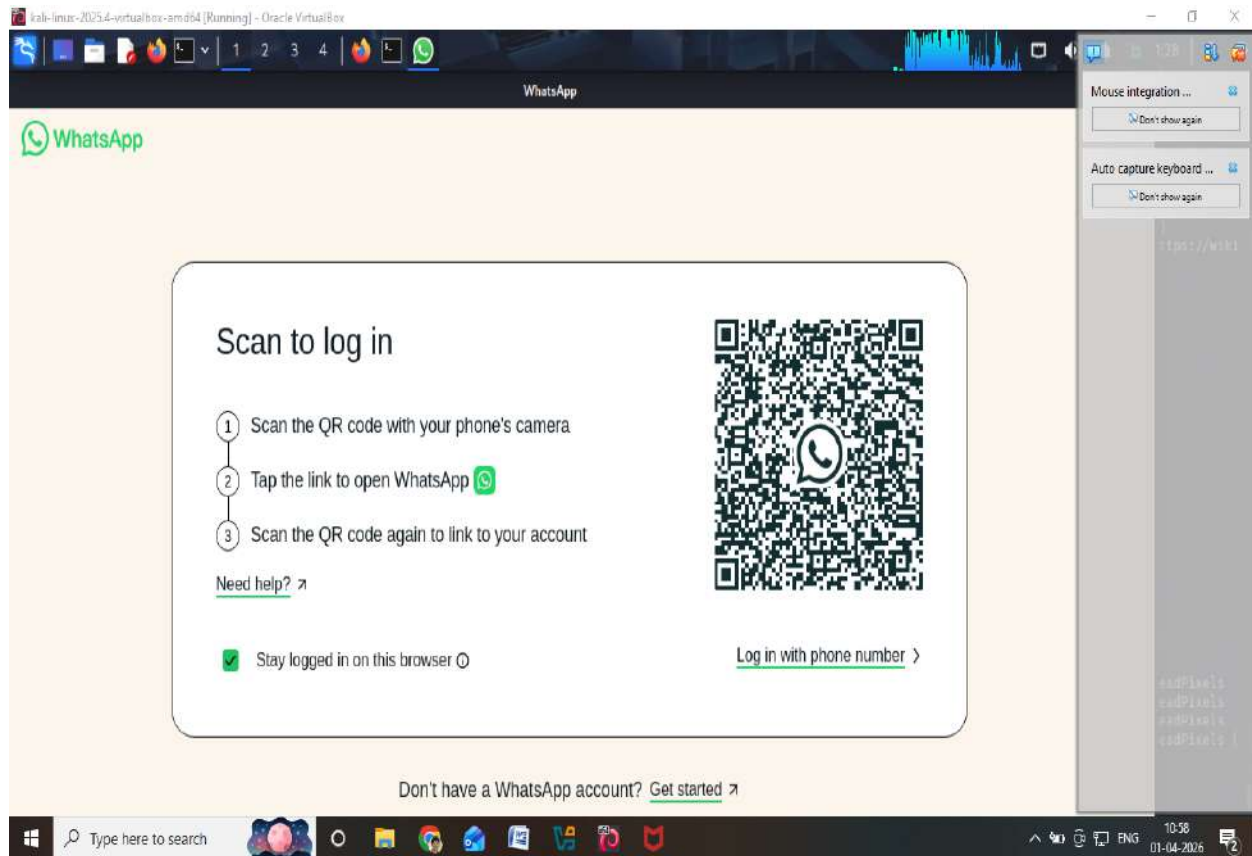
(kali@kali)~$ cd web-whatsapp-linux-x64
cd: no such file or directory: web-whatsapp-linux-x64

(kali@kali)~$ cd WhatsAppWeb-linux-x64

(kali@kali)~$ cd ~/WhatsAppWeb-linux-x64
$ ls
chrome_100_percent.pak  chrome-sandbox  libffmpeg.so  libvulkan.so.1  locales  snapshot_blob.bin  vk_swiftshader_icd.json
chrome_200_percent.pak  icudtl.dat      libGLSLv2.so  LICENSE          resources  v8_context_snapshot.bin  WhatsAppWeb
chrome_crashpad_handler  libEGL.so       libvk_swiftshader.so  LICENSES.chromium.html  resources.pak  version
```

✓ Step 5: Login

- Scan QR code using mobile



- Done

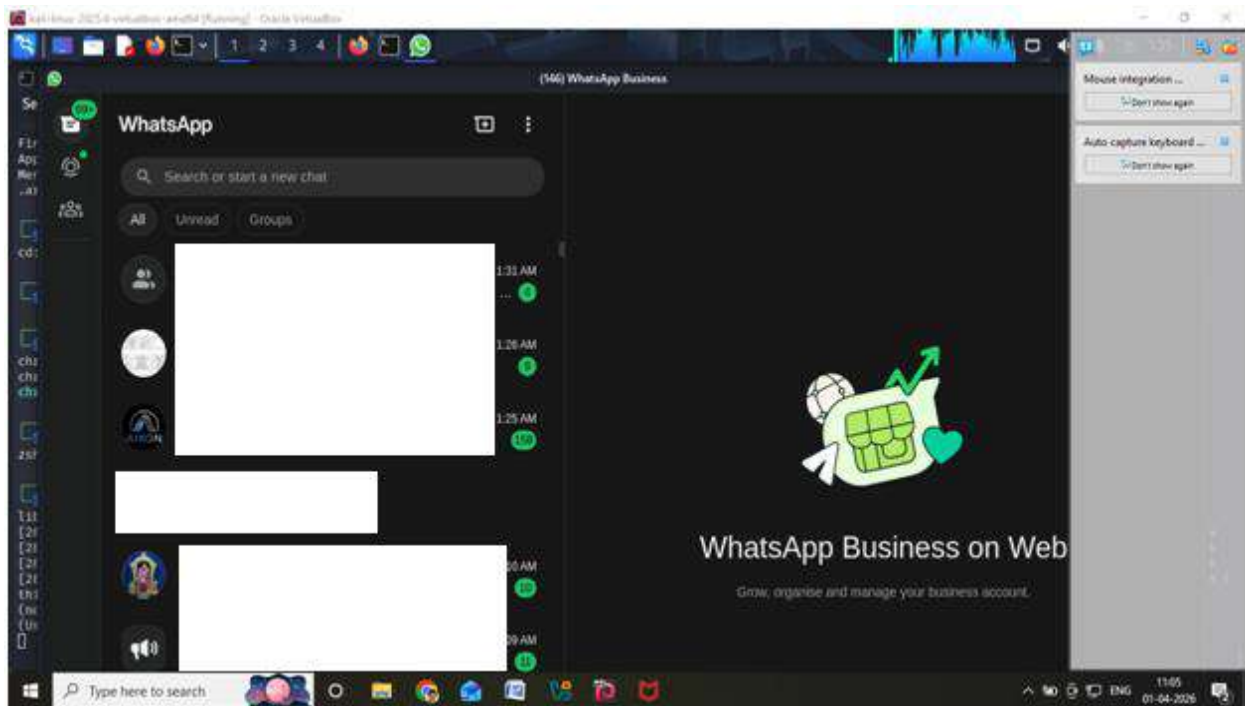
PART 1: Privacy Audit of Desktop WhatsApp

Step 1: Launch WhatsApp Desktop

cd WhatsAppWeb-linux-x64

./WhatsAppWeb

- QR Code appears
- Login using mobile → Linked Devices



Step 1: Install WhatsApp APK (Optional in Emulator)

- Download APK from trusted source or use your mobile

Step 2: Analyze Trackers using Exodus Privacy

1. Open browser in Kali
2. Go to: <https://reports.exodus-privacy.eu.org>

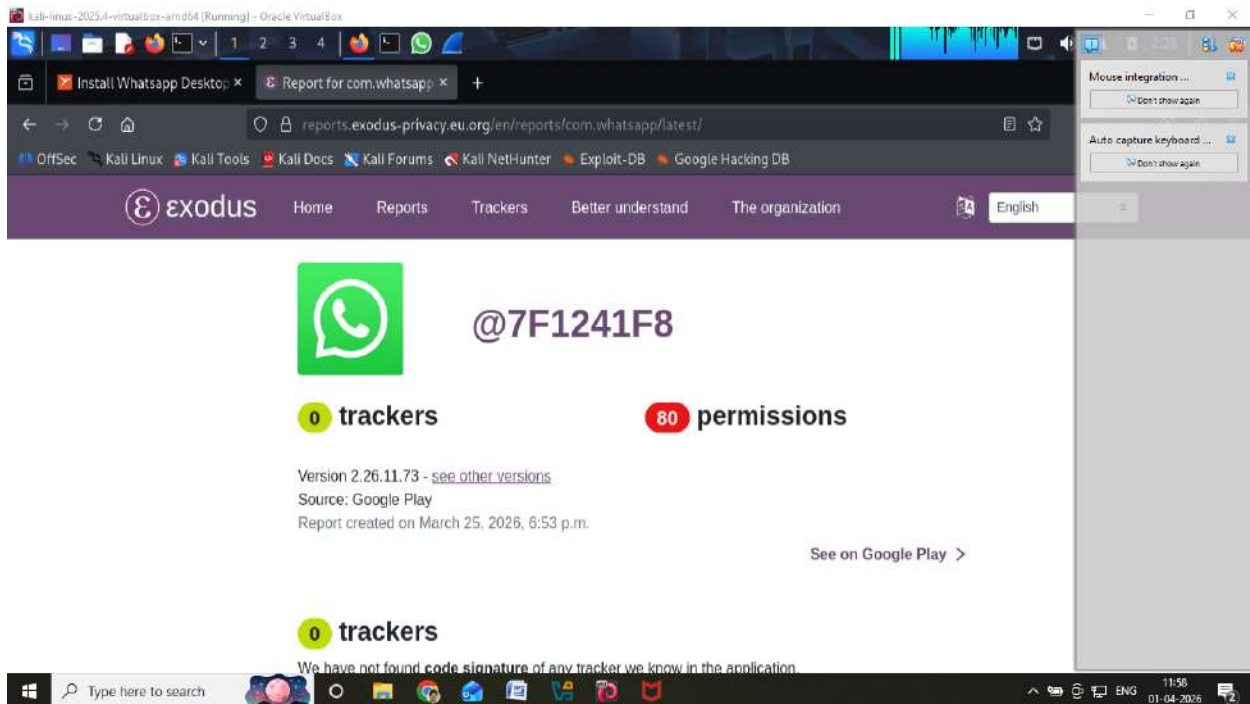


3. Search for **WhatsApp**

Observation:

- Detects trackers like:
 - Google Analytics
 - Crashlytics

- Facebook Analytics

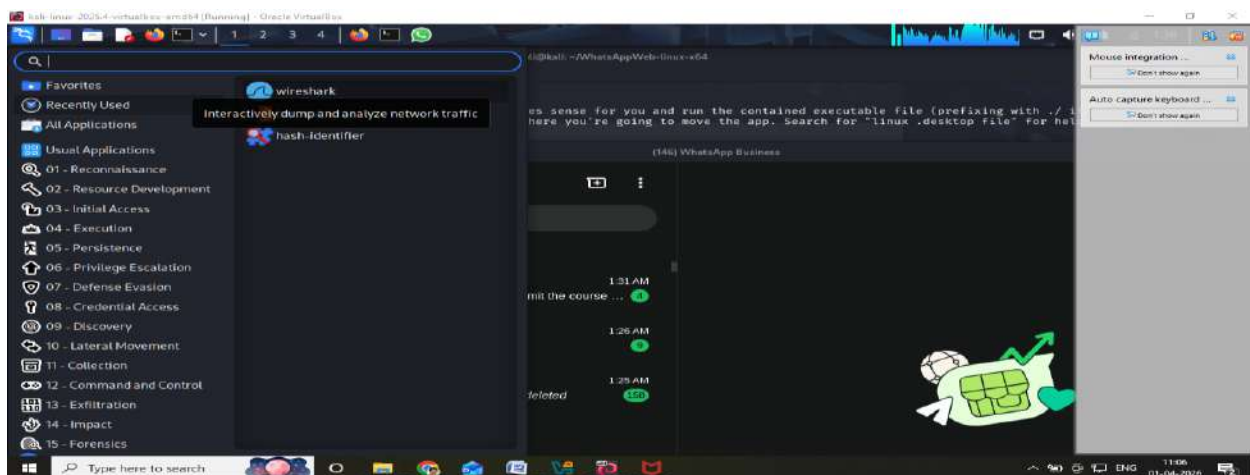


Record in lab:

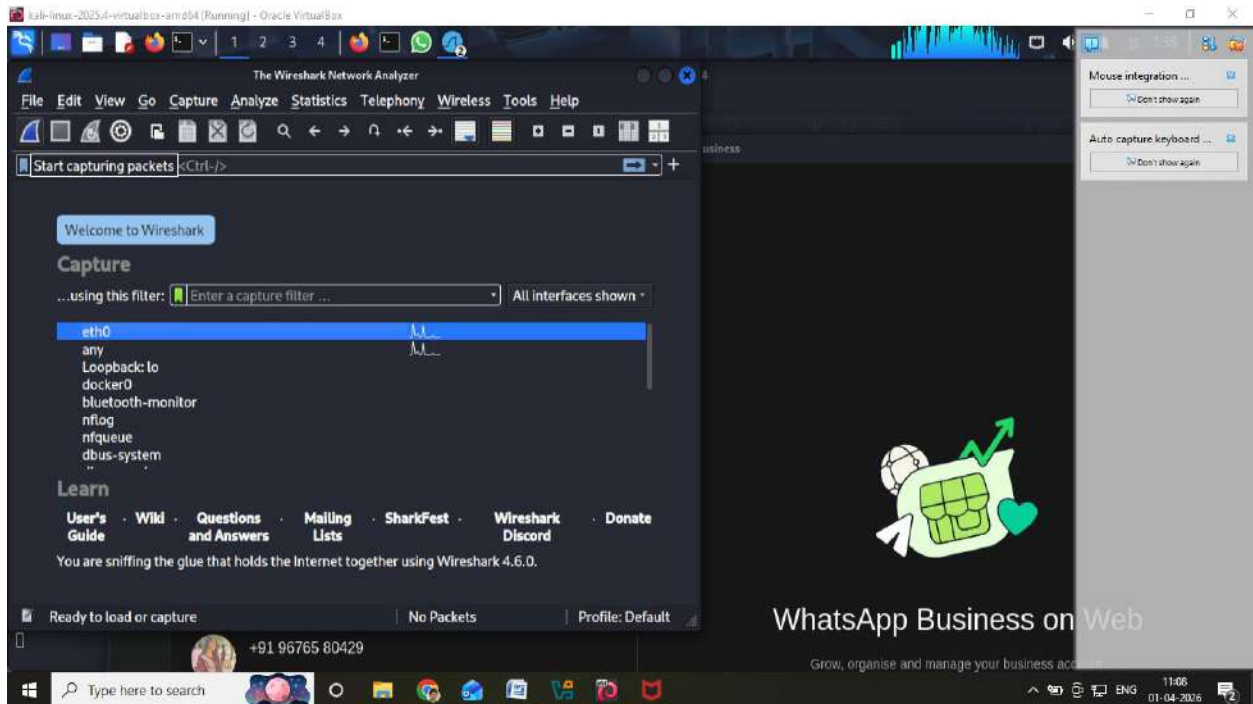
- Number of trackers
- Permissions used

Step 2: Start Wireshark

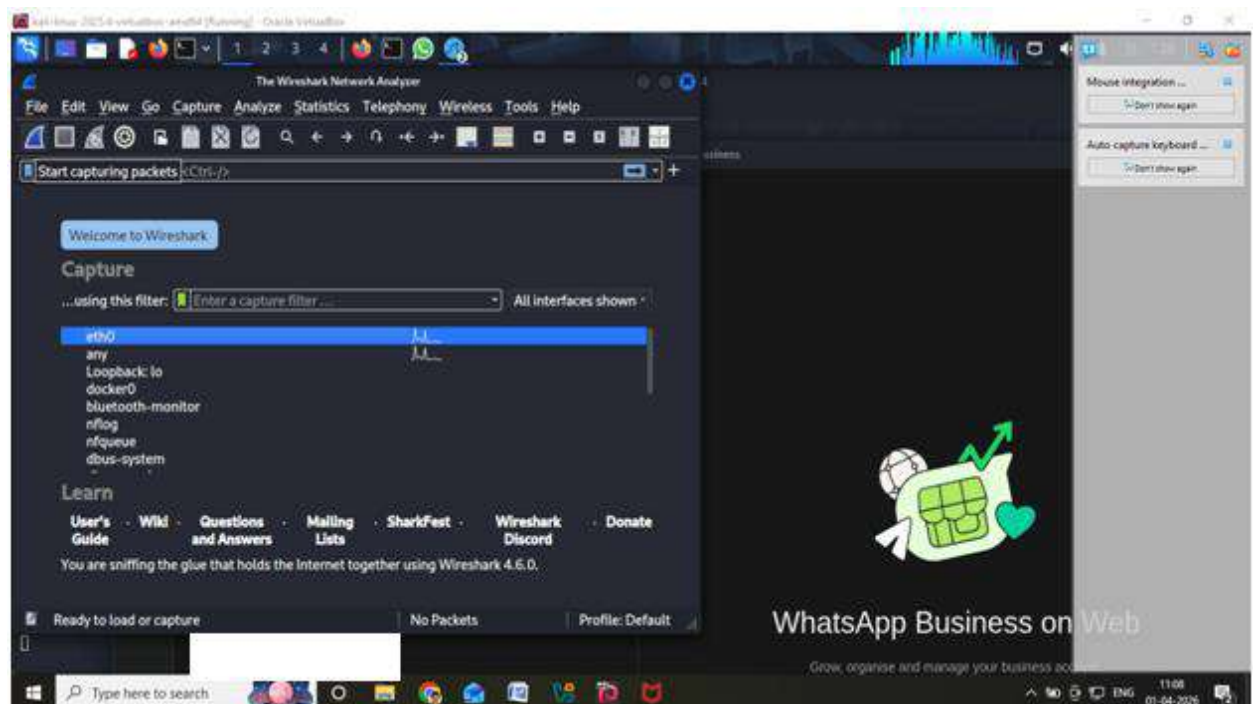
Wireshark



- Select interface (eth0 / wlan0)



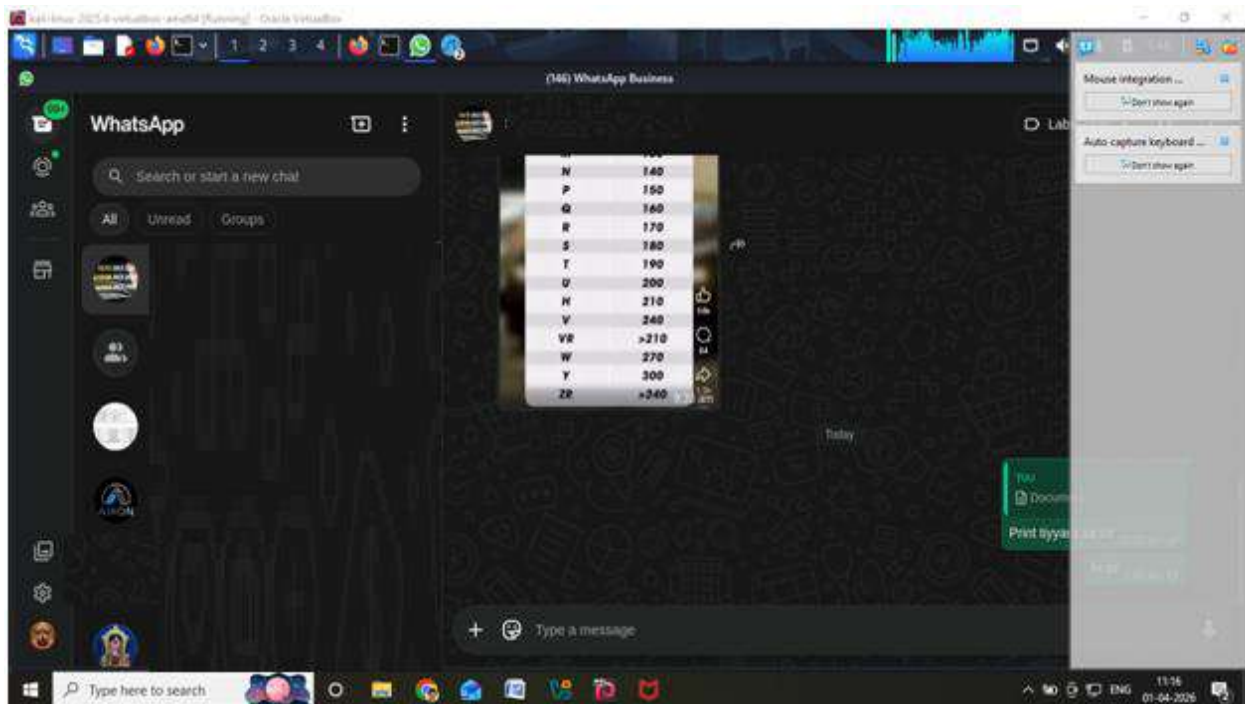
-
- Click **Start Capture**



Step 3: Generate Traffic

In WhatsApp Desktop:

- Send messages
- Send images
- Open chats

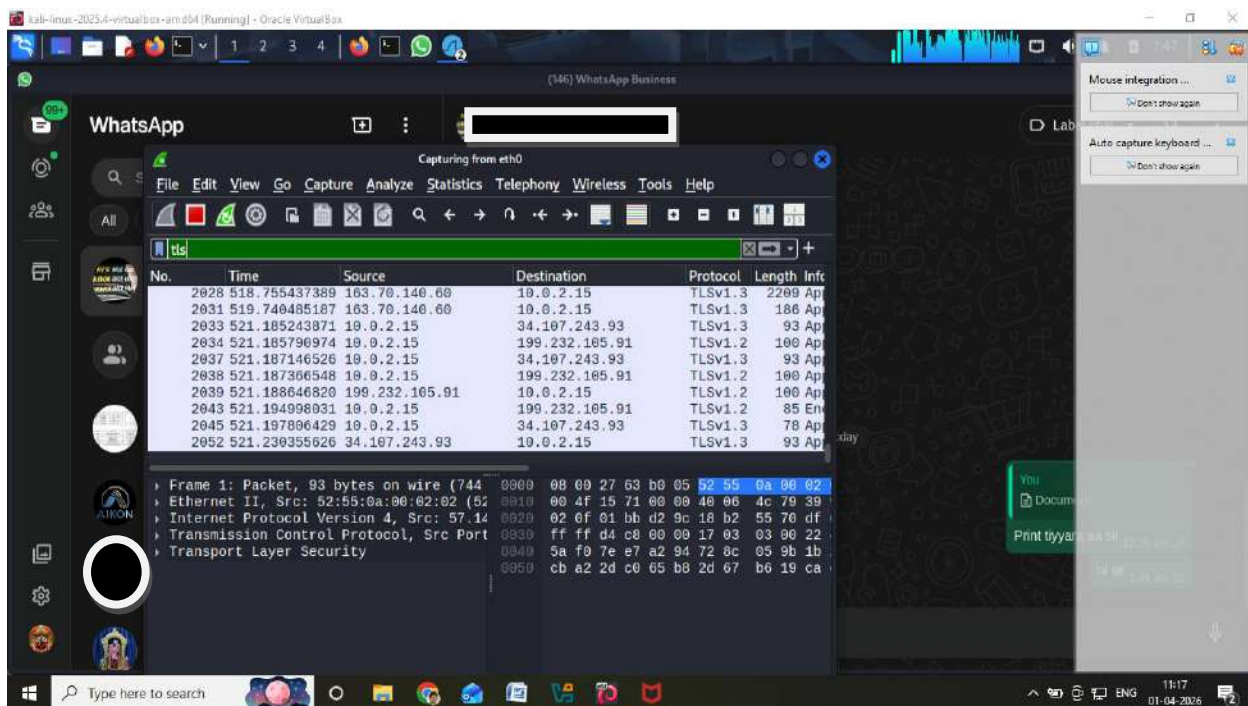


✓ Step 4: Apply Filters

Use filters:

tls

dns

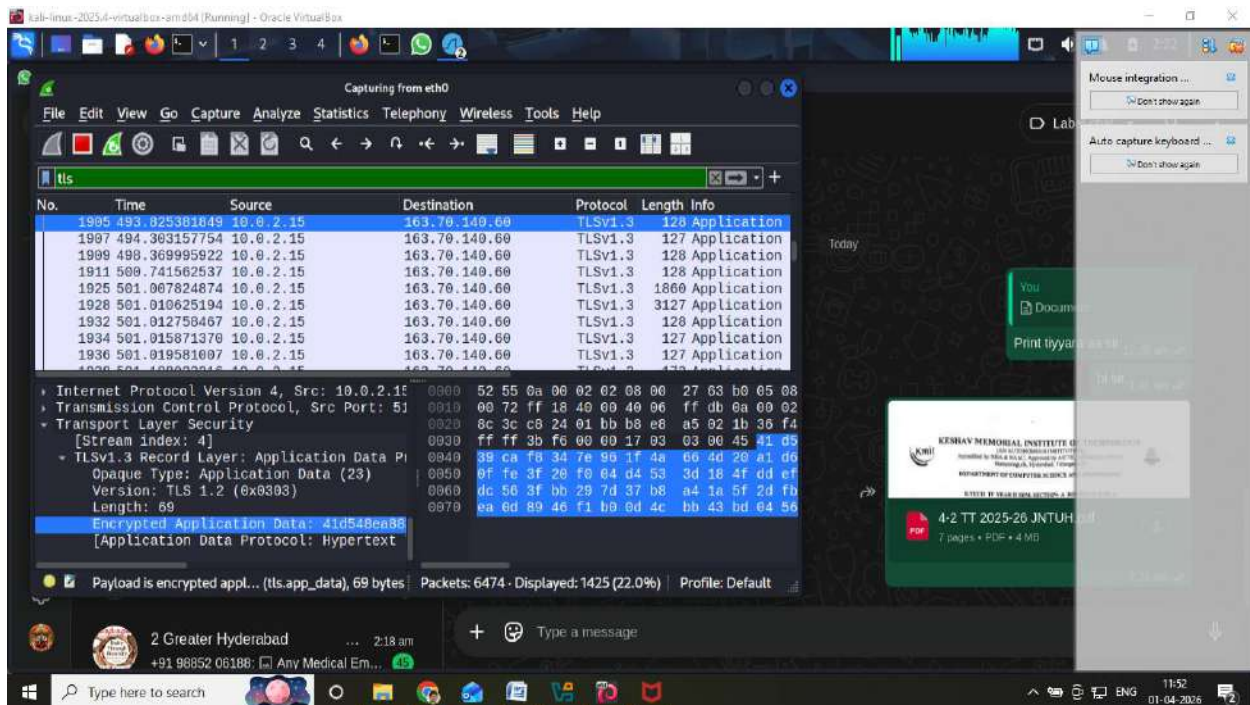


Observations

- Encrypted traffic (TLS packets)

- Domains:
 - whatsapp.net
 - facebook.com

Message content NOT visible



Findings

End-to-End Encryption

Secure communication

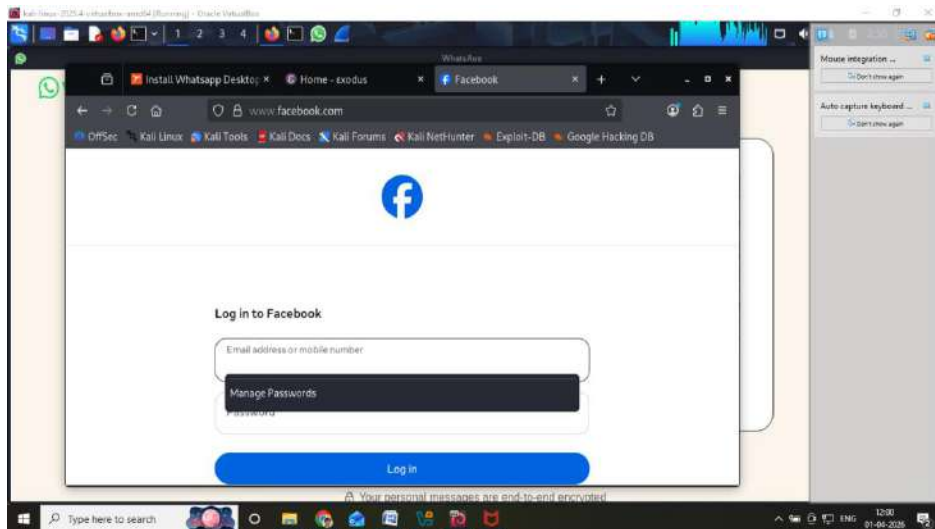
Metadata visible:

- IP address
- Time
- Server communication

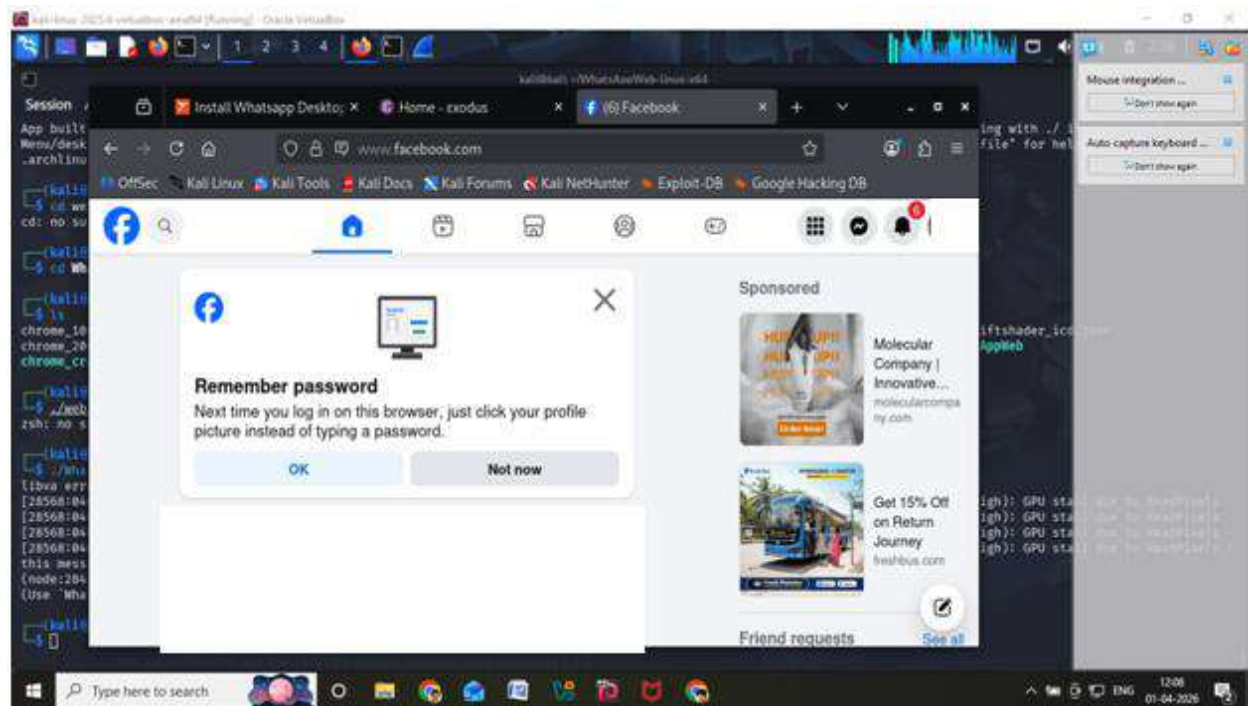
PART 2: Privacy Audit of Facebook Website

Step 1: Open Facebook

<https://www.facebook.com>

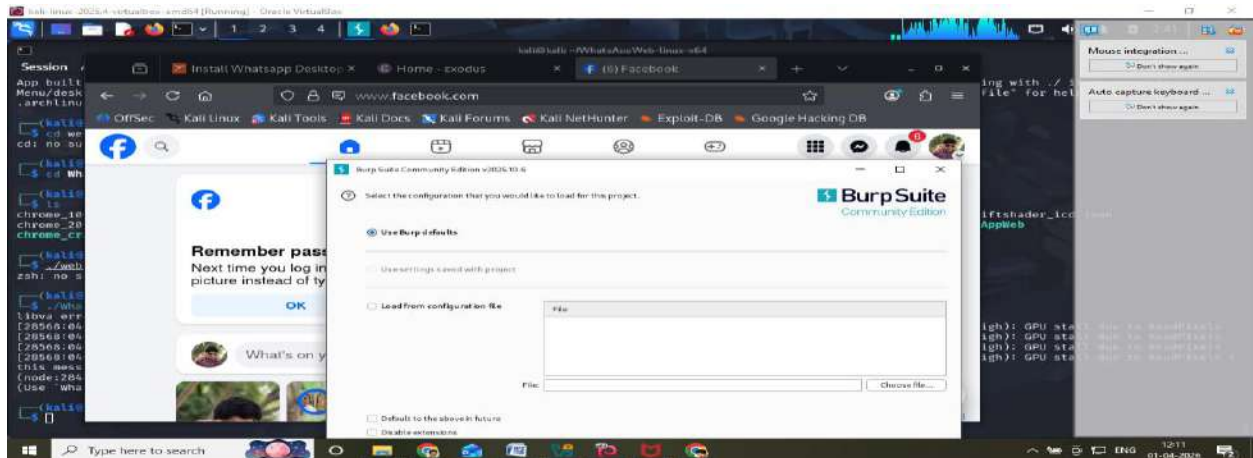
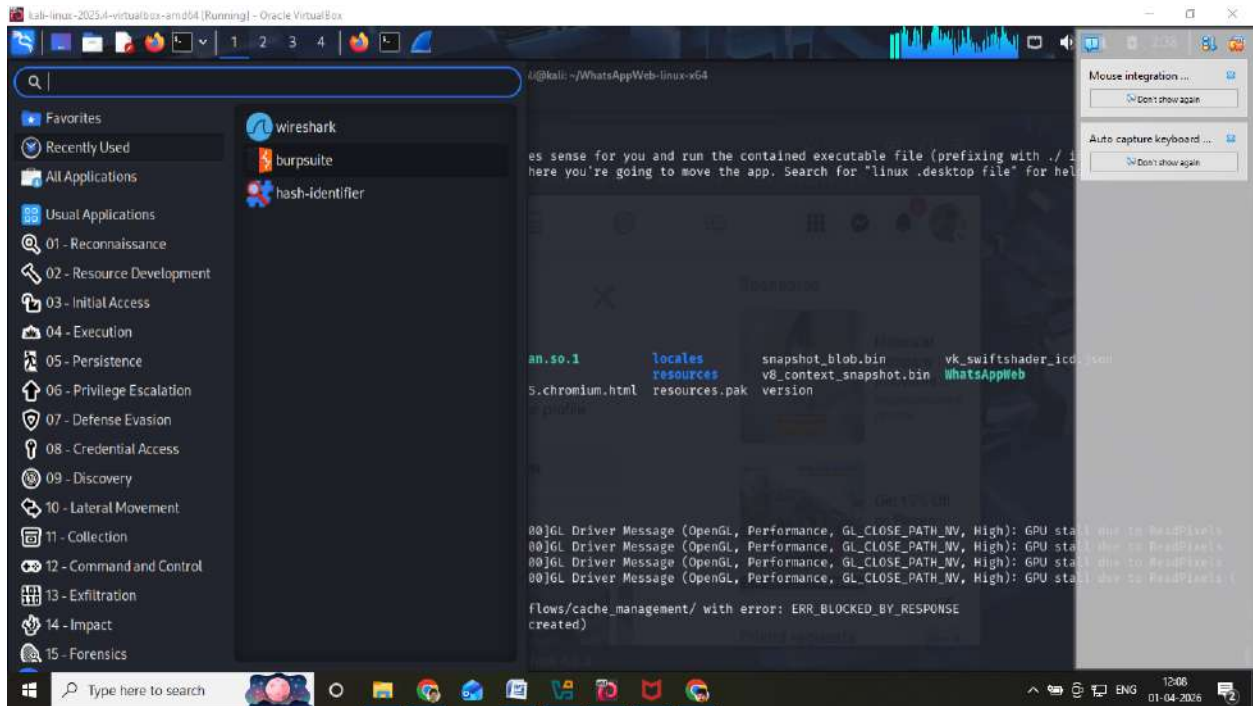


Login using test account



Step 2: Start Burp Suite

Burpsuite

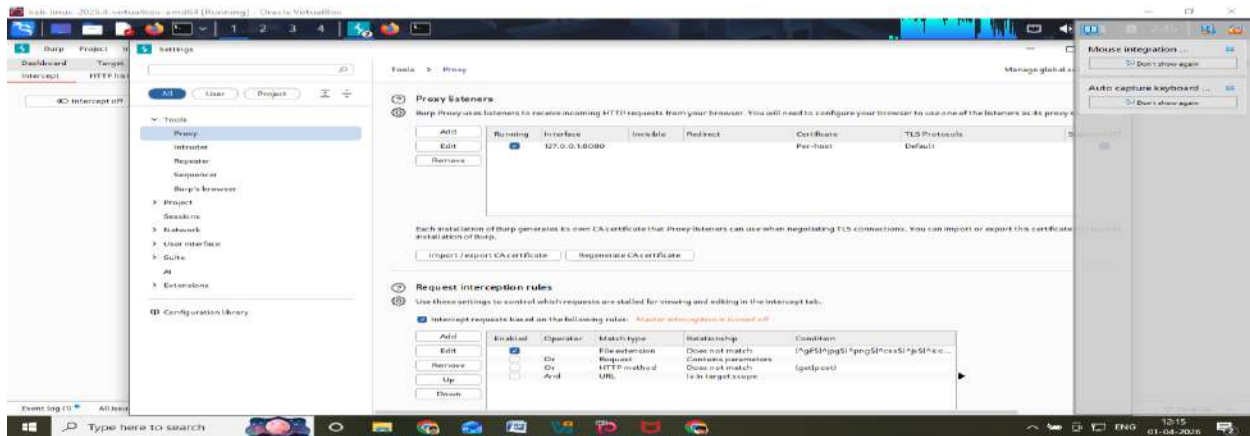


✓ Step 3: Configure Proxy

- IP: 127.0.0.1
- Port: 8080

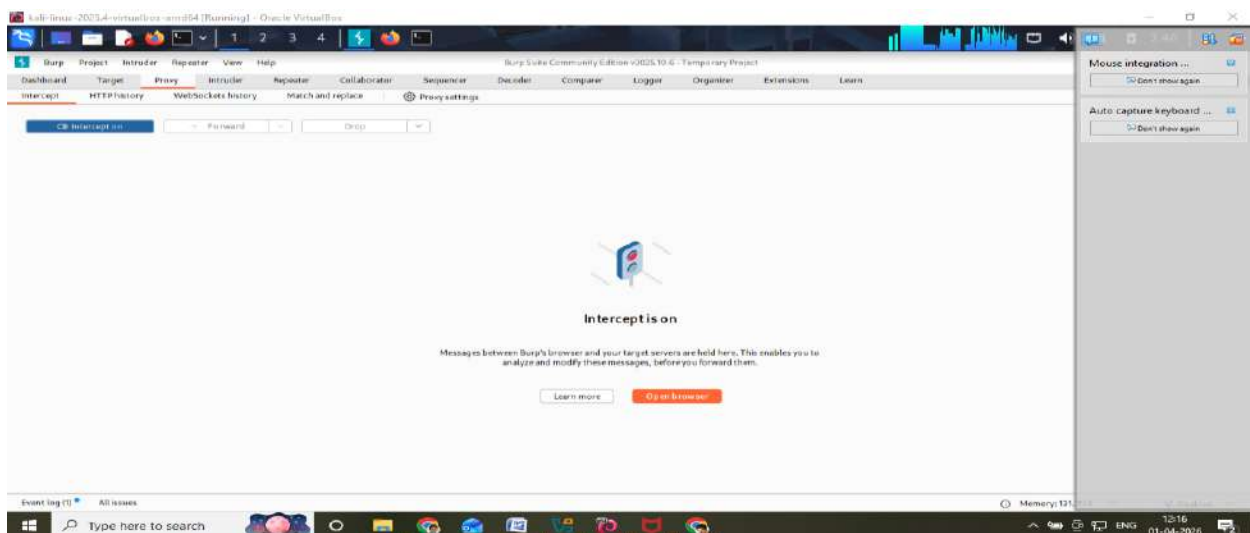
In browser:

- Settings → Network → Manual Proxy



✓ Step 4: Intercept Traffic

- Turn **Intercept ON**
- Reload Facebook



Observe:

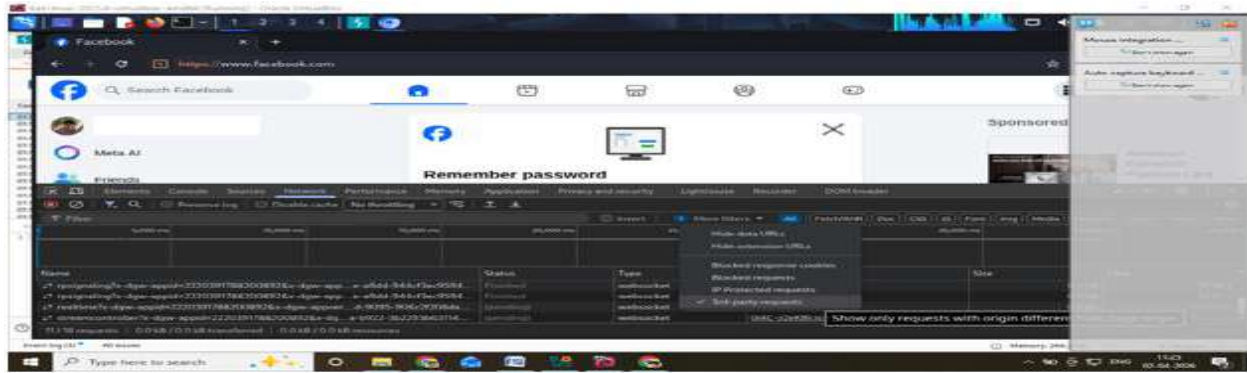
- Cookies (datr, fr)
- Session IDs
- Headers

Step 5: Analyze Tracking

Press **F12** → **Network** tab

Look for:

- Ads
- Analytics scripts
- Third-party requests



Findings

Heavy tracking

Uses cookies for personalization

Third-party integrations

PART 3: Data Breach Case Study

Case Study 1: Facebook Data Breach

- 533 million users affected
- Data leaked:
 - Phone numbers
 - Emails

Cause:

- API vulnerability

Case Study 2: WhatsApp Pegasus Attack

- Spyware attack via missed call
- Remote device access

Step 3: Check Email Breach

Go to:

👉 <https://haveibeenpwned.com>

Enter email → check breach status

Analysis Points

- Type of attack
- Vulnerability
- Data exposed
- Prevention

Observations

- WhatsApp uses strong encryption

- Facebook performs extensive tracking
- Data breaches occur due to vulnerabilities

Result

Privacy audit was successfully performed using Kali Linux tools. WhatsApp ensures secure communication through encryption, while Facebook shows extensive tracking behavior. Data breach analysis highlighted risks of poor security practices.

Experiment 21 :Conducting a Security Audit and Risk Assessment

1. Aim

To perform a security audit on a Windows system and identify potential vulnerabilities by conducting a risk assessment and suggesting mitigation strategies.

2. Tools Required

- Windows Operating System (Windows 10/11)
- Windows Security (Defender / Antivirus)
- Command Prompt
- Task Manager
- Web Browser (Google Chrome / Microsoft Edge)

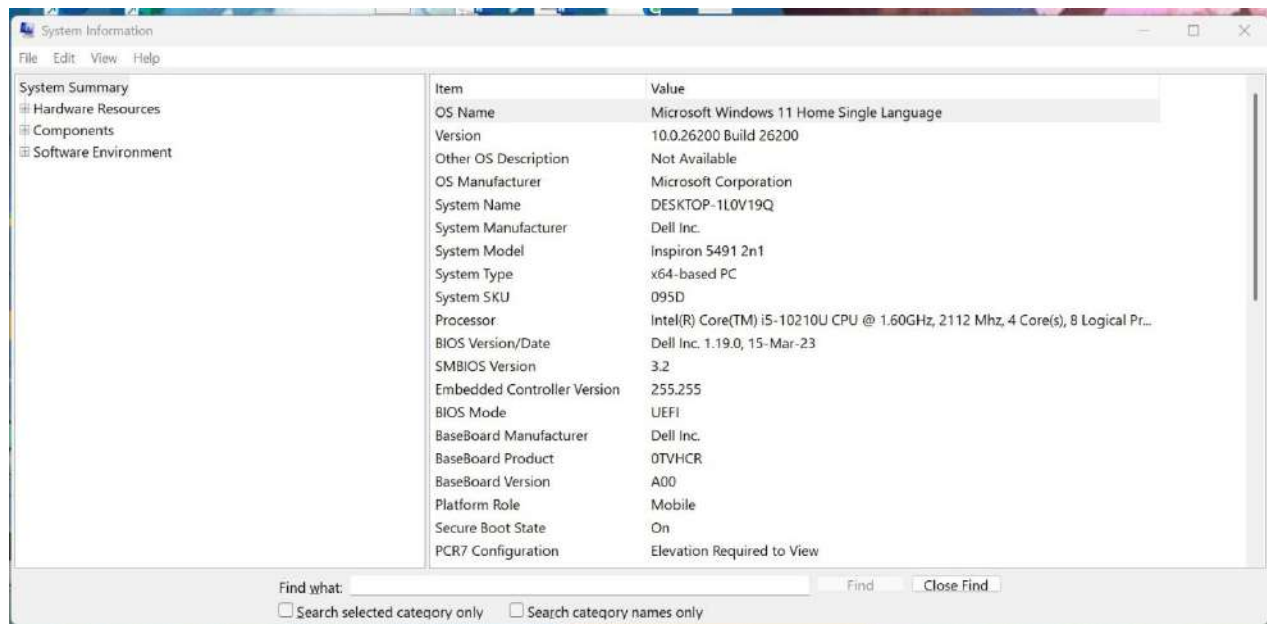
3. Introduction

A security audit is the process of evaluating a system's security measures to identify vulnerabilities. Risk assessment involves analyzing potential threats, vulnerabilities, and their impact on the system. It helps in implementing appropriate security controls to protect data and resources from cyber threats.

4. Procedure

STEP 1: System Information

- Press **Windows + R**
- Type: msinfo32 Note:
- OS Version
- System Type

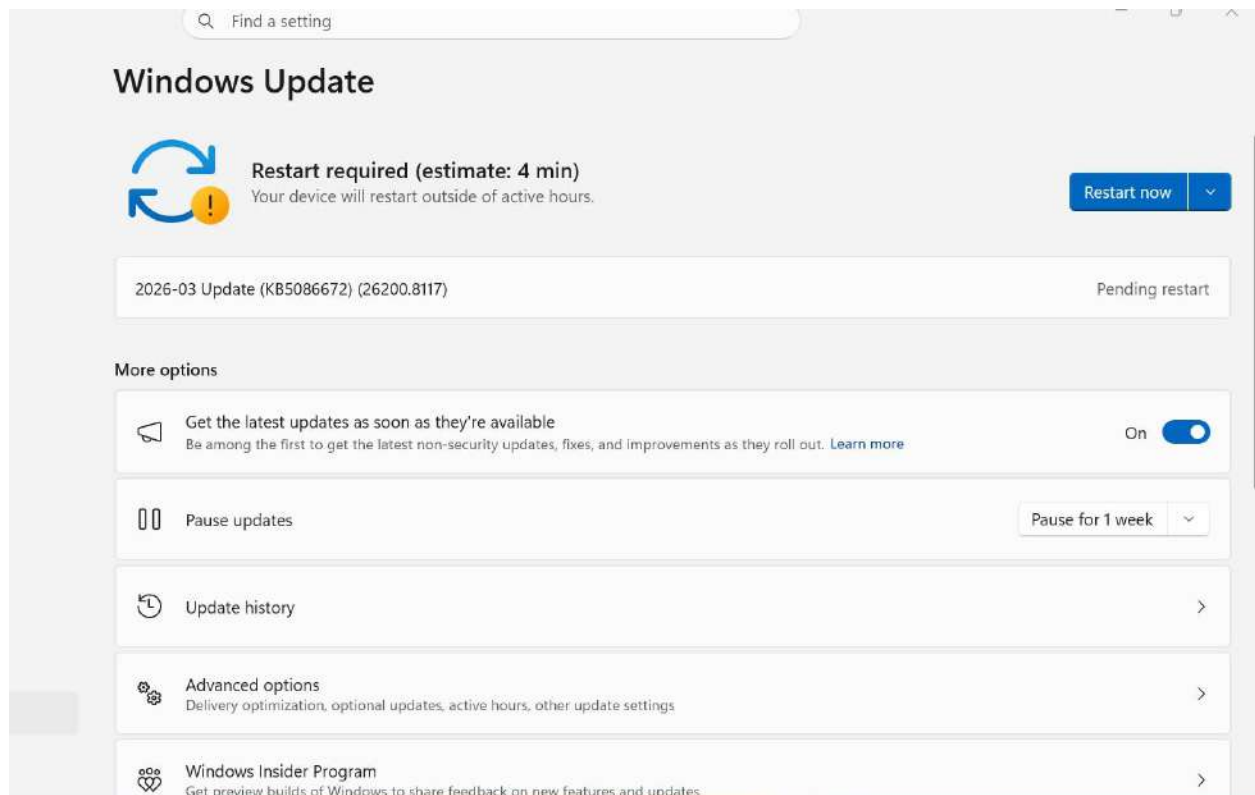


Observation:

The system information was obtained using the *msinfo32* tool. The system is running on Microsoft Windows 11 Home Single Language, 64-bit operating system, with Intel i5 processor and UEFI- based architecture. Secure Boot is enabled, indicating a secure boot configuration. Secure Boot being enabled enhances system security by preventing unauthorized boot loaders.

STEP 2: Check Windows Updates Steps:

- Open **Settings**
- Go to **Windows Update**
- Click **Check for updates**



Observation:

The Windows Update section was checked, and it was found that updates are available and pending installation. The system requires a restart to complete the update process.

Pending updates may expose the system to known vulnerabilities. Regular updates are essential to maintain system security and protect against threats.

STEP 3: Check Firewall Status Steps:

- Search: **Windows Defender Firewall**
- Click **Turn Windows Defender Firewall on or off**



Observation:

The firewall settings were checked using Windows Defender Firewall. It was observed that firewall settings are managed by Kaspersky antivirus, indicating that a third-party firewall is active on the system.

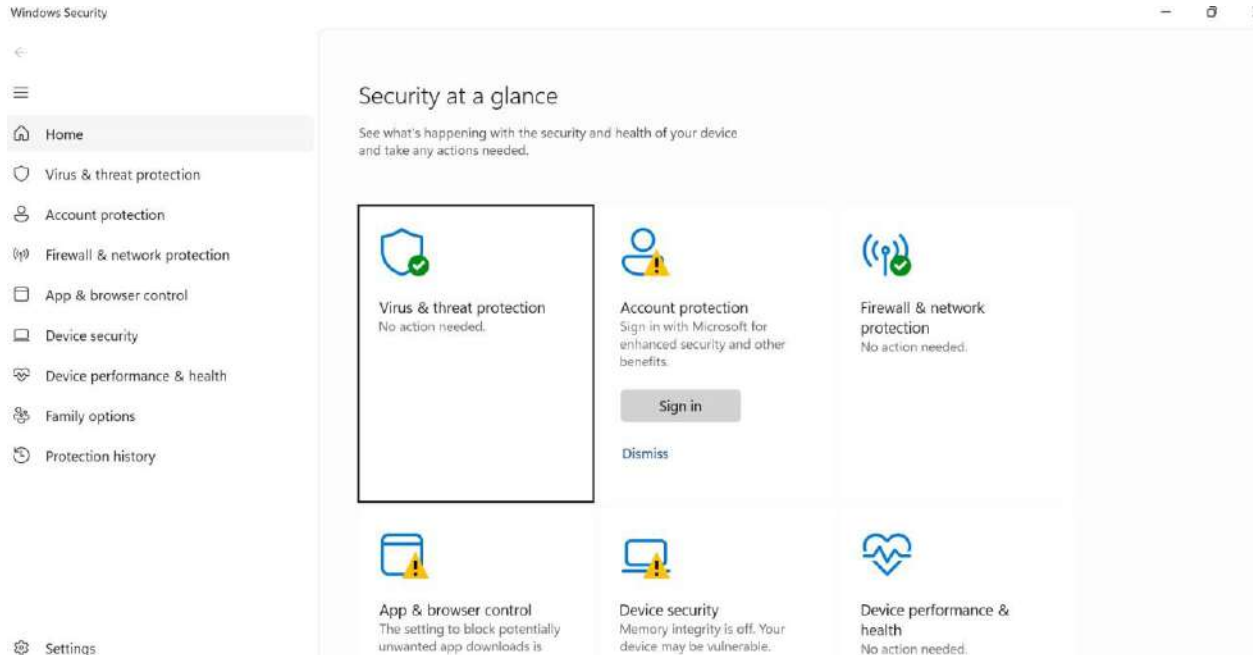
The presence of a third-party firewall ensures network protection; however, misconfiguration may still expose the system to risks.

STEP 4: Check Antivirus (Windows Security) Steps:

- Open **Windows Security**
- Go to **Virus & Threat Protection**

Check:

- Real-time protection → ON
- Last scan → Recent



Observation:

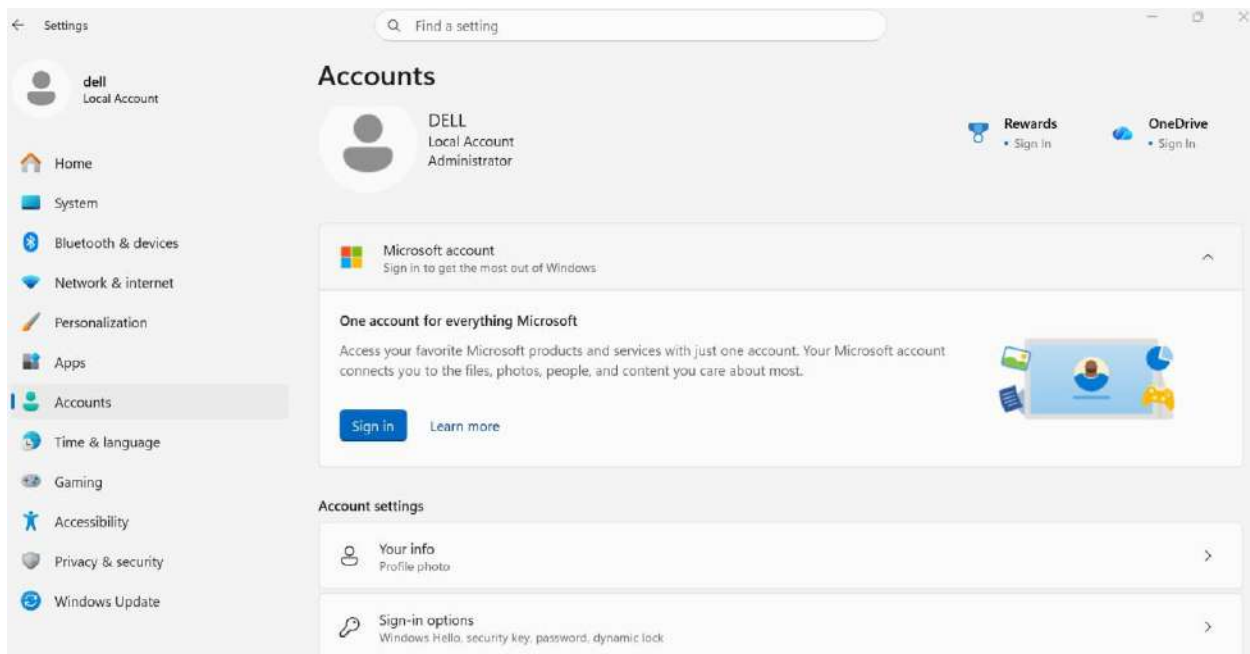
The antivirus status was verified under Virus and Threat Protection. It was observed that Kaspersky antivirus is active, and no current threats were detected. Protection settings and updates are functioning properly. A third-party antivirus like Kaspersky provides advanced protection against malware, phishing, and other cyber threats.

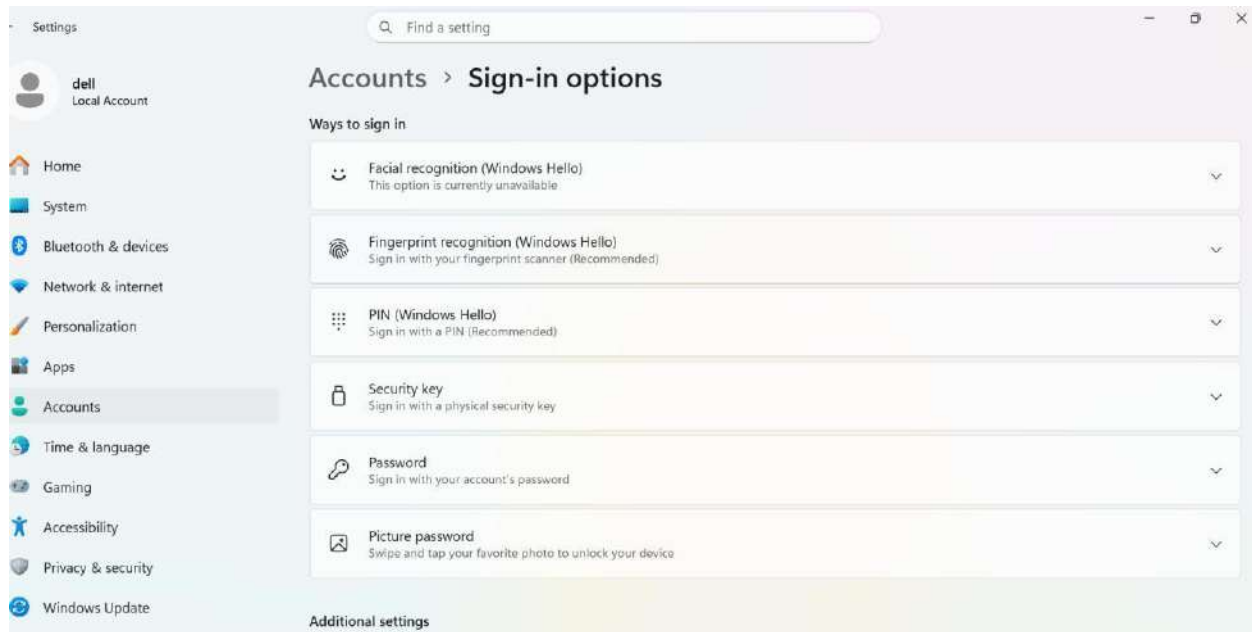
STEP 5: Check Password & Account Security Steps:

- Settings → **Accounts** → **Sign-in options**

Check:

- Password / PIN enabled
- Windows Hello (optional)





Observation:

The account settings and sign-in options were checked. The system uses a local administrator account with multiple authentication methods such as password and PIN (Windows Hello), ensuring secure access.

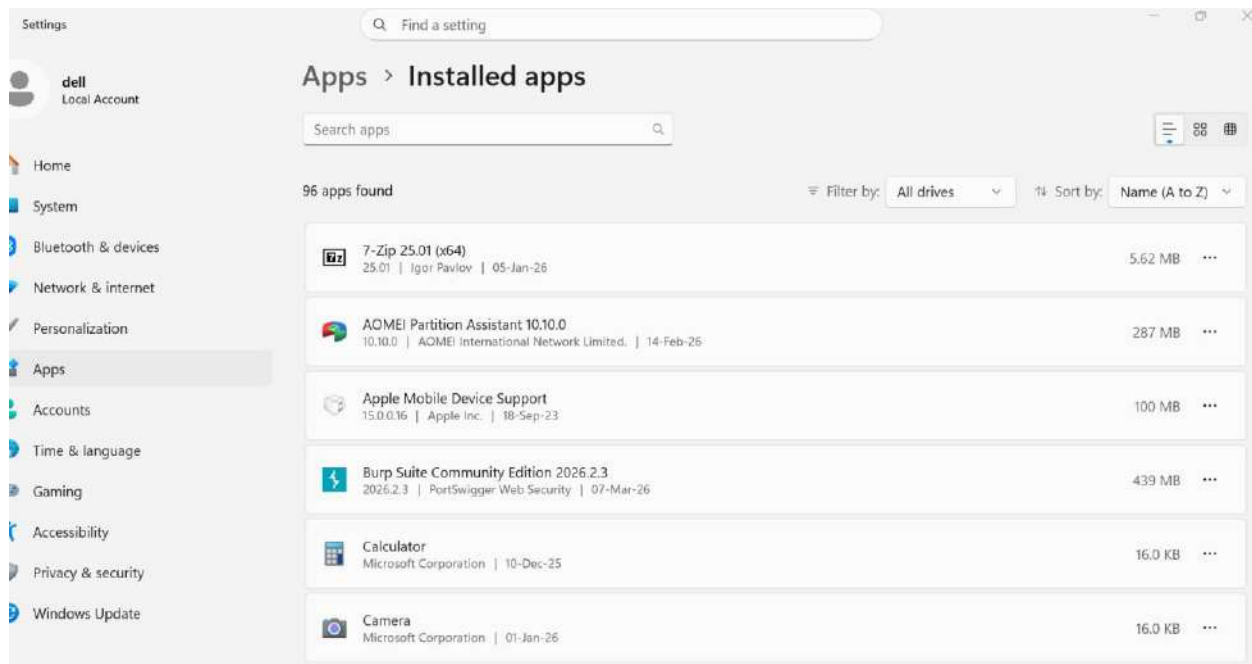
Strong authentication mechanisms like PIN and biometric options enhance system security and prevent unauthorized access.

STEP 6: Check Installed Applications Steps:

- Settings → **Apps** → **Installed apps**

Look for:

- Unknown software
- Cracked tools



Observation:

The list of installed applications was reviewed. No unauthorized or suspicious software was identified in the system.

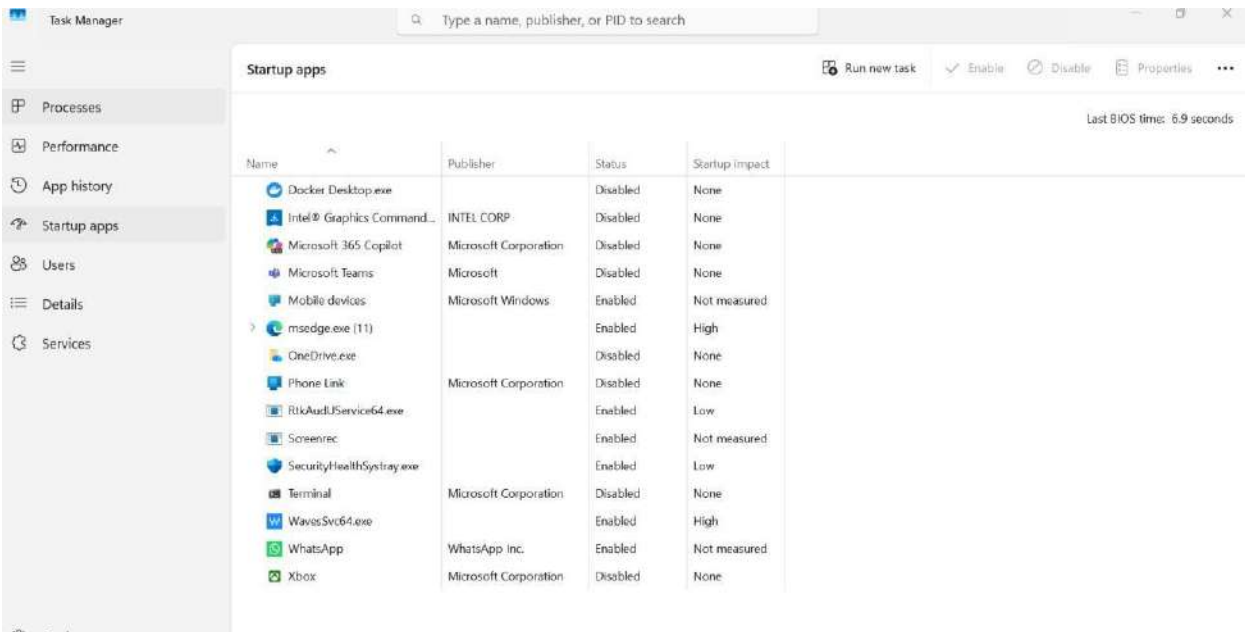
Unverified or cracked software may introduce malware and compromise system security.

STEP 7: Check Startup Programs Steps:

- Press **Ctrl + Shift + Esc**
- Go to **Startup tab**

Disable:

- Unknown apps

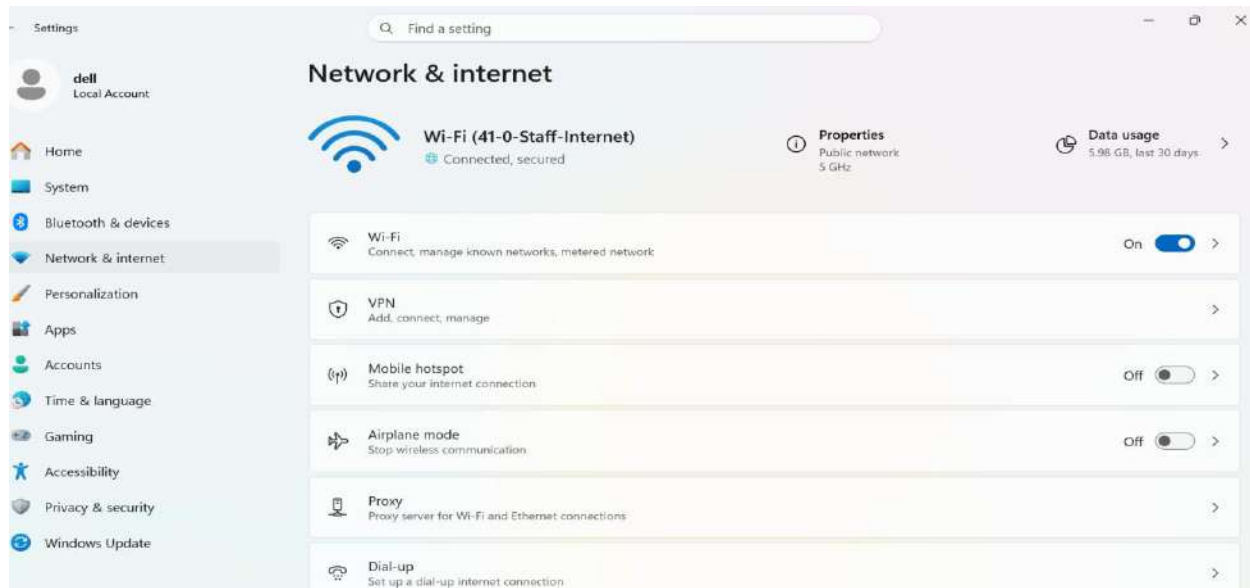


Observation:

Startup applications were analyzed using Task Manager. Some applications were enabled at startup, which may impact system performance and security. Disabling unnecessary startup applications reduces attack surface and improves system performance.

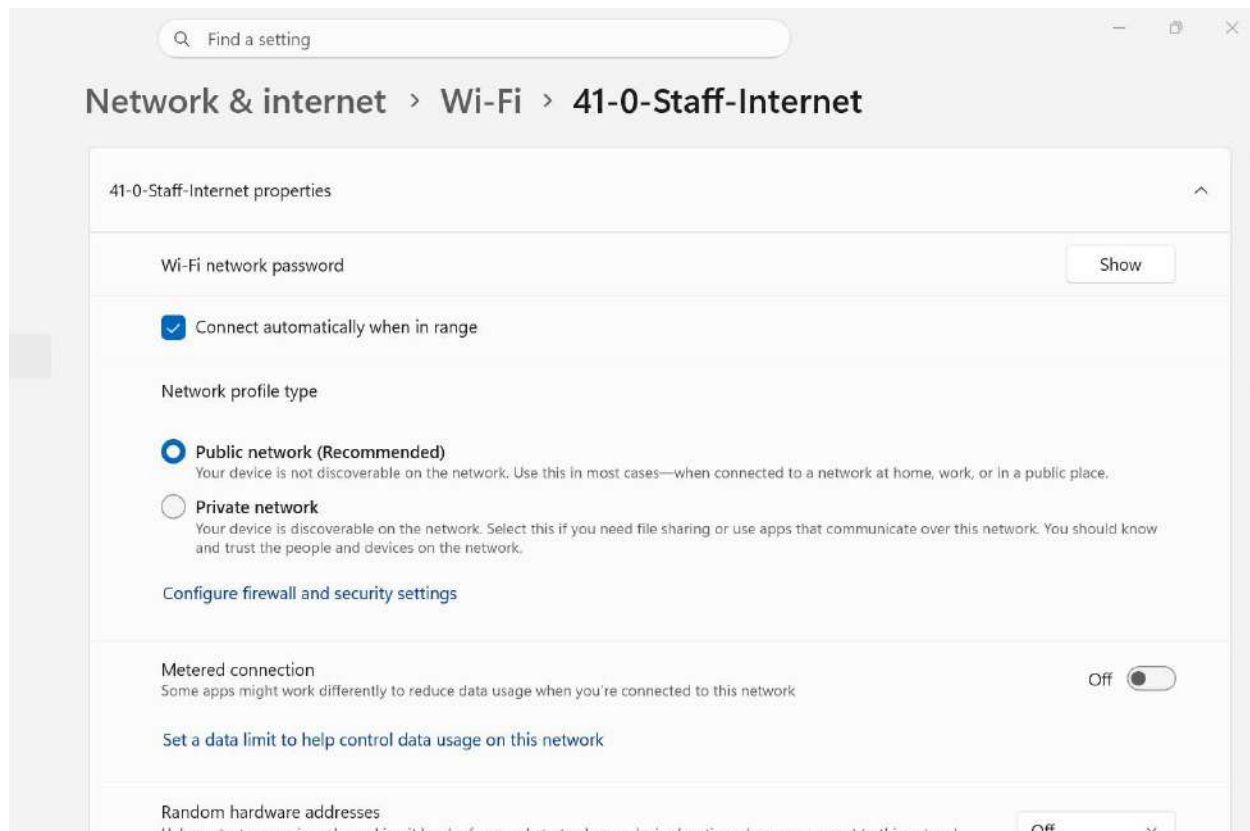
STEP 8: Check Network Security Steps:

- Settings → **Network & Internet**
- Check:
 - WiFi connection
 - Network type → Private/Public



Network Overview

- Wi-Fi connected → **41-0-Staff-Internet**
- Status → **Connected, secured**
- Shows general network info



Network Type

- Network profile → **Public network (selected)**
- Private option available

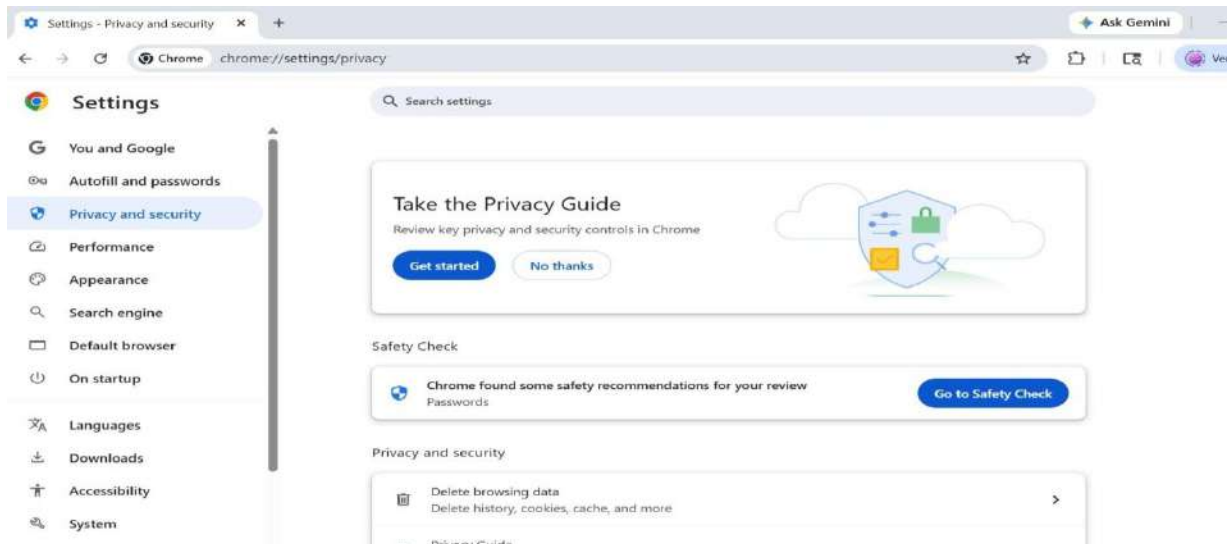
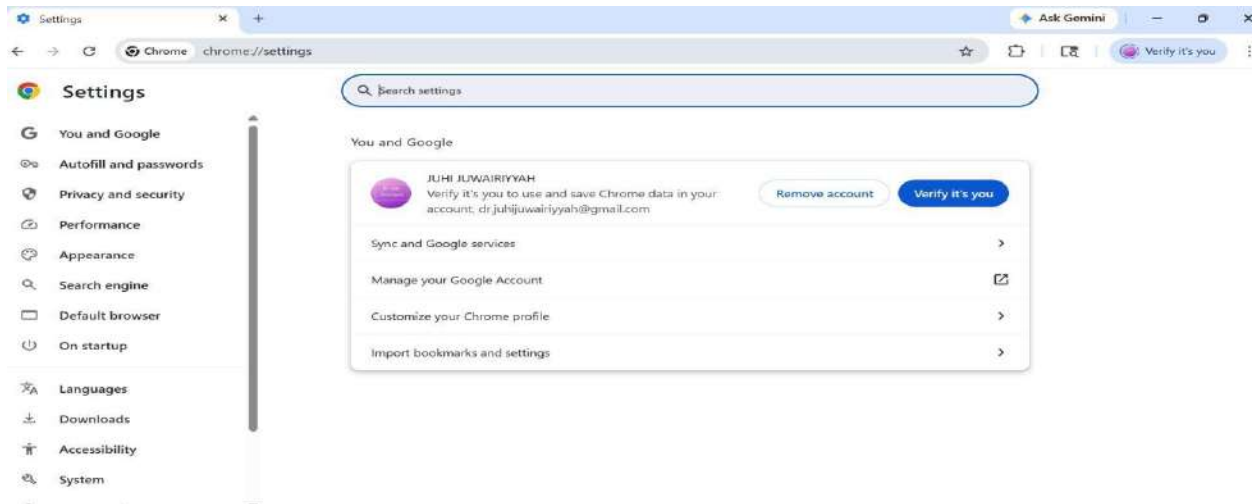
Observation:

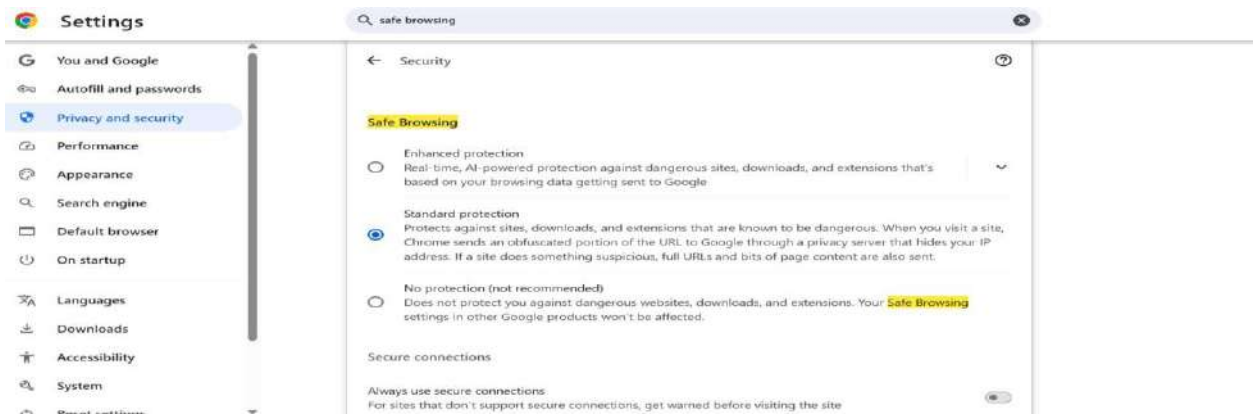
The network settings were analyzed, and the system is connected to a Wi-Fi network with a public network profile. This restricts device visibility and enhances security.

Using a public network profile improves security by limiting unauthorized access, especially in untrusted environments.

STEP 9: Check Browser Security Steps (Chrome/Edge):

- Go to **Settings → Privacy & Security**
- Check:
 - Safe browsing ON
 - Remove unknown extensions





Observation:

The browser security settings were examined under the Privacy and Security section. Safe Browsing was found to be enabled with standard protection, ensuring protection against malicious websites and unsafe downloads.

Safe Browsing helps prevent phishing attacks, malware infections, and unsafe browsing activities.

STEP 10: Check Data Backup Steps:

- Search: **Backup settings**
- Check:
 - OneDrive / External backup

: lab-user



Observation:

The backup settings were checked, and it was observed that OneDrive backup is not configured and system data is not being backed up.

Lack of proper backup mechanisms may lead to permanent data loss in case of system failure or cyber attacks such as ransomware.

STEP 11: Risk Assessment Table

Asset	Threat	Vulnerability	Risk Level	Solution
Personal Files	Data Loss	No Backup	High	Enable OneDrive
System	Malware	Unknown Apps	High	Remove apps
Network	Hacking	Public Network	Medium	Use Private
Account	Unauthorized Access	Weak Password	High	Strong password

STEP 12: Security Recommendations

✓ Technical Controls

- Enable firewall
- Enable antivirus
- Regular updates

Administrative Controls

- Strong passwords
- Avoid unknown downloads

User Awareness

- Avoid phishing links
- Safe browsing

STEP 13: Result

The system security audit identified potential vulnerabilities such as lack of backup and configuration risks. Suitable mitigation strategies were suggested to improve system security.

